YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 1 Introduction

Yazılım Mühendisliğine Giriş

Şu ana kadar işlenen: kitap s.3–19 (Bölüm 1 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 1, kitabın geri kalanını anlamak için temel bir çerçeve sunuyor. İki ana kısımdan oluşuyor:

- **1.1 FAQs about software engineering:** "Yazılım nedir?", "Yazılım mühendisliği nedir?", "Süreç nedir?", "İyi yazılımın özellikleri nelerdir?" gibi temel soruların cevapları.
- **1.2 Professional and ethical responsibility:** Yazılım mühendislerinin profesyonel ve etik sorumlulukları, ACM/IEEE etik kuralları.

Bölümün amaçları: (1) yazılım mühendisliğinin ne olduğunu ve neden önemli olduğunu anlamak, (2) konuya giriş sağlayan temel soruların cevaplarını bilmek, (3) yazılım mühendisleri için önemli etik ve profesyonel konuları anlamak.

Sayfa 4 Giriş · Yazılım krizi ve yazılım mühendisliğinin doğuşu

📖 Sayfada ne anlatılıyor (paragraf paragraf)

PARAGRAF 1

Ülkeler, altyapılar, elektrikli cihazlar, sanayi ve finans sistemi bilgisayar sistemlerine bağımlı. Bu yüzden yazılımı **düşük maliyetle (cost-effectively)** üretmek ve bakımını yapmak ekonomi için hayati.

PARAGRAF 2

Yazılım mühendisliği, kaliteli yazılımın düşük maliyetle geliştirilmesine odaklanır. Yazılım **soyut ve elle tutulamazdır (abstract, intangible)**. Malzeme, fizik kuralı ya da üretim süreci onu sınırlamaz. Bu işi kolaylaştırır, ama doğal sınır olmadığı için yazılım kolayca aşırı karmaşık ve anlaşılmaz olur.

PARAGRAF 3

"Yazılım mühendisliği" kavramı ilk kez **1968'de, "yazılım krizi" (software crisis)** üzerine bir konferansta önerildi. Krizin kaynağı **entegre devrelere (integrated circuits)** dayalı yeni donanımdı. Bu donanım eskiden imkânsız olan uygulamaları mümkün kıldı, yazılımlar kat kat büyüdü ve karmaşıklaştı.

PARAGRAF 4

Gayriresmi (informal) geliştirme yetersiz kaldı. Projeler yıllarca gecikti, tahminden pahalıya mal oldu, güvenilmezdi, bakımı zordu, performansı kötüydü. Donanım ucuzlarken yazılım pahalılaştı ve karmaşıklığı kontrol edecek yeni yöntemler gerekti.

PARAGRAF 5

Bu teknikler artık yaygın. Ama yazılım üretme yeteneğimiz arttıkça istenen sistemler de karmaşıklaştı: bilgisayar ve iletişim sistemleri birleşti, grafik arayüzler karmaşıklaştı. Teknikleri etkili uygulamayan şirketlerde projeler hâlâ **güvenilmez, geç ve bütçe üstü**.

PARAGRAF 6

Yazara göre 1968'den beri büyük ilerleme var. Geliştirme faaliyetleri daha iyi anlaşılıyor. Tanımlama, tasarım ve gerçekleştirme (specification, design, implementation) için etkili yöntemler geliştirildi. Yeni gösterimler (notations) ve araçlar emeği azaltıyor.

PARAGRAF 7

Tek bir "ideal" yaklaşım yok. Sistem ve kurum türleri çok çeşitli olduğu için farklı yaklaşımlar gerekir. Ama hepsinin temelinde **süreç (process)** ve **sistem organizasyonu** vardır. Yazılım mühendisliğinin özü budur.

👨‍🏫 Hoca anlatımı

Bu sayfa "bu ders neden var?" sorusunun cevabı. Bugün banka, elektrik, fabrika, her şey yazılımla dönüyor. Yazılım kötü ya da pahalı olursa ekonomi zarar görür. Yazılım mühendisliğinin işi bu yüzden basit: **kaliteli yazılımı makul maliyetle üretmek.**

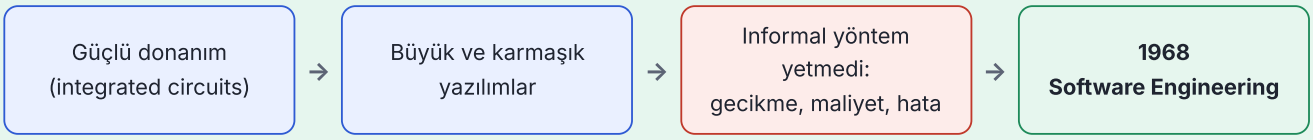
Yazılımın garip bir yanı var. Köprü yaparken çeliğin dayanımı, yer çekimi gibi sınırların vardır. Yazılımda böyle sınırlar yok, ne istersen yazabilirsin. Tuzak da burada: sınır olmayınca yazılım kontrolsüzce büyüyüp **karmaşıklaşıyor**.

1960'larda bilgisayarlar birden güçlendi ve insanlar dev yazılımlar yazmaya kalktı. "Otur, kodla" usulü çalışınca projeler felakete döndü: geç kaldı, pahalıya patladı, sürekli bozuldu. Buna **yazılım krizi** dendi ve **1968'de** "bu işi mühendislik gibi yapmalıyız" fikri doğdu.

Sorun bitmedi, çünkü biz geliştikçe istenen sistemler de büyüdü. Son mesaj da şu: **herkese uyan tek bir doğru yöntem yok**, ama hepsinin ortak temeli **süreç** ve **organizasyon**.

📝 Deftere not

- Ülkeler, altyapılar, üretim ve finans **bilgisayar sistemlerine bağımlı**. Bu yüzden yazılımı cost-effective üretmek ve bakımını yapmak ekonomi için şart.
- **Yazılım mühendisliği:** kaliteli yazılımın düşük maliyetle geliştirilmesine odaklanan mühendislik disiplini.
- Yazılım = abstract + intangible (soyut, elle tutulamaz)
  - Malzeme, fizik kuralı ya da üretim süreci sınırı yok.
  - (+) Fiziksel sınır yok. (–) Kolayca **aşırı karmaşık** ve anlaşılmaz olur.
- **1968,** software crisis**:** "software engineering" kavramı bu konferansta doğdu.
  - **Sebep:** integrated circuits ile güçlü donanım, yazılımlar kat kat büyüdü ve karmaşıklaştı.
  - **Sorun:** informal geliştirme yetmedi.
  - **Sonuç:** yıllarca gecikme, tahminden pahalı, güvenilmez, bakımı zor, kötü performans.
  - Donanım ucuzlarken yazılım pahalılaştı, yeni teknik ve yöntem gerekti.
- **Bugün:** teknikler yaygın, ama sistemler de karmaşıklaştı (bilgisayar ve iletişim sistemlerinin birleşmesi, grafik arayüzler).
  - Teknikleri doğru uygulamayan şirketlerde projeler hâlâ **güvenilmez, geç ve bütçe üstü**.
- **1968'den beri:** specification, design ve implementation için etkili yöntemler, yeni notasyonlar ve araçlar, daha az emek.
- ⭐ **Tek bir "ideal" yaklaşım YOK.** Farklı sistemler farklı yaklaşım ister.
- ⭐ Hepsinin ortak özü: **process + system organisation**



Şema: Yazılım krizinden yazılım mühendisliğine

Sayfa 5 1.1 FAQs · 1.1.1 What is software? (Yazılım nedir?)

📖 Sayfada ne anlatılıyor

GİRİŞ PARAGRAFI

Yazılım mühendisleri başarılarıyla gurur duyabilir. Uzay keşfi, internet, modern iletişim ve daha güvenli, daha ucuz ulaşım karmaşık yazılımlar sayesinde mümkün oldu. Yazar, disiplin olgunlaştıkça 21. yüzyıldaki katkısının daha da büyük olacağını düşünüyor.

1.1 FAQS

Bu kısım temel soruları **FAQ (Frequently Asked Questions, sık sorulan sorular)** formatında cevaplıyor. İnternet forumlarında yeni gelenlere bilgi vermek için kullanılan bu format, konuya kısa ve etkili bir giriş sağlıyor. Bütün cevapların özeti **Şekil 1.1**'de (sayfa 6).

1.1.1 YAZILIM NEDİR?

Birçok kişi yazılımı sadece "program" sanır. Yazar daha geniş bir tanımı tercih ediyor: yazılım **programlar + ilgili belgeler (documentation) + programların doğru çalışması için gereken yapılandırma verileri (configuration data)**. Bir yazılım sistemi genelde şunlardan oluşur: birden çok ayrı program, bunları kurmaya yarayan yapılandırma dosyaları, sistemin yapısını anlatan **sistem belgeleri**, kullanımını anlatan **kullanıcı belgeleri** ve güncel ürün bilgisi için web siteleri.

İKİ ÜRÜN TÜRÜ

Yazılım mühendisleri müşteriye satılabilen **yazılım ürünleri** geliştirir. İki temel tür var:
1) **Genel ürünler (Generic products):** Bir firmanın ürettiği, açık pazarda isteyen herkese satılan bağımsız (stand-alone) sistemler. Örnek: veritabanları, kelime işlemciler, çizim paketleri, proje yönetim araçları.
2) **Özel / ısmarlama ürünler (Customised / bespoke products):** Belirli bir müşterinin sipariş ettiği, yüklenici firmanın sadece o müşteri için geliştirdiği sistemler. Örnek: elektronik cihaz kontrol sistemleri, belirli bir iş sürecine özel sistemler, hava trafik kontrol sistemleri.

👨‍🏫 Hoca anlatımı

Burada hocaların çok sevdiği bir tuzak var: "Yazılım = program" **değildir!** Bir yazılımı satın aldığında sadece .exe dosyası gelmez. Kurulum ayarları, kullanım kılavuzu, teknik belgeler ve destek sitesi de gelir. Bunların hepsi yazılımın parçasıdır. Sınavda "What is software?" sorulursa mutlaka **programs + documentation + configuration data** de.

İkinci önemli konu iki ürün türü. Bunu market ile terzi gibi düşün. **Generic** marketteki hazır gömlek gibidir: firma kendi yapar, isteyen alır (Word, Photoshop). **Customised** ise terziye diktirilen takım elbise gibidir: tek bir müşteri için, onun ölçülerine göre yapılır (bir havalimanının hava trafik kontrol yazılımı).

📝 Deftere not

- Karmaşık yazılımlar sayesinde: uzay keşfi, internet, modern iletişim, daha güvenli ve ucuz ulaşım.
- Bölüm 1.1 **FAQ formatında** yazılmış, özeti **Şekil 1.1**'de.
- ⭐ **Software = programs + documentation + configuration data** (sadece program değil!)
  - Bir yazılım sistemi: ayrı programlar, yapılandırma dosyaları, sistem belgeleri, kullanıcı belgeleri, web siteleri.
- ⭐ **2 temel ürün türü:**
  - Generic products: herkese açık pazarda satılır, stand-alone. Ör: veritabanı, kelime işlemci, çizim programı, proje yönetim aracı.
  - Customised (bespoke) products: tek bir müşteri sipariş eder. Ör: elektronik cihaz kontrol sistemi, iş sürecine özel sistem, hava trafik kontrolü.

Sayfa 6 Şekil 1.1 FAQ tablosu · Generic ve Customised farkı

📖 Sayfada ne anlatılıyor

ŞEKİL 1.1: SIK SORULAN SORULAR TABLOSU

Bölüm 1.1'deki bütün soruların kısa cevapları tek tabloda toplanmış. Tablonun Türkçe özeti:

Soru	Kısa cevap
What is software? Yazılım nedir?	Programlar ve ilgili belgeler. Belirli bir müşteri ya da genel pazar için geliştirilebilir.
What is software engineering?	Yazılım üretiminin **tüm yönleriyle** ilgilenen mühendislik disiplini.

SE vs Computer science?	CS **teori ve temellerle**, SE işe yarar yazılım üretmenin **pratik** tarafıyla ilgilenir.
SE vs System engineering?	Sistem mühendisliği **donanım, yazılım ve süreç** dahil bilgisayar tabanlı sistemin tamamıyla ilgilenir. SE bu sürecin bir parçasıdır.
What is a software process?	Amacı yazılımı geliştirmek ya da evrimleştirmek olan faaliyetler kümesi.
What is a software process model?	Bir yazılım sürecinin belirli bir bakış açısından **basitleştirilmiş gösterimi**.
Costs of SE?	Kabaca **%60 geliştirme, %40 test**. Özel yazılımda evrim maliyeti çoğu zaman geliştirme maliyetini aşar.
SE methods?	Sistem modelleri, gösterimler, kurallar, tasarım tavsiyeleri ve süreç rehberliği içeren yapılandırılmış yaklaşımlar.
What is CASE?	Süreç faaliyetlerine **otomatik destek** veren yazılım sistemleri. Genelde yöntemleri desteklemek için kullanılır.
Attributes of good software?	İstenen işlev ve performansı sağlamalı. **Maintainable, dependable, usable** (bakımı kolay, güvenilir, kullanışlı) olmalı.
Key challenges?	Artan **çeşitlilikle** başa çıkmak, daha kısa **teslim süresi** talepleri, **güvenilir** yazılım geliştirmek.

TABLONUN ALTINDAKİ METİN

Genel ve özel ürün arasındaki **önemli fark** şu: genel üründe yazılımın tanımlamasını (specification) **geliştiren firma** kontrol eder. Özel üründe tanımlamayı genelde **satın alan kurum** geliştirir ve kontrol eder, geliştiriciler ona uymak zorundadır.
Ama bu iki tür arasındaki çizgi giderek bulanıklaşıyor. Birçok firma genel bir sistemle başlayıp onu müşteriye göre uyarlıyor. En iyi örnek **ERP (Enterprise Resource Planning) sistemleri, ör. SAP**. Büyük ve karmaşık bir sistem, şirketin iş kuralları, süreçleri ve istediği raporlar eklenerek o şirkete uyarlanıyor.

👩‍🏫 Hoca anlatımı

Bu tablo bütün bölümün **özeti**. Çoktan seçmeli sınavların çoğu buradan gelir. Her satırı sonraki sayfalarda ayrı ayrı açacağız, şimdilik tabloyu "harita" gibi düşün.

Generic ve customised farkının özü **"kuralları kim koyuyor?"** sorusu. Word'ü Microsoft istediği gibi tasarlar, sen sadece alırsın: **kontrol geliştiricide**. Hastaneye özel bir yazılımda ise hastane "şunu istiyorum" der, firma ona uyar: **kontrol müşteride**.

SAP örneği ise ikisinin karışımı. Hazır büyük bir paketi alıp şirkete göre ayarlıyorsun, yani yarı hazır, yarı terzi işi.

📝 Deftere not

- ⭐ **Şekil 1.1 tablosunu olduğu gibi deftere geçir** (yukarıdaki tablo).
- ⭐ Önemli rakam: maliyet ≈ **%60 development, %40 testing**. Custom yazılımda **evolution maliyeti > development maliyeti**.
- ⭐ İyi yazılım: **maintainable, dependable, usable** + istenen işlev ve performans.
- ⭐ Generic ve customised farkı, specification'ı kim kontrol ediyor?
  - **Generic:** geliştirici firma.
  - **Customised:** satın alan müşteri (geliştirici ona uyar).
- Çizgi bulanıklaşıyor: genel sistem alınıp müşteriye uyarlanıyor. Ör: **ERP sistemleri (SAP)**, iş kuralları, süreçler ve raporlar eklenerek uyarlanır.

Sayfa 7 1.1.2 Yazılım mühendisliği nedir? · 1.1.3 Bilgisayar bilimiyle farkı

📖 Sayfada ne anlatılıyor

1.1.2 TANIM

Yazılım mühendisliği, sistemin ilk tanımlanmasından kullanıma girdikten sonraki bakımına kadar **yazılım üretiminin tüm yönleriyle** ilgilenen bir mühendislik disiplinidir. Tanımda iki anahtar ifade var:
1) **Mühendislik disiplini (Engineering discipline):** Mühendisler işleri çalışır hale getirir. Teori, yöntem ve araçları uygun yerde ve seçerek kullanırlar. Uygun teori ya da yöntem olmasa bile çözüm ararlar. **Kurumsal ve finansal kısıtlar** içinde çalıştıklarını bilir, çözümü bu sınırlar içinde ararlar.
2) **Yazılım üretiminin tüm yönleri (All aspects of software production):** Sadece teknik geliştirme süreçleri değil, **yazılım proje yönetimi** ve yazılım üretimini destekleyen **araç, yöntem ve teorilerin geliştirilmesi** de buna dahil.

SİSTEMATİK YA DA YARATICI YAKLAŞIM

Genelde **sistematik ve düzenli** bir yaklaşım kaliteli yazılım için en etkilisidir. Ama mühendislik, duruma **en uygun yöntemi seçmektir**. Bazen daha yaratıcı ve daha az resmi bir yaklaşım daha etkili olur. Bu özellikle yazılım ve grafik tasarım becerisinin birlikte gerektiği **web tabanlı sistemler** için geçerli.

1.1.3 BİLGİSAYAR BİLİMİ İLE FARKI

Bilgisayar bilimi (computer science) bilgisayarların ve yazılımların altında yatan **teori ve yöntemlerle** ilgilenir. **Yazılım mühendisliği** ise yazılım üretmenin **pratik sorunlarıyla** ilgilenir. Elektrik mühendisinin biraz fizik bilmesi gerektiği gibi, yazılım mühendisinin de biraz bilgisayar bilimi bilmesi gerekir.
İdealde yazılım mühendisliğinin tamamı bilgisayar bilimi teorilerine dayanmalı, ama gerçekte öyle değil. Mühendisler çoğu zaman **ad hoc** (duruma özel, geçici) yaklaşımlar kullanmak zorunda kalır, çünkü şık teoriler gerçek ve karmaşık problemlere her zaman uygulanamaz.

👨‍🏫 Hoca anlatımı

"What is software engineering?" sorusu sınavın klasiğidir. Cevabın iki ayağı var, ikisini de yazmalısın:

1. Engineering discipline: Mühendis bilim insanı değildir, işi bitirmeye bakar. Sınırsız para ve zaman yoktur, patron ve bütçe vardır. Bu sınırlar içinde "çalışan" bir çözüm üretir. Teori yoksa bile bir yolunu bulur.

2. All aspects: Yazılım mühendisliği sadece kod yazmak değildir. Projeyi yönetmek, plan yapmak, araç geliştirmek de bu işin parçasıdır.

Bilgisayar bilimi ile farkını şöyle düşün: fizikçi elektriğin nasıl çalıştığını araştırır (teori), elektrik mühendisi o bilgiyle bina elektriğini döşer (pratik). **CS = fizikçi, SE = mühendis.** Gerçek hayatta teori her zaman yetmez, mühendis bazen o probleme özel **(ad hoc)** bir çözüm uydurur.

📝 Deftere not

- ⭐ **SE tanımı:** Yazılım üretiminin, tanımlamadan (specification) kullanıma girdikten sonraki bakıma (maintenance) kadar **tüm yönleriyle** ilgilenen mühendislik disiplini.
- ⭐ Tanımdaki **2 anahtar ifade:**
  - Engineering discipline: işi çalışır hale getirmek, teori ve yöntemi seçerek kullanmak, teori yoksa da çözüm aramak, **kurumsal ve finansal kısıtlar** içinde çalışmak.
  - All aspects of software production: teknik süreçler + **proje yönetimi** + araç, yöntem ve teori geliştirme.
- Genelde **sistematik ve düzenli** yaklaşım en iyisi. Ama bazen **yaratıcı ve az resmi** yaklaşım daha iyi (ör. web sistemleri, yazılım + grafik tasarım).
- ⭐ **CS ve SE farkı:**
  - **Computer science:** teori ve temel yöntemler.

- **Software engineering:** yazılım üretmenin pratik sorunları.
- Benzetme: elektrik mühendisi için fizik ne ise, yazılım mühendisi için CS o.
- Gerçekte SE çoğu zaman ad hoc yaklaşımlar kullanır, çünkü teoriler karmaşık gerçek problemlere her zaman uymaz.

Sayfa 7–8 1.1.4 Sistem mühendisliğiyle farkı · 1.1.5 Yazılım süreci nedir?

📖 Sayfada ne anlatılıyor

1.1.4 SİSTEM MÜHENDİSLİĞİ

Sistem mühendisliği (system engineering), içinde yazılımın önemli rol oynadığı karmaşık sistemlerin geliştirilmesi ve evriminin tüm yönleriyle ilgilenir. Yani **donanım geliştirme, politika ve süreç tasarımı, sistemin kurulumu (deployment)** ve yazılım mühendisliği bunun içindedir. Sistem mühendisleri sistemi tanımlar, **genel mimarisini** belirler, sonra farklı parçaları **birleştirerek (integrate)** bitmiş sistemi oluşturur. Bileşenlerin (donanım, yazılım vb.) kendi mühendisliğiyle daha az ilgilenirler.
Sistem mühendisliği yazılım mühendisliğinden **daha eski** bir disiplindir. İnsanlar yüz yılı aşkın süredir uçak, kimya fabrikası gibi karmaşık endüstriyel sistemleri tanımlayıp kuruyor. Sistemlerdeki yazılım oranı arttıkça **use-case modelleme** ve **konfigürasyon yönetimi** gibi yazılım mühendisliği teknikleri sistem mühendisliğinde de kullanılmaya başlandı. Detayı Bölüm 2'de.

1.1.5 YAZILIM SÜRECİ

Yazılım süreci (software process), bir yazılım ürününü ortaya çıkaran faaliyetler ve bunlarla ilişkili sonuçlardır. Bütün süreçlerde ortak **4 temel faaliyet** var:
1) **Software specification:** müşteri ve mühendisler üretilecek yazılımı ve çalışma kısıtlarını tanımlar.
2) **Software development:** yazılım tasarlanır ve programlanır.
3) **Software validation:** yazılımın müşterinin istediği şey olduğu kontrol edilir.
4) **Software evolution:** yazılım değişen müşteri ve pazar ihtiyaçlarına göre değiştirilir.

FARKLI SİSTEMLER, FARKLI SÜREÇLER

Farklı sistem türleri farklı süreçlere ihtiyaç duyar. Örneğin bir uçaktaki **gerçek zamanlı (real-time)** yazılım, geliştirme başlamadan önce **tamamen tanımlanmalıdır**. **E-ticaret** sistemlerinde ise tanımlama ve program genelde **birlikte** geliştirilir. Bu yüzden 4 faaliyet farklı şekillerde ve farklı detay seviyelerinde düzenlenebilir. Ama **uygun olmayan bir süreç** kullanmak kaliteyi ve kullanışlılığı düşürebilir, maliyeti artırabilir. Süreçler Bölüm 4'te, süreç iyileştirme Bölüm 28'de.

👨‍🏫 Hoca anlatımı

Sistem mühendisi orkestra şefi gibidir. Uçağı düşün: motor, kanat, elektronik, yazılım ve pilot prosedürleri var. Sistem mühendisi bunların hepsinin bir arada çalışmasını sağlar, genel mimariyi çizer ve parçaları birleştirir. Yazılım mühendisi ise orkestradaki bir enstrümandır: sadece yazılım kısmıyla uğraşır. Yani **SE, sistem mühendisliğinin bir parçasıdır.** Sistem mühendisliği daha eskidir, çünkü uçaklar ve fabrikalar yazılımdan önce de vardı.

4 temel faaliyet sınavın en çok sorulan konularından. Kolay hatırlamak için ev yapmak gibi düşün:

1. **Specification**, ne istediğine karar vermek ("3+1 ev, bahçeli"). 2. **Development**, evi çizmek ve inşa etmek. 3. **Validation**, teslimde "istediğim gibi mi?" diye kontrol etmek. 4. **Evolution**, yıllar sonra çocuk olunca bir oda eklemek.

Uçak ve e-ticaret örneği de önemli. Uçak yazılımında hata hayati olduğu için her şey baştan yazılı olmalı. Bir alışveriş sitesinde ise yaparken değiştirirsin. Yanlış süreci seçersen kalite düşer, maliyet artar.

📝 Deftere not

- ⭐ **System engineering:** bilgisayar tabanlı sistemin **tamamı** (donanım + yazılım + politika/süreç tasarımı + deployment).
  - System engineer: sistemi tanımlar, **genel mimariyi** belirler, parçaları **entegre eder**. Bileşen mühendisliğiyle az ilgilenir.
  - ⭐ **SE, system engineering'in bir parçasıdır.**
  - System eng. **daha eski** (100+ yıl: uçak, kimya fabrikası).
  - Artık SE teknikleri de kullanılıyor: **use-case modelling, configuration management**.
- ⭐ **Software process:** yazılım ürününü üreten faaliyetler + sonuçları.
- ⭐⭐ **4 temel faaliyet:**
  - **Specification:** yazılımı ve kısıtlarını tanımlamak.
  - **Development:** tasarım + programlama.
  - **Validation:** müşterinin istediği mi diye kontrol.
  - **Evolution:** değişen ihtiyaçlara göre değiştirmek.
- Farklı sistem, farklı süreç:
  - Uçak (**real-time**): önce **tamamen** tanımlanır.
  - **E-ticaret:** tanımlama ve program **birlikte** geliştirilir.
- Uygunsuz süreç: **kalite ve kullanışlılık düşer, maliyet artar.**

Sayfa 8–9 1.1.6 Yazılım süreç modeli nedir?

📖 Sayfada ne anlatılıyor

TANIM

Süreç modeli (software process model), bir yazılım sürecinin basitleştirilmiş tanımıdır ve sürece **tek bir bakış açısından** bakar. Modelde sürecin faaliyetleri, yazılım ürünleri ve çalışan insanların rolleri yer alabilir.

3 MODEL TÜRÜ (BAKIŞ AÇISI)

1) Workflow model (iş akışı): faaliyetlerin sırası, girdileri, çıktıları ve bağımlılıkları. Faaliyetler **insan eylemleridir**.
2) Dataflow / activity model (veri akışı / faaliyet): süreç, her biri **veri dönüştüren** faaliyetler kümesidir. Girdinin çıktıya nasıl dönüştüğünü gösterir (ör. specification → design). Dönüşümü insan da yapabilir bilgisayar da.
3) Role/action model (rol/eylem): süreçteki insanların **rolleri** ve sorumlu oldukları faaliyetler.

3 GENEL PARADİGMA

Çoğu süreç modeli şu üç genel yaklaşımdan birine dayanır:
1) Waterfall (şelale): faaliyetler ayrı aşamalar olarak sıralanır: gereksinim tanımlama, tasarım, uygulama, test. Her aşama bitip **onaylandıktan (signed-off)** sonra sıradakine geçilir.
2) Iterative development (yinelemeli): tanımlama, geliştirme ve geçerleme **iç içe** yürür. Çok soyut tanımlardan hızlıca bir ilk sistem yapılır, sonra **müşteri geri bildirimiyle** iyileştirilir. Sonuç ya teslim edilir ya da daha sağlam ve bakımı kolay olsun diye daha yapılandırılmış bir yaklaşımla **yeniden yazılır**.
3) Component-based SE, CBSE (bileşen tabanlı): sistemin parçalarının **zaten var olduğunu** varsayar. Süreç parçaları sıfırdan geliştirmeye değil, onları **birleştirmeye (integrating)** odaklanır (Bölüm 19). Bu modellere Bölüm 4 ve 17'de dönülecek.

👨‍🏫 Hoca anlatımı

Süreç ve süreç modeli farklı şeyler, dikkat! Süreç gerçekte yapılan iştir. Model ise o işin basitleştirilmiş bir **resmidir**, haritası gibi. Aynı şehrin metro haritası, trafik haritası ve turistik haritası olabilir. Model de sürece

tek bir açıdan bakar:

Workflow: "kim, hangi sırayla, ne yapıyor?" (insan eylemleri). **Dataflow:** "veri nasıl dönüşüyor?" (spec → tasarım → kod). **Role/action:** "kimin görevi ne?"

3 paradigma ise **Bölüm 4'ün temeli**, çok önemli:

Waterfall merdiven gibidir: bir basamağı bitirmeden ötekine geçemezsin, her basamakta imza (signed-off) atılır. **Iterative** heykeltıraş gibidir: önce kaba bir şekil yapılır, müşteri bakar, "burası olmamış" der, düzeltilir, tekrar bakılır. **CBSE** LEGO gibidir: parçalar hazır, sen sadece onları birleştirirsin.

📝 Deftere not

- ⭐ **Process model:** sürecin **tek bir bakış açısından** basitleştirilmiş gösterimi (faaliyetler, ürünler, roller).
- ⭐ **3 model türü:**
  - Workflow model: faaliyet sırası, girdi/çıktı, bağımlılıklar. Faaliyetler = **insan eylemleri**.
  - Dataflow / activity model: **veri dönüşümü** (ör. spec → design). İnsan ya da bilgisayar yapar.
  - Role/action model: insanların **rolleri** ve sorumlulukları.
- ⭐⭐ **3 genel paradigma:**
  - Waterfall: ayrı aşamalar (requirements → design → implementation → testing). Her aşama **signed-off** olunca sonrakine geçilir.
  - Iterative development: spec, dev ve validation **iç içe**. Soyut tanımdan hızlı ilk sistem, müşteri geri bildirimiyle iyileştirme, sonra teslim ya da daha sağlam şekilde yeniden yazma.
  - CBSE: parçalar **zaten var**, odak **entegrasyonda** (sıfırdan yazmak değil).

Sayfa 9–11 1.1.7 Yazılım mühendisliğinin maliyetleri (Şekil 1.2 ve 1.3)

📖 Sayfada ne anlatılıyor

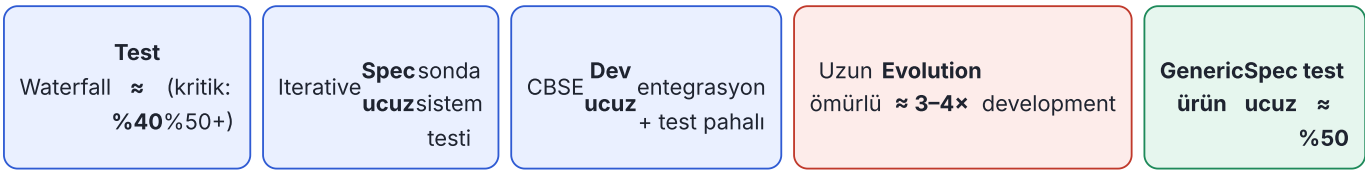
> GENEL DURUM
> Basit bir cevap yok, çünkü maliyetin faaliyetlere dağılımı **kullanılan sürece** ve **yazılımın türüne** bağlı. Örneğin gerçek zamanlı yazılım, web tabanlı sistemlere göre çok daha kapsamlı geçerleme ve test ister. Her yaklaşımın kendine özgü bir maliyet profili var. **Şekil 1.2** toplam maliyeti **100 birim** kabul ederek bunu gösteriyor.

Yaklaşım	Maliyet dağılımı (Şekil 1.2 ve 1.3)
Waterfall	Specification, design, development, integration & testing ayrı ölçülür. **En pahalısı entegrasyon ve test:** normalde toplamın yaklaşık **%40'ı**, bazı kritik sistemlerde **en az %50**.
Iterative	Aşamalar arasında kesin çizgi yok. Başta sadece üst düzey tanım yapıldığı için **specification maliyeti düşük**. Spec, tasarım, kodlama, entegrasyon ve test **paralel** yürür. İlk uygulama bitince yine **ayrı bir sistem testi** gerekir.
CBSE	Yeni olduğu için kesin rakam yok. **Geliştirme maliyeti düşer**, ama bileşenlerin tanımlamaya uyduğunu ve birlikte çalıştığını kontrol etmek gerektiği için **entegrasyon ve test maliyeti artar**.
Uzun ömürlü sistemler	Yazılım kullanıma girdikten sonra değiştirme (**evolution**) maliyeti de var. 10+ yıl kullanılan komuta-kontrol sistemlerinde evrim maliyeti geliştirme maliyetinin **3–4 katı** olabilir (grafikte 100 birim geliştirmeye karşı 400'e kadar). Küçük iş sistemlerinin ömrü kısa, evrim maliyetleri de düşük.

Generic ürün (Şekil 1.3)	Yukarıdakiler **özel (custom)** yazılım içindir. PC'ye satılan genel ürünler kabataslak bir tanımlamadan **evrimsel** yaklaşımla geliştirilir. **Specification ucuz**, ama farklı konfigürasyonlarda çalışacakları için **çok kapsamlı test** gerekir (grafikte yaklaşık yarısı sistem testi).

GENEL ÜRÜNLERDE EVRİM MALİYETİ

Genel ürünlerde evrim maliyetini ölçmek zordur. Ürün çıkınca bir sonraki sürüme başlanır ve pazarlama nedeniyle bu, kullanıcının aldığı ürünün değiştirilmiş hali olarak değil, **yeni (ama uyumlu) bir ürün** olarak sunulur. Bu yüzden evrim maliyeti ayrı hesaplanmaz, **sonraki sürümün geliştirme maliyeti** sayılır.



Şema: Maliyet profilleri (toplam geliştirme = 100 birim)

👨‍🏫 Hoca anlatımı

Bu sayfada ezberlenecek birkaç rakam ve bir mantık var. Mantık şu: **"Paranın çoğu nereye gidiyor?" sorusunun cevabı yönteme göre değişir.**

Waterfall: her şey en sonda birleştirilip test edilir, hatalar en sonda çıkar. Bu yüzden en pahalı kısım **test (%40)**. Hastane ya da uçak gibi kritik sistemlerde test daha da önemli olduğu için **%50'yi geçer**.

CBSE: LEGO parçaları hazır olduğu için yapmak ucuz. Ama parçalar birbirine uyuyor mu diye **test etmek** pahalı.

En şaşırtıcı bilgi: uzun yıllar kullanılan bir sistemde, yazılımı yaptıktan sonra onu değiştirmek ve güncellemek, yapmanın **3–4 katına** mal olabilir. Yani yazılımın asıl masrafı yapıldıktan sonra başlar.

Generic ürün (ör. Windows): milyonlarca farklı bilgisayarda çalışacağı için testi çok pahalıdır. Yeni sürüm çıkınca da bunu "güncelleme" değil, "yeni ürün" diye satarlar. Bu yüzden evrim maliyeti ayrıca hesaplanmaz.

📝 Deftere not

- Maliyet dağılımı **sürece ve yazılım türüne** bağlı (ör. real-time yazılım, web'den daha çok test ister).
- ⭐ **Waterfall:** en pahalı = **integration + testing ≈ %40** (kritik sistemlerde **≥ %50**).
- **Iterative:** spec ucuz (sadece üst düzey tanım), faaliyetler paralel, ama sonda **ayrı sistem testi** şart.
- **CBSE:** development ucuz, **integration + testing pahalı** (bileşenler uyuyor mu?).
- ⭐ **Uzun ömürlü sistem** (10+ yıl, ör. komuta-kontrol): **evolution maliyeti = development × 3–4**. Küçük iş sistemlerinde düşük.
- ⭐ **Generic ürün (Şekil 1.3):** evrimsel geliştirme, spec ucuz, **test çok pahalı** (farklı konfigürasyonlar).
  - Evolution maliyeti ayrı hesaplanmaz, **yeni sürüm = yeni ürün** olarak sayılır.

Sayfa 11–12 · 1.1.8 Yazılım mühendisliği yöntemleri (Şekil 1.4) · 1.1.9 CASE

📖 Sayfada ne anlatılıyor

1.1.8 YÖNTEM NEDİR?

Yazılım mühendisliği yöntemi (method), amacı kaliteli yazılımı **düşük maliyetle** üretmeyi kolaylaştırmak olan **yapılandırılmış** bir yazılım geliştirme yaklaşımıdır.

TARİHÇE

1970'ler: Structured Analysis (DeMarco, 1978) ve **JSD** (Jackson, 1983) gibi yöntemler. Sistemin temel **fonksiyonel bileşenlerini** bulmaya çalışırlar, yani **fonksiyon odaklıdırlar (function-oriented)** ve hâlâ kullanılırlar.
1980–90'lar: Bunlara **Booch** ve **Rumbaugh**'un önerdiği **nesne yönelimli (object-oriented, OO)** yöntemler eklendi.
Sonra: Bu farklı yaklaşımlar **UML (Unified Modeling Language)** etrafında tek bir birleşik yaklaşımda toplandı.

İDEAL YÖNTEM YOK

Her yöntemin uygun olduğu alan farklı. Örneğin OO yöntemler **etkileşimli (interactive)** sistemler için uygun, ama **katı gerçek zamanlı (stringent real-time)** gereksinimleri olan sistemler için uygun değil. Bütün yöntemler sistemin **(grafiksel) modellerini** geliştirip bunları tanımlama ya da tasarım olarak kullanma fikrine dayanır.

ŞEKİL 1.4: YÖNTEMİN BİLEŞENLERİ

Bileşen	Açıklama	Örnek
System model descriptions	Geliştirilecek sistem modelleri ve kullanılacak gösterim	Nesne modelleri, veri akış modelleri, durum makinesi modelleri
Rules	Modellere **her zaman** uygulanan kısıtlar	Modeldeki her varlığın **benzersiz bir adı** olmalı
Recommendations	İyi tasarım pratiğini tarif eden sezgisel tavsiyeler (heuristics). Uyulursa iyi düzenlenmiş model çıkar.	Bir nesnenin **7'den fazla alt nesnesi** olmamalı
Process guidance	Modelleri geliştirmek için izlenecek faaliyetler ve bunların düzeni	Nesne özellikleri, nesnenin operasyonlarından **önce** belgelenmeli

1.1.9 CASE NEDİR?

CASE = Computer-Aided Software Engineering (bilgisayar destekli yazılım mühendisliği). Gereksinim analizi, sistem modelleme, hata ayıklama (debugging) ve test gibi **süreç faaliyetlerini destekleyen** çok çeşitli programları kapsar. Artık bütün yöntemler kendi CASE teknolojisiyle gelir:
- yöntemin gösterimleri için **editörler**
- modeli yöntemin kurallarına göre kontrol eden **analiz modülleri**
- sistem belgelerini oluşturan **rapor üreticileri**
- modelden otomatik kaynak kod üreten **kod üreticisi (code generator)**
- yazılım mühendislerine **süreç rehberliği**

👨‍🏫 Hoca anlatımı

Yöntem bir yemek tarifi gibidir. Tarifte ne yapacağın (model), asla çiğnememen gereken kurallar ("tavuğu çiğ yeme" = **rules**), tavsiyeler ("biraz kekik iyi gider" = **recommendations**) ve adımların sırası (**process guidance**) vardır. Rules ile recommendations arasındaki farka dikkat: **rule zorunludur, recommendation tavsiyedir.**

Tarihçeyi şöyle hatırla: önce **fonksiyonlar** (1970'ler), sonra **nesneler** (1980–90'lar), sonra hepsi **UML** çatısında birleşti.

CASE araçları ise yazılımcının "elektrikli aletleri"dir. Diyagram çizmek için editör, hataları yakalamak için kontrol modülü, belge çıkarmak için rapor üreticisi, hatta diyagramdan otomatik kod yazan araçlar. Kısaca yazılım yapmaya yardım eden yazılımlar.

📝 Deftere not

- ⭐ **Method:** kaliteli yazılımı **cost-effective** üretmek için **yapılandırılmış (structured)** yaklaşım.
- Tarihçe:

- **1970'ler, function-oriented:** Structured Analysis (DeMarco), JSD (Jackson). Hâlâ kullanılıyor.
- **1980–90'lar, object-oriented:** Booch, Rumbaugh.
- Sonra hepsi **UML** altında birleşti.

- ⭐ **İdeal yöntem yok.** Ör: OO, interactive sistemlere uygun, katı real-time sistemlere uygun değil.
- Tüm yöntemler = sistemin **grafiksel modellerini** geliştirip spec ya da tasarım olarak kullanmak.
- ⭐ **Yöntemin 4 bileşeni (Şekil 1.4):**
  - **System model descriptions:** hangi modeller + gösterim (ör. nesne, veri akış, durum makinesi modelleri).
  - **Rules:** her zaman geçerli kısıt (ör. her varlığın benzersiz adı olmalı).
  - **Recommendations:** iyi tasarım tavsiyesi, heuristic (ör. bir nesnenin en fazla 7 alt nesnesi olmalı).
  - **Process guidance:** faaliyetlerin sırası (ör. nesne özellikleri operasyonlardan önce belgelenir).
- ⭐ **CASE = Computer-Aided Software Engineering:** süreç faaliyetlerini (gereksinim analizi, modelleme, debugging, test) destekleyen programlar.
  - Araçlar: **editörler, analiz/kontrol modülleri, rapor üreticileri, kod üreticileri, süreç rehberliği**.

Sayfa 12–13 1.1.10 İyi yazılımın özellikleri (Şekil 1.5)

📖 Sayfada ne anlatılıyor

FONKSİYONEL OLMAYAN ÖZELLİKLER

Yazılımın sunduğu hizmetlerin yanında, kalitesini gösteren başka özellikler de vardır. Bunlar yazılımın **ne yaptığıyla** değil, çalışırken **nasıl davrandığıyla** ve kaynak kodla belgelerin yapısıyla ilgilidir. Bunlara **fonksiyonel olmayan özellikler (non-functional attributes)** denir. Örnek: kullanıcı sorgusuna **yanıt süresi**, kodun **anlaşılabilirliği**.

UYGULAMAYA GÖRE DEĞİŞİR

Beklenen özellikler yazılımın kullanım alanına göre değişir. Özetle: **bankacılık sistemi → güvenli**, **etkileşimli oyun → hızlı tepki veren**, **telefon santrali sistemi → güvenilir** olmalı. Yazar bunları iyi tasarlanmış bir yazılımın 4 temel özelliğinde topluyor (Şekil 1.5).

Özellik	Türkçe açıklama
Maintainability bakım kolaylığı	Yazılım, müşterinin değişen ihtiyaçlarına göre **değiştirilebilecek** şekilde yazılmalı. Değişen iş ortamı yüzünden değişiklik **kaçınılmaz** olduğu için bu kritik bir özellik.
Dependability güvenilebilirlik	**Reliability (güvenilirlik), security (güvenlik) ve safety (emniyet)** özelliklerini kapsar. Güvenilir yazılım, sistem çöktüğünde **fiziksel ya da ekonomik zarara** yol açmamalı.
Efficiency verimlilik	Bellek ve işlemci gibi **sistem kaynaklarını israf etmemeli**. Yanıt verme hızı, işlem süresi, bellek kullanımı vb. bunun içinde.
Usability kullanılabilirlik	Hedef kullanıcının **fazla çaba harcamadan** kullanabileceği şekilde olmalı. Yani **uygun bir kullanıcı arayüzü** ve **yeterli belgesi** olmalı.

👨‍🏫 Hoca anlatımı

Bu sayfa bölümün en çok sorulan konusu, ezberle! "What are the attributes of good software?" sorusu neredeyse her sınavda çıkar.

Önce "fonksiyonel olmayan" ne demek, onu anla. Bir hesap makinesinin toplama yapması **fonksiyonel** özelliğidir (ne yaptığı). Sonucu 0.1 saniyede mi 10 saniyede mi verdiği **fonksiyonel olmayan** özelliğidir (nasıl yaptığı).

4 özelliği akılda tutmak için **"M-D-E-U"** kısaltmasını kullanabilirsin:

M – Maintainability: Yarın patron "şu özelliği ekle" diyecek. Kod kolayca değiştirilebiliyor mu? **D – Dependability:** Güvenebiliyor muyum? Çökmüyor mu, hacklenmiyor mu, kimseye zarar vermiyor mu? **E – Efficiency:** Belleği ve işlemciyi boşuna yemiyor mu? **U – Usability:** Normal bir kullanıcı zorlanmadan kullanabiliyor mu, kılavuzu var mı?

Dikkat: **Dependability tek başına 3 şeyi kapsar: reliability, security, safety.** Bu üçü Bölüm 3'te ayrıntılı gelecek.

📝 Deftere not

- **Non-functional attributes:** yazılımın **ne yaptığı değil, nasıl davrandığı** + kodun ve belgelerin yapısı. Ör: yanıt süresi, kodun anlaşılabilirliği.
- Uygulamaya göre değişir: banka = **güvenli**, oyun = **hızlı tepki**, telefon santrali = **güvenilir**.
- ⭐⭐ **İyi yazılımın 4 temel özelliği (Şekil 1.5):**
  - Maintainability: değişen ihtiyaçlara göre değiştirilebilmeli (değişim **kaçınılmaz**).
  - Dependability: **reliability + security + safety**. Çökünce fiziksel ya da ekonomik zarar vermemeli.
  - Efficiency: kaynakları (bellek, işlemci) israf etmemeli. Yanıt süresi, işlem süresi, bellek kullanımı.
  - Usability: hedef kullanıcı zorlanmadan kullanabilmeli. **Uygun arayüz + yeterli belge.**
- Hafıza ipucu: **M-D-E-U**

Sayfa 13–14 1.1.11 Yazılım mühendisliğinin temel zorlukları

📖 Sayfada ne anlatılıyor

21. YÜZYILIN 3 TEMEL ZORLUĞU

1) Heterogeneity challenge (çeşitlilik / farklılık zorluğu): Sistemlerin giderek, farklı türde bilgisayarları ve destek sistemlerini içeren **ağlar üzerinde dağıtık** çalışması gerekiyor. Yeni yazılımı çoğu zaman **farklı dillerde yazılmış eski (legacy) sistemlerle** birleştirmek gerekiyor. Zorluk, bu çeşitliliğe uyum sağlayacak kadar esnek ve güvenilir yazılım geliştirme tekniklerini bulmak.
2) Delivery challenge (teslim zorluğu): Geleneksel teknikler **zaman alıcı**, çünkü kalite için zaman gerekiyor. Ama bugün işletmeler çok hızlı değişmek zorunda, onları destekleyen yazılım da aynı hızla değişmeli. Zorluk, büyük ve karmaşık sistemlerde **kaliteden ödün vermeden teslim süresini kısaltmak**.
3) Trust challenge (güven zorluğu): Yazılım hayatımızın her alanına girdiği için ona **güvenebilmemiz** şart. Bu özellikle web sayfası ya da web servisi üzerinden erişilen **uzak sistemler** için geçerli. Zorluk, yazılımın kullanıcılarınca güvenilebilir olduğunu **gösteren** teknikler geliştirmek.

SAYFA 14 BAŞI

Bu zorluklar **birbirinden bağımsız değil**. Örneğin eski (legacy) bir sisteme web servisi arayüzü eklemek için hızlı değişiklik yapmak gerekebilir. Bu, hem heterogeneity hem delivery zorluğu demek. Bunlarla başa çıkmak için yeni araç ve teknikler ile mevcut yöntemleri birleştirmenin yaratıcı yolları gerekiyor.

👨‍🏫 Hoca anlatımı

Üç zorluğu basitçe şöyle düşün:

Heterogeneity, "herkes farklı": Bir bankanın 1980'lerde COBOL ile yazılmış eski sistemi var. Şimdi bunun telefondaki mobil uygulamayla, web sitesiyle ve farklı sunucularla konuşması lazım. Hepsi farklı dil, farklı makine. Bunları bir arada çalıştırmak zor.

Delivery, "hemen lazım": Rakip yeni bir özellik çıkardı, senin de iki haftada çıkarman lazım. Ama aceleye getirirsen kalite düşer. Hızlı ve kaliteli olmak zor.

Trust, "güvenebilir miyim?": Kart bilgini bir siteye giriyorsun. Bu yazılıma nasıl güveneceksin? Yazılımın güvenilir olduğunu **kanıtlamak** zor.

Sınavda üçünün **İngilizce adlarını** mutlaka bil: **heterogeneity, delivery, trust**.

📝 Deftere not

- ⭐⭐ **21. yüzyılın 3 temel zorluğu:**
  - Heterogeneity challenge: ağ üzerinde **dağıtık**, farklı bilgisayar ve sistemler. Yeni yazılımı farklı dillerdeki **legacy** sistemlerle birleştirme. Hedef: **esnek ve güvenilir** yazılım teknikleri.
  - Delivery challenge: geleneksel teknikler yavaş, işletmeler hızlı değişiyor. Hedef: **kaliteden ödün vermeden teslim süresini kısaltmak**.
  - Trust challenge: yazılım hayatın her yerinde, **güvenilmesi** şart (özellikle web üzerinden erişilen uzak sistemler). Hedef: güvenilir olduğunu **gösteren** teknikler.
- Zorluklar **bağımsız değil**. Ör: legacy sisteme hızlıca web servis arayüzü eklemek (heterogeneity + delivery).
- Çözüm: yeni araç ve teknikler + mevcut yöntemleri yaratıcı şekilde birleştirmek.

Sayfa 14 1.2 Profesyonel ve etik sorumluluk

📖 Sayfada ne anlatılıyor

GİRİŞ

Diğer mühendislik dalları gibi yazılım mühendisliği de mühendislerin özgürlüğünü sınırlayan **yasal ve sosyal bir çerçeve** içinde yapılır. Yazılım mühendisleri, işlerinin sadece teknik beceri uygulamaktan daha geniş sorumluluklar içerdiğini kabul etmeli. Profesyonel olarak saygı görmek için **etik ve ahlaki açıdan sorumlu** davranmalılar.

DÜRÜSTLÜK

Normal **dürüstlük ve doğruluk (honesty, integrity)** standartlarına her zaman uyulmalı. Beceriler dürüst olmayan şekilde ya da mesleğin itibarını zedeleyecek şekilde kullanılmamalı. Ama bazı alanlarda kabul edilebilir davranışın sınırını **yasalar değil**, daha belirsiz olan **mesleki sorumluluk** kavramı çizer. Bunlardan bazıları:

Alan	Açıklama
Confidentiality gizlilik	Resmi bir gizlilik sözleşmesi imzalanmış olsun ya da olmasın, işverenin ya da müşterinin **gizliliğine saygı** gösterilmeli.
Competence yetkinlik	Yetkinlik seviyeni **olduğundan farklı göstermemelisin**. Yetkinliğinin dışında kaldığını bildiğin işi bile bile kabul etmemelisin.
Intellectual property rights fikri mülkiyet hakları	**Patent ve telif hakkı (copyright)** gibi fikri mülkiyet kullanımını düzenleyen yerel yasaları bilmelisin. İşverenin ve müşterinin fikri mülkiyetinin korunmasına dikkat etmelisin.
Computer misuse bilgisayarın kötüye kullanımı	Teknik becerini başkalarının bilgisayarlarını kötüye kullanmak için kullanmamalısın. Bu, nispeten önemsiz şeylerden (işverenin bilgisayarında oyun oynamak) çok ciddi şeylere (**virüs yaymak**) kadar uzanır.

MESLEK KURULUŞLARI

Etik standartları belirlemede meslek kuruluşlarının önemli rolü var. **ACM**, **IEEE (Institute of Electrical and Electronic Engineers)** ve **British Computer Society** gibi kuruluşlar **mesleki davranış / etik kuralları (code of conduct / code of ethics)** yayınlar. Üyeler, üye olurken bu kurallara uymayı taahhüt eder. ACM ve IEEE birlikte ortak bir **etik ve mesleki uygulama kuralları** metni hazırlamış. Bu metnin hem **kısa** (Şekil 1.6) hem de ayrıntı ekleyen **uzun** bir versiyonu var.

👨‍🏫 Hoca anlatımı

Burada ana fikir şu: **iyi mühendis sadece iyi kod yazan kişi değildir, dürüst ve sorumlu da olmalıdır.** Bazı kötü davranışlar zaten yasaktır (hırsızlık gibi). Ama bazıları yasak olmasa da **"profesyonelce değildir"**.

4 alanı günlük hayattan örneklerle düşün:

Confidentiality: Çalıştığın şirketin yeni projesini rakibe anlatmazsın, sözleşme imzalamamış olsan bile. **Competence:** Hiç yapay zeka bilmiyorsan "ben uzmanım" deyip yapay zeka projesi almazsın. **Intellectual property:** Başkasının kodunu izinsiz kopyalayıp kendininmiş gibi satmazsın. **Computer misuse:** Bilgini başkasının bilgisayarına zarar vermek için kullanmazsın. İş bilgisayarında oyun oynamak bile küçük çaplı bir kötüye kullanımdır, virüs yaymak ise en ağırıdır.

Kuruluş isimlerini de bil: **ACM, IEEE, British Computer Society**. ACM ve IEEE **birlikte** bir etik kod hazırladı.

📝 Deftere not

- SE, **yasal ve sosyal çerçeve** içinde yapılır. Mühendisin sorumluluğu teknik becerinin ötesindedir, **etik ve ahlaki** davranmalıdır.
- Her zaman **honesty + integrity**. Mesleğin itibarını zedeleme.
- Bazı sınırları yasalar değil, **mesleki sorumluluk** belirler.
- ⭐⭐ **4 mesleki sorumluluk alanı:**
  - Confidentiality: sözleşme olsun olmasın, işveren ve müşteri gizliliğine saygı.
  - Competence: yetkinliğini farklı gösterme, yetkinliğin dışındaki işi bile bile kabul etme.
  - Intellectual property rights: patent ve copyright yasalarını bil, işveren ve müşterinin fikri mülkiyetini koru.
  - Computer misuse: başkalarının bilgisayarını kötüye kullanma (önemsizden ciddiye: oyun oynamak → virüs yaymak).
- ⭐ Etik kuralları yayınlayan kuruluşlar: **ACM, IEEE, British Computer Society**. Üyeler uymayı taahhüt eder.
- **ACM/IEEE ortak Code of Ethics: kısa** (Şekil 1.6) ve **uzun** versiyonu var.

Sayfa 15–16 Şekil 1.6 ACM/IEEE Etik Kuralları · Etik ikilemler

📖 Sayfada ne anlatılıyor

ŞEKİL 1.6: KISA VERSİYON

"Software Engineering Code of Ethics and Professional Practice" başlıklı metin, ACM/IEEE-CS ortak çalışma grubunun hazırladığı kısa versiyondur. Giriş kısmı (**preamble**) şunu söylüyor: kısa versiyon yüksek seviyede hedefleri (aspirations) özetler, uzun versiyondaki maddeler ise bunların pratikte nasıl uygulanacağını örnekler. Hedefler ayrıntı olmadan içi boş kalır, ayrıntılar da hedefler olmadan kuru ve kuralcı olur. İkisi birlikte bir bütün oluşturur. Yazılım mühendisleri, analiz, tanımlama, tasarım, geliştirme, test ve bakım işini **yararlı ve saygın bir meslek** haline getirmeyi taahhüt eder ve kamunun **sağlık, güvenlik ve refahını** gözeterek **8 ilkeye** uyar:

#	İlke	Özünde ne diyor?
1	**PUBLIC** kamu	Her zaman **kamu yararına** uygun davran.
2	**CLIENT AND EMPLOYER** müşteri ve işveren	Kamu yararıyla çelişmediği sürece müşterinin ve işverenin **çıkarına** davran.
3	**PRODUCT** ürün	Ürünlerin ve değişikliklerin **mümkün olan en yüksek mesleki standartları** karşılamasını sağla.

4	**JUDGMENT** yargı / karar	Mesleki kararlarında **dürüstlüğünü ve bağımsızlığını** koru.
5	**MANAGEMENT** yönetim	Yöneticiler ve liderler, yazılım geliştirme ve bakımının yönetiminde **etik bir yaklaşımı** benimseyip teşvik etmeli.
6	**PROFESSION** meslek	Kamu yararıyla uyumlu şekilde mesleğin **dürüstlüğünü ve itibarını** ilerlet.
7	**COLLEAGUES** meslektaşlar	Meslektaşlarına karşı **adil ve destekleyici** ol.
8	**SELF** kişinin kendisi	Meslekle ilgili **hayat boyu öğrenmeye** katıl ve etik bir yaklaşımı teşvik et.

KURALIN GEREKÇESİ (UZUN VERSİYONDAN)

Bilgisayarlar ticaret, sanayi, devlet, tıp, eğitim ve eğlencede giderek büyüyen bir rol oynuyor. Yazılım mühendisleri yazılımın analiz, tasarım, geliştirme, test ve bakımına doğrudan ya da öğreterek katkıda bulunuyor. Bu yüzden **iyilik yapma ya da zarar verme**, başkalarının bunu yapmasını sağlama ya da etkileme konusunda önemli fırsatları var. Emeklerinin iyilik için kullanılmasını sağlamak için bu kurallara bağlı kalmalılar. Kural, uygulayıcılar, eğitimciler, yöneticiler, politika yapıcılar, stajyerler ve öğrenciler dahil **yazılım mühendisi olan ya da olmak isteyen herkes** için geçerli. İlkeler, kişilerin, grupların ve kurumların içinde bulunduğu etik ilişkileri ve bu ilişkilerdeki temel yükümlülükleri tanımlar. Bu yükümlülüklerin temeli mühendisin **insanlığı** ve işinden etkilenen insanlara gösterilmesi gereken **özen**.

SAYFA 16: ETİK İKİLEMLER (ETHİCAL DİLEMMAS)

Farklı görüş ve hedeflerin olduğu her ortamda etik ikilemler çıkabilir. Örnekler:

• Üst yönetimin politikalarına **prensip olarak katılmıyorsan** ne yapmalısın? Kurum içinde mi savunmalısın, yoksa istifa mı etmelisin?

• Projede sorun görüyorsan bunu yönetime **ne zaman** söylemelisin? Sadece şüpheyken söylersen aşırı tepki vermiş olabilirsin. Çok geç bırakırsan sorun çözülemez hale gelebilir.

Çoğu ikilem küçüktür ve kolay çözülür. Çözülemezse **istifa** ilkeli bir seçenek olabilir, ama bu eş ve çocuklar gibi başkalarını da etkiler.

ÖRNEK: İŞVERENİN ETİK DIŞI DAVRANIŞI

Bir şirket **emniyet açısından kritik (safety-critical)** bir sistem geliştiriyor ve zaman baskısı yüzünden **emniyet doğrulama kayıtlarını sahte hazırlıyor**. Mühendisin görevi gizliliği korumak mı, yoksa müşteriyi uyarmak ya da sistemin güvensiz olabileceğini duyurmak mı?

Sorun şu ki **emniyet konusunda kesinlik yoktur**. Kriterler fazla katı olabilir ve sistem aslında ömrü boyunca güvenle çalışabilir. Ya da düzgün doğrulansa bile kaza çıkarabilir. Erken açıklamak işverene ve çalışanlara zarar verebilir. Açıklamamak ise başkalarına zarar verebilir.

Kararı sen vermelisin. Karar; **olası zararın büyüklüğüne** ve **etkilenecek insanlara** bağlı. Durum çok tehlikeliyse **basın yoluyla kamuoyuna duyurmak** haklı olabilir. Ama her zaman önce **işverenin haklarına saygı göstererek** çözmeye çalışmalısın.

ASKERİ VE NÜKLEER SİSTEMLER

Bir diğer etik konu askeri ve nükleer sistemlerde çalışmak. Bazı insanlar askeri sistemlerle ilgili hiçbir işte çalışmak istemez. Bazıları askeri sistemlerde çalışır ama **silah sistemlerinde** çalışmaz. Bazıları ise **ulusal güvenliği** öncelikli görür ve silah sistemlerinde çalışmayı etik açıdan sorun görmez.

👨‍🏫 Hoca anlatımı

8 ilkeyi ezberle, sınavda "list the principles" diye sorulabilir. Sırası mantıklıdır: en geniş halkadan en dara doğru gider. Önce **toplum (Public)**, sonra **müşteri ve işveren**, sonra yaptığın **ürün**, senin **kararların**, **yöneticiler**, **meslek**, **meslektaşlar** ve en sonda **sen (Self)**.

Hafıza ipucu: **"P-C-P-J-M-P-C-S"**, yani **P**ublic, **C**lient, **P**roduct, **J**udgment, **M**anagement, **P**rofession, **C**olleagues, **S**elf.

Bir de şuna dikkat: **1 numara Public (kamu)!** Yani müşterinin ya da patronun çıkarı kamu yararıyla çelişirse **kamu kazanır**. 2. ilkede de bu yüzden "kamu yararıyla tutarlı olduğu sürece" diye bir şart var.

Etik ikilem kısmı ezber değil, mantık konusu. Örnek olay şu: patronun sahte test raporu hazırladı. Susarsan insanlar zarar görebilir, konuşursan şirket ve iş arkadaşların zarar görebilir. Kitabın cevabı: **önce şirket içinde, işverenin haklarına saygı göstererek çöz. Çok tehlikeliyse kamuoyuna duyurmak haklı olabilir.** Karar, zararın büyüklüğüne ve kimi etkilediğine bağlıdır.

📝 Deftere not

- **ACM/IEEE Code of Ethics (Şekil 1.6):** kısa versiyon = yüksek seviye hedefler, uzun versiyon = ayrıntılar ve örnekler. İkisi birlikte bütün.
- Amaç: yazılım mühendisliğini **yararlı ve saygın** meslek yapmak, kamunun **sağlık, güvenlik ve refahını** korumak.
- ⭐⭐ **8 ilke (P-C-P-J-M-P-C-S):**
  - **1. Public:** kamu yararına davran.
  - **2. Client & Employer:** kamu yararıyla çelişmeden müşteri ve işverenin çıkarına davran.
  - **3. Product:** ürünler en yüksek mesleki standartta olsun.
  - **4. Judgment:** mesleki kararda dürüstlük ve bağımsızlık.
  - **5. Management:** yöneticiler etik yaklaşımı benimsesin ve teşvik etsin.
  - **6. Profession:** mesleğin dürüstlüğünü ve itibarını ilerlet.
  - **7. Colleagues:** meslektaşlara adil ve destekleyici ol.
  - **8. Self:** hayat boyu öğren, etik yaklaşımı teşvik et.
- Kural **herkes** için geçerli: uygulayıcı, eğitimci, yönetici, politika yapıcı, stajyer, öğrenci. Temeli: mühendisin **insanlığı** ve etkilenen insanlara **özen**.
- ⭐ **Etik ikilemler (ethical dilemmas):**
  - Yönetimle prensip anlaşmazlığı: içeride savun mu, istifa mı?
  - Projedeki sorunu ne zaman söylemeli? Erken (şüpheyken) = aşırı tepki riski, geç = çözülemez.
  - Çoğu ikilem küçük. İstifa ilkeli ama aileyi de etkiler.
- ⭐ **Örnek:** safety-critical sistemde şirket **doğrulama kayıtlarını sahte** hazırlıyor. Gizlilik mi, uyarmak mı?
  - **Emniyette kesinlik yok** (kriter fazla katı olabilir, ya da doğrulanmış sistem bile kaza yapabilir).
  - Erken açıklama = işverene ve çalışanlara zarar. Açıklamamak = başkalarına zarar.
  - Karar: **zararın büyüklüğü + etkilenen insanlar**. Çok tehlikeliyse **basına duyurmak** haklı olabilir, ama önce **işverenin haklarına saygıyla** çözmeye çalış.
- **Askeri ve nükleer sistemler:** (1) hiç çalışmayanlar, (2) askeri olur ama silah olmaz diyenler, (3) ulusal güvenlik öncelikli diyerek silah sistemlerinde de çalışanlar.

Sayfa 17 Etik tartışmanın sonu · Key Points (Bölümün ana noktaları)

📖 Sayfada ne anlatılıyor

ASKERİ İŞLER (DEVAM)

Bu durumda işveren ve çalışanın görüşlerini **önceden** birbirine açıkça belirtmesi önemli. Askeri ya da nükleer işler yapan bir kurum, çalışanların her görevi kabul etmesi gerektiğini baştan söyleyebilmeli. Aynı şekilde bir

çalışan işe girerken bu sistemlerde çalışmak istemediğini açıkça belirttiyse, işveren onu sonradan buna **zorlamamalı**.

ETİĞE YAKLAŞIM

Etik ve mesleki sorumluluk son yıllarda daha çok ilgi görüyor. Konuya **felsefi** açıdan bakılabilir: önce temel etik ilkeler ele alınır, sonra yazılım etiği bunlara göre tartışılır (Laudon 1995, daha az ölçüde Huff ve Martin 1995). Ama yazar bu yaklaşımı **fazla soyut** ve günlük hayatla bağ kurması zor buluyor. Davranış kuralları ve uygulamalardaki **daha somut** yaklaşımı tercih ediyor. Ona göre etik ayrı bir konu olarak değil, **yazılım mühendisliği bağlamında** tartışılmalı. Bu yüzden kitapta soyut etik tartışması yok, uygun yerlerde grup tartışmasına başlangıç olacak alıştırmalar var.

KEY POINTS: BÖLÜMÜN ANA NOKTALARI

• Yazılım mühendisliği, yazılım üretiminin **tüm yönleriyle** ilgilenen bir mühendislik disiplinidir.
• Yazılım ürünleri, geliştirilen **programlar ve ilgili belgelerden** oluşur. Temel ürün özellikleri: **maintainability, dependability, efficiency ve acceptability**.
• Yazılım süreci, geliştirmedeki **tüm faaliyetleri** içerir. Üst seviye faaliyetler olan **specification, development, validation ve evolution** bütün süreçlerin parçasıdır.
• Yöntemler, yazılım üretmenin **düzenli yollarıdır**. İzlenecek süreç, kullanılacak gösterimler, geliştirilecek modeller, bu modelleri yöneten kurallar ve tasarım önerileri içerirler.
• **CASE araçları**, süreçteki rutin işleri destekleyen yazılım sistemleridir: tasarım diyagramlarını düzenlemek, diyagram tutarlılığını kontrol etmek, çalıştırılan testleri takip etmek gibi.
• Yazılım mühendislerinin mesleğe ve topluma karşı sorumlulukları var. Sadece **teknik konularla** ilgilenmemelidirler.
• Meslek kuruluşları, üyelerinden beklenen davranış standartlarını belirleyen **davranış kuralları (codes of conduct)** yayınlar.

👨‍🏫 Hoca anlatımı

Askeri sistem meselesinin özü **"baştan açık ol"**. İşe girerken "ben silah projesinde çalışmam" dediysen, patron sonra seni zorlamamalı. Şirket de "burada her işi yaparsın" diyorsa bunu baştan söylemeli.

Yazarın etik konusundaki tavrı da şu: felsefe kitaplarındaki soyut tartışmalar yerine **somut kurallar ve gerçek örnekler** daha işe yarar.

Key Points kısmı bölümün resmi özetidir, sınavdan önce mutlaka oku! Bir şeye dikkat: Key Points'te 4. özellik **"acceptability"** olarak geçiyor, Şekil 1.5'te ise **"usability"**. İkisi aynı şeyi anlatıyor: yazılım kullanıcı tarafından **kabul edilebilir ve kullanılabilir** olmalı. Sınavda hangisini yazarsan yaz, yanına diğerini de parantez içinde ekle.

📝 Deftere not

- Askeri işler: işveren ve çalışan görüşlerini **önceden açıkça** belirtmeli. Çalışan baştan "çalışmam" dediyse **zorlanmamalı**.
- Etiğe yaklaşım: felsefi yaklaşım (Laudon, Huff & Martin) yazara göre **fazla soyut**. Yazar **somut** yaklaşımı (codes of conduct) tercih ediyor. Etik, **SE bağlamında** tartışılmalı.
- ⭐⭐ **KEY POINTS:**
  - SE = yazılım üretiminin **tüm yönleriyle** ilgilenen mühendislik disiplini.
  - Yazılım ürünü = **programlar + belgeler**. Özellikler: **maintainability, dependability, efficiency, acceptability (= usability)**.
  - Süreç = tüm faaliyetler. Üst seviye: **specification, development, validation, evolution**.
  - Method = düzenli üretim yolu: **süreç, notasyon, modeller, kurallar, tasarım önerileri**.
  - **CASE:** rutin işlere destek (diyagram düzenleme, tutarlılık kontrolü, test takibi).

- Mühendisin sorumluluğu **sadece teknik değil**, mesleğe ve topluma karşı da var.
- Meslek kuruluşları **codes of conduct** yayınlar.

Sayfa 18–19 Further Reading · Exercises (Alıştırmalar)

📖 Sayfada ne anlatılıyor

FURTHER READİNG (EK OKUMA)

Yazar ek kaynaklar öneriyor: Ghezi ve arkadaşlarının *Fundamentals of Software Engineering* kitabı, IEEE Software'in "uygulamanın durumu" özel sayısı, Peters ve Pedrycz'in vaka çalışmalı kitabı, Bott ve arkadaşlarının yasal ve etik konuları ele alan kitabı, ACM/IEEE etik kodunun onaylanmasını anlatan makale ve Brooks'un ünlü **"No Silver Bullet"** makalesi (1987). Bu makalenin mesajı: **yazılım mühendisliğinin sorunlarına tek ve basit bir çözüm (gümüş kurşun) yoktur.** *Sınav için önemli değil, sadece "No Silver Bullet" fikrini bil.*

EXERCİSES (ALIŞTIRMALAR)

Bölüm sonunda 10 alıştırma sorusu var. Hocalar sınav sorularını çoğu zaman bunlardan seçer. Türkçeleri:

#	Soru (Türkçe)
1.1	Maliyet dağılımına bakarak, yazılımı neden son kullanıcının çalıştırdığı programlardan **daha fazlası** olarak görmek gerektiğini açıklayın.
1.2	**Generic** ürün geliştirme ile **custom** yazılım geliştirme arasındaki farklar nelerdir?
1.3	Bütün yazılım ürünlerinin sahip olması gereken **4 önemli özellik** nedir? Bazen önemli olabilecek 4 özellik daha önerin.
1.4	**Süreç modeli** ile **süreç** arasındaki fark nedir? Süreç modelinin süreç iyileştirmede işe yarayabileceği 2 yol önerin.
1.5	Çok geniş pazara satılan generic ürünlerde **sistem testi maliyetinin** neden çok yüksek olduğunu açıklayın.
1.6	Yöntemler ancak CASE teknolojisi çıkınca yaygınlaştı. **CASE araçlarının** sağlayabileceği 5 tür yöntem desteği önerin.
1.7	Heterogeneity, delivery ve trust dışında yazılım mühendisliğinin 21. yüzyılda karşılaşacağı başka sorunlar nelerdir?
1.8	Mühendisler doktor ve avukatlar gibi **sertifikalı** olmalı mı? Tartışın.
1.9	Şekil 1.6'daki ACM/IEEE **etik kurallarının her biri** için uygun bir örnek önerin.
1.10	Terörle mücadele için vatandaşları izleyen sistemler geliştirmenin **etik** yönünü tartışın (mahremiyet).

👨‍🏫 Hoca anlatımı

Bu alıştırmaların en önemlilerinin (1.2, 1.3, 1.4, 1.5, 1.6, 1.9) **İngilizce cevap kalıplarını** bölüm sonundaki "Olası sınav soruları" kısmına koydum. Klasik sınavda büyük ihtimalle bunlara benzer sorular gelir.

1.7, 1.8 ve 1.10 gibi tartışma soruları daha çok yorum ister. Bunlarda tek doğru cevap yok, iki tarafı da yazman yeterli.

📝 Deftere not

- **"No Silver Bullet" (Brooks, 1987):** SE sorunlarına **tek ve basit bir çözüm yoktur**.

- ⭐ Sınavda en çok beklenen alıştırmalar: **1.2** (generic ve custom farkı), **1.3** (4 özellik), **1.4** (süreç ve süreç modeli farkı), **1.5** (generic üründe test neden pahalı), **1.6** (CASE desteği), **1.9** (etik ilke örnekleri). Cevapları bölüm sonunda.

BÖLÜM 1 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 1'deki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
software engineering	yazılım mühendisliği	Yazılım üretiminin tüm yönleriyle ilgilenen mühendislik disiplini	4, 7
cost-effective	maliyet açısından verimli	Harcanan paraya göre iyi sonuç veren	4
abstract / intangible	soyut / elle tutulamaz	Yazılımın fiziksel olmaması	4
complexity	karmaşıklık	Yazılımın en büyük sorunu	4
software crisis	yazılım krizi	1960'lardaki gecikme, maliyet ve hata dönemi (1968 konferansı)	4
integrated circuits	entegre devreler	Bilgisayarları çok güçlendiren donanım teknolojisi	4
informal development	gelişigüzel geliştirme	Kuralsız, plansız yazılım yapma	4
reliable / unreliable	güvenilir / güvenilmez	Hatasız çalışan / sık bozulan	4
over budget / late	bütçe aşımı / gecikme	Planlanandan pahalı / geç	4
documentation	belgeler, dokümantasyon	Sistem belgeleri ve kullanıcı belgeleri	5
configuration data	yapılandırma verisi	Programların doğru çalışması için gereken ayarlar	5
generic product	genel ürün	Açık pazarda herkese satılan yazılım	5
customised / bespoke product	özel / ısmarlama ürün	Tek müşteri için yapılan yazılım	5
stand-alone	bağımsız	Tek başına çalışan	5
contractor	yüklenici	Yazılımı sipariş üzerine geliştiren firma	5
specification	tanımlama, şartname	Yazılımın ne yapacağını ve kısıtlarını belirleme	6, 8
ERP	kurumsal kaynak planlama	Şirkete uyarlanan büyük genel sistem (ör. SAP)	6
engineering discipline	mühendislik disiplini	Kısıtlar içinde çalışan çözüm üretme	7
organisational / financial constraints	kurumsal / finansal kısıtlar	Mühendisin çalıştığı sınırlar	7
computer science	bilgisayar bilimi	Teori ve temel yöntemler	7
ad hoc	duruma özel, geçici	Teoriye dayanmayan, o probleme özel çözüm	7
system engineering	sistem mühendisliği	Donanım + yazılım + süreç: sistemin tamamı	7–8
architecture	mimari	Sistemin genel yapısı	8

integration / integrate	entegrasyon / birleştirme	Parçaları bir araya getirip bütün sistem yapmak	8, 9
deployment	kurulum, devreye alma	Sistemi kullanıma sokma	8
software process	yazılım süreci	Yazılımı üreten faaliyetler ve sonuçları	8
development	geliştirme	Tasarım + programlama	8
validation	geçerleme	Müşterinin istediği mi diye kontrol	8
evolution	evrim	Yazılımı değişen ihtiyaçlara göre değiştirme	8
real-time	gerçek zamanlı	Anında tepki vermesi gereken (ör. uçak yazılımı)	8
process model	süreç modeli	Sürecin tek bakış açısından basitleştirilmiş gösterimi	8
workflow model	iş akışı modeli	Faaliyet sırası, girdi/çıktı (insan eylemleri)	9
dataflow / activity model	veri akışı modeli	Veri dönüşümleri	9
role/action model	rol/eylem modeli	Kim hangi işten sorumlu	9
waterfall approach	şelale yaklaşımı	Ayrı ve sıralı aşamalar	9
signed-off	onaylanmış	Aşamanın bitip imzalanması	9
iterative development	yinelemeli geliştirme	Faaliyetler iç içe, sürekli iyileştirme	9
CBSE	bileşen tabanlı YM	Hazır parçaları birleştirerek geliştirme	9
cost distribution	maliyet dağılımı	Paranın faaliyetlere bölünmesi	10
long-lifetime system	uzun ömürlü sistem	10+ yıl kullanılan sistem	11
method	yöntem	Yapılandırılmış geliştirme yaklaşımı	11
function-oriented	fonksiyon odaklı	1970'lerin yöntemleri (Structured Analysis, JSD)	11
object-oriented (OO)	nesne yönelimli	1980–90'ların yöntemleri (Booch, Rumbaugh)	11
UML	Birleşik Modelleme Dili	Yöntemleri birleştiren standart gösterim	11
notation	gösterim	Model çizmek için semboller	4, 12
rules / recommendations	kurallar / öneriler	Zorunlu kısıt / tavsiye	12
heuristics	sezgisel kurallar	Deneyime dayalı tavsiyeler	12
process guidance	süreç rehberliği	Faaliyetlerin sırası	12
CASE	bilgisayar destekli YM	Süreci destekleyen yazılım araçları	12
code generator	kod üreticisi	Modelden otomatik kod yazan araç	12
non-functional attributes	fonksiyonel olmayan özellikler	Yazılımın nasıl davrandığı (hız, anlaşılabilirlik)	12
maintainability	bakım kolaylığı	Kolayca değiştirilebilme	13
dependability	güvenilebilirlik	Reliability + security + safety	13

reliability / security / safety	güvenilirlik / güvenlik / emniyet	Hatasız çalışma / saldırıya dayanıklılık / zarar vermeme	13
efficiency	verimlilik	Kaynakları israf etmeme	13
usability / acceptability	kullanılabilirlik / kabul edilebilirlik	Kullanıcının zorlanmadan kullanması	13, 17
heterogeneity	çeşitlilik, farklılık	Farklı sistem, dil ve makinelerin bir arada olması	13
distributed system	dağıtık sistem	Ağ üzerinde birden çok bilgisayarda çalışan sistem	13
legacy system	eski (miras) sistem	Uzun süredir kullanılan, eski teknolojili sistem	13
delivery	teslim	Yazılımı müşteriye verme	13
trust	güven	Kullanıcının yazılıma güvenmesi	13
ethical / professional responsibility	etik / mesleki sorumluluk	Teknik becerinin ötesindeki sorumluluklar	14
honesty / integrity	dürüstlük / doğruluk	Temel ahlaki standartlar	14
confidentiality	gizlilik	İşveren ve müşteri sırlarını koruma	14
competence	yetkinlik	Bir işi yapabilecek bilgi ve beceri	14
intellectual property rights	fikri mülkiyet hakları	Patent, telif hakkı (copyright)	14
computer misuse	bilgisayarın kötüye kullanımı	Başkasının bilgisayarını kötü amaçla kullanma	14
code of conduct / ethics	davranış / etik kuralları	Meslek kuruluşlarının yayınladığı kurallar	14
ACM / IEEE	meslek kuruluşları	Ortak etik kodu hazırlayan kuruluşlar	14
public interest	kamu yararı	Toplumun çıkarı (1 numaralı ilke)	15
ethical dilemma	etik ikilem	İki doğru arasında seçim yapmak zorunda kalmak	16
safety-critical system	emniyet açısından kritik sistem	Hatası insan hayatını tehlikeye atan sistem	16
falsify records	kayıtları sahte hazırlamak	Etik dışı davranış örneği	16

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 1'den sınavda en çok çıkabilecekler

1. **SE tanımı:** yazılım üretiminin **tüm yönleriyle** ilgilenen engineering discipline. (s.4, 7)
2. **1968, software crisis:** geç, pahalı, güvenilmez, bakımı zor yazılımlar. SE kavramı bu konferansta doğdu. (s.4)
3. **Software = programs + documentation + configuration data** (s.5)
4. **Generic** (herkese satılır, spec'i geliştirici kontrol eder) ve **Customised** (tek müşteri, spec'i müşteri kontrol eder). (s.5–6)
5. **CS = teori, SE = pratik. System eng. = bütün sistem** (donanım + yazılım + süreç), SE onun parçası. (s.7–8)
6. **4 temel süreç faaliyeti:** Specification, Development, Validation, Evolution (s.8)
7. **Process model:** sürecin tek bakış açısından basit gösterimi. Türleri: **workflow, dataflow, role/action** (s.8–9)
8. **3 paradigma:** Waterfall (sıralı, signed-off), Iterative (iç içe), CBSE (hazır parçalar) (s.9)
9. **Maliyet:** ≈ %60 geliştirme, %40 test. Waterfall'da en pahalısı test. Uzun ömürlü sistemde evolution = 3–4× development. Generic üründe test çok pahalı. (s.6, 10–11)
10. **Yöntemin 4 bileşeni:** model descriptions, rules, recommendations, process guidance (s.12)
11. **CASE = Computer-Aided Software Engineering**: editörler, analiz modülleri, rapor ve kod üreticiler (s.12)
12. **İyi yazılımın 4 özelliği (M-D-E-U):** Maintainability, Dependability, Efficiency, Usability (s.13)
13. **3 zorluk:** Heterogeneity, Delivery, Trust (s.13)
14. **4 mesleki sorumluluk:** Confidentiality, Competence, Intellectual property rights, Computer misuse (s.14)
15. **ACM/IEEE 8 ilke:** Public, Client & Employer, Product, Judgment, Management, Profession, Colleagues, Self (s.15)

Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What is software engineering?

Yazılım mühendisliği nedir? (s.4, 7)

> *Software engineering is an engineering discipline that is concerned with all aspects of software production, from system specification to maintenance. It focuses on the cost-effective development of high-quality software.*

Yazılım mühendisliği, sistem tanımlamasından bakıma kadar yazılım üretiminin tüm yönleriyle ilgilenen bir mühendislik disiplinidir. Kaliteli yazılımın düşük maliyetle geliştirilmesine odaklanır.

Q2. What was the software crisis?

Yazılım krizi neydi? (s.4)

> *New hardware based on integrated circuits made large and complex software possible. Informal development was not good enough. Projects were late, over budget, unreliable and difficult to maintain. The term "software engineering" was first proposed in 1968 because of this crisis.*

Entegre devreli donanım büyük ve karmaşık yazılımları mümkün kıldı. Gelişigüzel geliştirme yetmedi. Projeler gecikti, bütçeyi aştı, güvenilmez ve bakımı zordu. "Yazılım mühendisliği" terimi bu kriz yüzünden 1968'de ortaya çıktı.

Q3. What is software?

Yazılım nedir? (s.5)

> *Software is not only computer programs. It also includes the associated documentation and the configuration data that are needed to make these programs work correctly.*

Yazılım sadece bilgisayar programları değildir. Programların doğru çalışması için gereken belgeleri ve yapılandırma verilerini de içerir.

Q4. What are the differences between generic and customised software products? (Ex. 1.2)

Genel ve özel yazılım ürünleri arasındaki farklar nelerdir? (s.5–6)

> *Generic products are stand-alone systems that are sold on the open market to any customer, for example word processors. Customised products are developed for one particular customer, for example air traffic control systems. In generic products, the developer controls the specification. In customised products, the customer controls the specification.*

Genel ürünler açık pazarda herkese satılan bağımsız sistemlerdir (ör. kelime işlemci). Özel ürünler tek bir müşteri için geliştirilir (ör. hava trafik kontrolü). Genel üründe tanımlamayı geliştirici, özel üründe müşteri kontrol eder.

Q5. What is the difference between software engineering, computer science and system engineering?

YM, bilgisayar bilimi ve sistem mühendisliği arasındaki fark nedir? (s.7–8)

> *Computer science is concerned with theory and fundamentals. Software engineering is concerned with the practical problems of producing software. System engineering is concerned with all aspects of computer-based systems, including hardware, software and process. Software engineering is a part of system engineering.*

Bilgisayar bilimi teori ve temellerle ilgilenir. Yazılım mühendisliği yazılım üretmenin pratik sorunlarıyla ilgilenir. Sistem mühendisliği donanım, yazılım ve süreç dahil bilgisayar tabanlı sistemin tüm yönleriyle ilgilenir. YM, sistem mühendisliğinin bir parçasıdır.

Q6. What are the four fundamental activities of a software process?
Yazılım sürecinin 4 temel faaliyeti nelerdir? (s.8)

> *1) Software specification: the software and its constraints are defined. 2) Software development: the software is designed and programmed. 3) Software validation: the software is checked to make sure it is what the customer wants. 4) Software evolution: the software is changed for new customer and market needs.*

1) Tanımlama: yazılım ve kısıtları tanımlanır. 2) Geliştirme: tasarlanır ve programlanır. 3) Geçerleme: müşterinin istediği mi diye kontrol edilir. 4) Evrim: yeni müşteri ve pazar ihtiyaçlarına göre değiştirilir.

Q7. What is the difference between a software process and a software process model? (Ex. 1.4)
Süreç ve süreç modeli arasındaki fark nedir? (s.8–9)

> *A software process is the set of activities that produce a software product. A software process model is a simplified representation of a software process from one particular perspective. Examples are the workflow model, the dataflow model and the role/action model.*

Süreç, yazılım ürününü üreten faaliyetler kümesidir. Süreç modeli ise sürecin belirli bir bakış açısından basitleştirilmiş gösterimidir. Örnekler: iş akışı, veri akışı ve rol/eylem modeli.

Q8. Explain the three general process models (paradigms).
3 genel süreç paradigmasını açıklayın. (s.9)

> *1) Waterfall: activities are separate phases. Each phase is signed-off before the next one starts. 2) Iterative development: specification, development and validation are interleaved. A first system is developed quickly and then improved with customer feedback. 3) Component-based software engineering (CBSE): the system is built by integrating existing components.*

1) Şelale: faaliyetler ayrı aşamalardır, her aşama onaylanınca sonrakine geçilir. 2) Yinelemeli: tanımlama, geliştirme ve geçerleme iç içedir, hızlı ilk sistem müşteri geri bildirimiyle iyileştirilir. 3) CBSE: sistem hazır bileşenler birleştirilerek kurulur.

Q9. Why are system testing costs high for generic software products? (Ex. 1.5)
Generic ürünlerde sistem testi neden pahalıdır? (s.11)

> *Generic products are used in many different configurations, with different hardware and software. So they must be tested extensively for all of these configurations. Because of this, testing is the most expensive activity.*

Genel ürünler çok farklı donanım ve yazılım yapılandırmalarında kullanılır. Bu yüzden hepsi için kapsamlı test edilmeleri gerekir. Bu nedenle test en pahalı faaliyettir.

Q10. What are the components of a software engineering method?
Yöntemin bileşenleri nelerdir? (s.12)

> *1) System model descriptions: the models and the notation. 2) Rules: constraints that always apply to models. 3) Recommendations: advice for good design practice. 4) Process guidance: the activities to follow and their order.*

1) Model tanımları: modeller ve gösterim. 2) Kurallar: modellere her zaman uygulanan kısıtlar. 3) Öneriler: iyi tasarım tavsiyeleri. 4) Süreç rehberliği: izlenecek faaliyetler ve sırası.

Q11. What is CASE? Suggest types of support CASE tools can provide. (Ex. 1.6)
CASE nedir? CASE araçları ne tür destek sağlar? (s.12)

> *CASE (Computer-Aided Software Engineering) means software systems that support software process activities. CASE tools can provide: 1) diagram editors, 2) analysis tools that check models against the rules, 3) report generators for documentation, 4) code generators that create code from models, 5) process guidance.*

CASE, süreç faaliyetlerini destekleyen yazılım sistemleridir. Sağladıkları: 1) diyagram editörleri, 2) modeli kurallara göre kontrol eden analiz araçları, 3) belge için rapor üreticileri, 4) modelden kod üreten kod üreticileri, 5) süreç rehberliği.

Q12. What are the essential attributes of good software? (Ex. 1.3)
İyi yazılımın temel özellikleri nelerdir? (s.13)

> *1) Maintainability: software should be easy to change for new customer needs. 2) Dependability: software should be reliable, secure and safe. It should not cause damage if it fails. 3) Efficiency: software should not waste system resources like memory and processor. 4) Usability: software should be easy to use, with a good user interface and documentation.*

1) Bakım kolaylığı: yeni ihtiyaçlara göre kolayca değiştirilebilmeli. 2) Güvenilebilirlik: güvenilir, güvenli ve emniyetli olmalı, çökünce zarar vermemeli. 3) Verimlilik: bellek ve işlemciyi israf etmemeli. 4) Kullanılabilirlik: iyi arayüzü ve belgesiyle kolay kullanılmalı.

Q13. What are the key challenges facing software engineering?
YM'nin temel zorlukları nelerdir? (s.13)

> *1) Heterogeneity: systems must work on different computers and networks, together with old legacy systems. 2) Delivery: software must be delivered faster without reducing quality. 3) Trust: users must be able to trust the software.*

1) Çeşitlilik: sistemler farklı bilgisayar ve ağlarda, eski sistemlerle birlikte çalışmalı. 2) Teslim: yazılım kaliteden ödün vermeden daha hızlı teslim edilmeli. 3) Güven: kullanıcılar yazılıma güvenebilmeli.

Q14. What are the areas of professional responsibility?
Mesleki sorumluluk alanları nelerdir? (s.14)

> *1) Confidentiality: respect the confidentiality of employers and clients. 2) Competence: do not accept work outside your competence. 3) Intellectual property rights: protect patents and copyright. 4) Computer misuse: do not misuse other people's computers.*

1) Gizlilik: işveren ve müşteri gizliliğine saygı. 2) Yetkinlik: yetkinliğinin dışındaki işi kabul etme. 3) Fikri mülkiyet: patent ve telif haklarını koru. 4) Kötüye kullanım: başkalarının bilgisayarını kötüye kullanma.

Q15. List the eight principles of the ACM/IEEE Code of Ethics. (Ex. 1.9)
ACM/IEEE etik kodunun 8 ilkesini sayın. (s.15)

> *1) Public, 2) Client and Employer, 3) Product, 4) Judgment, 5) Management, 6) Profession, 7) Colleagues, 8) Self. For example: Public means acting in the public interest. Product means software must meet the highest professional standards. Self means lifelong learning.*

1) Kamu, 2) Müşteri ve işveren, 3) Ürün, 4) Yargı, 5) Yönetim, 6) Meslek, 7) Meslektaşlar, 8) Kişinin kendisi. Örneğin Public: kamu yararına davranmak. Product: en yüksek mesleki standart. Self: hayat boyu öğrenme.

Mini test

cevaplar en altta

1. The term "software engineering" was first proposed in:
   a) 1958
   b) 1968
   c) 1978
   d) 1988
2. Software consists of:
   a) only programs
   b) programs and hardware
   c) programs, documentation and configuration data
   d) only documentation
3. In generic products, who controls the specification?
   a) The customer
   b) The developer
   c) The government
   d) The user group
4. Which one is an example of a customised (bespoke) product?
   a) Word processor
   b) Database software
   c) Air traffic control system
   d) Drawing package
5. Computer science is mainly concerned with:
   a) theory and fundamentals
   b) practical problems
   c) hardware only
   d) project management
6. Which is NOT one of the four fundamental process activities?
   a) Specification
   b) Validation
   c) Marketing
   d) Evolution
7. In which approach is each phase "signed-off" before the next one starts?
   a) Iterative
   b) Waterfall
   c) CBSE
   d) Agile
8. CBSE assumes that:
   a) everything is written from scratch
   b) parts of the system already exist
   c) there is no testing
   d) there is no specification
9. In the waterfall model, the most expensive activity is usually:
   a) specification
   b) design
   c) integration and testing
   d) documentation
0. For long-lifetime systems, evolution costs can be about ___ development costs.
   a) half of
   b) equal to
   c) 3–4 times
   d) 10 times
1. "No object should have more than seven sub-objects" is an example of:
   a) a rule
   b) a recommendation
   c) process guidance
   d) a model description
2. CASE stands for:
   a) Computer-Aided Software Engineering
   b) Computer Application System Engineering
   c) Code Analysis System Editor
   d) Computer-Assisted System Evaluation
3. Dependability includes:
   a) speed, memory, cost
   b) reliability, security, safety
   c) interface, documentation
   d) design, code, test

4. "Software should not make wasteful use of memory and processor" describes:
   a) Maintainability
   b) Usability
   c) Efficiency
   d) Dependability
5. Integrating new software with old legacy systems in different languages is the ___ challenge.
   a) trust
   b) delivery
   c) heterogeneity
   d) security
6. "You should not knowingly accept work that is outside your competence" is about:
   a) Confidentiality
   b) Competence
   c) Computer misuse
   d) Intellectual property
7. The first principle of the ACM/IEEE Code of Ethics is:
   a) Self
   b) Product
   c) Public
   d) Colleagues
8. Which principle says software engineers should take part in lifelong learning?
   a) Profession
   b) Self
   c) Management
   d) Judgment

Cevap anahtarı: 1-b · 2-c · 3-b · 4-c · 5-a · 6-c · 7-b · 8-b · 9-c · 10-c · 11-b · 12-a · 13-b · 14-c · 15-c · 16-b · 17-c · 18-b

YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 3
Critical systems

Kritik Sistemler

Kitap s.43–62 (Bölüm 3 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 3, **kritik sistemleri** tanıtıyor. Bunlar en önemli özelliği **güvenilebilirlik (dependability)** olan sistemlerdir. Beş kısımdan oluşuyor:

- **3.1 A simple safety-critical system:** Kitap boyunca kullanılacak örnek, **insülin pompası**.
- **3.2 System dependability:** Güvenilebilirliğin 4 boyutu: availability, reliability, safety, security.
- **3.3 Availability and reliability:** Hazır bulunma ve güvenilirlik. Fault, error ve failure farkı.
- **3.4 Safety:** Emniyet, emniyet terimleri (hazard, risk, accident...).
- **3.5 Security:** Güvenlik, güvenlik terimleri (vulnerability, attack, threat...).

Bölümün amaçları: (1) kritik sistemde arızanın ciddi insani ya da ekonomik sonuçları olabileceğini anlamak, (2) güvenilebilirliğin 4 boyutunu bilmek, (3) güvenilebilirlik için geliştirmede hatalardan kaçınmak, kullanımda hataları bulup gidermek ve arızanın zararını sınırlamak gerektiğini anlamak.

Sayfa 44–45 Kritik sistemler nedir? · Türleri · Neden güvenilebilirlik?

📖 Sayfanın özeti

KRİTİK SİSTEM

Yazılım arızaları yaygındır ve çoğu sadece rahatsızlık verir. Ama bazı sistemlerde arıza büyük ekonomik kayba, fiziksel hasara ya da can kaybına yol açabilir. İnsanların ya da işletmelerin bağımlı olduğu bu teknik ya da sosyo-teknik sistemlere **kritik sistemler** denir. Üç ana türü vardır: **safety-critical** (kimya fabrikası kontrol sistemi), **mission-critical** (uzay aracı navigasyonu), **business-critical** (bankanın müşteri hesap sistemi).

DEPENDABİLİTY

Kritik bir sistemin en önemli emergent özelliği **güvenilebilirliktir**. Laprie'nin önerdiği bu terim availability, reliability, safety ve security'yi tek çatı altında toplar. En önemli olmasının üç nedeni var: kullanıcılar güvenmedikleri sistemi (ve aynı firmanın diğer ürünlerini) reddeder, arızanın maliyeti sistemin kendisinden çok daha yüksek olabilir (reaktör, uçak navigasyonu) ve güvenilmez sistemler değerli verilerin kaybına yol açabilir.

GELİŞTİRME YAKLAŞIMI

Arıza çok pahalı olduğu için kritik sistemlerde **denenmiş, eski teknikler** tercih edilir. Geliştiriciler muhafazakârdır: zayıf yönleri bilinen eski yöntem, uzun vadeli sorunları bilinmeyen yeni yönteme tercih edilir. Normal sistemlerde pahalı kaçan **formal (matematiksel) yöntemler** burada kullanılabilir, çünkü gereken test miktarını azaltır. Kritik sistemlerde doğrulama ve geçerleme (V&V) maliyeti genelde toplam maliyetin **%50'sinden fazladır**.

İNSANLAR VE 3 ARIZA KAYNAĞI

Kritik sistemlerin çoğu, insanların izleyip kontrol ettiği sosyo-teknik sistemlerdir, çünkü beklenmedik durumlarla başa çıkabilecek insanlara ihtiyaç vardır. Ama insanlar da hata yapabilir. Arızanın üç kaynağı vardır: **donanım** (tasarım, üretim hatası ya da ömrün dolması), **yazılım** (tanımlama, tasarım ya da kodlama hatası) ve **insan operatörler**. Donanım ve yazılım güvenilirleştikçe operatör hataları artık muhtemelen **en büyük tek arıza nedeni**. Bu kaynaklar birbirini tetikler (donanım arızası, operatörde stres, hata, yazılım arızası...). Bu yüzden tasarımcılar sisteme **bütüncül (holistic)** bakmalıdır, yoksa hatalar parçalar arasındaki **arayüzlerde** ortaya çıkar.

👨‍🏫 Hoca anlatımı

Telefonundaki oyun çökerse sinirlenirsin, o kadar. Ama bir uçağın otopilotu "çökerse" insanlar ölür. İşte ikincisi **kritik sistemdir**.

3 türü mutlaka ezberle, neredeyse kesin sorulur. Farkları "ne kaybedersin?" sorusuyla hatırla: **Safety-critical**, can ya da çevre (kimya fabrikası patlarsa). **Mission-critical**, görev (uzay aracının rotası şaşarsa görev başarısız olur). **Business-critical**, para (banka sistemi çökerse şirket büyük para kaybeder).

Dependability bir "şemsiye" kelimedir. Altında 4 şey var: availability, reliability, safety, security. Bunlar birbirine çok bağlı olduğu için tek bir kelimeyle anılıyor.

Kritik sistem geliştiricileri neden **"tutucu"**? Bir hastanede ameliyat robotu yaparken "yeni çıkmış, kimsenin denemediği bir yöntem kullanalım" demezsin. Eski ama nasıl davrandığını iyi bildiğin yöntemi seçersin.

Son olarak: bugün arızaların en büyük nedeni **insan hatası**. Donanım ve yazılım çok gelişti ama insanlar hâlâ stres altında hata yapıyor.

📝 Deftere not

- ⭐ **Critical systems:** arızası **ciddi ekonomik kayıp, fiziksel hasar ya da can tehlikesi** yaratan, insanların ya da işletmelerin bağımlı olduğu teknik veya sosyo-teknik sistemler.
- ⭐⭐ **3 tür kritik sistem:**
  - Safety-critical: arıza = yaralanma, ölüm, ciddi çevre hasarı. Ör: **kimya fabrikası kontrol sistemi**.
  - Mission-critical: arıza = hedefe yönelik bir faaliyetin başarısızlığı. Ör: **uzay aracı navigasyon sistemi**.
  - Business-critical: arıza = işletmeye çok yüksek maliyet. Ör: **bankanın müşteri hesap sistemi**.
- ⭐ En önemli emergent özellik = dependability (Laprie 1995): **availability + reliability + safety + security**.
- ⭐ **Dependability neden en önemli? (3 neden)**
  - Güvenilmez, emniyetsiz ya da güvensiz sistemler **kullanıcılarca reddedilir** (firmanın diğer ürünleri de).
  - **Arıza maliyeti devasa** olabilir (reaktör, uçak navigasyonu: arıza maliyeti sistem maliyetinden kat kat fazla).
  - Güvenilmez sistemler **bilgi kaybına** yol açar (veri, sistemden daha değerli olabilir).
- Kritik sistemlerde **denenmiş eski teknikler** kullanılır (muhafazakâr yaklaşım). **Formal yöntemler** kullanılabilir (test miktarını azaltır). **V&V maliyeti > %50**.
- Çoğu kritik sistem **sosyo-tekniktir**: beklenmedik durumlar için insan gerekir, ama insan da hata yapar.
- ⭐ **3 arıza kaynağı: hardware** (tasarım, üretim hatası, ömür sonu), **software** (spec, tasarım, kodlama hatası), **human operators**. Operatör hatası artık **en büyük tek neden**.

- Kaynaklar birbirini tetikler (donanım arızası, stres, insan hatası, yazılım arızası). Tasarımcı **holistic (bütüncül)** bakmalı, yoksa hatalar **arayüzlerde** çıkar.

Sayfa 46–47 3.1 Basit bir safety-critical sistem: İnsülin pompası

📖 Sayfanın özeti

NEDEN BU ÖRNEK?

Kritik sistemleri anlamak için uygulama alanını bilmek gerekir. Yazar herkesin az çok anladığı tıbbi bir örnek seçmiş: **pankreasın işini taklit eden**, şeker hastalarına yardım eden bir sistem.

TIBBİ ARKA PLAN

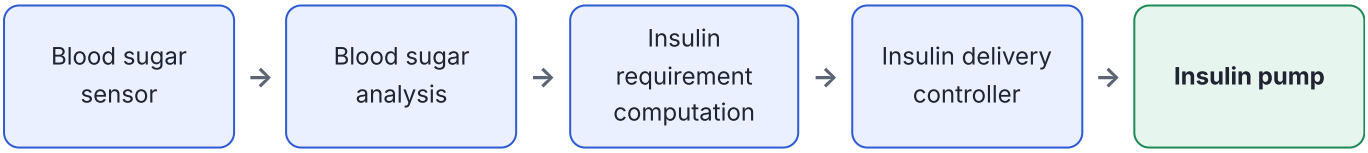
Diyabette pankreas yeterince **insülin** üretemez. İnsülin kandaki glikozu (şekeri) işler. Klasik tedavide hasta şekerini ölçüp gereken insülini hesaplar ve kendine iğne yapar. Sorun şu ki kandaki insülin seviyesi enjeksiyon zamanına da bağlıdır. Bu yüzden şeker **çok düşebilir** (fazla insülin: kısa vadede daha tehlikeli, beyin işlev bozukluğu, bilinç kaybı, ölüm) ya da **çok yükselebilir** (az insülin: uzun vadede göz, böbrek ve kalp hasarı).

OTOMATİK İNSÜLİN POMPASI

Minyatür sensörler sayesinde şekeri sürekli ölçüp gerektiğinde doğru dozda insülin veren sistemler yapılabiliyor. Hastanelerde zaten var, ileride hastanın vücuduna kalıcı takılabilir. Çalışma şekli: vücuttaki mikro-sensör şekerle orantılı bir kan parametresini ölçer, **kontrolcü (controller)** şeker seviyesini ve gereken insülini hesaplar, minyatür pompaya sinyal gönderir, pompa kalıcı bir iğneyle insülini verir.

ŞEKİL 3.1 VE 3.2

Şekil 3.1 pompanın bileşenlerini gösterir: insülin haznesi, iğne düzeneği, pompa, saat, sensör, kontrolcü, alarm, iki ekran ve güç kaynağı. **Şekil 3.2** bir **veri akış modelidir**: kan → şeker sensörü → kan parametreleri → şeker analizi → şeker seviyesi → insülin ihtiyacı hesabı → insülin dağıtım kontrolcüsü → pompa kontrol komutları → pompa → insülin.



Şekil 3.2 (sadeleştirilmiş): İnsülin pompasının veri akış modeli

İKİ ÜST DÜZEY GÜVENİLEBİLİRLİK GEREKSİNİMİ

(1) Sistem gerektiğinde insülin vermek için **hazır (available)** olmalı. (2) Sistem **güvenilir (reliable)** çalışmalı ve doğru miktarda insülin vermeli. Arıza aşırı doza yol açıp hastanın hayatını tehdit edebileceği için özellikle **aşırı doz (overdose) asla olmamalı**.

👨‍🏫 Hoca anlatımı

Bu örnek kitabın sonraki bölümlerinde de karşına çıkacak, o yüzden iyi anla. Mantık basit: **yapay pankreas**. Sensör şekeri ölçer, bilgisayar ne kadar insülin gerektiğini hesaplar, pompa iğneyle verir.

Neden **safety-critical**? Çünkü yanlış doz insanı öldürebilir. Çok insülin verirse şeker düşer ve hasta bayılıp ölebilir (kısa vadede en tehlikelisi bu). Az verirse şeker yükselir ve yıllar içinde göz, böbrek, kalp zarar görür.

İki gereksinimi hatırla: **available** (lazım olduğunda çalışsın) ve **reliable** (doğru dozu versin). En kritik kural: **asla aşırı doz verme!**

Şekil 3.2 bir **veri akış modeli (data-flow model)** örneği. Bölüm 1'deki dataflow model kavramını hatırla: veri bir uçtan girer, dönüşe dönüşe diğer uçtan çıkar. Burada kan girer, insülin çıkar.

📝 Deftere not

- ⭐ Kitabın örnek kritik sistemi: **insülin pompası** (pankreası taklit eden tıbbi sistem, şeker hastaları için).
- Diyabet: pankreas yeterli **insülin** üretemez. Klasik tedavi: ölç, hesapla, iğne yap. Sorun: insülin seviyesi enjeksiyon **zamanına** da bağlı.
  - **Düşük şeker** (fazla insülin): **kısa vadede daha tehlikeli**, beyin bozukluğu, bilinç kaybı, ölüm.
  - **Yüksek şeker** (az insülin): uzun vadede göz, böbrek, kalp hasarı.
- **Çalışma:** mikro-sensör ölçer → **controller** şekeri ve gereken insülini hesaplar → pompaya sinyal → iğneyle insülin.
- **Şekil 3.1 bileşenleri:** insulin reservoir, needle assembly, pump, clock, sensor, **controller**, alarm, display 1–2, power supply.
- **Şekil 3.2 (data-flow model):** blood → sugar sensor → sugar analysis → insulin requirement computation → delivery controller → pump → insulin.
- ⭐⭐ **2 üst düzey dependability gereksinimi:**
  - Gerektiğinde insülin verecek şekilde **available** olmalı.
  - **Reliable** olmalı, doğru miktarda insülin vermeli.
- ⭐ En kritik: **aşırı doz (overdose) ASLA olmamalı** (hayatı tehdit eder).

Sayfa 47–50 3.2 System dependability (Şekil 3.3 ve 3.4)

📖 Sayfanın özeti

GÜVENİLEBİLİRLİK = GÜVENİLİRLİK DERECESİ

Hepimiz sistemlerin sebepsiz çöktüğünü yaşamışızdır ve bilgisayarlarımıza tam güvenmeyiz. Bir sistemin güvenilebilirliği, kullanıcının sistemin beklendiği gibi çalışacağına ve normal kullanımda "çökmeyeceğine" duyduğu **güvenin derecesidir (trustworthiness)**. Sayıyla ifade edilemez, "güvenilmez", "çok güvenilir", "aşırı güvenilir" gibi göreli ifadelerle anlatılır. Güvenilebilirlik ile **kullanışlılık aynı şey değildir**: yazarın kitabı yazdığı kelime işlemci çok kullanışlı ama pek güvenilir değil. Bu yüzden yazar sık sık kaydedip yedek alarak olası zararı sınırlıyor.

4 ANA BOYUT (ŞEKİL 3.3)

Availability: sistemin herhangi bir anda çalışır ve hizmet verebilir durumda olma olasılığı. **Reliability:** belirli bir süre boyunca hizmetleri kullanıcının beklediği gibi doğru verme olasılığı. **Safety:** sistemin insanlara ya da çevreye zarar verme ihtimaline dair bir yargı. **Security:** sistemin kazara ya da kasıtlı saldırılara direnebilme ihtimaline dair bir yargı.

ALT ÖZELLİKLER VE BAĞLANTILAR

Bu boyutlar daha basit özelliklere bölünebilir. Security: **integrity** (program ve verinin zarar görmemesi) ve **confidentiality** (bilgiye sadece yetkililerin erişmesi). Reliability: **correctness**

(hizmetler tanımlandığı gibi), **precision** (bilgi uygun ayrıntı seviyesinde) ve **timeliness** (bilgi gerektiğinde verilir). Dört boyut birbirine bağlıdır: emniyet genelde availability ve reliability'ye bağlıdır, saldırgan veriyi bozarsa sistem güvenilmez olur, servis dışı bırakma (denial-of-service) saldırıları availability'yi hedefler, virüs bulaşmış bir sistemin emniyetli olduğu varsayılamaz. Bu sıkı bağlar yüzünden tek bir şemsiye kavram kullanılır.

DİĞER GÜVENİLEBİLİRLİK ÖZELLİKLERİ

Repairability: arızalar kaçınılmazdır ama hızlı onarımla aksama azaltılabilir. Bunun için teşhis, bileşene erişim ve değişiklik yapabilmek gerekir. Kaynak koda erişim ve beceri onarılabilirliği artırır, ama üçüncü taraf "kara kutu" bileşenlerle bu zorlaşıyor. **Maintainability:** yeni gereksinimlere düşük maliyetle ve yeni hata sokma olasılığı düşük şekilde uyarlanabilmek. **Survivability:** özellikle internet sistemleri için, saldırı altındayken ve bir kısmı devre dışıyken hizmet vermeye devam edebilmek. Güçlendirmek için üç strateji var: saldırıya direnç, saldırıyı tanıma ve saldırı hasarından kurtulma. **Error tolerance:** kullanılabilirliğin bir parçası. Kullanıcı hatalarından kaçınılması ve hataların tolere edilmesi, yani hatanın algılanıp otomatik düzeltilmesi ya da kullanıcıdan verinin yeniden istenmesi.

MALİYET VE ÖDÜNLEŞİM (ŞEKİL 3.4)

Bu dört boyut her sistemde aynı önemde değildir. İnsülin pompası için en önemlileri availability, reliability ve safety'dir. Ağa bağlı olmadığı ve gizli veri tutmadığı için security daha az önemlidir. Tasarımcılar genelde **performans ile güvenilebilirlik arasında ödünleşim** yapar: güvenilir yazılım istisnai durumları kontrol etmek ve hatalardan kurtulmak için fazladan (çoğu zaman tekrarlı, redundant) kod içerir. Bu performansı düşürür, bellek ihtiyacını ve geliştirme maliyetini artırır. Özellikle geçerleme pahalıdır, çünkü bazen sistemin güvenilir olduğunu dış bir düzenleyiciye (ör. ABD Federal Havacılık Otoritesi) kanıtlamak gerekir. **Şekil 3.4**: güvenilebilirlik arttıkça maliyet **üstel (exponential)** artar. Bu yüzden bir sistemin **%100 güvenilir** olduğunu göstermek imkânsızdır, maliyet sonsuz olurdu.

👨‍🏫 Hoca anlatımı

Şekil 3.3 bu bölümün kalbi! 4 boyutu ve kısa tanımlarını ezberle:

Availability, "şu an çalışıyor mu?" (ATM'ye gittiğinde açık mı?). **Reliability**, "doğru çalışıyor mu?" (ATM doğru parayı veriyor mu, bir ay boyunca hiç hata yaptı mı?). **Safety**, "zarar veriyor mu?" (insana ya da çevreye). **Security**, "saldırıya dayanıyor mu?" (hackerlar giremiyor mu?).

"Kullanışlı" ile "güvenilir" farkına dikkat: Word çok kullanışlıdır ama çökebilir. Sen de Ctrl+S'ye sürekli basarak bunu telafi edersin.

Diğer 4 özellik de (repairability, maintainability, survivability, error tolerance) çıkabilir. **Survivability** en ilginci: saldırı altında bile "en azından temel hizmeti vermeye" devam etmek. Mesela banka sitesi saldırı altındayken bile en azından bakiye görüntülemeyi sürdürmek.

Şekil 3.4'ün mesajı: %90 güvenilirlikten %99'a çıkmak pahalıdır, %99'dan %99.9'a çıkmak çok daha pahalıdır. **%100 imkânsızdır**, çünkü maliyet sonsuza gider. Sınavda "Why is the cost of assuring dependability exponential?" diye sorulabilir (alıştırma 3.4).

📝 Deftere not

- ⭐ **Dependability = trustworthiness:** kullanıcının sistemin beklendiği gibi çalışacağına ve "çökmeyeceğine" duyduğu **güvenin derecesi**. **Sayısal değil**, göreli ("not / very / ultra-

dependable").

- **Güvenilir ≠ kullanışlı** (ör. kelime işlemci kullanışlı ama güvenilir değil, yedek alarak zararı sınırla).
- ⭐⭐ **4 ana boyut (Şekil 3.3):**
  - Availability: **istendiğinde** hizmet verebilme. Herhangi bir anda çalışır olma olasılığı.
  - Reliability: hizmeti **tanımlandığı gibi** verebilme. Belirli süre boyunca doğru çalışma olasılığı.
  - Safety: **felaket arızası olmadan** çalışabilme. İnsana ya da çevreye zarar vermeme.
  - Security: kazara ya da **kasıtlı saldırılara** karşı kendini koruyabilme.
- ⭐ Alt özellikler: **Security** = integrity + confidentiality. **Reliability** = correctness + precision + timeliness.
- Boyutlar **birbirine bağlı** (safety, availability ve reliability'ye bağlı. Saldırgan veriyi bozarsa reliability düşer. **DoS** availability'yi bozar. Virüs varsa safety varsayılamaz).
- ⭐ **Diğer dependability özellikleri:**
  - Repairability: hızlı onarım (teşhis, erişim, değişiklik). Kaynak kod erişimi artırır, **black-box** bileşenler azaltır.
  - Maintainability: yeni gereksinime **ekonomik** uyum, yeni hata riski düşük.
  - Survivability: **saldırı altında** ve kısmen devre dışıyken bile hizmet. 3 strateji: **resistance to attack, attack recognition, recovery**.
  - Error tolerance: kullanıcı hatalarından kaçınma ve tolere etme (algıla, düzelt ya da yeniden giriş iste).
- İnsülin pompası için önemli: **availability, reliability, safety**. Security daha az önemli (ağa bağlı değil, gizli veri yok).
- ⭐ **Performans ve dependability ödünleşimi:** güvenilir yazılım fazladan (**redundant**) kontrol kodu içerir, bu yüzden performans düşer, bellek ve maliyet artar. Geçerleme pahalı (dış **düzenleyiciye** kanıt, ör. FAA).
- ⭐⭐ **Şekil 3.4:** maliyet, dependability arttıkça **üstel (exponential)** artar. **%100 dependable kanıtlanamaz** (maliyet sonsuz olur).

Sayfa 51–55 3.3 Availability and reliability · Fault, error, failure

📖 Sayfanın özeti

İKİ KAVRAM, SAYISAL OLASILIKLAR

Availability ve reliability yakın kavramlardır ve ikisi de olasılık olarak ifade edilebilir. Ama biri yüksekse diğerinin de yüksek olacağı varsayılamaz. Örneğin **telefon santralinde** availability daha kritiktir: ahize kaldırılınca çevir sesi gelmelidir. Bağlantı hatası olursa santral kendini hızla sıfırlar ve kullanıcı fark etmez bile. Availability ayrıca **onarım süresine** de bağlıdır. Yılda bir arızalanıp 3 günde kalkan A sistemi, ayda bir arızalanıp 10 dakikada kalkan B sisteminden daha güvenilirdir, ama B'nin availability'si çok daha iyidir (yılda 120 dakikaya karşı 4.320 dakika kesinti).

KESİN TANIMLAR

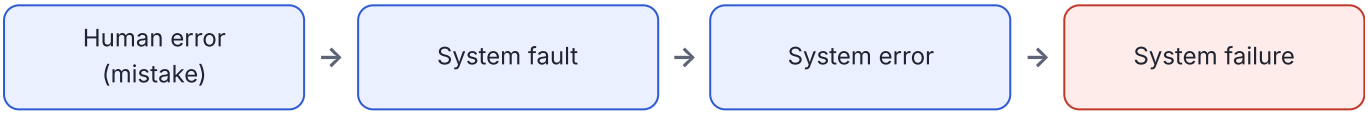
Reliability: belirli bir ortamda, belirli bir amaç için, belirli bir süre boyunca arızasız çalışma olasılığı.
Availability: sistemin belirli bir anda çalışır durumda olup istenen hizmeti verebilme olasılığı.

ORTAM VE ALGI FARKI

Güvenilirlik **ortama** bağlıdır: ofiste ölçülen kelime işlemci güvenilirliği, sistemi zorlayan öğrencilerin olduğu üniversitede farklı çıkar. **İnsan algısı** da önemlidir: yağmurda arıza yapan silecek, yağışlı Seattle'daki sürücüye güvenilmez gelir, kurak Las Vegas'taki sürücü fark etmeyebilir. Tanımlar arızanın **ciddiyetini** de hesaba katmaz: çalıştırınca bir kez stop edip sonra düzgün çalışan motor dert edilmez, ama yüksek hızda ayda bir stop eden motor hem güvenilmez hem emniyetsiz sayılır. Kesin tanıma göre sistem, tanımlamaya uyduğu sürece güvenilirdir. Ama tanımlama kullanıcı beklentisini karşılamıyorsa kullanıcılar onu güvenilmez bulur. Alan uzmanı olmayan mühendisler eksik bir tanımlamayı yanlış yorumlayabilir.

FAULT, ERROR, FAİLURE (ŞEKİL 3.5)

Güvenilirliği ve hazır bulunmayı **sistem arızaları** bozar. Bunların bir kısmı tanımlama hatalarından ya da ilişkili sistemlerdeki arızalardan, çoğu ise sistemdeki kusurlardan kaynaklanır. Dört terim ayrılır: **human error / mistake** (sisteme kusur sokan insan davranışı), **system fault** (hataya yol açabilecek yazılım özelliği, ör. değişkene ilk değer verilmemesi), **system error** (kullanıcının beklemediği davranışa yol açabilecek hatalı sistem durumu) ve **system failure** (sistemin beklenen hizmeti vermediği an). Bu bir zincirdir ama her halka bir sonrakini **kaçınılmaz olarak doğurmaz**. Kusur hiç kullanılmayan kodda olabilir, hatalı durum geçici olup kendiliğinden düzelebilir ya da sistemdeki korumalar hatayı hizmet etkilenmeden yakalayabilir.



Şekil 3.5'teki terimlerin zinciri (her adım bir sonrakine yol açmak zorunda değil)

GÜVENİLİRLİĞİ ARTIRMANIN 3 YOLU

Fault avoidance: hata yapma olasılığını azaltan ya da hataları kusura dönüşmeden yakalayan teknikler (ör. pointer gibi hataya açık yapılardan kaçınmak, statik analiz). **Fault detection and removal:** V&V ile kusurları sistem kullanılmadan önce bulup gidermek (ör. sistematik test ve debugging). **Fault tolerance:** kusurların hataya ya da hataların arızaya dönüşmesini engelleyen teknikler (ör. kendi kendini kontrol eden özellikler, yedekli (redundant) modüller).

GİRDİ-ÇIKTI EŞLEMESİ (ŞEKİL 3.6 VE 3.7)

Bir program, girdi kümesini çıktı kümesine eşleyen bir şey olarak düşünülebilir. Bazı girdiler hatalı çıktı üretir. Güvenilirlik, normal kullanımda bir girdinin bu **hatalı girdi kümesine** düşme olasılığıyla ilgilidir. Bu girdiler sık kullanılan kodla ilgiliyse arızalar sık görülür, nadir kullanılan kodla ilgiliyse kullanıcılar neredeyse hiç arıza görmez. Her kullanıcı sistemi farklı kullanır: **Şekil 3.7**'de 2. kullanıcının girdileri hatalı kümeyle kesişir ve arıza yaşar, 1. ve 3. kullanıcılar ise o girdileri hiç kullanmadığı için sistem onlara hep güvenilir görünür.

SAYFA 55: KUSUR SAYISI ≠ GÜVENİLİRLİK

Programın genel güvenilirliği çoğunlukla, kullanıcıların çoğunun **normal kullanımda** karşılaştığı hatalı girdi sayısına bağlıdır. Nadir kullanılan kısımlardaki kusurları gidermek güvenilirliği pek artırmaz. Mills ve arkadaşları bilinen hataların **%60'ını** gidermenin güvenilirliği sadece **%3** artırdığını buldu. Adams'ın IBM çalışmasında birçok hatanın ancak yüzlerce ya da binlerce aylık kullanımdan sonra arızaya yol açtığı görüldü. Kullanıcılar bilinen hataların etrafından dolanmayı da öğrenir (yazar örneğin kelime işlemcide otomatik numaralandırmayı kullanmıyor). Bu kusurları gidermek o kullanıcılar için fark yaratmaz.

👩‍🏫 Hoca anlatımı

Availability ve reliability farkı sınavın en sevilen sorularından. A/B örneğini mutlaka anla:

A sistemi: yılda 1 kez çöker ama 3 gün kapalı kalır. **B sistemi:** ayda 1 kez çöker ama 10 dakikada açılır. Hangisi daha **reliable**? **A**, çünkü daha az çöküyor. Hangisi daha **available**? **B**, çünkü yılda toplam 120 dakika kapalı, A ise 4.320 dakika. Yani availability sadece "ne sıklıkla çöktüğüne" değil, "ne kadar çabuk ayağa kalktığına" da bağlıdır.

Fault, error, failure zincirini yazım hatası örneğiyle düşün: Programcı dalgınlıkla bir değişkene başlangıç değeri vermeyi unuttu (**human error**). Kodda artık bir kusur var (**fault**). Program o satıra gelince değişken saçma bir değer alır (**error**, yanlış iç durum). Bu değer ekrana yanlış sonuç olarak çıkar (**failure**, kullanıcının gördüğü arıza). Ama zincir her zaman tamamlanmaz: o satır hiç çalışmayabilir ya da program yanlış değeri yakalayıp düzeltebilir.

3 yöntemi şöyle hatırla: **Avoidance**, hatayı hiç yapma. **Detection & removal**, yaptığın hatayı test ederek bul ve düzelt. **Tolerance**, hata kalsa bile sistemin çökmesini engelle (yedek parça gibi).

Son mesaj şaşırtıcı: **hataların %60'ını düzeltmek güvenilirliği sadece %3 artırdı!** Çünkü düzeltilen hataların çoğu kimsenin kullanmadığı yerlerdeydi. Önemli olan, insanların **sık kullandığı** yerlerdeki hatalardır.

📝 Deftere not

- Availability ve reliability yakın ama **farklı**. İkisi de **sayısal olasılık** olarak ifade edilebilir. Biri yüksekse diğeri yüksek olmak zorunda değil.
- ⭐ **Telefon santrali: availability** daha kritik (çevir sesi gelmeli). Hata olursa hızla sıfırlar, kullanıcı fark etmez.
- ⭐⭐ **A/B örneği:** A yılda 1 arıza, 3 gün kapalı. B ayda 1 arıza, 10 dk kapalı. A daha **reliable**, B daha **available** (yılda 120 dk ve 4.320 dk kesinti). Availability **onarım süresine** de bağlı.
- ⭐⭐ **Tanımlar:**
  - **Reliability:** belirli **ortamda**, belirli **amaç** için, belirli **süre** boyunca **arızasız çalışma** olasılığı.
  - **Availability:** belirli bir **anda** çalışır olup istenen hizmeti verebilme olasılığı.
- Güvenilirlik **ortama** bağlı (ofis ve üniversite). **İnsan algısı** önemli (silecek: Seattle ve Las Vegas). Tanımlar arızanın **ciddiyetini** hesaba katmaz (açılışta stop eden motor ve yüksek hızda stop eden motor).
- Kesin tanım: tanımlamaya uyuyorsa güvenilir. Ama spec beklentiye uymazsa kullanıcı güvenilmez bulur. Spec çoğu zaman **eksik ya da yanlış**.
- ⭐⭐ **Şekil 3.5 terimleri:**
  - Human error / mistake: sisteme kusur sokan insan davranışı.
  - System fault: hataya yol açabilecek yazılım özelliği (ör. **değişkene ilk değer verilmemesi**).
  - System error: beklenmeyen davranışa yol açabilecek **hatalı sistem durumu**.
  - System failure: sistemin beklenen hizmeti **vermediği olay ya da an**.

  Zincir: mistake → fault → error → failure. Ama **kaçınılmaz değil** (kod hiç kullanılmayabilir, durum geçici olabilir, koruma yakalayabilir).
- ⭐⭐ **Güvenilirliği artırmanın 3 yolu:**

- Fault avoidance: hata yapma olasılığını azalt. Ör: **pointer** gibi hataya açık yapılardan kaçın, **statik analiz**.
- Fault detection and removal: V&V ile kullanımdan önce bul ve gider. Ör: **sistematik test, debugging**.
- Fault tolerance: fault → error ya da error → failure dönüşümünü engelle. Ör: **self-checking**, **redundant** (yedekli) modüller.

- ⭐ **Şekil 3.6:** program = girdi kümesi → çıktı kümesi eşlemesi. Bazı girdiler (I_e) hatalı çıktı (O_e) üretir. Güvenilirlik = girdinin hatalı kümeye düşme olasılığı. Sık kullanılan kodda ise arıza sık, nadir kodda ise neredeyse hiç.
- ⭐ **Şekil 3.7:** her kullanıcı farklı kullanır. User 2 hatalı girdilerle kesişir ve arıza yaşar. User 1 ve 3 için sistem hep güvenilir.
- ⭐ Güvenilirlik, **normal kullanımdaki** hatalı girdilere bağlı. **Mills:** bilinen hataların **%60'ını** gidermek, güvenilirliği sadece **%3** artırdı. **Adams (IBM):** birçok hata yüzlerce ya da binlerce ay kullanımdan sonra arızaya yol açar.
- Kullanıcılar bilinen hataların etrafından **dolanır (work around)**. Bu hataları düzeltmek pratikte fark yaratmaz.

Sayfa 55–58 3.4 Safety (Emniyet) · Şekil 3.8 emniyet terimleri

📖 Sayfanın özeti

SAFETY-CRİTİCAL SİSTEMLER

Arızalansalar bile insanlara ya da çevreye **asla** zarar vermemesi gereken sistemlerdir: uçak kontrol sistemleri, kimya ve ilaç fabrikası süreç kontrolü, otomobil kontrol sistemleri. Donanımla kontrol daha basittir, ama çok sayıda sensör ve karmaşık kontrol kuralları gerektiği için artık yazılım şarttır. Ör: aerodinamik olarak kararsız askeri uçaklar, düşmemek için sürekli yazılım ayarına ihtiyaç duyar.

İKİ SINIF

Primary safety-critical software: sistemin içine kontrolcü olarak gömülü yazılım. Arızası donanım arızasına, o da yaralanmaya ya da çevre hasarına yol açar (kitap bununla ilgileniyor). **Secondary safety-critical software:** dolaylı olarak yaralanmaya yol açabilen yazılım. Ör: tasarlanan nesnede kusura yol açabilecek bilgisayar destekli mühendislik tasarım sistemleri ya da hastaya yanlış ilaç dozu verilmesine neden olabilecek tıbbi ilaç veritabanı.

GÜVENİLİR ≠ EMNİYETLİ

Reliability ve safety ilişkili ama **ayrı** özelliklerdir. Hata toleranslı bir sistem bile hâlâ kazaya yol açabilir. Hatasız olduğundan asla %100 emin olamayız. Ayrıca güvenilir bir sistemin emniyetli olmaması için üç neden vardır: (1) tanımlama kritik durumlarda eksik olabilir (Lutz'a göre emniyetle ilgili hataların temel kaynağı **gereksinim sorunlarıdır**), (2) arızalanmak üzere olan donanım yazılımın beklemediği sinyaller üretebilir, (3) operatör tek başına yanlış olmayan ama belirli durumda tehlikeli bir girdi verebilir (uçak yerdeyken teknisyenin iniş takımını kaldırma komutu vermesi ve yazılımın bunu uygulaması).

ŞEKİL 3.8: EMNİYET TERİMLERİ (LEVESON'DAN)

Terim	Kısaca anlamı ve örneği

Accident (mishap)	Ölüm, yaralanma, mal ya da çevre hasarıyla sonuçlanan **planlanmamış olay**. Ör: bilgisayar kontrollü makinenin operatörünü yaralaması.
Hazard	Kazaya yol açma **potansiyeli olan durum**. Ör: makinenin önündeki engeli algılayan sensörün bozulması.
Damage	Kazadan doğan **kaybın ölçüsü** (çok sayıda ölümden küçük yaralanmaya kadar).
Hazard severity	Bir tehlikenin yol açabileceği **en kötü zararın** değerlendirmesi (felaketten küçük hasara).
Hazard probability	Tehlikeyi yaratan olayların **gerçekleşme olasılığı** (olası, ör. 1/100, ile akla yatkın olmayan arası).
Risk	Sistemin kazaya yol açma olasılığının ölçüsü. **Hazard probability + hazard severity + tehlikenin kazaya dönüşme olasılığı** birlikte değerlendirilir.

EMNİYETİ SAĞLAMANIN 3 YOLU

Amaç kazaların olmaması ya da sonuçlarının küçük kalmasıdır. **Hazard avoidance:** tehlike hiç oluşmayacak şekilde tasarla (ör. kesme makinesinin çalışması için iki ayrı düğmeye aynı anda basılması, böylece eller bıçaktan uzak kalır). **Hazard detection and removal:** tehlikeyi kazaya dönüşmeden algıla ve gider (ör. kimya fabrikasında aşırı basıncı algılayıp tahliye vanasını açmak). **Damage limitation:** kazanın zararını azaltan korumalar (ör. uçak motorlarındaki otomatik yangın söndürücüler).

KAZALAR VE YAZILIMIN İKİ YÖNÜ

Kazalar genelde birkaç şey **aynı anda** ters gidince olur. Perrow'a göre ciddi kazaların neredeyse hepsi tek bir arızadan değil, **arızaların birleşiminden** kaynaklanır. Bütün kombinasyonları öngörmek imkânsız olduğu için karmaşık sistemlerde kazalar kaçınılmazdır. Yazılım karmaşıklığı artırdığı için kaza olasılığını artırabilir. Ama yazılım kontrolü emniyeti de artırabilir: daha geniş koşulları izleyebilir, kolay uyarlanabilir, çok güvenilir ve küçük donanım kullanır, gelişmiş emniyet kilitleri sağlar ve insanların tehlikeli ortamda geçirdiği süreyi azaltır.

SAYFA 58: ORANTI DUYGUSU

Bir sistemi %100 emniyetli yapmak imkânsızdır. Toplum, ara sıra olan bir kazanın sonuçlarının ileri teknolojinin faydalarına değip değmediğine karar vermelidir. Sınırlı ulusal kaynakların riski azaltmak için nasıl kullanılacağı da sosyal ve politik bir karardır.

👨‍🏫 Hoca anlatımı

Primary ve secondary farkı basit: **Primary**, yazılım doğrudan makineyi kontrol ediyor (ör. arabanın fren sistemi yazılımı). Hata yaparsa makine doğrudan zarar verir. **Secondary**, yazılım doğrudan bir şeyi kontrol etmiyor ama yanlış bilgi verip dolaylı zarara yol açıyor (ör. ilaç dozlarını tutan veritabanı yanlış doz gösterir, hemşire de o dozu verir).

"Güvenilir = emniyetli" değildir! Bu sınavda çok sorulur (alıştırma 3.7). İniş takımı örneğini hatırla: yazılım teknisyenin komutunu **kusursuz** uyguladı, yani güvenilir çalıştı. Ama uçak yerdeydi ve sonuç felaket oldu, yani emniyetli değildi. Yazılım "uçak yerdeyse bu komutu reddet" diye tasarlanmalıydı.

Emniyet terimlerini bir mutfak örneğiyle düşün: Ocakta yağ kızgın ve başında kimse yok: bu bir **hazard** (tehlikeli durum). Yağ alev alırsa: **accident**. Mutfağın yanması: **damage**. "En kötü ne olur?",

ev yanar: **hazard severity**. "Bunun olma ihtimali ne?": **hazard probability**. Hepsini birleştirince: **risk**.

3 emniyet yöntemi Bölüm 3.3'teki güvenilirlik yöntemlerine çok benzer: önle (**avoidance**), algıla ve gider (**detection & removal**), zararı sınırla (**damage limitation**). Örnekleri ezberle: iki düğmeli kesme makinesi, basınç tahliye vanası, uçak motorundaki yangın söndürücü.

Perrow'un fikri de önemli: büyük kazalar tek bir hatadan değil, **birden çok küçük hatanın aynı anda** olmasından çıkar.

📝 Deftere not

- ⭐ **Safety-critical:** arızalansa bile insanlara ya da çevreye **asla** zarar vermemeli. Ör: uçak kontrolü, kimya ve ilaç fabrikası süreç kontrolü, otomobil kontrolü.
- Donanım kontrolü daha basit ama artık **yazılım şart** (çok sensör, karmaşık kurallar). Ör: aerodinamik olarak kararsız askeri uçaklar.
- ⭐⭐ **2 sınıf:**
  - Primary safety-critical software: **gömülü kontrolcü**. Arıza → donanım arızası → yaralanma ya da çevre hasarı.
  - Secondary safety-critical software: **dolaylı** zarar. Ör: CAE tasarım sistemleri (tasarım kusuru), **tıbbi ilaç veritabanı** (yanlış doz).
- ⭐⭐ **Reliability ≠ safety.** Fault-tolerant sistem bile emniyetsiz olabilir. **Güvenilir ama emniyetsiz olmanın 3 nedeni:**
  - **1. Eksik spec:** kritik durumlardaki davranış tanımlanmamış (Lutz: emniyet hatalarının ana kaynağı **gereksinimler**).
  - **2. Donanım arızası:** beklenmedik sinyaller, yazılım öngörülmemiş ortamla karşılaşır.
  - **3. Operatör girdisi:** tek başına doğru ama durum için yanlış. Ör: **uçak yerdeyken iniş takımını kaldırma** komutu.
- ⭐⭐ **Şekil 3.8, emniyet terimleri (Leveson):**
  - **Accident (mishap):** ölüm, yaralanma, mal ya da çevre hasarı yaratan **planlanmamış olay**.
  - **Hazard:** kazaya yol açma **potansiyeli olan durum** (ör. engel sensörünün bozulması).
  - **Damage:** kazadan doğan kaybın ölçüsü.
  - **Hazard severity:** tehlikenin yol açabileceği **en kötü** zarar.
  - **Hazard probability:** tehlikeyi yaratan olayların gerçekleşme olasılığı.
  - **Risk:** sistemin kazaya yol açma olasılığı = hazard probability + severity + hazard'ın kazaya dönüşme olasılığı.
- ⭐⭐ **Emniyeti sağlamanın 3 yolu:**
  - Hazard avoidance: tehlike oluşmasın. Ör: **iki düğmeli kesme makinesi**.
  - Hazard detection and removal: kazadan önce algıla ve gider. Ör: **aşırı basınçta tahliye vanası**.
  - Damage limitation: zararı sınırla. Ör: **uçak motorunda otomatik yangın söndürücü**.
- ⭐ **Perrow:** ciddi kazalar tek arızadan değil, **arızaların birleşiminden**. Tüm kombinasyonlar öngörülemez, karmaşık sistemlerde kazalar **kaçınılmaz**.

- Yazılımın iki yönü: (–) karmaşıklık, kaza olasılığı artabilir. (+) geniş izleme, kolay uyarlama, güvenilir küçük donanım, **safety interlocks**, insanın tehlikeli ortamda geçirdiği süre azalır.
- **%100 emniyet imkânsız.** Fayda ve risk dengesi **sosyal ve politik** bir karar.

Sayfa 58–61 3.5 Security (Güvenlik) · Şekil 3.9 · Key Points

📖 Sayfanın özeti

GÜVENLİK VE İNTERNET

Güvenlik, sistemin kazara ya da kasıtlı **dış saldırılara** karşı kendini koruyabilme yeteneğidir. Sistemler internete bağlandıkça önemi arttı. İnternet yeni işlevler getirir (ör. banka hesabına doğrudan erişim), ama kötü niyetli kişilerin saldırmasına da kapı açar. Zafiyet bilgisi hızla yayılır, ama yamalar da hızla dağıtılabilir. Saldırılara örnek: virüsler, hizmetlerin izinsiz kullanımı, sistemin ya da verinin izinsiz değiştirilmesi.

GÜVENLİK NEDEN DİĞERLERİNİN TEMELİ?

Availability, reliability ve safety'yi güvence altına alan bütün yöntemler, çalışan sistemin **ilk kurulan sistemle aynı olduğunu** varsayar. Sistem bozulmuşsa (ör. yazılıma virüs eklenmişse) güvenilirlik ve emniyet iddiaları geçersizleşir. Tersine, geliştirme hataları da **güvenlik açıklarına** yol açar: beklenmedik girdilere tepki vermeyen ya da dizi sınırlarını kontrol etmeyen sistemler saldırıya açıktır. **Internet worm** (1989) ve 10 yıldan fazla sonra çıkan **Code Red** solucanı, C programlarının dizi sınırı kontrolü yapmamasından yararlanıp belleğin bir kısmının üzerine izinsiz erişim sağlayan kod yazdı.

ŞEKİL 3.9: GÜVENLİK TERİMLERİ (PFLEEGER'DEN)

Terim	Kısaca anlamı
Exposure	Sistemde olası **kayıp ya da zarar** (veri kaybı ya da hasarı, güvenlik ihlali sonrası kurtarma için zaman ve emek kaybı).
Vulnerability	Sistemde kayıp ya da zarar vermek için **istismar edilebilecek zayıflık**.
Attack	Bir zafiyetin **istismar edilmesi**. Genelde dışarıdan, kasıtlı zarar verme girişimi.
Threat	Kayıp ya da zarar verme **potansiyeli olan durum**. Saldırıya maruz kalan bir zafiyet gibi düşünülebilir.
Control	Zafiyeti azaltan **koruyucu önlem**. Ör: zayıf erişim kontrolü olan sistemde **şifreleme**.

GÜVENLİĞİN EN ÖNEMLİ OLDUĞU SİSTEMLER

Askeri sistemler, e-ticaret ve gizli bilgi işleyen sistemlerde güvenlik en önemli boyuttur. Ör: havayolu rezervasyon sistemi çalışmazsa sadece gecikme olur, ama güvensizse **sahte rezervasyonlar** yüzünden şirket çok para kaybedebilir.

DIŞ SALDIRININ 3 TÜR ZARARI

Denial of service: sistem normal hizmet veremez hale getirilir, bu availability'yi etkiler. **Corruption of programs or data:** bileşenler izinsiz değiştirilir, bu reliability ve safety'yi (ağırsa availability'yi de) etkiler. **Disclosure of confidential information:** gizli bilgi yetkisiz kişilere açılır, bu safety'yi etkileyebilir ve sonraki saldırılara zemin hazırlar.

EMNİYET BENZETMESİ VE 3 YÖNTEM

Emniyet terimleriyle açık bir benzerlik vardır: **exposure ≈ accident**, **vulnerability ≈ hazard**. Bu yüzden benzer üç yaklaşım vardır: **Vulnerability avoidance** (zafiyet hiç oluşmasın, ör. sistemi dış ağa bağlamamak), **Attack detection and neutralisation** (zafiyetleri istismar edilmeden bul ve gider, ör. gelen dosyaları tarayıp virüsü temizleyen antivirüs) ve **Exposure limitation** (başarılı saldırının sonuçlarını küçült, ör. düzenli yedekleme ve bozulan yazılımın yeniden oluşturulmasını sağlayan konfigürasyon yönetimi).

İNSAN FAKTÖRÜ

Zafiyetlerin çoğu teknik değil **insan** hatalarından kaynaklanır: tahmin edilebilir şifreler, şifrelerin yazılıp görünür yerlerde bırakılması, sistem yöneticilerinin erişim kontrolü ya da yapılandırma dosyalarını ayarlarken yaptığı hatalar. Bu yüzden sadece teknik özelliklere değil, sistemin **gerçekte nasıl kullanıldığına** da bakmak gerekir (detaylar Bölüm 30'da).

KEY POİNTS (BÖLÜM ÖZETİ)

Kritik sistemde arıza büyük ekonomik kayıp, fiziksel hasar ya da can tehlikesi yaratır. Üç sınıfı vardır: safety-, mission- ve business-critical. Güvenilebilirlik, kullanıcının sisteme duyduğu güvenin derecesidir ve en önemli boyutları availability, reliability, safety ve security'dir. Availability istendiğinde hizmet verebilme, reliability hizmetin tanımlandığı gibi verilme olasılığıdır. Bu ikisi genelde en önemli boyutlardır, çünkü sistem güvenilmezse emniyet ve güvenlik de sağlanamaz. Güvenilirlik, kullanımda hata olma olasılığıyla ilgilidir. Bilinen kusurları olan program bile kullanıcılarına güvenilir görünebilir. Safety, sistemin normal ya da anormal çalışırken insanları ya da çevreyi tehdit etmemesidir. Security bütün kritik sistemler için önemlidir, çünkü güvenlik yoksa diğer üç boyut da tehlikeye girer. Güvenilebilirliği artırmak için insanları, donanımı ve yazılımı birlikte ele alan **sosyo-teknik** bir yaklaşım gerekir.

👨‍🏫 Hoca anlatımı

Güvenlik terimlerini bir ev hırsızlığı örneğiyle düşün: Arka kapının kilidi bozuk: bu bir **vulnerability** (zafiyet). Mahallede hırsızlar dolaşıyor ve bozuk kilidi fark edebilirler: bu bir **threat** (tehdit). Hırsız bozuk kapıdan içeri giriyor: bu bir **attack** (saldırı). Evden çalınanlar ve kırılan eşyalar: **exposure** (kayıp, zarar). Kapıya yeni ve sağlam kilit takmak: **control** (önlem).

Attack ve threat farkı sınavda soruldu (alıştırma 3.10): **threat** = zarar **verebilecek** durum (potansiyel). **Attack** = zafiyetin **gerçekten istismar edilmesi** (eylem).

Emniyet ve güvenlik benzetmesi çok şık: **vulnerability ↔ hazard** (ikisi de "kötü bir şeyin olabileceği durum"), **exposure ↔ accident** (ikisi de "kötü şeyin gerçekleşip zarar vermesi"). Yöntemler de paralel: önle, algıla ve gider, zararı sınırla.

Neden güvenlik diğerlerinin temeli? Çünkü sisteme virüs girerse, artık kurduğun sistem değil, değiştirilmiş bir sistem çalışıyor. "Bu sistem güvenilir ve emniyetli" diye yaptığın bütün testler boşa gider.

Availability ve security ilişkisi (alıştırma 3.9): DoS saldırısı sistemi çalışamaz hale getirir. Yani güvenlik zayıfsa availability de düşer.

Son olarak: **güvenlik açıklarının çoğu insandan gelir**, "123456" şifresi ya da monitöre yapıştırılmış şifre kâğıdı gibi.

📝 Deftere not

- ⭐ **Security:** sistemin **kazara ya da kasıtlı dış saldırılara** karşı kendini koruyabilme yeteneği. İnternetle önemi arttı (yeni işlevler ama saldırı riski. Zafiyet bilgisi hızlı yayılır, yamalar da).
- Saldırı örnekleri: **virüsler**, hizmetlerin izinsiz kullanımı, sistemin ya da verinin izinsiz değiştirilmesi.
- ⭐ Güvenlik yoksa **availability, reliability ve safety tehlikeye girer**, çünkü bu yöntemler sistemin **kurulduğu gibi kaldığını** varsayar (virüs varsa iddialar geçersiz).
- Geliştirme hataları güvenlik açığı yaratır: beklenmedik girdi kontrolü yok, **dizi sınırı kontrolü yok**. Ör: **Internet worm (1989), Code Red** (C'nin dizi sınırı kontrolü yapmamasından yararlandı).
- ⭐⭐ **Şekil 3.9, güvenlik terimleri (Pfleeger):**
  - **Exposure:** olası kayıp ya da zarar (veri, kurtarma emeği).
  - **Vulnerability:** istismar edilebilecek **zayıflık**.
  - **Attack:** zafiyetin **istismarı**, genelde dışarıdan ve kasıtlı.
  - **Threat:** zarar verme **potansiyeli olan durum** (saldırıya maruz zafiyet).
  - **Control:** zafiyeti azaltan önlem. Ör: **şifreleme (encryption)**.
- Güvenliğin **en önemli** olduğu sistemler: askeri, e-ticaret, gizli bilgi. Ör: havayolu rezervasyonu güvensizse **sahte rezervasyon** ve büyük para kaybı.
- ⭐⭐ **Dış saldırının 3 tür zararı:**
  - Denial of service: normal hizmet verilemez, **availability** etkilenir.
  - Corruption of programs or data: izinsiz değişiklik, **reliability ve safety** (ağırsa availability) etkilenir.
  - Disclosure of confidential information: gizli bilgi açığa çıkar, safety etkilenebilir, sonraki saldırılara zemin hazırlar.
- ⭐ **Benzetme:** exposure ≈ accident, vulnerability ≈ hazard.
- ⭐⭐ **Güvenliği sağlamanın 3 yolu:**
  - Vulnerability avoidance: zafiyet oluşmasın. Ör: **dış ağa bağlanmamak**.
  - Attack detection and neutralisation: zafiyetleri bul ve gider. Ör: **virüs tarayıcı**.
  - Exposure limitation: başarılı saldırının sonucunu küçült. Ör: **düzenli yedekleme**, konfigürasyon yönetimi.
- ⭐ Zafiyetlerin çoğu **insan hatası**: tahmin edilebilir şifre, yazılı şifre, yöneticinin erişim ve yapılandırma hataları. Sistemin **gerçek kullanımını** da düşün.
- ⭐ **KEY POINTS:** kritik sistem = ekonomik kayıp, fiziksel hasar, can tehlikesi · 3 sınıf (safety, mission, business) · dependability = **güven derecesi**, 4 boyut · availability ve reliability **en önemli** (yoksa safety ve security sağlanamaz) · bilinen kusurlu program bile güvenilir görünebilir · safety = normal ya da anormal çalışırken zarar vermeme · security her kritik sistem için önemli · **sosyo-teknik yaklaşım** şart.

Sayfa 61–62 Further Reading · Exercises (Alıştırmalar)

📖 Sayfanın özeti

> **FURTHER READİNG**
> Bilgi güvencesi üzerine bir makale, safety-critical sistem tasarımı üzerine bir kitap, Schneier'in *Secrets and Lies* güvenlik kitabı, survivability üzerine bir giriş makalesi ve otomatik sistemlerdeki olayları derleyen *Computer-related Risks* kitabı. *Sınav için önemli değil.*

> **ALIŞTIRMALAR (KONULARI)**

#	Konu
3.1	Kritik sistemin **3 ana türü** nedir? Farklarını açıklayın.
3.2	Kritik sistemlerde güvenilebilirliğin neden önemli olduğuna dair 6 neden önerin.
3.3	Güvenilebilirliğin **en önemli boyutları** nelerdir?
3.4	Güvenilebilirliği sağlamanın maliyeti neden **üsteldir**?
3.5	Şu sistemler için hangi güvenilebilirlik özellikleri en kritik? ISS internet sunucusu, kapalı ameliyatta bilgisayar kontrollü neşter, uydu fırlatma aracının yön kontrolü, internet tabanlı kişisel finans sistemi.
3.6	Safety-critical yazılım içeren (ya da içerebilecek) 6 tüketici ürünü belirleyin.
3.7	Reliability ve safety farkı nedir? Güvenilir bir sistem neden emniyetsiz olabilir (ve tersi)?
3.8	Tümör tedavisi için radyasyon veren tıbbi sistemde bir **hazard** ve bunu önleyecek bir yazılım özelliği önerin.
3.9	Availability ile security arasında neden yakın ilişki var?
3.10	Güvenlik terimleriyle **attack** ve **threat** farkı nedir?
3.11	Bilinen hataları olan yazılımı teslim etmek etik mi? (tartışma)
3.12	İnsan hakları örgütü için bir şirketin sistemine izinsiz girme talebinin etik ikilemi (tartışma).

👨‍🏫 Hoca anlatımı

Bu bölümün alıştırmaları sınav için **altın değerinde**, çünkü çoğu kısa tanım ve fark sorusu. 3.1, 3.3, 3.4, 3.5, 3.7, 3.8, 3.9 ve 3.10'un cevap kalıplarını bölüm sonuna koydum.

3.5 gibi "hangi özellik kritik?" sorularında mantık şu: **Neşter** insana dokunuyor, o yüzden **safety**. **İnternet sunucusu** binlerce müşteri kullanıyor, o yüzden **availability**. **Fırlatma aracı** tek seferlik görev, o yüzden **reliability**. **Finans sistemi** para ve kişisel veri tutuyor, o yüzden **security**.

📝 Deftere not

- ⭐ Sınavda en olası alıştırmalar: **3.1** (3 tür), **3.3** (4 boyut), **3.4** (üstel maliyet), **3.5** (hangi özellik kritik), **3.7** (reliability ve safety), **3.8** (hazard örneği), **3.9** (availability ve security), **3.10** (attack ve threat).

- ⭐ **3.5 için kısa cevaplar:** ISS sunucusu → **availability** · neşter → **safety** (+ reliability) · fırlatma aracı → **reliability** (+ safety) · finans sistemi → **security** (+ reliability).

BÖLÜM 3 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 3'teki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
critical system	kritik sistem	Arızası ciddi kayıp ya da can tehlikesi yaratan sistem	44
safety-critical system	emniyet-kritik sistem	Arızası yaralanma, ölüm ya da çevre hasarı yaratan (kimya fabrikası)	44
mission-critical system	görev-kritik sistem	Arızası bir görevin başarısızlığına yol açan (uzay aracı navigasyonu)	44
business-critical system	iş-kritik sistem	Arızası işletmeye çok yüksek maliyet çıkaran (banka hesap sistemi)	44
dependability	güvenilebilirlik	Kullanıcının sisteme duyduğu güvenin derecesi (4 boyutlu şemsiye kavram)	44, 48
trustworthiness	güvenilir olma	Sistemin beklendiği gibi çalışacağına dair güven	48
formal methods	formal (matematiksel) yöntemler	Matematiksel geliştirme yöntemleri, test miktarını azaltır	45
verification & validation (V&V)	doğrulama ve geçerleme	Sistemin doğru yapıldığını ve doğru şeyin yapıldığını kontrol	45
holistic	bütüncül	Sistemin tamamına bakan	45
insulin pump	insülin pompası	Kitabın safety-critical örnek sistemi	46
pancreas / diabetes	pankreas / diyabet	İnsülin üreten organ / şeker hastalığı	46
controller	kontrolcü, denetleyici	Hesaplayıp komut veren bileşen	46–47
overdose	aşırı doz	İnsülin pompasında asla olmaması gereken	47
availability	hazır bulunma, erişilebilirlik	İstendiğinde hizmet verebilme olasılığı	48, 51

reliability	güvenilirlik	Belirli sürede arızasız ve tanımlandığı gibi çalışma olasılığı	48, 51
safety	emniyet	İnsana ya da çevreye zarar vermeden çalışabilme	48, 55
security	güvenlik	Kazara ya da kasıtlı saldırılara karşı korunabilme	48, 58
integrity	bütünlük	Program ve verinin zarar görmemesi	48–49
confidentiality	gizlilik	Bilgiye sadece yetkililerin erişmesi	49
correctness / precision / timeliness	doğruluk / kesinlik / zamanlılık	Reliability'nin alt özellikleri	49
denial-of-service (DoS)	hizmet engelleme	Sistemi hizmet veremez hale getiren saldırı	49, 59
repairability	onarılabilirlik	Arızadan sonra hızlı onarılabilme	49
black-box component	kara kutu bileşen	İçi görülemeyen, kaynak kodu olmayan bileşen	49
maintainability	bakım kolaylığı	Yeni gereksinimlere ekonomik uyarlanabilme	49
survivability	hayatta kalabilirlik	Saldırı altında bile hizmet vermeye devam edebilme	49
error tolerance	hata toleransı	Kullanıcı hatalarını önleme ve tolere etme	49
redundant	yedekli, fazladan	Hata durumunda devreye giren ekstra kod ya da modül	50, 53
trade-off	ödünleşim	Performans ve güvenilebilirlik dengesi	50
regulator	düzenleyici kurum	Sistemin güvenilir olduğunu kanıtlamak gereken kurum (ör. FAA)	50
exponential	üstel	Maliyetin güvenilebilirlikle hızla artması	50
down time	kesinti süresi	Sistemin çalışmadığı süre	51
human error / mistake	insan hatası	Sisteme kusur sokan insan davranışı	53
system fault	sistem kusuru	Hataya yol açabilecek yazılım özelliği	53

system error	sistem hatası	Beklenmeyen davranışa yol açabilecek hatalı durum	53
system failure	sistem arızası	Beklenen hizmetin verilmediği olay	53
fault avoidance	kusurdan kaçınma	Hata yapmayı önleyen teknikler	53
fault detection and removal	kusur bulma ve giderme	V&V ile kusurları bulup düzeltme	53
fault tolerance	kusur toleransı	Kusurun arızaya dönüşmesini engelleme	53
static analysis	statik analiz	Kodu çalıştırmadan inceleme	53
input/output mapping	girdi-çıktı eşlemesi	Programı girdiden çıktıya dönüşüm olarak görmek	54
erroneous inputs / outputs	hatalı girdiler / çıktılar	Arızaya yol açan girdi ve çıktı kümesi	54
usage pattern	kullanım deseni	Kullanıcının sistemi kullanma biçimi	54
primary safety-critical software	birincil emniyet-kritik yazılım	Gömülü kontrolcü, doğrudan zarar	55
secondary safety-critical software	ikincil emniyet-kritik yazılım	Dolaylı zarar (ilaç veritabanı)	55–56
accident (mishap)	kaza	Zarar veren planlanmamış olay	57
hazard	tehlike	Kazaya yol açma potansiyeli olan durum	57
damage	hasar, zarar	Kazadan doğan kayıp	57
hazard severity	tehlike şiddeti	En kötü olası zarar	57
hazard probability	tehlike olasılığı	Tehlikenin gerçekleşme ihtimali	57
risk	risk	Kazaya yol açma olasılığının ölçüsü	57
hazard avoidance	tehlikeden kaçınma	Tehlike oluşmasın diye tasarlama	56
hazard detection and removal	tehlike algılama ve giderme	Kazadan önce yakalama	57
damage limitation	hasar sınırlama	Kazanın zararını azaltma	57
safety interlock	emniyet kilidi	Tehlikeli işlemi engelleyen mekanizma	58
worm / virus	solucan / virüs	Kötü amaçlı yazılım (Internet worm, Code Red)	58

array bound checking	dizi sınırı kontrolü	C'de olmayan, saldırıya açık bırakan kontrol	58
exposure	maruziyet, kayıp	Olası kayıp ya da zarar	59
vulnerability	zafiyet	İstismar edilebilecek zayıflık	59
attack	saldırı	Zafiyetin istismarı	59
threat	tehdit	Zarar verme potansiyeli olan durum	59
control	önlem, kontrol	Zafiyeti azaltan koruma (ör. şifreleme)	59
encryption	şifreleme	Veriyi okunamaz hale getirme	59
corruption of programs or data	program ya da veri bozulması	İzinsiz değişiklik	59
disclosure of confidential information	gizli bilginin ifşası	Gizli bilginin yetkisiz kişilere açılması	59
vulnerability avoidance	zafiyetten kaçınma	Zafiyet oluşmasını engelleme	60
attack detection and neutralisation	saldırı algılama ve etkisizleştirme	Ör. virüs tarayıcı	60
exposure limitation	kayıp sınırlama	Ör. düzenli yedekleme	61
backup	yedek	Verinin kopyası	61

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 3'ten sınavda en çok çıkabilecekler

1. **3 tür kritik sistem:** safety-critical (kimya fabrikası), mission-critical (uzay aracı), business-critical (banka) (s.44)
2. **Dependability neden en önemli:** kullanıcı reddeder, arıza maliyeti devasa, bilgi kaybı (s.44)
3. **3 arıza kaynağı:** hardware, software, human operators (en büyük neden: insan) (s.45)
4. **İnsülin pompası:** available + reliable olmalı, **asla aşırı doz yok** (s.46–47)
5. **4 boyut (Şekil 3.3):** availability, reliability, safety, security + kısa tanımları (s.48)
6. **Diğer özellikler:** repairability, maintainability, survivability, error tolerance (s.49)
7. **Maliyet üstel**, %100 dependability kanıtlanamaz (Şekil 3.4) (s.50)
8. **A/B örneği:** A daha reliable, B daha available. Availability onarım süresine bağlı. (s.51)
9. **Reliability ve availability tanımları** (s.51)
10. **Fault → error → failure** zinciri ve tanımları (Şekil 3.5) (s.53)
11. **Güvenilirliği artırmanın 3 yolu:** fault avoidance, detection & removal, tolerance (s.53)
12. **%60 hata giderme → %3 güvenilirlik artışı** (Mills) (s.55)
13. **Primary ve secondary** safety-critical yazılım (s.55)
14. **Reliable ≠ safe**, 3 neden (eksik spec, donanım, operatör: iniş takımı) (s.56)
15. **Emniyet terimleri (Şekil 3.8):** accident, hazard, damage, severity, probability, risk (s.57)
16. **Emniyetin 3 yolu:** hazard avoidance, detection & removal, damage limitation (s.56–57)
17. **Güvenlik terimleri (Şekil 3.9):** exposure, vulnerability, attack, threat, control (s.59)
18. **Saldırının 3 zararı:** DoS, corruption, disclosure (s.59)
19. **Güvenliğin 3 yolu:** vulnerability avoidance, attack detection & neutralisation, exposure limitation (s.60–61)

C Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What are the three principal types of critical system? Explain the differences. (Ex. 3.1)
Kritik sistemin 3 ana türü nedir? (s.44)

> *1) Safety-critical systems: failure may cause injury, loss of life or serious environmental damage, for example a chemical plant control system. 2) Mission-critical systems: failure may cause the failure of a goal-directed activity, for example a spacecraft navigation system. 3) Business-critical systems: failure may cause very high costs for the business, for example a bank's customer accounting system.*

1) Emniyet-kritik: arıza yaralanma, ölüm ya da çevre hasarı yaratır (kimya fabrikası). 2) Görev-kritik: arıza bir görevin başarısızlığına yol açar (uzay aracı navigasyonu). 3) İş-kritik: arıza işletmeye çok yüksek maliyet çıkarır (banka hesap sistemi).

Q2. Why is dependability the most important property of critical systems? (Ex. 3.2)
Güvenilebilirlik neden en önemli özelliktir? (s.44)

> *1) Systems that are unreliable, unsafe or insecure are often rejected by their users. 2) The cost of system failure may be enormous, much higher than the cost of the system itself. 3) Untrustworthy systems may cause loss of valuable data.*

1) Güvenilmez, emniyetsiz ya da güvensiz sistemler kullanıcılarca reddedilir. 2) Arızanın maliyeti sistemin kendisinden çok daha yüksek olabilir. 3) Güvenilmez sistemler değerli veri kaybına yol açabilir.

Q3. What are the most important dimensions of system dependability? (Ex. 3.3)
Güvenilebilirliğin en önemli boyutları nelerdir? (s.48)

> *1) Availability: the ability of the system to deliver services when requested. 2) Reliability: the ability of the system to deliver services as specified. 3) Safety: the ability of the system to operate without catastrophic failure. 4) Security: the ability of the system to protect itself against accidental or deliberate intrusion.*

1) Availability: istendiğinde hizmet verebilme. 2) Reliability: hizmeti tanımlandığı gibi verebilme. 3) Safety: felaket arızası olmadan çalışabilme. 4) Security: kazara ya da kasıtlı saldırılara karşı kendini koruyabilme.

Q4. Why is the cost of assuring dependability exponential? (Ex. 3.4)
Güvenilebilirlik maliyeti neden üsteldir? (s.50)

> *Dependable software needs extra, often redundant, code for checking and recovery, and much more testing and validation. The higher the dependability you need, the more you have to spend on testing to show that you have reached that level. Because of this, it is impossible to show that a system is 100% dependable, as the cost would be infinite.*

Güvenilir yazılım kontrol ve kurtarma için fazladan (yedekli) kod ve çok daha fazla test ve geçerleme ister. Ne kadar yüksek güvenilebilirlik istersen o seviyeye ulaştığını göstermek için o kadar çok test harcaman gerekir. Bu yüzden %100 güvenilir olduğu gösterilemez, maliyet sonsuz olurdu.

Q5. What is the difference between availability and reliability? Give an example.
Availability ve reliability farkı nedir? (s.51)

> *Reliability is the probability of failure-free operation over a specified time, in a given environment, for a specific purpose. Availability is the probability that a system, at a point in time, will be operational and able to deliver the requested services. Example: system A fails once a year but needs three days to restart, system B fails once a month but restarts in ten minutes. A is more reliable, but B is more available. A telephone exchange needs high availability more than high reliability.*

Reliability, belirli ortamda, amaçta ve sürede arızasız çalışma olasılığıdır. Availability, belirli bir anda çalışır olup hizmet verebilme olasılığıdır. Örnek: A yılda bir arızalanıp 3 günde açılır, B ayda bir arızalanıp 10 dakikada açılır. A daha güvenilir, B daha erişilebilirdir. Telefon santrali için availability daha önemlidir.

Q6. Explain the terms human error, system fault, system error and system failure.
Bu terimleri açıklayın. (s.53)

> *A human error (mistake) is human behaviour that introduces faults into a system. A system fault is a characteristic of the software that can lead to a system error, for example a variable that is not initialised. A system error is an erroneous system state that can lead to unexpected behaviour. A system failure is an event when the system does not deliver a service as expected by its users. Faults do not always lead to errors, and errors do not always lead to failures.*

İnsan hatası, sisteme kusur sokan insan davranışıdır. Sistem kusuru, hataya yol açabilecek yazılım özelliğidir (ör. ilk değer verilmemiş değişken). Sistem hatası, beklenmeyen davranışa yol açabilecek hatalı durumdur. Sistem arızası, sistemin beklenen hizmeti vermediği olaydır. Kusurlar her zaman hataya, hatalar her zaman arızaya yol açmaz.

Q7. What are the three approaches to improve system reliability?
Güvenilirliği artırmanın 3 yolu nedir? (s.53)

> *1) Fault avoidance: techniques that minimise mistakes, for example avoiding error-prone constructs like pointers and using static analysis. 2) Fault detection and removal: verification and validation to find and remove faults before the system is used, for example systematic testing and debugging. 3) Fault tolerance: techniques so that faults do not cause failures, for example self-checking and redundant modules.*

1) Kusurdan kaçınma: hataları azaltan teknikler (pointer'dan kaçınma, statik analiz). 2) Kusur bulma ve giderme: kullanımdan önce V&V (sistematik test, debugging). 3) Kusur toleransı: kusurun arızaya dönüşmemesi (kendi kendini kontrol, yedekli modüller).

Q8. Why can a reliable system be unsafe? (Ex. 3.7)

Güvenilir bir sistem neden emniyetsiz olabilir? (s.56)

> *Reliability means the system conforms to its specification, but safety means it does not cause harm. A reliable system can be unsafe because: 1) the specification may be incomplete for some critical situations, 2) hardware malfunctions may create unexpected inputs, 3) operators may give inputs that are correct alone but dangerous in some situations. For example, software raised the undercarriage of an aircraft while it was on the ground.*

Güvenilirlik sistemin tanımlamaya uyması, emniyet ise zarar vermemesidir. Güvenilir sistem emniyetsiz olabilir çünkü: 1) spec bazı kritik durumlar için eksik olabilir, 2) donanım arızası beklenmedik girdiler üretebilir, 3) operatör tek başına doğru ama bazı durumlarda tehlikeli girdi verebilir. Ör: uçak yerdeyken yazılımın iniş takımını kaldırması.

Q9. Define hazard, accident and risk. How can safety be achieved?

Hazard, accident ve risk nedir? Emniyet nasıl sağlanır? (s.56–57)

> *A hazard is a condition that can cause an accident. An accident is an unplanned event that causes death, injury or damage. Risk is the probability that the system will cause an accident, based on hazard probability and severity. Safety can be achieved by: 1) hazard avoidance, for example a cutting machine that needs two buttons, 2) hazard detection and removal, for example a pressure relief valve, 3) damage limitation, for example fire extinguishers in an aircraft engine.*

Hazard, kazaya yol açabilecek durumdur. Accident, ölüm, yaralanma ya da hasar yaratan planlanmamış olaydır. Risk, tehlikenin olasılığı ve şiddetine göre sistemin kazaya yol açma olasılığıdır. Emniyet: 1) tehlikeden kaçınma (iki düğmeli kesme makinesi), 2) algılama ve giderme (basınç tahliye vanası), 3) hasar sınırlama (uçak motorunda yangın söndürücü).

Q10. Suggest a hazard in a radiation therapy system and a software feature to prevent an accident. (Ex. 3.8)

Radyasyon tedavi sisteminde bir tehlike ve önleyici yazılım özelliği önerin.

> *Hazard: the system may deliver an overdose of radiation to the patient. Software feature: the software should check every dose against a maximum safe limit and refuse to deliver any dose above it. It should also stop the treatment and raise an alarm.*

Tehlike: sistem hastaya aşırı doz radyasyon verebilir. Yazılım özelliği: yazılım her dozu güvenli üst sınırla karşılaştırmalı, üstündeki dozu vermeyi reddetmeli, tedaviyi durdurup alarm vermeli.

Q11. Why is there a close relationship between availability and security? (Ex. 3.9)

Availability ile security neden yakından ilişkilidir? (s.49, 59)

> *If a system is not secure, attackers can make it unavailable, for example with a denial-of-service attack. Also, if the system or its data is corrupted by an attack, it may have to be taken out of service. So poor security reduces availability.*

Sistem güvenli değilse saldırganlar onu kullanılamaz hale getirebilir (ör. DoS saldırısı). Ayrıca saldırıyla sistem ya da veri bozulursa sistem hizmetten çekilmek zorunda kalabilir. Yani zayıf güvenlik, availability'yi düşürür.

Q12. What is the difference between an attack and a threat? (Ex. 3.10)
Attack ve threat farkı nedir? (s.59)

> *A threat is a circumstance that has the potential to cause loss or harm. It is a vulnerability that may be attacked. An attack is the actual exploitation of a vulnerability, usually a deliberate attempt from outside to cause damage. A threat is potential, an attack is an action.*

Threat, kayıp ya da zarar verme potansiyeli olan durumdur, saldırıya uğrayabilecek bir zafiyettir. Attack ise bir zafiyetin gerçekten istismar edilmesidir, genelde dışarıdan kasıtlı bir girişimdir. Threat potansiyeldir, attack eylemdir.

Q13. What types of damage can an external attack cause?
Dış saldırı ne tür zararlar verebilir? (s.59)

> *1) Denial of service: normal services become unavailable. 2) Corruption of programs or data: unauthorised changes affect reliability and safety. 3) Disclosure of confidential information: information is exposed to unauthorised people.*

1) Hizmet engelleme: normal hizmetler kullanılamaz. 2) Program ya da veri bozulması: izinsiz değişiklik güvenilirliği ve emniyeti etkiler. 3) Gizli bilginin ifşası: bilgi yetkisiz kişilere açılır.

Q14. How can the security of a system be assured?
Sistemin güvenliği nasıl sağlanır? (s.60–61)

> *1) Vulnerability avoidance: design the system so that vulnerabilities do not occur, for example do not connect it to a public network. 2) Attack detection and neutralisation: detect and remove vulnerabilities before they cause an exposure, for example a virus checker. 3) Exposure limitation: minimise the consequences of a successful attack, for example regular backups.*

1) Zafiyetten kaçınma: zafiyet oluşmayacak şekilde tasarla (dış ağa bağlama). 2) Saldırı algılama ve etkisizleştirme: zafiyetleri zarar vermeden bul ve gider (virüs tarayıcı). 3) Kayıp sınırlama: başarılı saldırının sonuçlarını azalt (düzenli yedekleme).

D Mini test

cevaplar en altta

1. A spacecraft navigation system is an example of a ___ system.
   a) safety-critical
   b) mission-critical
   c) business-critical
   d) legacy

2. A bank's customer accounting system is:
   a) safety-critical
   b) mission-critical
   c) business-critical
   d) not critical

3. Today, probably the largest single cause of system failures is:
   a) hardware
   b) software
   c) human operators
   d) power supply

4. "The ability of the system to deliver services when requested" is:
   a) Reliability
   b) Availability
   c) Safety
   d) Security

5. Integrity and confidentiality are parts of:
   a) Reliability
   b) Safety
   c) Security
   d) Availability
6. Continuing to deliver services while under attack is called:
   a) survivability
   b) repairability
   c) maintainability
   d) error tolerance
7. As dependability increases, cost increases:
   a) linearly
   b) exponentially
   c) not at all
   d) and then decreases
8. System A fails once a year (3 days to restart), B once a month (10 minutes). Which is more available?
   a) A
   b) B
   c) Both are equal
   d) Cannot say
9. "A variable that is not initialised" is an example of a:
   a) system failure
   b) system fault
   c) human error
   d) hazard
0. Using redundant modules is an example of:
   a) fault avoidance
   b) fault detection
   c) fault tolerance
   d) hazard avoidance
1. Mills found that removing 60% of known errors improved reliability by only:
   a) 3%
   b) 30%
   c) 60%
   d) 90%
2. A medical drug database is ___ safety-critical software.
   a) primary
   b) secondary
   c) not
   d) mission
3. "A condition with the potential for causing an accident" is a:
   a) risk
   b) damage
   c) hazard
   d) mishap
4. A cutting machine that needs two buttons pressed at the same time is an example of:
   a) hazard avoidance
   b) hazard detection
   c) damage limitation
   d) fault tolerance
5. Fire extinguishers in an aircraft engine are an example of:
   a) hazard avoidance
   b) hazard detection and removal
   c) damage limitation
   d) exposure
6. "A weakness that may be exploited to cause loss or harm" is a:
   a) threat
   b) vulnerability
   c) attack
   d) control
7. Encryption is an example of a:
   a) control
   b) threat
   c) exposure
   d) attack

8. Regular system backups are an example of:

a) vulnerability avoidance
b) attack detection
c) exposure limitation
d) hazard avoidance

Cevap anahtarı: 1-b · 2-c · 3-c · 4-b · 5-c · 6-a · 7-b · 8-b · 9-b · 10-c · 11-a · 12-b · 13-c · 14-a · 15-c · 16-b · 17-a · 18-c

YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 4
Software processes

Yazılım Süreçleri

Kitap s.63–91 (Bölüm 4 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 4, **yazılım süreci** kavramını tanıtıyor. Yazılım süreci, yazılım üretmek için yapılan tutarlı faaliyetler bütünüdür. Bu bölüm sınav açısından **en önemli bölümlerden biri**. Beş kısımdan oluşuyor:

- **4.1 Software process models:** Waterfall, evolutionary development, CBSE (+ formal geliştirme ve Cleanroom).
- **4.2 Process iteration:** Incremental delivery ve spiral model.
- **4.3 Process activities:** Specification, design & implementation, validation (test), evolution.
- **4.4 The Rational Unified Process (RUP):** 4 faz, 9 iş akışı, 6 iyi uygulama.
- **4.5 Computer-Aided Software Engineering (CASE):** CASE araçlarının sınıflandırılması (tools, workbenches, environments).

Bölümün amaçları: (1) yazılım süreci ve süreç modeli kavramlarını anlamak, (2) 3 genel süreç modelini ve ne zaman kullanılacaklarını bilmek, (3) gereksinim mühendisliği, geliştirme, test ve evrim faaliyetlerini ana hatlarıyla anlamak, (4) RUP'un iyi uygulamaları nasıl birleştirdiğini anlamak, (5) CASE teknolojisini tanımak.

Sayfa 64–66 Yazılım süreci · 4.1 Software process models · Formal geliştirme

📖 Sayfanın özeti

SÜREÇ VE OTOMASYON

Yazılım süreci, bir yazılım ürününü ortaya çıkaran faaliyetler kümesidir. Yazılım giderek sıfırdan değil, mevcut sistemleri genişleterek ve hazır bileşenleri birleştirerek yapılıyor. Süreçler karmaşıktır ve insan yargısına ve yaratıcılığına dayandığı için otomatikleştirme çabaları sınırlı başarı göstermiştir. CASE araçları bazı faaliyetleri destekler ama yaratıcı tasarımın yerini alamaz. Bunun bir nedeni de süreçlerin çok çeşitli olmasıdır: **ideal süreç yoktur**. Kritik sistemler çok yapılandırılmış, hızlı değişen iş sistemleri ise esnek ve çevik (agile) süreç ister.

ORTAK 4 FAALİYET

Bütün süreçlerde: **specification** (işlev ve kısıtlar tanımlanır), **design and implementation** (tanımlamaya uyan yazılım üretilir), **validation** (müşterinin istediğini yaptığı doğrulanır), **evolution** (değişen ihtiyaçlara göre geliştirilir).

SÜREÇ STANDARDİZASYONU

İdeal süreç olmasa da çoğu kurumun süreci iyileştirilebilir. Kurum içindeki süreç çeşitliliğini azaltan **standardizasyon** iletişimi iyileştirir, eğitim süresini kısaltır, otomatik süreç desteğini ekonomik hale getirir ve yeni yöntemlerin benimsenmesine zemin hazırlar.

4.1 SÜREÇ MODELLERİ (PARADİGMALAR)

Süreç modeli, süreci belirli bir bakış açısından gösteren soyut bir temsildir ve sadece kısmi bilgi verir. Bu bölümdeki genel modeller sürecin **mimari çerçevesini** gösterir, ayrıntısını değil. Uyarlanıp genişletilebilecek çerçevelerdir. Üç model: **waterfall** (faaliyetler ayrı aşamalar), **evolutionary development** (faaliyetler iç içe, hızlı ilk sistem müşteri girdisiyle iyileştirilir), **CBSE** (çok sayıda yeniden kullanılabilir bileşeni birleştirme). Bu modeller birbirini dışlamaz, büyük sistemlerde genelde **birlikte** kullanılır (RUP üçünü birleştirir).

FORMAL SİSTEM GELİŞTİRME

En önemli varyant: sistemin matematiksel modeli oluşturulur ve tutarlılığı koruyan matematiksel dönüşümlerle çalıştırılabilir koda çevrilir. En bilinen örnek IBM'in **Cleanroom** sürecidir. Her artım formal olarak tanımlanıp gerçekleştirmeye dönüştürülür, doğruluk formal yaklaşımla gösterilir, kusur testi

yapılmaz ve sistem testi güvenilirliği ölçmeye odaklanır. Cleanroom ve **B yöntemi**, katı emniyet, güvenilirlik ya da güvenlik gereksinimi olan sistemlere uygundur, çünkü müşterilere ya da sertifika kurumlarına sistemin gereksinimleri karşıladığını göstermeyi kolaylaştırır. Bu alanların dışında pek kullanılmaz: özel uzmanlık ister ve çoğu sistem için maliyet ya da kalite avantajı sağlamaz.

👩‍🏫 Hoca anlatımı

Bölüm 1'de gördüğün **4 temel faaliyet** burada tekrar geliyor: specification, design & implementation, validation, evolution. Bu bölüm bunların farklı modellerde **nasıl düzenlendiğini** anlatıyor.

Süreç modelleri bir evin **iskeleti** gibidir: duvarların yerini gösterir ama mobilyayı göstermez. Her şirket bu iskeleti kendine göre döşer. Üç ana model var (waterfall, evolutionary, CBSE) ve gerçek hayatta çoğu zaman **karışık** kullanılır.

Cleanroom adını temiz odadan alır: çip üretilen tozsuz odalar gibi, hatanın en baştan hiç girmemesini amaçlar. Matematikle "bu program doğru" diye ispat edilir. Çok pahalı ve uzmanlık istediği için sadece uçak, nükleer santral gibi çok kritik sistemlerde kullanılır.

📝 Deftere not

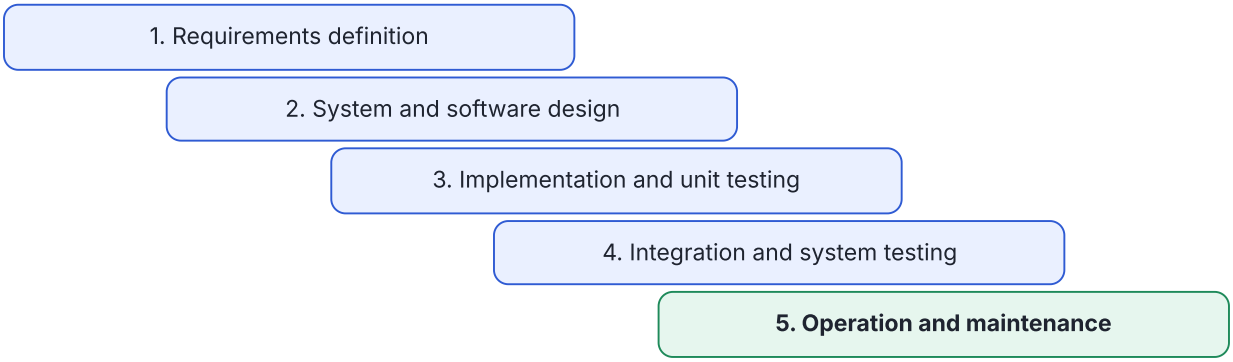
- **Software process:** yazılım ürününü ortaya çıkaran faaliyetler kümesi. Giderek **mevcut sistemleri genişletme** ve **hazır bileşen** birleştirme ile yapılıyor.
- Süreçler insan yargısı ve yaratıcılığına dayanır, bu yüzden **otomasyon sınırlı**. CASE bazı faaliyetleri destekler, yaratıcı tasarımın yerini alamaz.
- ⭐ **İdeal süreç yok.** Kritik sistemler → **yapılandırılmış** süreç. Hızlı değişen iş sistemleri → **esnek / agile** süreç.
- ⭐⭐ **Ortak 4 faaliyet:** Specification · Design & Implementation · Validation · Evolution.
- **Standardizasyon**: iletişim iyileşir, eğitim süresi azalır, otomasyon ekonomik olur, yeni yöntemlere zemin hazırlar.
- ⭐ **Process model:** süreci belirli bir bakış açısından gösteren **soyut temsil** (kısmi bilgi). Genel modeller = **process paradigms**, sürecin **çerçevesi**.
- ⭐⭐ **3 genel model: Waterfall** (ayrı aşamalar) · **Evolutionary development** (iç içe, hızlı ilk sistem + müşteri girdisi) · **CBSE** (yeniden kullanılabilir bileşenleri birleştirme). Birbirini **dışlamaz**, birlikte kullanılır (RUP üçünü birleştirir).
- ⭐ **Formal system development:** matematiksel model → tutarlılığı koruyan dönüşümler → çalıştırılabilir kod.
  - Cleanroom (IBM): her artım formal tanımlanır, doğruluk formal gösterilir, **kusur testi yok**, sistem testi **güvenilirlik** ölçer.
  - Cleanroom ve **B method**: katı safety, reliability ve security gereksinimli sistemler için (sertifika kurumuna kanıt kolaylaşır).
  - Dışında pek kullanılmaz: **uzmanlık** ister, çoğu sistemde maliyet ya da kalite avantajı yok.

Sayfa 66–68 4.1.1 The waterfall model (Şelale modeli, Şekil 4.1)

📖 Sayfanın özeti

KÖKENİ

Yayınlanan ilk yazılım süreci modelidir ve genel sistem mühendisliği süreçlerinden türetilmiştir (**Royce, 1970**). Aşamalar şelale gibi birbirine aktığı için **waterfall** ya da **software life cycle** (yazılım yaşam döngüsü) denir.



Şekil 4.1: Yazılım yaşam döngüsü (geri oklar önceki aşamalara dönüşü gösterir)

5 AŞAMA

1) Requirements analysis and definition: sistemin hizmetleri, kısıtları ve hedefleri kullanıcılarla görüşülerek belirlenir ve ayrıntılı olarak tanımlanıp sistem tanımlaması olur. **2) System and software design:** gereksinimler donanım ya da yazılım sistemlerine ayrılır, genel mimari kurulur, temel yazılım soyutlamaları ve ilişkileri tanımlanır. **3) Implementation and unit testing:** tasarım programlara ya da program birimlerine dönüştürülür. Birim testi her birimin kendi tanımlamasına uyduğunu doğrular. **4) Integration and system testing:** birimler birleştirilip tam sistem olarak test edilir ve sonra müşteriye teslim edilir. **5) Operation and maintenance:** genelde **en uzun** aşamadır. Sistem kurulup kullanılır. Bakım; önceki aşamalarda bulunamayan hataları düzeltmeyi, birimlerin gerçekleştirmesini iyileştirmeyi ve yeni gereksinimlere göre hizmetleri geliştirmeyi içerir.

TEORİ VE PRATİK

Teoride her aşamanın sonucu **onaylanmış (signed off)** belgelerdir ve bir aşama bitmeden diğeri başlamaz. Pratikte aşamalar **örtüşür** ve birbirini besler (tasarımda gereksinim sorunları, kodlamada tasarım sorunları bulunur). Süreç doğrusal değil, yinelemelerden oluşur. Belge üretip onaylamak pahalı olduğu için birkaç yinelemeden sonra bazı kısımlar (ör. tanımlama) **dondurulur** ve sorunlar sonraya bırakılır, görmezden gelinir ya da etrafından dolanılır. Bu **erken dondurma**, sistemin kullanıcının istediğini yapmamasına ve kötü yapılandırılmış sistemlere yol açabilir. Son aşamada hatalar ve yeni ihtiyaçlar ortaya çıkar, bakım önceki aşamaları tekrarlamayı gerektirebilir.

AVANTAJ, DEZAVANTAJ, NE ZAMAN?

Avantajları: her aşamada belge üretilir ve diğer mühendislik süreç modelleriyle uyumludur. **En büyük sorunu:** projenin aşamalara **katı (inflexible)** şekilde bölünmesi. Kararlar çok erken verilmek zorunda olduğu için değişen müşteri gereksinimlerine yanıt vermek zordur. **Ne zaman kullanılmalı?** Gereksinimler **iyi anlaşılmış** ve geliştirme sırasında köklü değişmesi **beklenmiyorsa**. Diğer mühendislik projelerindeki modele benzediği için, özellikle yazılım daha büyük bir sistem mühendisliği projesinin parçasıysa hâlâ kullanılır.

👩‍🏫 Hoca anlatımı

Waterfall bu dersin en klasik sorusu! Neredeyse her sınavda aşamaları, avantajları ve dezavantajları sorulur.

Adı şelaleden geliyor: su yukarıdan aşağı akar ve geri dönmez. Model de öyle: bir aşama bitip **imzalanmadan** sonrakine geçilmez. Bir ev inşaatı gibi düşün: önce planı çizersin (gereksinim), sonra projeyi yaparsın (tasarım), sonra duvarları örersin (kodlama ve birim testi), sonra her şeyi birleştirip kontrol edersin (entegrasyon ve sistem testi), sonra evde yaşarsın ve tamir edersin (işletim ve bakım).

5 aşamayı sırasıyla ezberle. Kısa hali: **Requirements → Design → Implementation & unit testing → Integration & system testing → Operation & maintenance.** En uzun aşama **operation & maintenance**, bu da sorulabilir.

Sorun nedir? Esnek değildir. Evin temeli atıldıktan sonra müşteri "aslında havuz da istiyorum" derse çok pahalıya patlar. Yazılımda müşteriler sürekli fikir değiştirdiği için bu ciddi bir dezavantaj.

Ne zaman kullanılır? Gereksinimler **net** ve **değişmeyecekse**. Ör: uçağın içindeki yazılım. Uçak mühendisliği de böyle aşamalı çalıştığı için uyumludur.

📝 Deftere not

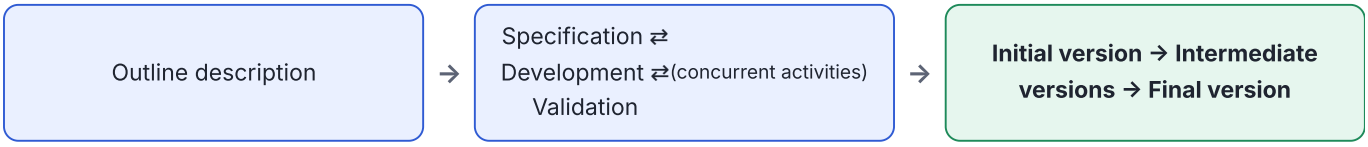
- ⭐ İlk yayınlanan süreç modeli, sistem mühendisliğinden türedi (**Royce, 1970**). Diğer adı: **software life cycle**.
- ⭐⭐ **5 aşama (Şekil 4.1):**
  - **1. Requirements analysis & definition:** hizmetler, kısıtlar, hedefler kullanıcılarla belirlenir, ayrıntılı tanımlanır, **system specification** olur.
  - **2. System & software design:** gereksinimler donanım ya da yazılıma ayrılır, **genel mimari**, temel soyutlamalar ve ilişkiler.
  - **3. Implementation & unit testing:** tasarım → program birimleri. **Unit test**: her birim spec'ine uyuyor mu?
  - **4. Integration & system testing:** birimler birleştirilir, tam sistem test edilir, **müşteriye teslim**.
  - **5. Operation & maintenance:** genelde **EN UZUN** aşama. Kurulum ve kullanım. Bakım = gizli hataları düzeltme, iyileştirme, yeni gereksinimlere göre geliştirme.
- Teoride her aşamanın sonu **signed-off belgeler**, önceki bitmeden sonraki başlamaz. Pratikte aşamalar **örtüşür** ve birbirini besler, süreç **doğrusal değil**.
- Belge maliyeti yüzünden bazı kısımlar (spec) **erken dondurulur (premature freezing)**. Sonuç: sistem kullanıcının istediğini yapmaz, kötü yapılandırılmış olur.
- ⭐⭐ **Avantajlar:** her aşamada **belge** üretilir · diğer **mühendislik süreç modelleriyle uyumlu**.
- ⭐⭐ **Dezavantaj:** projenin aşamalara **esnek olmayan (inflexible)** bölünmesi. Kararlar **erken** verilir, **değişen gereksinimlere** yanıt vermek zor.
- ⭐⭐ **Ne zaman?** Gereksinimler **iyi anlaşılmış** ve köklü değişmesi **beklenmiyorsa**. Ayrıca yazılım büyük bir **sistem mühendisliği projesinin parçasıysa**.

Sayfa 68–69 4.1.2 Evolutionary development (Evrimsel geliştirme, Şekil 4.2)

📖 Sayfanın özeti

FİKİR

Bir ilk sürüm geliştirilir, kullanıcı yorumuna açılır ve yeterli bir sistem ortaya çıkana kadar **birçok sürümle** iyileştirilir. Tanımlama, geliştirme ve geçerleme ayrı değil, **iç içe ve eşzamanlı** yürür ve aralarında hızlı geri bildirim vardır.



Şekil 4.2: Evrimsel geliştirme

İKİ TÜR

Exploratory development (keşifsel geliştirme): amaç müşteriyle birlikte gereksinimleri keşfedip **son sistemi teslim etmektir**. Önce **iyi anlaşılan** kısımlardan başlanır, sistem müşterinin önerdiği yeni özelliklerle büyür. **Throwaway prototyping (atılacak prototip):** amaç müşterinin gereksinimlerini **anlamak** ve daha iyi bir gereksinim tanımı yapmaktır. Prototip, **iyi anlaşılmayan** gereksinimlerle deney yapmaya odaklanır.

AVANTAJ VE SORUNLAR

Müşterinin acil ihtiyaçlarını karşılamada genelde waterfall'dan **daha etkilidir**. Avantajı, tanımlamanın **artımlı** geliştirilebilmesidir: kullanıcılar problemi daha iyi anladıkça bu sisteme yansır. Mühendislik ve yönetim açısından iki sorunu vardır: **1) Süreç görünür değildir:** yöneticiler ilerlemeyi ölçmek için düzenli çıktılara ihtiyaç duyar. Sistem hızlı geliştirildiği için her sürümü belgelemek ekonomik değildir. **2) Sistemler genelde kötü yapılandırılır:** sürekli değişiklik yapıyı bozar, değişiklik eklemek giderek zorlaşır ve pahalılaşır.

NE ZAMAN? KARMA YAKLAŞIM

Yazara göre **küçük ve orta boyutlu** sistemler (**500.000 satıra kadar**) için en iyi yaklaşımdır. **Büyük, karmaşık, uzun ömürlü** sistemlerde sorunlar ağırlaşır: farklı ekipler farklı parçaları geliştirirken kararlı bir mimari kurmak ve katkıları birleştirmek zorlaşır. Büyük sistemler için **karma** süreç önerilir: belirsizlikleri çözmek için atılacak prototip yapılır, sonra sistem daha yapılandırılmış bir yaklaşımla yeniden yazılır. İyi anlaşılan kısımlar waterfall ile, tanımlaması zor kısımlar (ör. kullanıcı arayüzü) keşifsel programlamayla geliştirilir. Detaylar Bölüm 17'de, evrimsel geliştirme RUP'a da dahil.

👨‍🏫 Hoca anlatımı

Evrimsel geliştirme **heykel yapmak** gibidir: önce kaba bir şekil çıkarırsın, müşteri bakar, "burası olmamış" der, düzeltirsin, tekrar bakar... Sonunda müşterinin istediği şekil ortaya çıkar.

İki türü karıştırma: Exploratory, yaptığın şey **son ürün olacak**. İyi bildiğin yerden başlar, büyütürsün. **Throwaway**, yaptığın şey **çöpe atılacak**. Amaç sadece müşterinin ne istediğini anlamak, yani anlamadığın yerden başlarsın. Bir arabanın kil maketi gibi: maketi görürsün, beğenirsin, sonra asıl araba ayrıca üretilir.

2 sorunu ezberle (alıştırma 4.2 bunu soruyor): (1) **Görünürlük yok:** yönetici "ne kadar bitti?" diye sorduğunda gösterecek belge yok. (2) **Yapı bozulur:** sürekli yama yapılan kod spagettiye döner, bakımı zorlaşır.

500.000 satır sınırını hatırla: küçük ve orta projelerde en iyisi evrimsel. Büyükte karma yaklaşım.

📝 Deftere not

- ⭐ **Evolutionary development:** ilk sürüm → kullanıcı yorumu → birçok sürümle iyileştirme → yeterli sistem. Spec, development, validation **iç içe (concurrent)**, hızlı geri bildirim.
- ⭐⭐ **2 tür:**
  - Exploratory development: müşteriyle gereksinimleri keşfet, **son sistemi teslim et**. **İyi anlaşılan** kısımlardan başla, müşterinin önerdiği özelliklerle büyüt.
  - Throwaway prototyping: gereksinimleri **anlamak** için, daha iyi gereksinim tanımı. **İyi anlaşılmayan** gereksinimlerle deney.
- ⭐ **Avantaj:** acil ihtiyaçları karşılamada waterfall'dan **daha etkili**. Spec **artımlı** gelişir, kullanıcının anlayışı sisteme yansır.
- ⭐⭐ **2 sorun:**

- **Process is not visible:** yönetici ilerlemeyi ölçemez, her sürümü belgelemek ekonomik değil.
- **Systems are often poorly structured:** sürekli değişiklik yapıyı bozar, değişiklik zorlaşır ve pahalılaşır.

- ⭐ **Ne zaman? Küçük ve orta** sistemler (**≤ 500.000 satır**) için en iyi.
- **Büyük, karmaşık, uzun ömürlü** sistemlerde sorunlu (kararlı mimari ve ekip katkılarını birleştirmek zor), bu yüzden **karma süreç**: throwaway prototip, sonra yapılandırılmış yeniden yazım. İyi anlaşılan kısımlar **waterfall**, belirsiz kısımlar (ör. **UI**) **exploratory**.

Sayfa 69–71 4.1.3 Component-based software engineering (CBSE, Şekil 4.3)

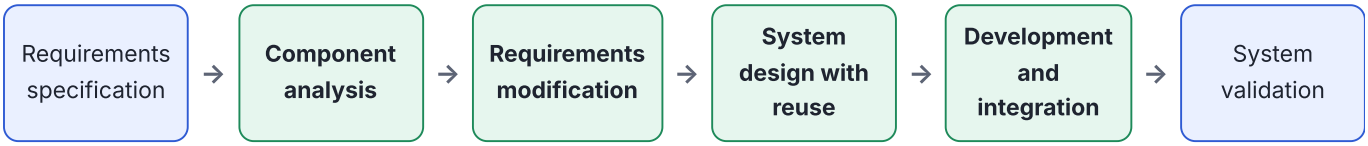
📖 Sayfanın özeti

YENİDEN KULLANIM

Çoğu projede bir miktar yeniden kullanım olur. Genelde bu **gayriresmi**dir: ekip benzer tasarım ya da kod bilir, bulup uyarlar ve sisteme ekler. Evrimsel yaklaşımda hızlı geliştirme için çoğu zaman şarttır. Bu tür yeniden kullanım süreçten bağımsız olarak yapılır. Son yıllarda ise yeniden kullanıma **dayanan** CBSE yaklaşımı ortaya çıktı ve yaygınlaşıyor (detay Bölüm 19).

YAKLAŞIM

CBSE geniş bir yeniden kullanılabilir bileşen tabanına ve bunları birleştiren bir çerçeveye dayanır. Bileşenler bazen metin biçimlendirme ya da sayısal hesaplama gibi işlevler sunan kendi başına sistemlerdir (**COTS**). İlk aşama (gereksinim tanımı) ve son aşama (geçerleme) diğer süreçlere benzer. Aradaki aşamalar farklıdır.



Şekil 4.3: CBSE süreci (yeşiller CBSE'ye özgü ara aşamalar)

ARA AŞAMALAR

1) Component analysis: tanımlamayı karşılayacak bileşenler aranır. Genelde tam eşleşme yoktur, bileşenler işlevin sadece bir kısmını sağlar. **2) Requirements modification:** bulunan bileşenlere göre gereksinimler analiz edilip **değiştirilir**. Değişiklik mümkün değilse bileşen analizine dönülüp başka çözümler aranır. **3) System design with reuse:** sistemin çerçevesi tasarlanır ya da mevcut bir çerçeve yeniden kullanılır. Bileşenlere uygun düzenleme yapılır, uygun bileşen yoksa yeni yazılım tasarlanır. **4) Development and integration:** dışarıdan alınamayan yazılım geliştirilir, bileşenler ve COTS sistemleri birleştirilir. Bu modelde entegrasyon ayrı bir faaliyet değil, geliştirmenin parçası olabilir.

AVANTAJ VE DEZAVANTAJ

Avantaj: geliştirilecek yazılım miktarı azalır, dolayısıyla **maliyet ve risk düşer**. Genelde **daha hızlı teslim** sağlar. **Dezavantaj:** gereksinimlerden **ödün verilmesi kaçınılmazdır** ve sistem kullanıcıların gerçek ihtiyaçlarını karşılamayabilir. Ayrıca bileşenlerin yeni sürümleri kurumun kontrolünde olmadığı için **sistem evrimi üzerindeki kontrolün bir kısmı kaybedilir**. CBSE, farklı tedarikçilerin **web servislerini** birleştirmeye dayanan yeni yaklaşıma benzer (Bölüm 12).

👨‍🏫 Hoca anlatımı

CBSE = **LEGO ile yazılım yapmak**. Her şeyi sıfırdan yazmazsın, hazır parçaları bulur ve birleştirirsin.

Buradaki en ilginç adım **Requirements modification**: **Gereksinimleri parçalara göre değiştirirsin!** Normalde "ne istiyorsam onu yaparım" dersin. CBSE'de ise "elimde şu parçalar var, istediğimi biraz

bunlara uydurayım" dersin. Hazır takım elbise almak gibi: terziye diktirmek kadar tam oturmaz ama ucuz ve hızlıdır.

Avantaj ve dezavantajı da bu benzetmeden çıkar. **Artısı:** ucuz, hızlı, az risk. **Eksisi:** tam istediğin olmaz (ödün verirsin) ve parçaların yeni sürümlerini sen kontrol edemezsin. Firma parçayı değiştirirse ya da desteği keserse sen de etkilenirsin.

📝 Deftere not

- Yeniden kullanım çoğu projede **gayriresmi (informal)** olarak var. CBSE ise yeniden kullanıma **dayanan** sistematik yaklaşım.
- ⭐ **CBSE:** geniş bir **yeniden kullanılabilir bileşen** tabanı + **birleştirme çerçevesi (integrating framework)**. Bileşenler bazen **COTS** sistemlerdir.
- ⭐⭐ **Şekil 4.3, CBSE aşamaları:** Requirements specification → **Component analysis** → **Requirements modification** → **System design with reuse** → **Development and integration** → System validation.
  - **Component analysis:** uygun bileşen ara. Genelde **tam eşleşme yok**.
  - **Requirements modification:** gereksinimleri **bileşenlere göre değiştir**. Olmazsa analize geri dön.
  - **System design with reuse:** çerçeveyi tasarla ya da yeniden kullan. Bileşen yoksa yeni yazılım tasarla.
  - **Development and integration:** alınamayanı geliştir, bileşen ve COTS'u birleştir. Entegrasyon geliştirmenin **parçası**.
- ⭐⭐ **Avantajlar:** daha az yazılım geliştirilir, **maliyet ve risk düşer**, **daha hızlı teslim**.
- ⭐⭐ **Dezavantajlar: gereksinimlerden ödün** kaçınılmaz, gerçek ihtiyacı karşılamayabilir · bileşenlerin yeni sürümleri kontrolde değil, **evrim kontrolü kaybı**.
- CBSE, **web servisleri** birleştirmeye dayanan yaklaşıma benzer (Bölüm 12).

Sayfa 71–73 4.2 Process iteration · 4.2.1 Incremental delivery (Şekil 4.4)

📖 Sayfanın özeti

DEĞİŞİM KAÇINILMAZ

Büyük projelerde değişim kaçınılmazdır: iş ortamı, yönetim öncelikleri ve teknolojiler değişir. Bu yüzden süreç tek seferlik değildir, faaliyetler değişiklik taleplerine göre düzenli olarak tekrarlanır. Yinelemeyi desteklemek için tasarlanmış iki model vardır: **incremental delivery** (tanımlama, tasarım ve gerçekleştirme bir dizi artıma bölünür ve sırayla geliştirilir) ve **spiral development** (sistem ilk taslaktan son sisteme doğru spiral şeklinde dışa doğru gelişir). Yinelemeli süreçlerin özü, tanımlamanın yazılımla **birlikte** geliştirilmesidir. Bu, tam tanımlamanın sözleşmenin parçası olduğu birçok kurumun **tedarik modeliyle çelişir**. Artımlı yaklaşımda son artıma kadar tam tanımlama yoktur. Bu yüzden devlet kurumları gibi büyük müşterilerin zor kabul edebileceği yeni tür sözleşmeler gerekir.

WATERFALL VE EVRİMSEL ARASINDA BİR ORTA YOL

Waterfall, gereksinimlerin tasarımdan, tasarım kararlarının gerçekleştirmeden önce kesinleşmesini ister. Değişiklik yeniden çalışma gerektirir, ama iyi belgelenmiş ve değiştirilebilir sistemler üretir. Evrimsel yaklaşım kararları erteletir, ama kötü yapılandırılmış ve bakımı zor yazılıma yol açabilir. **Incremental delivery** ikisinin avantajlarını birleştirir.





Şekil 4.4: Artımlı teslim (sistem tamamlanmadıysa "develop increment"e geri dönülür)

NASIL İŞLER?

Müşteriler hizmetleri ana hatlarıyla belirler ve hangilerinin en önemli olduğunu söyler. Her biri sistemin işlevlerinin bir alt kümesini sunan teslim artımları tanımlanır. **En yüksek öncelikli hizmetler ilk teslim edilir.** İlk artımın gereksinimleri ayrıntılı tanımlanıp geliştirilir. Bu sırada sonraki artımların analizi devam edebilir, ama mevcut artım için gereksinim değişikliği kabul edilmez. Teslim edilen artım hemen kullanılır, müşteriler onunla deney yaparak sonraki artımlar için gereksinimlerini netleştirir. Yeni artımlar mevcutlarla birleştirilir ve sistem her teslimde gelişir.

4 AVANTAJ

1) Müşteriler değer elde etmek için bütün sistemi beklemez, ilk artım en kritik ihtiyaçları karşılar. 2) İlk artımlar prototip gibi kullanılıp sonraki gereksinimleri netleştirir. 3) **Projenin tamamen başarısız olma riski düşüktür**, bazı artımlar mutlaka başarıyla teslim edilir. 4) En öncelikli hizmetler önce teslim edildiği ve sonrakiler onlarla birleştirildiği için **en önemli hizmetler en çok test edilir**.

SAYFA 73: SORUNLAR VE XP

Artımlar **küçük** olmalı (**en fazla 20.000 satır**) ve her biri bir işlev sunmalı. Müşteri gereksinimlerini doğru boyutta artımlara eşlemek zor olabilir. Ayrıca çoğu sistem farklı parçaların kullandığı **ortak temel olanaklara** ihtiyaç duyar, gereksinimler artımın sırası gelene kadar ayrıntılandırılmadığı için bu ortak olanakları belirlemek zordur. Bu yaklaşımın bir varyantı **extreme programming (XP)**'dir (Beck, 2000): çok küçük işlev artımları, sürece müşteri katılımı, sürekli kod iyileştirme ve **pair programming (eşli programlama)** (Bölüm 17).

👨‍🏫 Hoca anlatımı

Incremental delivery = sistemi **parça parça teslim etmek**. Bir alışveriş sitesi düşün. 1. teslim: ürün listeleme ve sepet (en önemli şeyler). 2. teslim: ödeme. 3. teslim: kullanıcı yorumları. 4. teslim: öneri sistemi. Müşteri 1. teslimden itibaren siteyi kullanmaya başlar, son teslimi beklemez.

4 avantajı ezberle: (1) erken kullanım, (2) ilk parçalar prototip gibi işe yarar, (3) bütün proje batmaz, (4) en önemli parçalar en çok test edilir (en başta teslim edildikleri için her yeni parçayla birlikte tekrar tekrar test edilirler).

Evolutionary ile farkı ne? Evrimselde sürekli aynı sistemi düzeltirsin, plan yoktur. Artımlıda ise **önce genel mimari çizilir** (Şekil 4.4'teki 3. adım), sonra parçalar bu mimariye takılır. Yani daha düzenlidir.

Sorunlar: parçalar küçük olmalı (≤ 20.000 satır) ve herkesin kullandığı ortak altyapıyı (ör. giriş sistemi) önceden belirlemek zordur.

XP (Extreme Programming) bunun uç bir versiyonu: çok çok küçük parçalar, müşteri sürekli ekipte, iki programcı tek bilgisayarda (pair programming).

📝 Deftere not

- ⭐ Büyük projelerde **değişim kaçınılmaz**, faaliyetler tekrarlanır (**iteration**).
- ⭐⭐ Yinelemeyi destekleyen **2 model:**
  - Incremental delivery: spec, tasarım ve gerçekleştirme **artımlara** bölünür, sırayla geliştirilir.
  - Spiral development: ilk taslaktan son sisteme **spiral** şeklinde gelişir.

- Yinelemeli süreçlerin özü: spec yazılımla **birlikte** gelişir. Sorun: tam spec'in sözleşmenin parçası olduğu **tedarik modeliyle çelişir**, yeni tür sözleşme gerekir (devlet kurumları zorlanır).
- ⭐ **Waterfall:** belgeli ve değiştirilebilir ama erken taahhüt. **Evolutionary:** kararları erteler ama kötü yapı. **Incremental:** ikisinin **avantajlarını birleştirir**.
- ⭐ **Şekil 4.4:** Define outline requirements → Assign requirements to increments → Design system architecture → [Develop increment → Validate increment → Integrate increment → Validate system] → tamamlanınca Final system.
- Müşteri hizmetleri **önceliklendirir**. **En yüksek öncelik ilk teslim edilir.** Mevcut artım geliştirilirken o artımın gereksinimleri **değiştirilemez**. Teslim edilen artım hemen kullanılır.
- ⭐⭐ **4 avantaj:**
  - 1. Bütün sistemi beklemeden değer: ilk artım **en kritik** ihtiyacı karşılar.
  - 2. İlk artımlar **prototip** gibi, sonraki gereksinimleri netleştirir.
  - 3. **Projenin tamamen başarısız olma riski düşük.**
  - 4. **En önemli hizmetler en çok test edilir** (önce teslim edilip her artımla birleştirildiği için).
- ⭐ **Sorunlar:** artımlar **küçük** olmalı (**≤ 20.000 satır**) · gereksinimleri doğru boyutlu artımlara eşlemek zor · **ortak temel olanakları** belirlemek zor.
- Extreme programming (XP) (Beck 2000): çok küçük artımlar, **müşteri katılımı**, sürekli kod iyileştirme, **pair programming**. Agile yöntem (Bölüm 17).

Sayfa 73–74 4.2.2 Spiral development (Boehm'in spiral modeli, Şekil 4.5)

📖 Sayfanın özeti

FİKİR

Boehm (1988) tarafından önerildi. Süreci geri dönüşlü bir faaliyet dizisi olarak değil, bir **spiral** olarak gösterir. Spiralin **her döngüsü sürecin bir aşamasıdır**: en içteki döngü sistemin **fizibilitesiyle**, sonraki gereksinim tanımıyla, sonraki sistem tasarımıyla ilgilenebilir.

HER DÖNGÜNÜN 4 SEKTÖRÜ

1) Objective setting: o aşamanın hedefleri belirlenir. Süreç ve ürün kısıtları belirlenir, ayrıntılı yönetim planı yapılır, proje riskleri tanımlanır ve bu risklere göre alternatif stratejiler planlanabilir. **2) Risk assessment and reduction:** belirlenen her risk ayrıntılı analiz edilir ve riski azaltmak için adım atılır. Ör: gereksinimlerin uygunsuz olma riski varsa bir prototip geliştirilir. **3) Development and validation:** risk değerlendirmesinden sonra bir geliştirme modeli seçilir. Kullanıcı arayüzü riskleri baskınsa **evrimsel prototipleme**, emniyet riskleri baskınsa **formal dönüşümler**, ana risk alt sistem entegrasyonuysa **waterfall** uygun olabilir. **4) Planning:** proje gözden geçirilir ve spiralin bir döngüsüne daha devam edilip edilmeyeceğine karar verilir. Devam edilecekse sonraki aşama planlanır.



Şekil 4.5 (sadeleştirilmiş): Her spiral döngüsü bu 4 sektörden geçer ve dışa doğru büyür

ŞEKİL 4.5'TEKİ AYRINTILAR

İç döngülerde risk analizi, prototip 1-2-3 ve operasyonel prototip, gereksinim planı ve yaşam döngüsü planı, kullanım konsepti (concept of operation), yazılım gereksinimleri ve gereksinim geçerleme yer alır. Dış döngülerde geliştirme planı, entegrasyon ve test planı, tasarım ve V&V, ürün tasarımı, ayrıntılı tasarım, kod, birim testi, entegrasyon testi, kabul testi ve hizmet (service) yer alır.

TEMEL FARK: RİSK

Spiral modelin diğer modellerden en önemli farkı **riskin açıkça tanınmasıdır**. Risk, kabaca ters gidebilecek bir şeydir. Ör: yeni bir programlama dili kullanılacaksa derleyicilerin güvenilmez olması ya da yeterince verimli kod üretmemesi bir risktir. Riskler takvim ve maliyet aşımı gibi proje sorunlarına yol açtığı için **risk azaltma** çok önemli bir yönetim faaliyetidir (Bölüm 5). Bir döngü performans ve işlevsellik gibi hedeflerin belirlenmesiyle başlar, alternatifler ve kısıtlar sıralanır, her alternatif hedeflere göre değerlendirilir ve risk kaynakları belirlenir. Riskler analiz, prototipleme ve simülasyon gibi bilgi toplama faaliyetleriyle çözülür, sonra geliştirme yapılır ve sonraki aşama planlanır.

👨‍🏫 Hoca anlatımı

Spiral model = **"önce riski düşün"** modeli. Salyangoz kabuğu gibi içten dışa doğru döner. Her turda proje biraz daha büyür ve olgunlaşır.

4 sektörü ezberle, kesin sorulur: (1) **Hedef koy**, (2) **Riski değerlendir ve azalt**, (3) **Geliştir ve doğrula**, (4) **Planla** (devam mı, dur mu?).

Bu modelin en güzel yanı, 3. sektörde **riske göre** diğer modellerden birini seçmen. Spiral aslında diğer modelleri içine alan bir "üst model" (meta-model) gibidir. Arayüz riskliyse evrimsel prototip, emniyet riskliyse formal yöntem, entegrasyon riskliyse waterfall. Alıştırma 4.3 tam olarak bunu soruyor: "Waterfall ve prototipleme spiral modelin içine nasıl sığar?" Cevap: spiralin her döngüsünün 3. sektöründe, o döngünün riskine göre seçilirler.

Diğer modellerden **en önemli farkı**: **riski açıkça tanıması**. Sınavda "What is the main difference between the spiral model and other models?" sorulursa cevap budur.

📝 Deftere not

- ⭐ **Spiral model: Boehm (1988)**. Süreç = **spiral**. **Her döngü = bir süreç aşaması** (iç döngü: fizibilite → gereksinim tanımı → sistem tasarımı...).
- ⭐⭐ **Her döngünün 4 sektörü:**
  - **1. Objective setting:** hedefler, kısıtlar, ayrıntılı yönetim planı, **riskler**, alternatif stratejiler.
  - **2. Risk assessment and reduction:** her risk ayrıntılı analiz edilir ve azaltılır (ör. gereksinim riski varsa **prototip**).
  - **3. Development and validation:** riske göre geliştirme modeli seçilir: UI riski → **evolutionary prototyping**, safety riski → **formal transformations**, alt sistem entegrasyon riski → **waterfall**.
  - **4. Planning:** gözden geçir, **bir döngü daha?** Evetse sonraki aşamayı planla.
- ⭐⭐ **Diğer modellerden ana fark: riskin açıkça tanınması (explicit recognition of risk)**.
- **Risk:** ters gidebilecek bir şey (ör. yeni dilin derleyicisi güvenilmez ya da verimsiz). Takvim ve maliyet aşımına yol açar, bu yüzden **risk azaltma** önemli yönetim faaliyeti (Bölüm 5).
- Döngü akışı: hedefler (performans, işlev) → alternatifler ve kısıtlar → değerlendirme, risk kaynakları → riskleri **analiz, prototipleme, simülasyon** ile çöz → geliştirme → sonraki aşamayı planla.
- Spiral, waterfall ve prototiplemeyi **içine alabilir** (3. sektörde seçilirler, alıştırma 4.3).

Sayfa 74–76 — 4.3 Process activities · 4.3.1 Software specification (Şekil 4.6)

📖 Sayfanın özeti

4.3 SÜREÇ FAALİYETLERİ

Dört temel faaliyet farklı süreçlerde farklı düzenlenir: waterfall'da **sıralı**, evrimsel geliştirmede **iç içe**. Nasıl yapılacakları yazılımın türüne, insanlara ve kurum yapısına bağlıdır. Doğru ya da yanlış bir düzenleme yoktur.

4.3.1 TANIMLAMA = GEREKSİNİM MÜHENDİSLİĞİ

Yazılım tanımlaması ya da **gereksinim mühendisliği**, sistemden hangi hizmetlerin beklendiğini ve çalışma ile geliştirme kısıtlarını anlama ve tanımlama sürecidir. Bu aşamadaki hatalar kaçınılmaz olarak sonraki tasarım ve gerçekleştirme sorunlarına yol açtığı için **özellikle kritik** bir aşamadır. Sonucunda sistemin tanımlaması olan bir **gereksinim belgesi** çıkar. Gereksinimler genelde iki ayrıntı seviyesinde sunulur: son kullanıcılar ve müşteriler **üst seviye** bir ifadeye, geliştiriciler daha **ayrıntılı** bir sistem tanımlamasına ihtiyaç duyar.



Şekil 4.6: Gereksinim mühendisliği süreci (alt satırlar her faaliyetin ürünü)

4 ANA AŞAMA

1) Feasibility study (fizibilite çalışması): kullanıcı ihtiyaçlarının mevcut yazılım ve donanım teknolojileriyle karşılanıp karşılanamayacağı tahmin edilir. Sistemin iş açısından **maliyet-etkin** olup olmadığı ve **bütçe** içinde yapılıp yapılamayacağı incelenir. **Ucuz ve hızlı** olmalı ve daha ayrıntılı analize geçilip geçilmeyeceği kararına yön vermelidir. **2) Requirements elicitation and analysis (gereksinim çıkarma ve analiz):** mevcut sistemleri gözlemleyerek, olası kullanıcı ve alıcılarla görüşerek, görev analiziyle gereksinimler çıkarılır. Analistin sistemi anlamasına yardım eden **sistem modelleri ve prototipler** geliştirilebilir. **3) Requirements specification (gereksinim tanımlama):** toplanan bilgi bir belgeye dönüştürülür. İki tür gereksinim içerebilir: **user requirements** (müşteri ve son kullanıcı için soyut ifadeler) ve **system requirements** (sağlanacak işlevin daha ayrıntılı tanımı). **4) Requirements validation (gereksinim geçerleme):** gereksinimler **gerçekçilik, tutarlılık ve eksiksizlik** açısından kontrol edilir. Belgedeki hatalar kaçınılmaz olarak bulunur ve düzeltilir.

SAYFA 76: SIRA KATI DEĞİL

Bu faaliyetler katı bir sırayla yapılmaz: analiz, tanımlama ve belgeleme sırasında da devam eder, yeni gereksinimler süreç boyunca ortaya çıkar. Yani analiz, tanım ve tanımlama **iç içedir**. Extreme programming gibi **agile** yöntemlerde gereksinimler kullanıcı önceliklerine göre **artımlı** geliştirilir ve geliştirme ekibinin parçası olan kullanıcılardan çıkarılır.

👨‍🏫 Hoca anlatımı

Gereksinim mühendisliği = "müşteri tam olarak ne istiyor?" sorusunu cevaplamak. En kritik aşamadır, çünkü burada yapılan hata bütün projeye yayılır. Yanlış bir binanın temelini atarsan üstüne ne kadar güzel kat çıksan da bina yanlış olur.

4 aşamayı bir düğün salonu kurmak isteyen girişimci örneğiyle düşün: (1) **Feasibility**, "bu iş kâr eder mi, param yeter mi?" Hızlı ve ucuz bir ön araştırma. (2) **Elicitation & analysis**, gelin ve damat adaylarıyla konuşmak, başka salonları gezmek. (3) **Specification**, toplanan bilgiyi yazıya dökmek. (4) **Validation**, "yazdıklarım gerçekçi mi, birbiriyle çelişiyor mu, eksik var mı?" diye kontrol etmek.

User ve system requirements farkı (alıştırma 4.5): User requirements müşteri için yazılır, sade ve soyuttur ("Sistem müşteri faturalarını göstermeli"). System requirements geliştirici için yazılır, ayrıntılı ve tekniktir ("Fatura ekranı son 24 ayı listeler, 2 saniyede açılır..."). İki farklı okuyucu olduğu için iki seviye gerekir. Bu konu Bölüm 6'da ayrıntılı gelecek.

Validation'daki **3 kriteri** unutma: **realism, consistency, completeness** (gerçekçilik, tutarlılık, eksiksizlik).

📝 Deftere not

- 4 faaliyet farklı süreçlerde farklı düzenlenir: waterfall'da **sıralı**, evrimselde **iç içe**. Doğru ya da yanlış düzenleme yok.
- ⭐ **Software specification = requirements engineering:** beklenen hizmetleri ve çalışma ile geliştirme kısıtlarını anlama ve tanımlama.
- ⭐ **Özellikle kritik** aşama: buradaki hatalar sonraki tasarım ve kodlama sorunlarına yol açar.
- Çıktı: **gereksinim belgesi**. İki ayrıntı seviyesi: müşteri ve kullanıcı için **üst seviye**, geliştirici için **ayrıntılı**.
- ⭐⭐ **Şekil 4.6, 4 aşama:**
  - Feasibility study: mevcut teknolojiyle karşılanabilir mi? **Maliyet-etkin** mi? **Bütçe** içinde mi? **Ucuz ve hızlı** olmalı. Çıktı: **feasibility report**.
  - Requirements elicitation & analysis: mevcut sistemleri gözlem, kullanıcı ve alıcılarla görüşme, görev analizi. **Sistem modelleri ve prototipler**.
  - Requirements specification: bilgiyi belgeye dönüştür. **User requirements** (soyut, müşteri ve kullanıcı için) + **system requirements** (ayrıntılı işlev).
  - Requirements validation: **realism, consistency, completeness** kontrolü. Hatalar bulunur ve düzeltilir.
- Faaliyetler **katı sırayla değil**, iç içe. Yeni gereksinimler süreç boyunca çıkar.
- **Agile (XP):** gereksinimler kullanıcı önceliklerine göre **artımlı**, ekipteki kullanıcılardan çıkarılır.

Sayfa 76–79 4.3.2 Software design and implementation (Şekil 4.7 ve 4.8)

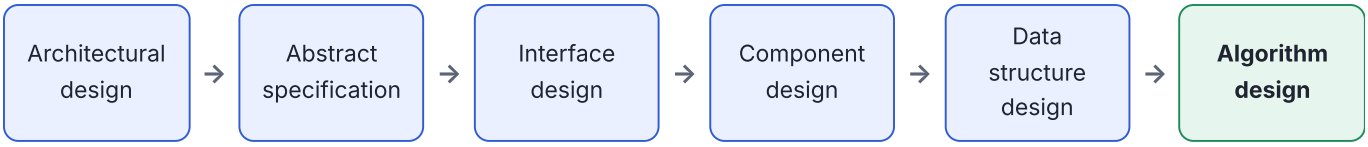
📖 Sayfanın özeti

GERÇEKLEŞTİRME VE TASARIM

Gerçekleştirme aşaması, sistem tanımlamasını **çalıştırılabilir bir sisteme** dönüştürme sürecidir. Her zaman tasarım ve programlama içerir, evrimsel yaklaşımda tanımlamanın iyileştirilmesini de içerebilir. **Yazılım tasarımı**; gerçekleştirilecek yazılımın yapısının, sistemin parçası olan verinin, bileşenler arası arayüzlerin ve bazen kullanılan algoritmaların tanımıdır. Tasarım bir anda bitmez, **yinelemeli** olarak birkaç sürümle geliştirilir, giderek daha resmi ve ayrıntılı hale gelir ve sürekli geri dönülerek düzeltilir. Farklı soyutlama seviyelerinde modeller geliştirilebilir. Şekil 4.7 aşamaları sıralı gösterse de gerçekte **iç içedir** ve geri bildirim ile yeniden çalışma kaçınılmazdır.

TASARIMIN 6 FAALİYETİ (ŞEKİL 4.7)

Her faaliyetin çıktısı bir sonraki aşamanın tanımlamasıdır ve giderek ayrıntılanır. **1) Architectural design:** sistemi oluşturan alt sistemler ve ilişkileri belirlenip belgelenir → *system architecture*. **2) Abstract specification:** her alt sistemin hizmetleri ve kısıtları için soyut bir tanımlama → *software specification*. **3) Interface design:** her alt sistemin diğer alt sistemlerle arayüzü tasarlanır. Bu tanımlama, alt sistemin iç işleyişi bilinmeden kullanılabilmesi için **belirsizlik içermemelidir**. Formal tanımlama yöntemleri kullanılabilir → *interface specification*. **4) Component design:** hizmetler bileşenlere dağıtılır ve bileşen arayüzleri tasarlanır → *component specification*. **5) Data structure design:** veri yapıları ayrıntılı tasarlanır → *data structure specification*. **6) Algorithm design:** algoritmalar ayrıntılı tasarlanır → *algorithm specification*.



Şekil 4.7: Tasarım süreci (her faaliyet bir "specification" ürünü verir)

UYARLAMALAR VE AGİLE

Bu genel model pratikte uyarlanır: son iki aşama (veri yapısı ve algoritma) kodlamaya ertelenebilir, keşifsel yaklaşımda arayüzler veri yapılarından sonra tasarlanabilir, soyut tanımlama atlanabilir (ama kritik sistemlerde genelde şarttır). Agile yöntemlerde tasarım çıktıları ayrı belgeler değil, **program kodunun** kendisidir. Mimariden sonraki aşamalar artımlıdır.

YAPILANDIRILMIŞ YÖNTEMLER (STRUCTURED METHODS)

Karşıt yaklaşım, sistemin **grafiksel modellerini** üretip çoğu zaman bunlardan otomatik kod üretmeye dayanır. **1970'lerde** fonksiyon odaklı tasarım için icat edildiler. Nesne yönelimli yöntemler **1990'larda UML** altında birleşti. Bir yapılandırılmış yöntem; tasarım süreç modeli, gösterimler, rapor formatları, kurallar ve tasarım yönergeleri içerir. Desteklenebilecek modeller: **object model** (nesne sınıfları ve bağımlılıkları), **sequence model** (nesnelerin çalışma sırasında etkileşimi), **state transition model** (sistem durumları ve geçiş tetikleyicileri), **structural model** (bileşenler ve birleşimleri), **data-flow model** (veri dönüşümleri: nesne yönelimli yöntemlerde pek kullanılmaz, gerçek zamanlı ve iş sistemlerinde sık kullanılır).

SAYFA 79: PROGRAMLAMA VE DEBUGGİNG (ŞEKİL 4.8)

Yapılandırılmış yöntemler aslında standart gösterimler ve iyi uygulama örnekleridir. Yönergelere uymak makul bir tasarım getirir ama yaratıcılık hâlâ gerekir. Tasarımcılar yöntemleri **harfiyen izlemez**, duruma göre seçerler. Programlama tasarımın doğal devamıdır. Emniyet-kritik sistemler kodlamadan önce ayrıntılı tasarlanır, ama genelde sonraki tasarım aşamaları ile programlama iç içedir. CASE araçları tasarımdan iskelet program üretebilir. **Programlama kişisel bir faaliyettir** ve genel bir süreç yoktur (kimi iyi bildiği bileşenden başlar, kimi en sona bırakır). Programcılar kendi kodlarını test eder. **Test, kusurların varlığını gösterir. Debugging ise kusurları bulup düzeltmektir.** Debugging süreci (Şekil 4.8): **Locate error → Design error repair → Repair error → Retest program**. Debugging sırasında gözlemlenen davranış hakkında **hipotezler** kurulup test edilir. Kod elle izlenebilir, yeni test durumları yazılabilir, değişken değerlerini ve çalışan komutların izini gösteren etkileşimli hata ayıklama araçları kullanılabilir.

👩‍🏫 Hoca anlatımı

Tasarımın 6 faaliyeti büyükten küçüğe gider, bir şehir planlaması gibi: (1) **Mimari**, şehrin mahallelerini belirle. (2) **Soyut tanımlama**, her mahalle ne işe yarayacak? (3) **Arayüz**, mahalleler arası yollar nasıl bağlanacak? (4) **Bileşen**, mahalledeki binalar. (5) **Veri yapısı**, binaların içindeki odalar. (6) **Algoritma**, odada işler nasıl yapılacak? Sınavda sırayla yazmanı isteyebilirler (alıştırma 4.6).

Interface design'da önemli bir nokta: arayüz tanımlaması **belirsizlik içermemeli**, çünkü başka ekipler alt sistemin içini bilmeden sadece arayüze bakarak onu kullanacak.

5 model türü (structured methods): **object** (hangi sınıflar var), **sequence** (kim kime hangi sırayla mesaj gönderiyor), **state transition** (sistem hangi durumlardan hangisine geçiyor), **structural** (parçalar nasıl birleşiyor), **data-flow** (veri nasıl dönüşüyor). Bunlar Bölüm 8'de ayrıntılı gelecek.

Test ve debugging farkı çok sorulur: **Test** = "hata var mı?" (alarm çalar). **Debugging** = "hata nerede ve nasıl düzeltirim?" (dedektif işi). Debugging'in 4 adımı: **bul → onarımı tasarla → onar → tekrar test et**.

📝 Deftere not

- **Implementation:** sistem spec'ini **çalıştırılabilir sisteme** dönüştürmek. Her zaman **tasarım + programlama**. Evrimselde spec iyileştirmesi de olabilir.
- ⭐ **Software design:** yazılımın **yapısı**, **veri**, bileşenler arası **arayüzler** ve bazen **algoritmalar**. **Yinelemeli** geliştirilir, resmiyet ve ayrıntı artar. Aşamalar iç içe, geri bildirim kaçınılmaz.
- ⭐⭐ **Tasarımın 6 faaliyeti (Şekil 4.7) → ürünleri:**
  - **1. Architectural design** → system architecture (alt sistemler ve ilişkiler).
  - **2. Abstract specification** → software specification (her alt sistemin hizmetleri ve kısıtları).
  - **3. Interface design** → interface specification (alt sistemler arası arayüz, **belirsizlik içermemeli**, formal yöntem kullanılabilir).
  - **4. Component design** → component specification (hizmetler bileşenlere, bileşen arayüzleri).
  - **5. Data structure design** → data structure specification.
  - **6. Algorithm design** → algorithm specification.
- Uyarlamalar: (1) veri yapısı ve algoritma tasarımı **kodlamaya ertelenebilir**, (2) keşifsel yaklaşımda arayüzler veri yapılarından **sonra**, (3) soyut tanımlama **atlanabilir** (kritik sistemlerde değil).
- **Agile:** tasarım çıktısı = **program kodu** (ayrı belge yok). Mimariden sonrası artımlı.
- ⭐ **Structured methods:** grafiksel modeller + çoğu zaman **otomatik kod üretimi**. 1970'ler function-oriented, 1990'lar OO, sonra **UML**. İçerik: tasarım süreç modeli, gösterimler, rapor formatları, kurallar, yönergeler.
- ⭐⭐ **5 model: Object** (sınıflar ve bağımlılıklar) · **Sequence** (çalışırken nesne etkileşimi) · **State transition** (durumlar ve tetikleyiciler) · **Structural** (bileşenler ve birleşimleri) · **Data-flow** (veri dönüşümleri, OO'da az, real-time ve iş sistemlerinde çok).
- Tasarımcılar yöntemleri **harfiyen izlemez**, yaratıcılık gerekir.
- Emniyet-kritik sistemler kodlamadan **önce** ayrıntılı tasarlanır. Normalde tasarım ve programlama **iç içe**. CASE **iskelet program** üretebilir.
- ⭐ **Programlama kişisel** bir faaliyet, genel süreç yok.
- ⭐⭐ **Testing ≠ Debugging:** test **kusurların varlığını** gösterir. Debugging kusurları **bulup düzeltir**.
- ⭐ **Debugging (Şekil 4.8):** Locate error → Design error repair → Repair error → Retest program. **Hipotez** kur ve test et. Kodu elle izle, yeni test durumları, **etkileşimli debugger**.

Sayfa 80–82 — 4.3.3 Software validation (Şekil 4.9, 4.10) · 4.3.4 Software evolution (Şekil 4.11)

📖 Sayfanın özeti

V&V

Geçerleme, daha genel adıyla **doğrulama ve geçerleme (V&V)**, sistemin tanımlamasına uyduğunu ve müşterinin beklentilerini karşıladığını göstermeyi amaçlar. Her aşamada inceleme ve gözden geçirme (inspections, reviews) gibi kontroller içerir, ama maliyetin çoğu gerçekleştirmeden sonraki **testte** oluşur. Küçük programlar dışında sistemler tek parça halinde test edilmez.



Şekil 4.9: Test süreci (oklar geri dönüşü gösterir, süreç yinelemelidir)

3 TEST AŞAMASI

İdealde bileşen kusurları erken, arayüz sorunları entegrasyonda bulunur. Ama kusur bulundukça debugging gerekir ve diğer aşamalar tekrarlanabilir, yani süreç **yinelemelidir**. **1) Component (unit) testing:** bileşenler diğer bileşenlerden **bağımsız** olarak test edilir. Bileşen bir fonksiyon, nesne sınıfı ya da bunların tutarlı bir grubu olabilir. **2) System testing:** bileşenler birleştirilir. Beklenmedik etkileşimlerden doğan hatalar ve **arayüz sorunları** aranır, fonksiyonel ve fonksiyonel olmayan gereksinimler ile **emergent özellikler** test edilir. Büyük sistemlerde önce alt sistemler, sonra son sistem test edilir. **3) Acceptance testing:** sistem kullanıma kabul edilmeden önceki **son aşama**. Simüle veri yerine **müşterinin sağladığı gerçek veriyle** test edilir. Gerçek veri sistemi farklı şekillerde zorladığı için gereksinimlerdeki hata ve eksikleri, işlevlerin kullanıcı ihtiyacını karşılamadığı ya da performansın kabul edilemez olduğu durumları ortaya çıkarabilir.

TEST PLANLARI (ŞEKİL 4.10, V-MODELİ)

Normalde bileşen geliştirme ve testi **iç içedir**: programcı kodu en iyi bildiği için kendi test verisini üretir. Artımlı yaklaşımda her artım geliştirildikçe kendi gereksinimlerine göre test edilir. **XP**'de testler geliştirme başlamadan önce gereksinimlerle birlikte yazılır. Sonraki test aşamaları önceden planlanmalı ve **bağımsız bir test ekibi**, tanımlama ve tasarımdan çıkarılan test planlarıyla çalışmalıdır. Şekil 4.10'da (V şekli) her geliştirme aşaması bir test planı üretir: requirements specification → **acceptance test plan**, system specification → **system integration test plan**, system design → **sub-system integration test plan**. Detailed design → module and unit code and test. Sonra sırasıyla sub-system integration test → system integration test → acceptance test → service.

ALPHA VE BETA TESTİ

Kabul testine bazen **alpha testing** denir. Tek müşteri için yapılan özel sistemlerde, geliştirici ile müşteri sistemin kabul edilebilir olduğunda anlaşana kadar sürer. Sistem bir **yazılım ürünü** olarak piyasaya sunulacaksa **beta testing** kullanılır: sistem, kullanmayı kabul eden birkaç potansiyel müşteriye verilir, onlar sorunları bildirir. Bu, ürünü gerçek kullanıma açar ve geliştiricilerin öngörmediği hataları bulur. Geri bildirimden sonra sistem değiştirilip ya yeni bir beta testine ya da genel satışa çıkarılır.

4.3.4 YAZILIM EVRİMİ

Yazılımın **esnekliği**, büyük sistemlere giderek daha çok yazılım eklenmesinin ana nedenlerinden biridir. Donanım alındıktan sonra tasarımını değiştirmek çok pahalıdır, yazılım ise geliştirme sırasında ya da sonrasında her zaman değiştirilebilir ve büyük değişiklikler bile donanımdakinden çok daha ucuzdur. Tarihsel olarak geliştirme ve evrim (bakım) ayrı görülmüştür: geliştirme yaratıcı, bakım sıkıcı sayılır. Oysa bakım maliyetleri çoğu zaman ilk geliştirme maliyetinin **birkaç katıdır**. Bu ayrım giderek **anlamsızlaşıyor**, çünkü artık çok az sistem tamamen yenidir. Geliştirme ve bakımı bir **süreklilik (continuum)** olarak görmek daha mantıklıdır. Yazılım mühendisliği, yazılımın ömrü boyunca sürekli değiştiği **evrimsel bir süreç** olarak düşünülmelidir (Şekil 4.11: Define system requirements → Assess existing systems → Propose system changes → Modify systems → New system).

👨‍🏫 Hoca anlatımı

3 test aşaması araba üretimi gibidir: (1) **Component testing**, her parçayı (motor, fren, far) ayrı ayrı test et. (2) **System testing**, parçaları birleştirip arabayı bir bütün olarak test et (motor ile fren uyumlu mu?). (3) **Acceptance testing**, müşteri arabayı kendi gerçek yolunda sürer ve "tamam, alıyorum" der.

Acceptance testing'in farkı **gerçek veriyle** yapılması. Laboratuvarda her şey güzel çalışır ama gerçek hayat başkadır.

Alpha ve beta farkı sınavda sorulabilir: **Alpha**, tek müşteriye özel sistemde müşteriyle birlikte yapılan kabul testi. **Beta**, piyasaya çıkacak ürünü önce gönüllü kullanıcılara vermek (oyunların "beta sürümü" gibi). Kullanıcılar hataları bildirir, sonra ürün düzeltilip satışa çıkar.

Şekil 4.10 (V modeli) şunu söyler: her geliştirme aşamasında, ileride yapılacak testin planı da hazırlanır. Gereksinimleri yazarken kabul testinin planı, sistem tanımlamasını yazarken sistem entegrasyon testinin planı yazılır. Böylece test ekibi bağımsız çalışabilir.

Evrim kısmındaki ana fikir: "geliştirme bitti, şimdi bakım" diye bir ayrım artık yok. Yazılım **hiç bitmeyen** bir süreç gibi sürekli değişir. Ayrıca bakım, geliştirmeden **daha pahalıdır**.

📝 Deftere not

- ⭐ **V&V (verification & validation):** sistem **spec'ine uyuyor mu** ve **müşteri beklentisini karşılıyor mu**? Her aşamada inspection ve review var, ama maliyetin çoğu **gerçekleştirme sonrası testte**.
- Sistem tek parça (monolithic) test edilmez. Süreç **yinelemeli** (kusur bulununca debugging, aşamalar tekrar).
- ⭐⭐ **3 test aşaması (Şekil 4.9):**
  - Component (unit) testing: bileşenler **bağımsız** test edilir (fonksiyon, nesne sınıfı ya da grup).
  - System testing: birleştirilmiş sistem. **Beklenmedik etkileşimler, arayüz sorunları**, fonksiyonel ve fonksiyonel olmayan gereksinimler, **emergent özellikler**. Büyükte önce alt sistem, sonra sistem.
  - Acceptance testing: kullanıma kabulden önce **son aşama**. **Müşterinin gerçek verisiyle**. Gereksinim hata ve eksiklerini, ihtiyaç uyumsuzluğunu ve performans sorunlarını ortaya çıkarır.
- Bileşen geliştirme ve testi **iç içe** (programcı kendi test verisini üretir). **XP**: testler **geliştirmeden önce**, gereksinimlerle birlikte yazılır.
- ⭐ Sonraki test aşamaları **önceden planlanır**, **bağımsız test ekibi** çalışır.
- ⭐ **Şekil 4.10 (V-model):** Requirements spec → **Acceptance test plan** · System spec → **System integration test plan** · System design → **Sub-system integration test plan** · Detailed design → Module & unit code and test. Test sırası: sub-system integration test → system integration test → acceptance test → service.
- ⭐⭐ **Alpha testing:** kabul testi, **tek müşteriye özel** sistemde, müşteri ve geliştirici anlaşana kadar.
- ⭐⭐ **Beta testing: piyasaya** çıkacak üründe, birkaç **potansiyel müşteriye** verilir, sorun bildirirler, düzeltilir, yeni beta ya da satış.
- ⭐ **4.3.4 Evolution:** yazılımın **esnekliği** büyük sistemlerde kullanılmasının ana nedeni. Donanım değişikliği çok pahalı, yazılım her zaman değiştirilebilir.
- Geliştirme (yaratıcı) ve bakım (sıkıcı) ayrımı tarihsel. Ama **bakım maliyeti ilk geliştirmenin birkaç katı**.
- ⭐ Ayrım artık **anlamsız**: geliştirme ve bakım bir **continuum**. SE = **evrimsel süreç**. Şekil 4.11: Define system requirements → Assess existing systems → Propose system changes → Modify systems → New system.

Sayfa 82–85 4.4 The Rational Unified Process (RUP) · Şekil 4.12 ve 4.13

📖 Sayfanın özeti

RUP NEDİR?

UML ve Unified Software Development Process çalışmalarından türetilmiş **modern** bir süreç modelidir. **Hibrit** bir modeldir: genel süreç modellerinin öğelerini birleştirir, **yinelemeyi** destekler ve tanımlama ile tasarımda iyi uygulamaları gösterir. Geleneksel modeller sürece tek bir açıdan bakarken RUP **üç bakış açısıyla** anlatılır: **dynamic** (zaman içindeki fazlar), **static** (yapılan süreç faaliyetleri) ve **practice** (önerilen

iyi uygulamalar). Çoğu anlatım statik ve dinamik bakışı tek diyagramda birleştirir, yazar anlaşılmasını kolaylaştırmak için ayrı anlatıyor.

DİNAMİK BAKIŞ: 4 FAZ (ŞEKİL 4.12)

RUP **fazlı** bir modeldir. Ama waterfall'dan farklı olarak fazlar teknik faaliyetlere değil, **iş (business) konularına** bağlıdır. **1) Inception (başlangıç):** sistem için bir **iş gerekçesi (business case)** oluşturulur. Sistemle etkileşecek bütün dış varlıklar (insanlar ve sistemler) belirlenir ve etkileşimler tanımlanır. Bu bilgiyle sistemin işe katkısı değerlendirilir. Katkı küçükse proje bu fazdan sonra **iptal edilebilir**. **2) Elaboration (detaylandırma):** problem alanı anlaşılır, **mimari çerçeve** kurulur, proje planı yapılır ve temel **riskler** belirlenir. Sonunda bir gereksinim modeli (**UML use case**'leri), mimari tanım ve geliştirme planı olmalıdır. **3) Construction (inşa):** esas olarak tasarım, programlama ve test. Parçalar paralel geliştirilip entegre edilir. Sonunda kullanıcılara teslime hazır, belgeleri olan **çalışan bir yazılım** olmalıdır. **4) Transition (geçiş):** sistem geliştirme topluluğundan **kullanıcı topluluğuna** taşınır ve gerçek ortamda çalıştırılır. Çoğu modelde göz ardı edilen ama pahalı ve bazen sorunlu bir faaliyettir. Sonunda gerçek ortamında doğru çalışan, belgelenmiş bir sistem olmalıdır.



Şekil 4.12: RUP fazları (↻ = faz içi yineleme; ayrıca Transition'dan Inception'a dönen faz yinelemesi)

YİNELEME İKİ ŞEKİLDE

(1) Her faz kendi içinde yinelemeli yürütülüp sonuçlar artımlı geliştirilebilir. (2) Bütün faz kümesi de artımlı olarak tekrarlanabilir (Transition'dan Inception'a dönen ok).

STATİK BAKIŞ: İŞ AKIŞLARI (ŞEKİL 4.13)

Geliştirme sırasında yapılan faaliyetlere RUP'ta **workflow** denir. **6 temel** ve **3 destekleyici** iş akışı vardır. RUP UML ile birlikte tasarlandığı için iş akışları UML modellerine göre düzenlenmiştir.

Workflow	Kısaca
Business modelling	İş süreçleri **business use case**'lerle modellenir.
Requirements	Sistemle etkileşen **aktörler** belirlenir, gereksinimleri modellemek için **use case**'ler geliştirilir.
Analysis and design	Mimari, bileşen, nesne ve sıralama (sequence) modelleriyle tasarım modeli oluşturulur.
Implementation	Bileşenler gerçekleştirilip alt sistemlere yapılandırılır. Tasarım modellerinden **otomatik kod üretimi** süreci hızlandırır.
Testing	Gerçekleştirmeyle birlikte yürüyen yinelemeli süreç. Gerçekleştirme bitince sistem testi yapılır.
Deployment	Ürün sürümü oluşturulur, kullanıcılara dağıtılır ve iş yerlerine kurulur.
Configuration and change management (destekleyici)	Sistemdeki değişiklikleri yönetir (Bölüm 29).
Project management (destekleyici)	Sistem geliştirmeyi yönetir (Bölüm 5).
Environment (destekleyici)	Ekibin uygun yazılım araçlarına sahip olmasını sağlar.

NEDEN İKİ AYRI BAKIŞ?

Fazlar belirli iş akışlarına bağlı değildir. İlke olarak bütün iş akışları her fazda aktif olabilir. Tabii ki ilk fazlarda en çok emek iş modelleme ve gereksinimlere, son fazlarda test ve kuruluma harcanır.

PRATİK BAKIŞ: 6 İYİ UYGULAMA

1) Develop software iteratively: müşteri önceliklerine göre artımlar planla, en öncelikli özellikleri erken teslim et. **2) Manage requirements:** müşteri gereksinimlerini açıkça belgele, değişiklikleri izle, değişikliklerin etkisini kabul etmeden önce analiz et. **3) Use component-based architectures:** mimariyi bileşenler halinde yapılandır. **4) Visually model software:** statik ve dinamik görünümler için grafiksel **UML** modelleri kullan. **5) Verify software quality:** yazılımın kurumun kalite standartlarını karşıladığından emin ol. **6) Control changes to software:** değişiklikleri bir değişiklik yönetim sistemi ve konfigürasyon yönetimi prosedürleriyle yönet.

SAYFA 85: DEĞERLENDİRME

RUP her tür geliştirme için uygun değildir, ama yeni nesil genel süreçleri temsil eder. En önemli yenilikleri: **fazların ve iş akışlarının ayrılması** ve yazılımı kullanıcı ortamına **kurmanın (deployment) sürecin parçası** sayılması. Fazlar **dinamiktir ve hedefleri vardır**. İş akışları **statiktir**, tek bir faza bağlı olmayan teknik faaliyetlerdir ve her fazın hedeflerine ulaşmak için geliştirme boyunca kullanılabilir.

👨‍🏫 Hoca anlatımı

RUP sınavda çok sorulur! Üç şeyi bilmen lazım: **3 bakış açısı**, **4 faz** ve **6 iyi uygulama**. İş akışlarını da en azından adlarıyla bil.

4 fazı bir film çekimi gibi düşün: (1) **Inception**, "Bu film para kazandırır mı?" Senaryo fikri ve yatırımcı ikna etme. Kazandırmayacaksa proje iptal. (2) **Elaboration**, senaryo detaylanır, çekim planı yapılır, riskler belirlenir (ör. "oyuncu hastalanırsa ne yaparız?"). (3) **Construction**, filmin çekilmesi ve montajı. (4) **Transition**, filmin sinemalara dağıtılması ve izleyiciye ulaşması.

Kısaltma: **I-E-C-T** ("İnce Ekmek Çok Tatlı" diye ezberleyebilirsin 😄).

Waterfall ile en önemli farkı: waterfall'un aşamaları **teknik** (tasarım, kodlama, test), RUP'un fazları ise **iş odaklı** (iş gerekçesi, mimari, inşa, teslim). Ayrıca RUP'ta tasarım, kodlama, test gibi teknik işler (**workflow**) her fazda yapılabilir. Sadece oranları değişir.

Statik ve dinamik bakışın avantajı (alıştırma 4.4): Fazlar ile iş akışları birbirinden ayrı olduğu için her iş akışı her fazda kullanılabilir. Bu da süreci esnek yapar. Bir projenin "ne zaman, hangi aşamada" olduğunu (dinamik) ve "ne iş yapıldığını" (statik) ayrı ayrı görebilirsin.

6 iyi uygulama için kısa hatırlatıcı: **iterative, requirements, components, visual (UML), quality, change control**.

📝 Deftere not

- ⭐ **RUP:** UML ve Unified Process'ten türemiş **modern, hibrit** süreç modeli. Genel modellerin öğelerini birleştirir, **yinelemeyi** destekler, iyi uygulamaları gösterir.
- ⭐⭐ **3 bakış açısı: Dynamic** (zaman içindeki fazlar) · **Static** (faaliyetler, yani workflows) · **Practice** (iyi uygulamalar).
- ⭐ Fazlı model ama fazlar **iş konularına** bağlı, **teknik değil** (waterfall'dan farkı).
- ⭐⭐ **4 faz (Şekil 4.12):**
  - Inception: **business case**. Dış varlıklar (insan, sistem) ve etkileşimler belirlenir. Katkı küçükse **iptal**.
  - Elaboration: problem alanını anla, **mimari çerçeve**, proje planı, **temel riskler**. Çıktı: gereksinim modeli (**UML use cases**), mimari tanım, geliştirme planı.

- Construction: tasarım, programlama, test. Paralel geliştirme ve entegrasyon. Çıktı: **çalışan yazılım** + belgeler.
- Transition: sistemi **kullanıcı topluluğuna** taşı, gerçek ortamda çalıştır (çoğu modelde göz ardı edilir, pahalı). Çıktı: gerçek ortamda doğru çalışan belgeli sistem.

- Yineleme: (1) **faz içi** yineleme, (2) **bütün fazların** artımlı tekrarı (Transition → Inception).
- ⭐ **Statik bakış = workflows (Şekil 4.13): 6 temel + 3 destekleyici.**
  - Temel: **Business modelling** (business use cases) · **Requirements** (aktörler, use cases) · **Analysis & design** (mimari, bileşen, nesne, sequence modelleri) · **Implementation** (bileşenler, otomatik kod üretimi) · **Testing** (yinelemeli, gerçekleştirmeyle birlikte) · **Deployment** (sürüm, dağıtım, kurulum).
  - Destekleyici: **Configuration & change management** · **Project management** · **Environment** (araçlar).
- ⭐ Fazlar iş akışlarına **bağlı değil**. Her iş akışı her fazda aktif olabilir (başta iş modelleme ve gereksinim, sonda test ve kurulum ağırlıklı).
- ⭐⭐ **6 iyi uygulama (practice):**
  - 1. **Develop software iteratively** (müşteri önceliğine göre artımlar).
  - 2. **Manage requirements** (belgele, değişiklikleri izle, etkisini analiz et).
  - 3. **Use component-based architectures.**
  - 4. **Visually model software** (UML).
  - 5. **Verify software quality** (kurum standartları).
  - 6. **Control changes to software** (değişiklik ve konfigürasyon yönetimi).
- ⭐ RUP her geliştirme için uygun değil, ama yeni nesil. **Yenilikleri: fazlar ve iş akışlarının ayrılması · deployment'ın sürecin parçası** olması. Fazlar = **dinamik**, hedefli. Workflows = **statik**, teknik, her fazda kullanılır.

Sayfa 85–89 4.5 Computer-Aided Software Engineering (CASE) · Şekil 4.14, 4.15, 4.16

📖 Sayfanın özeti

CASE NEDİR?

Gereksinim mühendisliği, tasarım, program geliştirme ve test gibi süreç faaliyetlerini desteklemek için kullanılan yazılımlardır: tasarım editörleri, veri sözlükleri, derleyiciler, hata ayıklayıcılar, sistem oluşturma araçları vb. CASE, bazı faaliyetleri **otomatikleştirerek** ve geliştirilen yazılım hakkında **bilgi sağlayarak** destek verir. Otomatikleştirilebilecek faaliyetlere örnekler: (1) gereksinim tanımlaması ya da tasarımın parçası olarak **grafiksel sistem modelleri** geliştirmek, (2) tasarımdaki varlıklar ve ilişkiler hakkında bilgi tutan bir **veri sözlüğüyle (data dictionary)** tasarımı anlamak, (3) kullanıcının etkileşimli oluşturduğu grafik tanımdan **kullanıcı arayüzü üretmek**, (4) çalışan program hakkında bilgi vererek **debugging**, (5) programları COBOL gibi eski bir dil sürümünden yenisine **otomatik çevirmek**.

BEKLENTİ VE GERÇEK

CASE artık çoğu rutin faaliyet için var ve kalite ile verimliliği bir miktar artırdı, ama ilk savunucuların beklediğinden az. Kat kat iyileşme beklenirken gerçekte iyileşme **yaklaşık %40** civarında kaldı (Huff, 1992). İyileşmeyi sınırlayan **iki faktör** vardır: (1) yazılım mühendisliği temelde **yaratıcı düşünceye** dayalı bir tasarım faaliyetidir. CASE rutin işleri otomatikleştirir, tasarımı desteklemek için yapay zekâ kullanma girişimleri başarılı olmamıştır. (2) Yazılım mühendisliği bir **ekip faaliyetidir** ve mühendisler çok zaman ekip üyeleriyle etkileşerek geçirir. CASE bunu pek desteklemez.

4.5.1 SINIFLANDIRMA: 3 BAKIŞ AÇISI

Functional perspective (araçların belirli işlevine göre), **process perspective** (destekledikleri süreç faaliyetlerine göre) ve **integration perspective** (bir ya da daha çok faaliyeti destekleyen entegre birimlere nasıl düzenlendiklerine göre).

ŞEKİL 4.14: İŞLEVE GÖRE SINIFLANDIRMA

Araç türü	Örnekler
Planning tools	PERT araçları, tahmin araçları, tablolama programları
Editing tools	Metin editörleri, diyagram editörleri, kelime işlemciler
Change management tools	Gereksinim izlenebilirlik araçları, değişiklik kontrol sistemleri
Configuration management tools	Sürüm yönetim sistemleri, sistem oluşturma araçları
Prototyping tools	Çok yüksek seviyeli diller, kullanıcı arayüzü üreticileri
Method-support tools	Tasarım editörleri, veri sözlükleri, kod üreticiler
Language-processing tools	Derleyiciler, yorumlayıcılar
Program analysis tools	Çapraz referans üreticileri, statik ve dinamik analizciler
Testing tools	Test verisi üreticileri, dosya karşılaştırıcılar
Debugging tools	Etkileşimli hata ayıklama sistemleri
Documentation tools	Sayfa düzeni programları, resim editörleri
Re-engineering tools	Çapraz referans sistemleri, program yeniden yapılandırma sistemleri

ŞEKİL 4.15: FAALİYETE GÖRE SINIFLANDIRMA

Hangi araç türünün hangi süreç aşamasını (specification, design, implementation, V&V) desteklediğini gösterir. **Planlama, düzenleme (editing), değişiklik yönetimi ve belgeleme** araçları **bütün aşamalarda** kullanılır. Örneğin test, debugging ve program analizi araçları gerçekleştirme ile V&V'de, yöntem destek ve prototipleme araçları ise tanımlama ve tasarımda öne çıkar.

DESTEK GENİŞLİĞİNE GÖRE: FUGGETTA'NIN 3 SINIFI

Tools: tasarım tutarlılığını kontrol etmek, program derlemek, test sonuçlarını karşılaştırmak gibi **tek tek süreç görevlerini** destekler. Genel amaçlı bağımsız araçlar olabilir (kelime işlemci) ya da tezgâhlarda gruplanabilir. **Workbenches:** tanımlama ya da tasarım gibi **süreç aşamalarını ya da faaliyetlerini** destekler. Genelde az ya da çok entegre bir araç kümesidir. **Environments:** yazılım sürecinin **tamamını ya da önemli bir kısmını** destekler. Genelde birkaç entegre tezgâh içerir.

ŞEKİL 4.16 VE SINIRLARIN BULANIKLIĞI

Şekil 4.16 bu sınıfları örneklerle gösterir. **Tools:** editörler, derleyiciler, dosya karşılaştırıcılar. **Workbenches:** analiz ve tasarım (çok yöntemli ve tek yöntemli), programlama, test (genel amaçlı ve dile özgü). **Environments: integrated environments** (veri, kontrol ve sunum entegrasyonu için altyapı sağlar) ve **process-centred environments** (süreç bilgisi ve süreç modelini kullanarak mühendislere hangi aracı ya da tezgâhı ne zaman kullanacaklarını öneren bir süreç motoru içerir, daha geneldir). Genel amaçlı araçları ne zaman kullanacağına mühendis karar verir. Tezgâhlar ise genelde bir süreç modeli ve kurallar içeren bir **yöntemi** destekler. Pratikte sınırlar bulanıktır: kelime işlemcilerin içinde diyagram editörü vardır, tasarım tezgâhları programlama ve testi de destekler. Yine de sınıflandırma, bir aracın ne kadar süreç desteği verdiğini anlamada faydalı ilk adımdır.

👨‍🏫 Hoca anlatımı

CASE'i Bölüm 1'den hatırla: **yazılım yapmaya yardım eden yazılımlar**. Bu bölüm onları sınıflandırıyor.

Neden beklenen devrim olmadı? Önce %40 rakamını hatırla, sonra iki nedeni: (1) Yazılım **yaratıcı** bir iştir, bilgisayar yaratıcılığı otomatikleştiremez. (2) Yazılım **ekip işidir**, CASE insanlar arasındaki iletişime yardım etmez.

Tools, workbenches, environments farkını bir marangozla anlat: **Tool** = tek bir alet (çekiç, testere), tek bir iş yapar. **Workbench** = marangoz tezgâhı, bir işin (ör. masa yapımı) bütün aletleri bir arada. **Environment** = bütün atölye, mobilyanın baştan sona yapımını destekleyen birden çok tezgâh.

Yazılımdan örnek: **Tool** = derleyici. **Workbench** = bir programlama IDE'si (editör + derleyici + debugger). **Environment** = baştan sona tüm süreci destekleyen entegre platform.

Şekil 4.14'teki tabloyu çok iyi ezberlemen gerekmez ama birkaç örnek bil: planning → PERT, prototyping → UI generators, language-processing → compilers, testing → test data generators.

📝 Deftere not

- ⭐ **CASE:** süreç faaliyetlerini (gereksinim, tasarım, programlama, test) destekleyen yazılım. Ör: tasarım editörü, **data dictionary**, derleyici, debugger, sistem oluşturma araçları.
- Destek şekli: faaliyetleri **otomatikleştirme** + yazılım hakkında **bilgi sağlama**.
- ⭐ **Otomatikleştirilebilecek 5 faaliyet:** grafiksel sistem modelleri · data dictionary ile tasarımı anlama · grafik tanımdan **UI üretimi** · çalışan program bilgisiyle **debugging** · eski dilden (COBOL) yeniye **otomatik çeviri**.
- ⭐ İyileşme beklenenden az: **~%40** (Huff 1992).
- ⭐⭐ **İyileşmeyi sınırlayan 2 faktör:** (1) SE **yaratıcı düşünceye** dayalı tasarım işi (yapay zekâ ile tasarım desteği başarısız). (2) SE bir **ekip faaliyeti**, CASE ekip etkileşimini pek desteklemez.
- ⭐ **3 sınıflandırma bakışı: Functional** (işleve göre) · **Process** (desteklediği faaliyete göre) · **Integration** (entegre birimlere nasıl düzenlendiğine göre).
- ⭐ **Şekil 4.14 (işleve göre):** Planning (PERT, tahmin, tablolama) · Editing (metin ve diyagram editörü) · Change mgmt (izlenebilirlik, değişiklik kontrolü) · Configuration mgmt (sürüm yönetimi, system building) · Prototyping (çok yüksek seviyeli diller, UI üreticileri) · Method-support (tasarım editörü, data dictionary, kod üretici) · Language-processing (derleyici, yorumlayıcı) · Program analysis (cross-reference, statik ve dinamik analiz) · Testing (test verisi üretici, dosya karşılaştırıcı) · Debugging (etkileşimli debugger) · Documentation (sayfa düzeni, resim editörü) · Re-engineering (cross-reference, yeniden yapılandırma).
- **Şekil 4.15 (faaliyete göre):** planning, editing, change mgmt ve documentation **tüm aşamalarda** (spec, design, implementation, V&V).
- ⭐⭐ **Fuggetta'nın 3 sınıfı (destek genişliği):**
  - Tools: **tek tek görevler** (tutarlılık kontrolü, derleme, test sonucu karşılaştırma). Bağımsız ya da tezgâhta.
  - Workbenches: **süreç aşamaları ya da faaliyetleri** (spec, design). Entegre araç kümesi.
  - Environments: sürecin **tamamı ya da büyük kısmı**. Birkaç entegre tezgâh.
- **Şekil 4.16:** Tools (editors, compilers, file comparators) · Workbenches (analysis & design: multi-method ve single-method; programming; testing: general-purpose ve language-specific) · Environments (**integrated**: veri, kontrol, sunum entegrasyonu altyapısı · **process-centred**: süreç modeli ve **process engine** ile hangi aracın ne zaman kullanılacağını önerir).

- Pratikte sınırlar **bulanık** (kelime işlemcide diyagram editörü). Yine de sınıflandırma faydalı ilk adım.

Sayfa 89–91 Key Points · Further Reading · Exercises

📖 Sayfanın özeti

KEY POİNTS (BÖLÜM ÖZETİ)

Yazılım süreçleri yazılım üretmek için yapılan faaliyetlerdir, süreç modelleri bunların soyut temsilleridir. Bütün süreçler tanımlama, tasarım ve gerçekleştirme, geçerleme ve evrim içerir. Genel modeller (waterfall, evrimsel, CBSE) süreçlerin düzenini anlatır. Yinelemeli modeller süreci faaliyet döngüsü olarak sunar ve bir tanımlamaya ya da tasarıma **erken bağlanmayı (premature commitment)** önler. Örnekleri artımlı geliştirme ve spiral modeldir. Gereksinim mühendisliği tanımlama geliştirme sürecidir ve müşteri ihtiyaçlarını geliştiricilere iletmeyi amaçlar. Tasarım ve gerçekleştirme, tanımlamayı çalışan sisteme dönüştürür ve sistematik tasarım yöntemleri kullanılabilir. Geçerleme, sistemin tanımlamaya uyduğunu ve kullanıcıların gerçek ihtiyaçlarını karşıladığını kontrol eder. Evrim, mevcut sistemlerin yeni gereksinimlere göre değiştirilmesidir ve küçük ve orta sistemler için olağan yaklaşım haline geliyor. RUP, fazlara (inception, elaboration, construction, transition) ayrılmış ama faaliyetleri (gereksinim, analiz ve tasarım vb.) fazlardan ayıran modern bir genel modeldir. CASE süreçlere otomatik destek verir: araçlar tek tek faaliyetleri, tezgâhlar ilişkili faaliyet kümelerini, ortamlar sürecin tamamını ya da çoğunu destekler.

FURTHER READİNG

Beck'in *Extreme Programming Explained* kitabı, Krutchen'in RUP giriş kitabı, Ould'un süreç modelleri bölümü ve Fuggetta'nın CASE sınıflandırma makalesi. *Sınav için önemli değil.*

ALIŞTIRMALAR (KONULARI)

#	Konu
4.1	Şu sistemler için en uygun genel süreç modeli hangisi? Arabada **ABS fren kontrolü**, yazılım bakımı için **sanal gerçeklik** sistemi, eskisinin yerine geçecek **üniversite muhasebe** sistemi, istasyonlarda yolcuların tren saatlerini bulduğu **etkileşimli** sistem.
4.2	Evrimsel geliştirmeyle yapılan programların **bakımı neden zordur**?
4.3	Waterfall ve prototipleme modelleri **spiral modelin içine** nasıl sığar?
4.4	RUP'taki gibi **statik ve dinamik bakış** sunmanın avantajları nelerdir?
4.5	Gereksinim mühendisliğinde **user ve system requirements** ayrımı neden önemli?
4.6	Yazılım **tasarım sürecinin ana faaliyetleri** ve çıktıları nelerdir? Diyagramla ilişkilerini gösterin.
4.7	Bir tasarım yönteminin **5 bileşeni** nedir? (Bölüm 1'deki yöntem bileşenleriyle bağlantılı)
4.8	Sistem testlerini yürütüp sonuçlarını kaydeden bir süreç modeli tasarlayın.
4.9	Gerçek ortamda kullanılan yazılım neden **değişmek zorundadır**, yoksa giderek işe yaramaz hale gelir?
4.10	CASE sınıflandırması, CASE satın almaktan sorumlu yöneticilere nasıl yardımcı olur?
4.11	Kendi ortamınızdaki araçları işlev, faaliyet ve destek genişliğine göre sınıflandırın.
4.12	Gelişmiş CASE teknolojisi yazılımcıların işlerini elinden alır mı? Etik tartışma.

👨‍🏫 Hoca anlatımı

4.1 tipinde soru ("hangi model uygun?") sınavda çok popülerdir. Mantık şu:

ABS fren: emniyet-kritik, gereksinimler net, donanımla birlikte geliştirilir → **waterfall** (ya da formal geliştirme). **Sanal gerçeklik sistemi:** yeni, ne isteneceği belirsiz, arayüz ağırlıklı → **evrimsel (prototipleme)**. **Üniversite muhasebe sistemi (eskisinin yerine):** gereksinimler eski sistemden biliniyor → **waterfall** ya da **CBSE** (hazır muhasebe paketleri var). **Tren saati etkileşimli sistemi:** kullanıcı arayüzü kritik, kullanıcı tepkisi önemli → **evrimsel** (ya da artımlı).

4.9 (yazılım neden değişmeli): iş ortamı, yasalar, donanım ve kullanıcı ihtiyaçları değişir. Yazılım değişmezse gerçek dünyayla uyumu kaybolur ve işe yaramaz hale gelir.

Bu bölümün en önemli soruların cevap kalıpları bölüm sonunda.

📝 Deftere not

- ⭐ **KEY POINTS:** süreç = faaliyetler, model = soyut temsil · 4 faaliyet her süreçte · genel modeller: waterfall, evolutionary, CBSE · yinelemeli modeller **erken taahhüdü (premature commitment) önler**: incremental, spiral · RE = spec geliştirme (müşteri ihtiyacını geliştiriciye iletir) · design & implementation = spec'i çalışan sisteme dönüştür · validation = spec'e uyum + gerçek ihtiyaç · evolution = küçük ve orta sistemlerde olağan yaklaşım · RUP = fazlar ve faaliyetler ayrı · CASE: tools, workbenches, environments.
- ⭐ **Alıştırma 4.1 kısa cevaplar:** ABS → **waterfall** (net gereksinim, emniyet-kritik, donanımla) · Sanal gerçeklik → **evolutionary** (belirsiz gereksinim) · Üniversite muhasebe → **waterfall / CBSE** (gereksinimler biliniyor) · Tren saati sistemi → **evolutionary / incremental** (UI kritik).
- ⭐ Sınavda en olası: **4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.9**.

BÖLÜM 4 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 4'teki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
software process	yazılım süreci	Yazılım üreten faaliyetler kümesi	64
agile process	çevik süreç	Esnek, hızlı değişime uygun süreç	64
process standardisation	süreç standardizasyonu	Kurumdaki süreç çeşitliliğini azaltma	64
process paradigm	süreç paradigması	Genel süreç modeli (çerçeve)	65
formal system development	formal sistem geliştirme	Matematiksel model + dönüşümlerle koda	65–66
Cleanroom process	Cleanroom süreci	IBM'in formal süreci, kusur testi yok	66
B method	B yöntemi	Başka bir formal geliştirme yaklaşımı	66
waterfall model / software life cycle	şelale modeli / yaşam döngüsü	Ayrı, sıralı aşamalar (Royce 1970)	66
requirements analysis and definition	gereksinim analizi ve tanımı	Waterfall 1. aşama	67
unit testing	birim testi	Her birimin spec'ine uyduğunu doğrulama	67
operation and maintenance	işletim ve bakım	Waterfall'un en uzun aşaması	67

signed off	onaylanmış	Aşama belgelerinin imzalanması	67
premature freezing / commitment	erken dondurma / bağlanma	Kararları çok erken kesinleştirme	67, 89
inflexible partitioning	esnek olmayan bölümleme	Waterfall'un ana sorunu	67
evolutionary development	evrimsel geliştirme	İlk sürüm + kullanıcı yorumu + sürümler	68
concurrent activities	eşzamanlı faaliyetler	Aynı anda yürüyen faaliyetler	68
exploratory development	keşifsel geliştirme	Gereksinimleri keşfedip son sistemi teslim	68
throwaway prototyping	atılacak prototip	Gereksinimleri anlamak için prototip	68
visible process	görünür süreç	İlerlemesi ölçülebilen süreç	69
poorly structured	kötü yapılandırılmış	Sürekli değişiklikle bozulan yapı	69
reuse	yeniden kullanım	Var olan tasarım ya da kodu kullanma	69
CBSE	bileşen tabanlı YM	Hazır bileşenleri birleştirerek geliştirme	69–70
component analysis	bileşen analizi	Uygun bileşen arama	70
requirements modification	gereksinim değiştirme	Gereksinimleri bileşenlere uydurma	70
system design with reuse	yeniden kullanımlı tasarım	Çerçeveyi bileşenlere göre tasarlama	70

web services	web servisleri	CBSE'ye benzeyen servis tabanlı geliştirme	71
process iteration	süreç yinelemesi	Faaliyetlerin tekrarlanması	71
incremental delivery	artımlı teslim	Sistemi parça parça teslim etme	71
increment	artım	Sistemin teslim edilen bir parçası	72
extreme programming (XP)	uç programlama	Çok küçük artımlar, müşteri katılımı, pair programming	73
pair programming	eşli programlama	İki programcı tek bilgisayarda	73
spiral model	spiral model	Boehm 1988, risk odaklı döngüler	73
objective setting	hedef belirleme	Spiralin 1. sektörü	73
risk assessment and reduction	risk değerlendirme ve azaltma	Spiralin 2. sektörü	73
development and validation	geliştirme ve geçerleme	Spiralin 3. sektörü	73
planning	planlama	Spiralin 4. sektörü	73
risk	risk	Ters gidebilecek bir şey	73–74
requirements engineering	gereksinim mühendisliği	Hizmetleri ve kısıtları anlama ve tanımlama	75
feasibility study	fizibilite çalışması	Yapılabilir ve maliyet-etkin mi?	75
requirements elicitation and analysis	gereksinim çıkarma ve analiz	Gözlem, görüşme,	75

		görev analizi	
requirements specification	gereksinim tanımlama	Bilgiyi belgeye dökme	76
requirements validation	gereksinim geçerleme	Gerçekçilik, tutarlılık, eksiksizlik kontrolü	76
user / system requirements	kullanıcı / sistem gereksinimleri	Soyut (müşteri için) / ayrıntılı (geliştirici için)	76
requirements document	gereksinim belgesi	Gereksinim mühendisliğinin çıktısı	75
architectural design	mimari tasarım	Alt sistemler ve ilişkileri	77
abstract specification	soyut tanımlama	Alt sistem hizmetleri ve kısıtları	77
interface design	arayüz tasarımı	Alt sistemler arası arayüzler (belirsizlik olmamalı)	77
component design	bileşen tasarımı	Hizmetleri bileşenlere dağıtma	77
data structure / algorithm design	veri yapısı / algoritma tasarımı	Tasarımın son iki faaliyeti	77
structured methods	yapılandırılmış yöntemler	Grafiksel modeller + otomatik kod üretimi	78
object / sequence / state transition / structural / data-flow model	nesne / sıralama / durum geçiş / yapısal / veri akış modeli	Yapılandırılmış yöntemlerin modelleri	78
skeleton program	iskelet program	CASE'in tasarımdan ürettiği taslak kod	79
debugging	hata ayıklama	Kusurları bulup düzeltme	79

defect testing	kusur testi	Kusurların varlığını gösterme	79
hypothesis	hipotez	Debugging'de hatanın yeri hakkında tahmin	79
inspections / reviews	denetimler / gözden geçirmeler	Test dışı V&V kontrolleri	80
component (unit) testing	bileşen (birim) testi	Bileşenleri bağımsız test etme	80
system testing	sistem testi	Birleştirilmiş sistemi test etme	80
acceptance testing	kabul testi	Müşterinin gerçek verisiyle son test	80
test plan	test planı	Önceden hazırlanan test planı	81
alpha testing	alfa testi	Tek müşteriye özel sistemde kabul testi	81
beta testing	beta testi	Ürünü potansiyel müşterilere deneme için verme	81
software evolution / maintenance	yazılım evrimi / bakımı	Yazılımı sürekli değiştirme	81–82
continuum	süreklilik	Geliştirme ve bakımın kesintisiz bütünlüğü	82
Rational Unified Process (RUP)	Rational Birleşik Süreci	Modern, hibrit, fazlı süreç modeli	82
dynamic / static / practice perspective	dinamik / statik / uygulama bakışı	RUP'un 3 bakış açısı	82

inception	başlangıç	RUP 1. faz: business case	83
elaboration	detaylandırma	RUP 2. faz: mimari, plan, riskler	83
construction	inşa	RUP 3. faz: tasarım, kodlama, test	83
transition	geçiş	RUP 4. faz: kullanıcıya teslim	83
business case	iş gerekçesi	Sistemin işe katkısının gerekçesi	83
use case	kullanım durumu	UML'de gereksinim modelleme	83–84
workflow	iş akışı	RUP'ta faaliyet (6 temel + 3 destekleyici)	84
actor	aktör	Sistemle etkileşen kişi ya da sistem	84
deployment	kurulum, dağıtım	Ürünü kullanıcılara dağıtıp kurma	84
configuration and change management	konfigürasyon ve değişiklik yönetimi	Değişiklikleri yönetme	84
CASE	bilgisayar destekli YM	Süreci destekleyen yazılımlar	85
data dictionary	veri sözlüğü	Tasarımdaki varlık ve ilişki bilgisi	85
PERT tools	PERT araçları	Planlama araçları	87
compiler / interpreter	derleyici / yorumlayıcı	Language-processing araçları	87
re-engineering tools	yeniden mühendislik araçları	Program yeniden yapılandırma	87

tools / workbenches / environments	araçlar / tezgâhlar / ortamlar	Fuggetta'nın 3 CASE sınıfı	87
integrated / process-centred environment	entegre / süreç merkezli ortam	Ortam türleri (Şekil 4.16)	88–89
process engine	süreç motoru	Hangi aracın ne zaman kullanılacağını öneren bileşen	88

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 4'ten sınavda en çok çıkabilecekler

1. **4 temel faaliyet:** specification, design & implementation, validation, evolution (s.64)
2. **3 genel model:** waterfall, evolutionary, CBSE. Birbirini dışlamaz. (s.65)
3. **Cleanroom** = formal geliştirme, kusur testi yok (s.66)
4. **Waterfall 5 aşaması** + avantaj (belge, mühendislik uyumu) + dezavantaj (esnek değil) + ne zaman (gereksinimler net) (s.66–68)
5. **Evolutionary 2 türü:** exploratory ve throwaway prototyping. **2 sorun:** görünür değil, kötü yapı. ≤ 500.000 satır. (s.68–69)
6. **CBSE aşamaları** (component analysis, requirements modification, design with reuse, development & integration) + artı ve eksi (s.70)
7. **Incremental delivery:** en öncelikli önce. **4 avantaj.** Artım ≤ 20.000 satır. XP. (s.71–73)
8. **Spiral (Boehm):** her döngü bir aşama. **4 sektör.** Ana fark: **riskin açıkça tanınması**. (s.73–74)
9. **RE'nin 4 aşaması:** feasibility, elicitation & analysis, specification, validation (s.75–76)
10. **Tasarımın 6 faaliyeti:** architectural, abstract spec, interface, component, data structure, algorithm (s.77)
11. **Structured methods 5 modeli:** object, sequence, state transition, structural, data-flow (s.78)
12. **Testing ≠ debugging.** Debugging 4 adım. (s.79)
13. **3 test aşaması:** component, system, acceptance. **Alpha ve beta** farkı. (s.80–81)
14. **Geliştirme ve bakım ayrımı anlamsız**, evrimsel süreç (s.82)
15. **RUP:** 3 bakış · **4 faz (I-E-C-T)** · 6+3 iş akışı · **6 iyi uygulama** · fazlar dinamik, iş akışları statik (s.82–85)
16. **CASE:** %40 iyileşme, 2 sınırlayıcı faktör (yaratıcılık, ekip işi) · **tools / workbenches / environments** (s.85–89)

C Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What are the fundamental activities common to all software processes?

Tüm süreçlerde ortak temel faaliyetler nelerdir? (s.64)

> *1) Software specification: the functionality and constraints of the software are defined. 2) Software design and implementation: software that meets the specification is produced. 3) Software validation: the software is checked to ensure it does what the customer wants. 4) Software evolution: the software evolves to meet changing customer needs.*

1) Tanımlama: yazılımın işlevi ve kısıtları tanımlanır. 2) Tasarım ve gerçekleştirme: tanımlamayı karşılayan yazılım üretilir. 3) Geçerleme: müşterinin istediğini yaptığı kontrol edilir. 4) Evrim: değişen müşteri ihtiyaçlarına göre geliştirilir.

Q2. Describe the stages of the waterfall model. What are its advantages and disadvantages?

Waterfall aşamaları, avantaj ve dezavantajları (s.66–68)

> *Stages: 1) requirements analysis and definition, 2) system and software design, 3) implementation and unit testing, 4) integration and system testing, 5) operation and maintenance. Advantages: documentation is produced at each phase and it fits with other engineering process models. Disadvantage: the inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements. It should be used only when the requirements are well understood and unlikely to change.*

Aşamalar: 1) gereksinim analizi ve tanımı, 2) sistem ve yazılım tasarımı, 3) gerçekleştirme ve birim testi, 4) entegrasyon ve sistem testi, 5) işletim ve bakım. Avantaj: her aşamada belge üretilir, diğer mühendislik modelleriyle uyumludur. Dezavantaj: aşamalara katı bölünme yüzünden değişen gereksinimlere yanıt vermek zordur. Sadece gereksinimler iyi anlaşılmışsa ve değişmeyecekse kullanılmalı.

Q3. What are the two types of evolutionary development?

Evrimsel geliştirmenin 2 türü (s.68)

> *1) Exploratory development: the developer works with the customer to explore the requirements and deliver a final system. It starts with well-understood parts and adds new features. 2) Throwaway prototyping: the aim is to understand the customer's requirements. The prototype experiments with poorly understood requirements and is then thrown away.*

1) Keşifsel geliştirme: geliştirici müşteriyle gereksinimleri keşfeder ve son sistemi teslim eder. İyi anlaşılan kısımlardan başlayıp yeni özellikler ekler. 2) Atılacak prototip: amaç gereksinimleri anlamaktır. Prototip iyi anlaşılmayan gereksinimlerle deney yapar, sonra atılır.

Q4. Why are programs developed using evolutionary development likely to be difficult to maintain? (Ex. 4.2)

Evrimsel geliştirilen programların bakımı neden zordur? (s.69)

> *Because of continual change, the software structure becomes corrupted, so the system is often poorly structured. Also, the process is not visible: systems are developed quickly and it is not cost-effective to produce documents for every version. Poor structure and missing documentation make maintenance difficult and costly.*

Sürekli değişiklik yazılımın yapısını bozar, sistem kötü yapılandırılmış olur. Ayrıca süreç görünür değildir: sistem hızlı geliştirildiği için her sürümü belgelemek ekonomik değildir. Kötü yapı ve eksik belge bakımı zor ve pahalı yapar.

Q5. Describe component-based software engineering. What are its advantages and disadvantages?
CBSE'yi anlatın, artı ve eksileri (s.69–70)

> *CBSE is based on a large base of reusable components and an integrating framework. Its stages are requirements specification, component analysis, requirements modification, system design with reuse, development and integration, and system validation. Advantages: less software to develop, lower cost and risk, faster delivery. Disadvantages: requirements compromises are inevitable, so the system may not meet the real needs of users, and some control over system evolution is lost.*

CBSE, geniş bir yeniden kullanılabilir bileşen tabanına ve birleştirme çerçevesine dayanır. Aşamaları: gereksinim tanımı, bileşen analizi, gereksinim değiştirme, yeniden kullanımlı tasarım, geliştirme ve entegrasyon, sistem geçerleme. Artıları: daha az yazılım, daha düşük maliyet ve risk, hızlı teslim. Eksileri: gereksinimlerden ödün kaçınılmazdır, gerçek ihtiyacı karşılamayabilir ve evrim kontrolünün bir kısmı kaybolur.

Q6. What are the advantages of incremental delivery?
Artımlı teslimin avantajları (s.72)

> *1) Customers do not have to wait for the entire system, because the first increment satisfies their most critical requirements. 2) Early increments act as prototypes and help to clarify later requirements. 3) There is a lower risk of overall project failure. 4) The highest priority services are delivered first and receive the most testing.*

1) Müşteri bütün sistemi beklemez, ilk artım en kritik ihtiyaçları karşılar. 2) İlk artımlar prototip gibi sonraki gereksinimleri netleştirir. 3) Projenin tamamen başarısız olma riski düşüktür. 4) En öncelikli hizmetler önce teslim edilir ve en çok test edilir.

Q7. Describe Boehm's spiral model. How is it different from other models?
Spiral modeli anlatın. Diğerlerinden farkı nedir? (s.73–74)

> *In the spiral model, the process is represented as a spiral and each loop represents a phase of the process. Each loop has four sectors: 1) objective setting, 2) risk assessment and reduction, 3) development and validation, 4) planning. The main difference from other models is the explicit recognition of risk.*

Spiral modelde süreç bir spiraldir ve her döngü bir aşamadır. Her döngünün 4 sektörü vardır: 1) hedef belirleme, 2) risk değerlendirme ve azaltma, 3) geliştirme ve geçerleme, 4) planlama. Diğer modellerden ana farkı riskin açıkça tanınmasıdır.

Q8. How can the waterfall model and prototyping be accommodated in the spiral model? (Ex. 4.3)
Waterfall ve prototipleme spiralin içine nasıl sığar? (s.73)

> *In the development and validation sector of each loop, a development model is chosen according to the main risk. If user interface risks are dominant, evolutionary prototyping can be used. If the main risk is sub-system integration, the waterfall model can be used. So the spiral model can include both of them.*

Her döngünün geliştirme ve geçerleme sektöründe ana riske göre bir geliştirme modeli seçilir. Kullanıcı arayüzü riski baskınsa evrimsel prototipleme, ana risk alt sistem entegrasyonuysa waterfall kullanılır. Böylece spiral ikisini de içerebilir.

Q9. What are the main phases of the requirements engineering process?
Gereksinim mühendisliği sürecinin ana aşamaları (s.75–76)

> *1) Feasibility study: is the system possible with current technology, cost-effective and within budget? 2) Requirements elicitation and analysis: requirements are derived by observing existing systems and talking to users. 3) Requirements specification: the information is written into a document with user and system requirements. 4) Requirements validation: requirements are checked for realism, consistency and completeness.*

1) Fizibilite: mevcut teknolojiyle yapılabilir, maliyet-etkin ve bütçe içinde mi? 2) Gereksinim çıkarma ve analiz: mevcut sistemleri gözlemleme ve kullanıcılarla görüşme. 3) Gereksinim tanımlama: bilgi kullanıcı ve sistem gereksinimleriyle belgeye yazılır. 4) Gereksinim geçerleme: gerçekçilik, tutarlılık ve eksiksizlik kontrolü.

Q10. Why is it important to distinguish between user requirements and system requirements? (Ex. 4.5)
User ve system requirements ayrımı neden önemli? (s.75–76)

> *They have different readers. User requirements are abstract, high-level statements for customers and end-users, who do not need technical details. System requirements are detailed descriptions for the system developers, who need to know exactly what to build.*

Okuyucuları farklıdır. Kullanıcı gereksinimleri, teknik ayrıntıya ihtiyacı olmayan müşteri ve son kullanıcılar için soyut, üst seviye ifadelerdir. Sistem gereksinimleri ise tam olarak ne yapacaklarını bilmesi gereken geliştiriciler için ayrıntılı tanımlardır.

Q11. Describe the main activities in the software design process. (Ex. 4.6)
Tasarım sürecinin ana faaliyetleri (s.77)

> *1) Architectural design: sub-systems and their relationships, giving the system architecture. 2) Abstract specification of each sub-system's services and constraints. 3) Interface design: unambiguous interfaces between sub-systems. 4) Component design: services are allocated to components. 5) Data structure design. 6) Algorithm design. The output of each activity is a specification for the next stage.*

1) Mimari tasarım: alt sistemler ve ilişkileri. 2) Her alt sistemin hizmet ve kısıtlarının soyut tanımlaması. 3) Arayüz tasarımı: alt sistemler arası belirsizlik içermeyen arayüzler. 4) Bileşen tasarımı: hizmetlerin bileşenlere dağıtılması. 5) Veri yapısı tasarımı. 6) Algoritma tasarımı. Her faaliyetin çıktısı sonraki aşamanın tanımlamasıdır.

Q12. What is the difference between testing and debugging?
Test ve debugging farkı (s.79)

> *Testing establishes the existence of defects. Debugging is concerned with locating and correcting these defects. The debugging process is: locate error, design error repair, repair error and retest the program.*

Test, kusurların varlığını gösterir. Debugging ise bu kusurları bulup düzeltir. Debugging süreci: hatayı bul, onarımı tasarla, hatayı onar, programı tekrar test et.

Q13. Describe the stages of the testing process. What is the difference between alpha and beta testing?

Test aşamaları ve alpha ile beta farkı (s.80–81)

> *1) Component (unit) testing: individual components are tested independently. 2) System testing: the integrated system is tested for interface problems and emergent properties. 3) Acceptance testing: the system is tested with real data from the customer before it is accepted. Alpha testing is acceptance testing for a custom system with a single client. Beta testing means delivering a software product to some potential customers, who report problems before general sale.*

1) Bileşen testi: bileşenler bağımsız test edilir. 2) Sistem testi: birleşik sistem arayüz sorunları ve emergent özellikler için test edilir. 3) Kabul testi: sistem kabulden önce müşterinin gerçek verisiyle test edilir. Alpha testi, tek müşteriye özel sistemde kabul testidir. Beta testi ise ürünü genel satıştan önce sorun bildiren birkaç potansiyel müşteriye vermektir.

Q14. Describe the phases of the Rational Unified Process.

RUP'un fazlarını açıklayın (s.83)

> *1) Inception: establish a business case for the system and identify external entities. 2) Elaboration: understand the problem domain, establish an architectural framework, develop the project plan and identify key risks. 3) Construction: system design, programming and testing, producing a working system. 4) Transition: move the system from the development community to the user community and make it work in a real environment.*

1) Başlangıç: iş gerekçesi oluştur, dış varlıkları belirle. 2) Detaylandırma: problem alanını anla, mimari çerçeve kur, proje planı yap, temel riskleri belirle. 3) İnşa: tasarım, programlama ve test ile çalışan sistem üret. 4) Geçiş: sistemi geliştiricilerden kullanıcılara taşı ve gerçek ortamda çalıştır.

Q15. What are the advantages of providing static and dynamic views of the process, as in RUP? (Ex. 4.4)

Statik ve dinamik bakışın avantajı (s.84–85)

> *The phases (dynamic view) are not associated with specific workflows (static view). So all workflows can be active at all stages of the process. This separation makes the process more flexible and shows both when things happen and what activities are done.*

Fazlar (dinamik bakış) belirli iş akışlarına (statik bakış) bağlı değildir. Bu yüzden bütün iş akışları sürecin her aşamasında aktif olabilir. Bu ayrım süreci daha esnek yapar ve hem ne zaman ne olduğunu hem de hangi faaliyetlerin yapıldığını gösterir.

Q16. What are the six best practices recommended by RUP?

RUP'un 6 iyi uygulaması (s.84–85)

> *1) Develop software iteratively. 2) Manage requirements. 3) Use component-based architectures. 4) Visually model software with UML. 5) Verify software quality. 6) Control changes to software.*

1) Yinelemeli geliştir. 2) Gereksinimleri yönet. 3) Bileşen tabanlı mimari kullan. 4) Yazılımı UML ile görsel modelle. 5) Yazılım kalitesini doğrula. 6) Yazılım değişikliklerini kontrol et.

Q17. What limits the improvements from CASE? How are CASE systems classified by breadth of support?

CASE'i sınırlayan faktörler ve sınıfları (s.86–87)

> *Two factors limit CASE: software engineering is a design activity based on creative thought, and it is a team activity that CASE does not support well. By breadth of support, CASE is classified into tools (support individual tasks), workbenches (support process phases or activities) and environments (support all or most of the software process).*

CASE'i iki faktör sınırlar: yazılım mühendisliği yaratıcı düşünceye dayalı bir tasarım işidir ve CASE'in iyi desteklemediği bir ekip işidir. Destek genişliğine göre CASE: araçlar (tek tek görevler), tezgâhlar (süreç aşamaları) ve ortamlar (sürecin tamamı ya da çoğu).

D Mini test

cevaplar en altta

1. The waterfall model was first published by:
   a) Boehm
   b) Royce
   c) Beck
   d) Fuggetta
2. Normally the longest phase of the waterfall model is:
   a) requirements definition
   b) implementation
   c) system testing
   d) operation and maintenance
3. The main problem of the waterfall model is:
   a) no documentation
   b) inflexible partitioning into stages
   c) too much testing
   d) no planning
4. Throwaway prototyping is used to:
   a) deliver the final system
   b) understand the customer's requirements
   c) test hardware
   d) write documentation
5. Which is a problem of evolutionary development?
   a) The process is not visible
   b) Customers wait too long
   c) Too many documents
   d) No customer feedback
6. In CBSE, which stage changes the requirements to fit available components?
   a) Component analysis
   b) Requirements modification
   c) System validation
   d) Development and integration
7. In incremental delivery, which services are delivered first?
   a) the easiest
   b) the cheapest
   c) the highest priority
   d) the least tested
8. Increments should be no more than about:
   a) 2,000 lines
   b) 20,000 lines
   c) 200,000 lines
   d) 500,000 lines
9. The spiral model was proposed by:
   a) Royce
   b) Boehm
   c) Krutchen
   d) Mills

0. The main difference between the spiral model and others is:
   a) no testing
   b) explicit recognition of risk
   c) no planning
   d) one single phase

1. Which is the first stage of the requirements engineering process?
   a) Requirements validation
   b) Feasibility study
   c) Requirements specification
   d) Architectural design

2. Requirements validation checks requirements for:
   a) speed, size, cost
   b) realism, consistency, completeness
   c) colour, layout, font
   d) syntax only

3. "Locating and correcting defects" is called:
   a) testing
   b) debugging
   c) validation
   d) deployment

4. Testing with real data supplied by the customer is:
   a) unit testing
   b) system testing
   c) acceptance testing
   d) component testing

5. Giving a product to potential customers to report problems before sale is:
   a) alpha testing
   b) beta testing
   c) unit testing
   d) debugging

6. Which RUP phase establishes a business case?
   a) Inception
   b) Elaboration
   c) Construction
   d) Transition

7. In RUP, the static view describes the:
   a) phases
   b) workflows (activities)
   c) best practices
   d) risks

8. CASE systems that support all or most of the software process are called:
   a) tools
   b) workbenches
   c) environments
   d) compilers

Cevap anahtarı: 1-b · 2-d · 3-b · 4-b · 5-a · 6-b · 7-c · 8-b · 9-b · 10-b · 11-b · 12-b · 13-b · 14-c · 15-b · 16-a · 17-b · 18-c

YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 6
Software requirements

Yazılım Gereksinimleri

Kitap s.115–141 (Bölüm 6 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 6, kitabın **Part 2: Requirements** kısmının ilk bölümü. Yazılım gereksinimlerini ve bunları ifade etmenin yollarını anlatıyor. Beş kısımdan oluşuyor:

- **6.1 Functional and non-functional requirements:** Fonksiyonel, fonksiyonel olmayan ve alan (domain) gereksinimleri.
- **6.2 User requirements:** Kullanıcı gereksinimleri nasıl yazılır?
- **6.3 System requirements:** Sistem gereksinimleri, yapılandırılmış dil, tablolar, sıralama diyagramları.
- **6.4 Interface specification:** Arayüz tanımlama.
- **6.5 The software requirements document:** Gereksinim belgesi (SRS) ve IEEE standardı.

Part 2'nin genel fikri: Büyük sistemlerde belki de en büyük sorun gereksinim mühendisliğidir. Gereksinim mühendisliği, müşteriler ve kullanıcılar ile geliştiriciler arasındaki bir **iletişim sürec**idir. Sadece teknik değildir: gereksinimler kullanıcıların beğenileri, önyargıları ve politik ya da kurumsal konulardan etkilenir. Kitap bu bölümde ikinci örnek sistem olarak bir **kütüphane sistemi (LIBSYS)** kullanıyor.

Bölümün amaçları: (1) kullanıcı ve sistem gereksinimi kavramlarını ve neden farklı yazılmaları gerektiğini anlamak, (2) fonksiyonel ve fonksiyonel olmayan gereksinimlerin farkını anlamak, (3) gereksinimlerin bir gereksinim belgesinde nasıl düzenlendiğini bilmek.

Sayfa 118–120 — Gereksinim nedir? · User ve system requirements · Gereksinim türleri

📖 Sayfanın özeti

TANIM

Bir sistemin gereksinimleri, sistemin sağlayacağı **hizmetlerin** ve **çalışma kısıtlarının** tanımlarıdır. Bir cihazı kontrol etmek, sipariş vermek ya da bilgi bulmak gibi bir sorunu çözmeye yardım eden sistem için müşteri ihtiyaçlarını yansıtırlar. Bu hizmet ve kısıtları bulma, analiz etme, belgeleme ve kontrol etme sürecine **gereksinim mühendisliği (requirements engineering, RE)** denir.

"GEREKSİNİM" KELİMESİ TUTARSIZ KULLANILIR

Bazen bir hizmetin ya da kısıtın **üst seviye, soyut** bir ifadesidir, bazen de bir sistem fonksiyonunun **ayrıntılı, resmi** tanımıdır. **Davis** bunun nedenini şöyle açıklar: büyük bir proje ihaleye çıkarılırken ihtiyaçlar, birden fazla yüklenicinin farklı çözümler önerebileceği kadar **soyut** yazılmalıdır. Sözleşme verildikten sonra yüklenici, müşterinin anlayıp doğrulayabilmesi için daha **ayrıntılı** bir sistem tanımı yazmalıdır. İkisi de "gereksinim belgesi" diye anılabilir.

İKİ SEVİYE

Bu seviyeleri ayırmamak birçok soruna yol açar. Yazar iki terim kullanıyor: **User requirements**, sistemin hangi hizmetleri vereceğinin ve hangi kısıtlar altında çalışacağının **doğal dil ve diyagramlarla** yazılmış ifadeleridir. **System requirements**, sistemin fonksiyonlarını, hizmetlerini ve çalışma kısıtlarını **ayrıntılı** olarak ortaya koyar. Sistem gereksinim belgesi (bazen **functional specification** denir) kesin olmalı, tam olarak neyin gerçekleştirileceğini tanımlamalıdır ve alıcı ile geliştirici arasındaki **sözleşmenin parçası** olabilir.

ÖRNEK (ŞEKİL 6.1)

Kütüphane sisteminden bir kullanıcı gereksinimi: sistem, telif hakkı lisans kurumlarının istediği bütün verileri takip etmelidir. Bu tek gereksinim birkaç sistem gereksinimine açılır: belge isteyen kişiye kullanıcı ve istek bilgilerini kaydeden bir form gösterilmesi, formların **5 yıl** saklanması, formların kullanıcıya, materyal adına ve tedarikçiye göre indekslenmesi, bütün isteklerin kaydının tutulması, yazar ödünç hakları olan materyallerde ödünç bilgilerinin her ay lisans kurumlarına gönderilmesi. Kullanıcı gereksinimi daha soyut, sistem gereksinimleri daha ayrıntılıdır.

OKUYUCULAR (ŞEKİL 6.2)

Farklı seviyeler farklı okuyuculara hitap eder. **User requirements** okuyucuları: müşteri yöneticileri, son kullanıcılar, müşteri mühendisleri, yüklenici yöneticileri, sistem mimarları. Bunlar genelde sistemin nasıl gerçekleştirileceğiyle ilgilenmez. **System requirements** okuyucuları: son kullanıcılar, müşteri mühendisleri, sistem mimarları, **yazılım geliştiriciler**. Bunlar sistemin tam olarak ne yapacağını bilmek zorundadır.

6.1 ÜÇ GEREKSİNİM TÜRÜ

Functional requirements: sistemin vermesi gereken hizmetler, belirli girdilere nasıl tepki vereceği ve belirli durumlarda nasıl davranacağı. Bazen sistemin ne **yapmaması** gerektiğini de söyler. **Non-functional requirements:** hizmet ve fonksiyonlar üzerindeki **kısıtlar**: zamanlama, geliştirme süreci ve standart kısıtları. Genelde tek bir özelliğe değil **sistemin bütününe** uygulanır. **Domain requirements:** sistemin **uygulama alanından** gelen, o alanın özelliklerini ve kısıtlarını yansıtan gereksinimler. Fonksiyonel ya da fonksiyonel olmayan olabilir. Gerçekte bu ayrım net değildir: güvenlikle ilgili bir kullanıcı gereksinimi fonksiyonel olmayan gibi görünür, ama ayrıntılandırılınca kullanıcı kimlik doğrulama gibi açıkça fonksiyonel gereksinimler doğurabilir.

👩‍🏫 Hoca anlatımı

Gereksinim = **"sistem ne yapmalı ve hangi sınırlar içinde?"** sorusunun cevabı.

User ve system requirements farkı çok önemli, Bölüm 4'te de görmüştük. Bir restoran örneği: **User requirement** (müşterinin dili): "Vejetaryen yemek seçeneği olsun." **System requirement** (aşçının dili): "Menüde en az 3 et içermeyen ana yemek bulunmalı, et ile aynı tavada pişirilmemeli, menüde yeşil yaprak simgesiyle işaretlenmeli." Biri patron ve müşteri için, diğeri yapacak kişi için.

Şekil 6.2'deki okuyucu farkına dikkat: **yazılım geliştiriciler sadece system requirements'ı okur**, **yöneticiler** (client ve contractor managers) ise sadece user requirements'ı.

3 gereksinim türü sınavın en klasik sorusu: **Functional** = sistem **ne yapar** (ör. "kullanıcı kitap arayabilmeli"). **Non-functional** = sistem **nasıl / hangi kısıtla** yapar (ör. "arama 2 saniyede sonuç vermeli"). **Domain** = **alanın kendisinden** gelen kurallar (ör. kütüphanecilik standardı, telif yasası).

📝 Deftere not

- ⭐ **Requirements:** sistemin sağlayacağı **hizmetlerin** ve **çalışma kısıtlarının** tanımları. Müşteri ihtiyaçlarını yansıtır.
- ⭐ **Requirements engineering (RE):** bu hizmet ve kısıtları **bulma, analiz etme, belgeleme ve kontrol etme** süreci.
- "Requirement" tutarsız kullanılır: soyut üst seviye ifade ya da ayrıntılı resmi tanım. **Davis:** ihale için **soyut** (farklı yükleniciler farklı çözüm önerebilsin), sözleşmeden sonra **ayrıntılı** (müşteri doğrulayabilsin).

- ⭐⭐ **2 seviye:**
  - User requirements: **doğal dil + diyagramlar**. Hangi hizmetler ve hangi kısıtlar.
  - System requirements: fonksiyonlar, hizmetler ve kısıtlar **ayrıntılı**. Belge = **functional specification**. **Kesin** olmalı, **sözleşmenin parçası** olabilir.
- **Şekil 6.1 (LIBSYS):** 1 user requirement (telif lisans verilerini takip et) → 5 system requirement (form, 5 yıl saklama, indeksleme, istek kaydı, aylık rapor).
- ⭐ **Şekil 6.2, okuyucular:**
  - **User req.:** client managers, system end-users, client engineers, **contractor managers**, system architects.
  - **System req.:** system end-users, client engineers, system architects, **software developers**.
- ⭐⭐ **6.1, 3 gereksinim türü:**
  - Functional: verilecek **hizmetler**, girdilere tepki, durumlara göre davranış (bazen ne **yapmaması** gerektiği).
  - Non-functional: hizmet ve fonksiyonlar üzerindeki **kısıtlar** (zamanlama, süreç, standartlar). Genelde **sistemin bütününe** uygulanır.
  - Domain: **uygulama alanından** gelir, alanın özellik ve kısıtları. Functional ya da non-functional olabilir.
- Ayrım **net değil**: güvenlik (non-functional) ayrıntılanınca kimlik doğrulama (functional) gereksinimleri doğurur.

Sayfa 120–121 6.1.1 Functional requirements (Fonksiyonel gereksinimler)

📖 Sayfanın özeti

TANIM

Fonksiyonel gereksinimler sistemin **ne yapması gerektiğini** anlatır. Yazılımın türüne, beklenen kullanıcılara ve kurumun gereksinim yazma yaklaşımına bağlıdır. Kullanıcı gereksinimi olarak yazıldıklarında oldukça **soyut** olurlar, sistem gereksinimi olarak yazıldıklarında fonksiyonu, **girdilerini, çıktılarını ve istisnalarını** ayrıntılı anlatırlar.

LIBSYS ÖRNEKLERİ

LIBSYS, öğrencilerin ve öğretim üyelerinin başka kütüphanelerden kitap ve belge sipariş ettiği, birçok makale veritabanına tek arayüz sunan ve makale kopyası indirmeyi sağlayan bir üniversite kütüphane sistemidir. Fonksiyonel gereksinim örnekleri: (1) kullanıcı başlangıçtaki veritabanlarının hepsinde ya da seçtiği bir alt kümede **arama** yapabilmeli, (2) sistem, belge deposundaki belgeleri okumak için uygun **görüntüleyiciler** sağlamalı, (3) her siparişe kullanıcının kendi kalıcı alanına kopyalayabileceği **benzersiz bir kimlik (ORDER_ID)** verilmeli. Bu örnekler fonksiyonel gereksinimlerin **farklı ayrıntı seviyelerinde** yazılabildiğini de gösterir (1 ile 3'ü karşılaştır).

BELİRSİZLİK (İMPRECİSİON) SORUNU

Tanımlamadaki **belirsizlik** birçok yazılım mühendisliği sorununun nedenidir. Geliştirici, uygulamayı kolaylaştırmak için belirsiz bir gereksinimi kendi işine geldiği gibi yorumlayabilir, ama müşterinin istediği bu olmayabilir. O zaman yeni gereksinimler ve değişiklikler gerekir, bu da teslimi geciktirir ve maliyeti artırır. Ör: "uygun görüntüleyiciler" gereksiniminde amaç sistemin desteklediği **her belge**

formatı için görüntüleyici olmasıdır, ama ifade bunu açıkça söylemez. Takvim baskısındaki bir geliştirici sadece bir **metin görüntüleyici** verip gereksinimin karşılandığını iddia edebilir.

COMPLETENESS VE CONSİSTENCY

İlke olarak fonksiyonel gereksinimler hem **eksiksiz (complete)** hem **tutarlı (consistent)** olmalıdır. Eksiksizlik, kullanıcının istediği **bütün hizmetlerin** tanımlanmasıdır. Tutarlılık, gereksinimlerin birbiriyle **çelişmemesidir**. Pratikte büyük, karmaşık sistemlerde bunu sağlamak neredeyse **imkânsızdır**, çünkü: (1) büyük sistemlerin tanımlamasını yazarken hata ve eksik yapmak kolaydır, (2) farklı paydaşların (stakeholders) farklı ve çoğu zaman tutarsız ihtiyaçları vardır. Bu tutarsızlıklar ilk tanımlamada fark edilmeyebilir ve ancak daha derin analizde ya da bazen sistem teslim edildikten sonra ortaya çıkar.

👩‍🏫 Hoca anlatımı

Fonksiyonel gereksinim = sistemin **yaptığı işler**. "Kullanıcı arama yapabilmeli", "sistem fatura üretmeli" gibi.

"Uygun görüntüleyici" örneği çok öğretici: müşteri "her formatı açabilsin" demek istemiş ama "uygun" kelimesi belirsiz. Tembel ya da acelesi olan geliştirici "bir metin görüntüleyici koydum, uygun işte" der. Kim haklı? İkisi de! İşte belirsiz gereksinim böyle kavgaya ve ek maliyete yol açar. Ders: **gereksinimleri net yaz.**

Complete ve consistent kavramlarını iyi bil: **Complete**, hiçbir şey eksik değil (müşterinin istediği her şey yazılmış). **Consistent**, hiçbir şey çelişmiyor (bir yerde "herkes görebilir", başka yerde "sadece yönetici görebilir" yazmıyor). Büyük sistemlerde ikisini birden tam sağlamak **pratikte imkânsızdır**, çünkü çok fazla gereksinim ve farklı isteklere sahip çok fazla insan vardır.

📝 Deftere not

- ⭐ **Functional requirements:** sistemin **ne yapması gerektiği**. Yazılım türüne, kullanıcılara, kurumun yaklaşımına bağlı.
  - User req. olarak: **soyut**. System req. olarak: fonksiyon, **girdi, çıktı, istisnalar** ayrıntılı.
- **LIBSYS:** üniversite kütüphanesi, başka kütüphanelerden kitap ve belge sipariş, makale veritabanlarına tek arayüz.
- **Örnekler:** (1) tüm ya da seçilen veritabanlarında **arama**, (2) belgeler için uygun **viewer**'lar, (3) her siparişe benzersiz **ORDER_ID**. Farklı ayrıntı seviyelerinde yazılabilir.
- ⭐ **Imprecision (belirsizlik)** = birçok SE sorununun nedeni. Geliştirici kolay yolu seçecek şekilde yorumlar, yeni gereksinim çıkar, **gecikme ve maliyet artar**.
  - Ör: "**appropriate viewers**": amaç her format için viewer, ama geliştirici sadece **metin viewer** verip "karşılandı" diyebilir.
- ⭐⭐ **Complete:** kullanıcının istediği **tüm hizmetler** tanımlanmış. **Consistent:** gereksinimler **çelişmiyor**.
- ⭐ Büyük sistemlerde ikisi pratikte **imkânsız**: (1) büyük spec'te hata ve eksik yapmak kolay, (2) farklı **stakeholder**'ların tutarsız ihtiyaçları. Tutarsızlıklar geç (analizde ya da teslimden sonra) ortaya çıkar.

Sayfa 121–125 6.1.2 Non-functional requirements · Şekil 6.3, 6.4, 6.5, 6.6

📖 Sayfanın özeti

TANIM

Fonksiyonel olmayan gereksinimler, sistemin verdiği belirli fonksiyonlarla **doğrudan ilgili olmayan** gereksinimlerdir. Güvenilirlik, yanıt süresi ve bellek kullanımı gibi **emergent özelliklerle** ilgili olabilir ya da G/Ç cihazlarının yetenekleri ve arayüzlerdeki veri gösterimleri gibi sistem **kısıtlarını** tanımlayabilir. Nadiren tek bir özelliğe bağlıdırlar, sistemin bütün olarak özelliklerini tanımlar ya da kısıtlarlar.

NEDEN DAHA KRİTİK?

Fonksiyonel olmayan gereksinimler çoğu zaman tek tek fonksiyonel gereksinimlerden **daha kritiktir**. Kullanıcılar ihtiyacını tam karşılamayan bir fonksiyonun etrafından dolanmanın yolunu genelde bulur. Ama fonksiyonel olmayan bir gereksinimi karşılamamak **bütün sistemi kullanılamaz** hale getirebilir. Ör: bir uçak sistemi güvenilirlik gereksinimini karşılamazsa uçuş için güvenli sertifikası alamaz. Gerçek zamanlı bir kontrol sistemi performans gereksinimini karşılamazsa kontrol fonksiyonları doğru çalışmaz.

SÜREÇ KISITLARI VE KAYNAKLARI

Bunlar sadece yazılımla ilgili değildir, bazıları **geliştirme sürecini** kısıtlar: kullanılacak kalite standartları, tasarımın belirli bir CASE aracıyla yapılması ya da izlenecek süreç gibi. Fonksiyonel olmayan gereksinimler kullanıcı ihtiyaçlarından, **bütçe** kısıtlarından, **kurum politikalarından**, diğer yazılım ve donanımlarla **birlikte çalışabilirlik** ihtiyacından ya da emniyet yönetmelikleri ve **gizlilik yasaları** gibi dış etkenlerden doğar.

ŞEKİL 6.3: SINIFLANDIRMA (3 ANA DAL)

Product requirements: ürünün davranışını belirtir. Alt türleri: **efficiency** (→ **performance** ve **space**), **reliability**, **portability**, **usability**. Ör: sistemin ne kadar hızlı çalışacağı, ne kadar bellek kullanacağı, kabul edilebilir arıza oranı. **Organisational requirements:** müşteri ve geliştirici kurumun politika ve prosedürlerinden türer. Alt türleri: **delivery** (ürün ve belgelerin ne zaman teslim edileceği), **implementation** (programlama dili, tasarım yöntemi), **standards** (izlenecek süreç standartları). **External requirements:** sisteme ve geliştirme sürecine dışarıdan gelen bütün gereksinimler. Alt türleri: **interoperability** (başka kurumların sistemleriyle etkileşim), **ethical** (sistemin kullanıcılar ve kamu tarafından kabul edilebilir olması), **legislative** (yasalara uygunluk, alt türleri **privacy** ve **safety**).

ŞEKİL 6.4: LIBSYS ÖRNEKLERİ

Product: kullanıcı arayüzü çerçeve ve Java applet kullanmadan **basit HTML** ile yapılmalı (bir fonksiyon değil, tasarımcının özgürlüğünü kısıtlayan bir kısıt, farklı tarayıcılarda çalışmayı kolaylaştırmak için). **Organisational:** geliştirme süreci ve belgeler şirketin belirli bir standart sürecine uymalı. **External:** gizlilik yasası gereği sistem, kullanıcıların adı ve kütüphane numarası dışında **kişisel bilgilerini** kütüphane personeline göstermemeli.

DOĞRULANABİLİRLİK SORUNU (ŞEKİL 6.5)

Fonksiyonel olmayan gereksinimlerin yaygın sorunu **doğrulanmalarının zor** olmasıdır. Kullanıcılar bunları "kolay kullanım", "hızlı tepki" gibi **genel hedefler (goals)** olarak ifade eder. Bu belirsiz hedefler yoruma açıktır ve teslimden sonra anlaşmazlık çıkarır. Örnek: bir trafik kontrol sistemi için "deneyimli kontrolörler için kolay kullanılmalı ve hata en aza inmeli" bir **hedeftir**. Bunun **test edilebilir** hali şöyledir: "Deneyimli kontrolörler toplam **iki saatlik** eğitimden sonra bütün fonksiyonları

kullanabilmeli, eğitimden sonra ortalama hata sayısı **günde ikiyi** geçmemeli." Bu, sistem simülatöründe hataları sayarak test edilebilir.

Özellik	Ölçüt (Şekil 6.6)
Speed	Saniyede işlenen işlem sayısı, kullanıcı/olay yanıt süresi, ekran yenileme süresi
Size	Kilobayt, RAM çipi sayısı
Ease of use	Eğitim süresi, yardım ekranı sayısı
Reliability	Arızalar arası ortalama süre (mean time to failure), kullanılamama olasılığı, arıza oranı, availability
Robustness	Arızadan sonra yeniden başlama süresi, arızaya yol açan olay yüzdesi, arızada veri bozulma olasılığı
Portability	Hedefe bağımlı komut yüzdesi, hedef sistem sayısı

NİCEL YAZIM, HEDEFLER, ÇATIŞMALAR

Mümkünse fonksiyonel olmayan gereksinimler **nicel (quantitatively)** yazılmalıdır ki **nesnel olarak test** edilebilsin. Pratikte müşteriler hedeflerini nicel gereksinimlere çeviremeyebilir. Bakım kolaylığı gibi bazı hedefler için ölçüt yoktur, müşteriler sayıların günlük deneyimdeki anlamını bilmeyebilir ve nicel doğrulamanın maliyeti çok yüksek olabilir. Bu yüzden belgelerde hedefler ve gereksinimler karışık bulunur. Hedefler müşteri önceliklerini gösterdiği için faydalıdır, ama müşteriye bunların yoruma açık olduğu ve nesnel doğrulanamayacağı söylenmelidir. Fonksiyonel olmayan gereksinimler birbiriyle **çatışabilir**: ör. bellek en fazla **4 MB** olmalı (gömülü sistemlerde yaygın), ama sistem kritik gerçek zamanlı yazılım dili **Ada** ile yazılmalı. Ada programı 4 MB'a sığmayabilir, bu yüzden bir **ödünleşim (trade-off)** gerekir: başka bir dil ya da daha fazla bellek.

SAYFA 125: BELGEDE AYIRMAK

Fonksiyonel ve fonksiyonel olmayan gereksinimleri belgede ayırmak faydalıdır ama zordur. Ayrı yazılırsa aralarındaki ilişki görülmez, birlikte yazılırsa ayırt etmek zorlaşır. Performans ya da güvenilirlik gibi emergent özelliklerle ilgili gereksinimler açıkça vurgulanmalıdır (ayrı bölümde ya da başka bir şekilde ayırt edilerek). Emniyet ve güvenlik gereksinimleri kritik sistemler için özellikle önemlidir (Bölüm 9).

👨‍🏫 Hoca anlatımı

Fonksiyonel olmayan gereksinim = sistemin **"nasıl"** olması gerektiği: hızlı mı, güvenilir mi, güvenli mi, hangi dilde yazılacak, hangi yasaya uyacak?

Neden daha kritik? Bir telefon uygulaması düşün: "fotoğraf filtresi" özelliği eksik olsa idare edersin. Ama uygulama her 5 dakikada çöküyorsa (reliability) ya da 30 saniyede açılıyorsa (performance) silersin. Uçak örneği de öyle: güvenilirlik şartını karşılamazsa uçmaya izin verilmez.

Şekil 6.3 ağacını ezberle. 3 ana dal: **Product** (ürünün kendisi: hız, boyut, güvenilirlik, taşınabilirlik, kullanılabilirlik), **Organisational** (şirketin kuralları: teslim, gerçekleştirme dili, standartlar), **External** (dış dünya: birlikte çalışabilirlik, etik, yasalar ve altında gizlilik ile emniyet).

Goal ve verifiable requirement farkı çok önemli (Şekil 6.5): "Kolay kullanılsın" bir **hedeftir**, test edemezsin, herkes farklı anlar. "2 saatlik eğitimden sonra günde en fazla 2 hata yapılsın" ise **doğrulanabilir** bir gereksinimdir, sayıp kontrol edebilirsin. Sınavda "bu belirsiz gereksinimi test

edilebilir hale getirin" diye sorabilirler. Şekil 6.6'daki ölçütleri (saniye, MB, eğitim süresi, MTTF) kullan.

Çatışma örneği: "4 MB'tan az bellek" ve "Ada dilinde yaz" birbiriyle çelişebilir. O zaman biri feda edilir.

📝 Deftere not

- ⭐ **Non-functional requirements:** belirli fonksiyonlarla **doğrudan ilgili değil**. **Emergent özellikler** (reliability, response time, store occupancy) ya da **kısıtlar** (G/Ç yetenekleri, veri gösterimi). **Sistemin bütününe** uygulanır.
- ⭐⭐ Genelde functional'dan **daha kritik**: kullanıcı eksik fonksiyonun etrafından dolanır, ama non-functional karşılanmazsa **bütün sistem kullanılamaz**. Ör: uçak, reliability yoksa **sertifika yok**. Real-time sistem, performance yoksa kontrol çalışmaz.
- **Süreci** de kısıtlayabilir (kalite standartları, belirli CASE aracı, süreç).
- Kaynakları: kullanıcı ihtiyaçları, **bütçe**, **kurum politikaları**, **birlikte çalışabilirlik**, dış etkenler (emniyet yönetmelikleri, **gizlilik yasaları**).
- ⭐⭐ **Şekil 6.3, sınıflandırma:**
  - Product requirements: ürün davranışı. **Efficiency** (performance, space), **reliability**, **portability**, **usability**.
  - Organisational requirements: müşteri ve geliştirici kurum politikaları. **Delivery**, **implementation** (dil, yöntem), **standards**.
  - External requirements: dış etkenler. **Interoperability**, **ethical**, **legislative** (**privacy**, **safety**).
- ⭐ **Şekil 6.4 (LIBSYS):** Product: arayüz **basit HTML** (frame ve applet yok) · Organisational: şirket **süreç standardına** uy · External: gizlilik, personel kullanıcıların sadece **ad ve kütüphane numarasını** görsün.
- ⭐⭐ **Sorun: doğrulamak zor.** Kullanıcılar **hedef (goal)** olarak yazar ("kolay kullanım"), yoruma açık, anlaşmazlık çıkar.
  - **System goal:** "deneyimli kontrolörler için kolay, hata en az".
  - **Verifiable NFR:** "**2 saatlik eğitimden** sonra tüm fonksiyonları kullanabilsin, ortalama hata **≤ 2 / gün**". Simülatörde hatalar sayılarak test edilir.
- ⭐ Mümkünse **nicel (quantitative)** yaz, böylece **nesnel test** edilebilir.
- ⭐⭐ **Şekil 6.6, ölçütler:** Speed (işlem/sn, yanıt süresi, ekran yenileme) · Size (KB, RAM çipi) · Ease of use (**eğitim süresi**, yardım ekranı sayısı) · Reliability (**MTTF**, kullanılamama olasılığı, arıza oranı, availability) · Robustness (yeniden başlama süresi, arızaya yol açan olay %, veri bozulma olasılığı) · Portability (hedefe bağımlı komut %, hedef sistem sayısı).
- Nicel yazım her zaman mümkün değil: bazı hedeflere ölçüt yok (**maintainability**), müşteri sayıları anlamaz, doğrulama **pahalı**. Belgede hedef ve gereksinim karışık olur. Hedefler öncelik gösterir ama **yoruma açıktır**.
- ⭐ NFR'ler **çatışabilir**: bellek **≤ 4 MB** ve dil = **Ada** → sığmayabilir → **trade-off** (başka dil ya da daha çok bellek).
- Belgede ayırmak faydalı ama zor. Emergent özellik gereksinimlerini **açıkça vurgula** (ayrı bölüm). Safety ve security kritik sistemlerde önemli (Bölüm 9).

Sayfa 125–126 6.1.3 Domain requirements (Alan gereksinimleri)

📖 Sayfanın özeti

TANIM

Alan gereksinimleri, kullanıcıların özel ihtiyaçlarından değil sistemin **uygulama alanından** türer. Genelde alana özgü **terminoloji** ya da alan kavramlarına atıf içerirler. Kendi başına yeni fonksiyonel gereksinimler olabilir, mevcut fonksiyonel gereksinimleri **kısıtlayabilir** ya da belirli hesaplamaların **nasıl yapılacağını** belirleyebilir. Özelleşmiş oldukları için yazılım mühendisleri bunların diğer gereksinimlerle ilişkisini anlamakta zorlanır.

NEDEN ÖNEMLİ?

Alan gereksinimleri çoğu zaman uygulama alanının **temel gerçeklerini** yansıtır. Karşılanmazlarsa sistemin tatmin edici çalışması **imkânsız** olabilir.

LIBSYS ÖRNEKLERİ

(1) Bütün veritabanlarına, belirli bir kütüphane standardına (**Z39.50**) dayalı standart bir kullanıcı arayüzü olmalı. Bu bir **tasarım kısıtıdır**: geliştiriciler arayüz tasarımına başlamadan bu standardı öğrenmelidir. (2) **Telif hakkı** kısıtları yüzünden bazı belgeler geldiği anda silinmeli ve kullanıcının isteğine göre ya sunucuda yazdırılıp elle iletilmeli ya da ağ yazıcısına gönderilmeli. Bu, kütüphanelerdeki materyale uygulanan telif yasalarından gelir ve sistemin bazı belge sınıfları için otomatik **"yazdır ve sil"** özelliği içermesini gerektirir. Yani kullanıcılar bu belgelerin kendi elektronik kopyasına sahip olamaz.

TREN ÖRNEĞİ (ŞEKİL 6.7)

Bir hesaplamanın nasıl yapılacağını belirten alan gereksinimine örnek, **otomatik tren koruma sistemidir**. Bu sistem tren kırmızı sinyali geçerse treni otomatik durdurur. Gereksinim, trenin yavaşlamasının bir **matematiksel formülle** nasıl hesaplanacağını söyler (kontrol yavaşlaması ile yol eğiminden gelen yavaşlamanın toplamı, eğim terimi yerçekimi ivmesine ve tren türüne bağlı sabitlere dayanır). Alana özgü terminoloji kullanır, anlamak için demiryolu ve tren özelliklerini bilmek gerekir.

ANA SORUN

Tren örneği alan gereksinimlerinin **büyük bir sorununu** gösterir: uygulama alanının dilinde (burada matematik denklemleri) yazılırlar ve yazılım mühendislerinin anlaması zordur. **Alan uzmanları** bazı bilgileri kendilerine çok açık geldiği için gereksinime **yazmayabilir**. Ama geliştiriciye açık olmayabilir ve gereksinim yanlış gerçekleştirilebilir.

SAYFA 127 BAŞI (ŞEKİL 6.8)

LIBSYS'ten bir kullanıcı gereksinimi: sistem, kullanıcıların yaptığı bütün ödemelerin kaydını tutan bir **muhasebe sistemi** sağlamalı ve yöneticiler bunu düzenli kullanıcılara **indirim** yapacak şekilde yapılandırabilmeli. (Bu örnek 6.2'de, kullanıcı gereksinimlerindeki sorunları göstermek için kullanılıyor.)

👨‍🏫 Hoca anlatımı

Domain requirement = **işin doğasından gelen kurallar**. Müşteri istese de istemese de o alanda böyledir. Bir eczane yazılımı yapıyorsan "reçeteli ilaç reçetesiz satılamaz" kuralı müşterinin keyfi değil, eczacılığın ve yasaların kuralıdır.

LIBSYS'teki iki örnek güzel: **Z39.50 standardı** (kütüphanecilik dünyasının standardı) ve **telif yasası** (bazı belgeleri yazdırıp hemen silmek). Bunları kullanıcı istemedi, alan zorunlu kıldı.

Tren örneğindeki sorun sınavda sorulabilir: alan uzmanı (demiryolu mühendisi) formülü kendi diliyle yazar ve "herkes bunu bilir" dediği şeyleri atlar. Yazılımcı ise demiryolundan anlamaz, bu yüzden yanlış kodlayabilir. Yani sorun: **anlaşılması zor, eksik yazılan gereksinimler**.

📝 Deftere not

- ⭐ **Domain requirements:** kullanıcı ihtiyacından değil **uygulama alanından** türer. Alana özgü **terminoloji**.
  - Yeni fonksiyonel gereksinim olabilir, mevcutları **kısıtlayabilir** ya da hesaplamanın **nasıl** yapılacağını belirleyebilir.
- ⭐ Önemli: alanın **temel gerçeklerini** yansıtır. Karşılanmazsa sistem **çalışamaz**.
- ⭐ **LIBSYS örnekleri:**
  - **Z39.50** standardına dayalı ortak arayüz → **tasarım kısıtı**.
  - **Telif hakkı** → bazı belgeler gelince silinir, yazdırılır (**delete-on-print**), kullanıcının elektronik kopyası olmaz.
- **Şekil 6.7, tren koruma sistemi:** kırmızı sinyal geçilirse treni durdurur. Gereksinim yavaşlamanın **matematiksel formülünü** verir (alan terminolojisi).
- ⭐⭐ **Ana sorun:** alan dilinde (ör. denklemler) yazılır, yazılımcı **anlamakta zorlanır**. **Alan uzmanları** "çok açık" bilgileri **atlar**, geliştirici yanlış gerçekleştirebilir.
- **Şekil 6.8:** LIBSYS muhasebe sistemi gereksinimi (ödemelerin kaydı + düzenli kullanıcıya indirim yapılandırma).

Sayfa 127–129 6.2 User requirements (Kullanıcı gereksinimleri) · Şekil 6.9, 6.10

📖 Sayfanın özeti

NASIL YAZILMALI?

Kullanıcı gereksinimleri fonksiyonel ve fonksiyonel olmayan gereksinimleri, ayrıntılı teknik bilgisi olmayan kullanıcıların **anlayabileceği** şekilde anlatmalıdır. Sadece sistemin **dış davranışını** belirtmeli ve mümkün olduğunca **tasarım özelliklerinden kaçınmalıdır**. Yani yazılım jargonu, yapılandırılmış ya da resmi gösterimler kullanılmamalı, gereksinim sistemin gerçekleştirilmesi anlatılarak tarif edilmemelidir. **Basit dil**, basit tablolar, formlar ve sezgisel diyagramlar kullanılmalıdır.

DOĞAL DİLİN 3 SORUNU

Gereksinimler bir metin belgesinde doğal dil cümleleriyle yazılınca çeşitli sorunlar çıkar: **1) Lack of clarity (açıklık eksikliği):** belgeyi uzun ve zor okunur yapmadan dili kesin ve belirsizlikten uzak kullanmak zordur. **2) Requirements confusion (gereksinim karışıklığı):** fonksiyonel gereksinimler, fonksiyonel olmayan gereksinimler, sistem hedefleri ve tasarım bilgisi açıkça ayrılamayabilir. **3) Requirements amalgamation (gereksinim birleşmesi):** birkaç farklı gereksinim **tek bir gereksinim** gibi yazılabilir.

ŞEKİL 6.8 VE 6.9 ÖRNEKLERİ

Şekil 6.8'deki muhasebe gereksinimi hem **kavramsal** bilgiyi (LIBSYS'in parçası olarak bir muhasebe sistemi olmalı) hem de **ayrıntılı** bilgiyi (düzenli kullanıcılara indirim) karıştırır. Ayrıntı sistem gereksinimlerine bırakılmalıydı. Kullanıcı ve sistem gereksinimlerini ayırmak iyi bir uygulamadır, yoksa teknik olmayan okuyucular sadece teknisyenleri ilgilendiren ayrıntılarda boğulur. Şekil 6.9, tasarım

modellerini düzenleyen bir CASE aracı için gerçek bir gereksinim belgesinden alınmış bir **ızgara (grid)** gereksinimidir. İlk cümlesi **üç tür gereksinimi** karıştırır: (1) editörün bir ızgara sağlaması (kavramsal, fonksiyonel gereksinim ve gerekçesi), (2) ızgara birimlerinin santimetre ya da inç olması (ayrıntılı, fonksiyonel olmayan), (3) ızgaranın kullanıcı tarafından nasıl açılıp kapatılacağı (fonksiyonel olmayan arayüz gereksinimi). Ayrıca başlangıç bilgisinin bir kısmını verir (başta kapalı) ama tamamını vermez (çizgiler arası mesafe yok).

DAHA İYİ HALİ (ŞEKİL 6.10)

Çok fazla bilgi içeren kullanıcı gereksinimleri geliştiricinin **yenilikçi çözümler** bulma özgürlüğünü kısıtlar ve anlaşılması zordur. Kullanıcı gereksinimi **temel özelliklere** odaklanmalıdır. Yeniden yazılmış hali: editör, yatay ve dikey çizgilerden oluşan bir ızgara sağlamalı ve bu **pasif** bir ızgara olmalı (nesnelerin hizalanması kullanıcının sorumluluğu). Yanında bir **gerekçe (rationale)** vardır: ızgara düzenli diyagram çizmeye yardım eder. Nesnelerin çizgilere otomatik yapıştığı aktif ızgara faydalı olabilir ama konumlandırma kesin değildir, nesnelerin yerine en iyi kullanıcı karar verir. Ayrıca ayrıntılı tanımlamanın yeri (**specification**) ve gereksinimi öneren kişi (**source**) belirtilir.

GEREKÇENİN (RATİONALE) ÖNEMİ

Mümkünse her kullanıcı gereksinimiyle birlikte bir gerekçe yazılmalıdır. Gerekçe gereksinimin **neden** konduğunu açıklar ve özellikle gereksinimler **değiştirilirken** faydalıdır. Ör: ileride aktif ızgaraya geçiş önerilirse pasif ızgaranın bilinçli bir tercih olduğu, gerçekleştirme kararı olmadığı anlaşılır.

YANLIŞ ANLAŞILMAYI AZALTMAK İÇİN 4 KURAL

1) Standart bir **format** belirle ve bütün gereksinimleri ona uydur. Standart format eksikleri azaltır ve kontrolü kolaylaştırır. Yazarın formatında başlangıç gereksinimi kalın yazılır, her biri bir gerekçe ve ayrıntılı tanımlamaya bir referans içerir, kaynak bilgisi (kim önerdi) de eklenebilir. **2)** Dili **tutarlı** kullan: **zorunlu (mandatory)** gereksinimler "**shall**" ile, **istenen (desirable)** ama şart olmayan gereksinimler "**should**" ile yazılır. **3)** Önemli kısımları **vurgulamak** için kalın, italik ya da renk kullan. **4)** Bilgisayar **jargonundan** mümkün olduğunca kaçın (ama bazı teknik terimler kaçınılmaz olarak girer).

VOLERE

Robertson'ların **VOLERE** gereksinim mühendisliği yöntemi, kullanıcı gereksinimlerinin önce **kartlara**, her kartta bir gereksinim olacak şekilde yazılmasını önerir. Kartlarda gerekçe, diğer gereksinimlere bağımlılıklar, kaynak, destekleyici materyaller gibi alanlar bulunur. Bu, yazarın formatını genişletir ve hem kullanıcı hem sistem gereksinimleri için kullanılabilir.

👨‍🏫 Hoca anlatımı

Kullanıcı gereksinimi **müşterinin anlayacağı dilde** yazılmalı: teknik terim yok, tasarım detayı yok, sadece "sistem dışarıdan ne yapacak".

Doğal dilin 3 sorunu sınavın sevdiği bir liste: (1) **Lack of clarity**, net yazmaya çalıştıkça metin uzar ve okunmaz. (2) **Confusion**, fonksiyonel, fonksiyonel olmayan, hedef ve tasarım birbirine karışır. (3) **Amalgamation**, birkaç gereksinim tek cümleye sıkıştırılır.

Grid örneği üçünü birden gösteriyor: tek cümlede "ızgara olsun" (fonksiyonel), "santim ya da inç olsun" (detay) ve "kontrol panelinden açılsın" (arayüz) karışmış.

"Shall" ve "should" farkı çok önemli ve sınavda sorulabilir: **shall** = zorunlu ("sistem şifre istemeli"), **should** = istenen ama şart değil ("sistem renkli tema da sunabilir").

Rationale (gerekçe) neden önemli? İki yıl sonra biri "neden ızgara otomatik yapışmıyor, düzeltelim" derse, gerekçe "bilerek böyle yaptık, çünkü..." diye cevap verir.

📝 Deftere not

- ⭐ **User requirements:** teknik bilgisi olmayan kullanıcıların anlayacağı şekilde. Sadece **dış davranış**, **tasarım özelliklerinden kaçın**. Jargon yok, resmi gösterim yok. **Basit dil, tablo, form, sezgisel diyagram**.
- ⭐⭐ **Doğal dilin 3 sorunu:**
  - Lack of clarity: kesin ve belirsizliksiz yazmak zor, belge uzar.
  - Requirements confusion: functional, non-functional, hedefler ve tasarım **karışır**.
  - Requirements amalgamation: birkaç gereksinim **tek gereksinim** gibi yazılır.
- **Şekil 6.8:** muhasebe gereksinimi kavramsal + ayrıntıyı (indirim) karıştırır. Ayrıntı system req.'e bırakılmalı.
- ⭐ User ve system req. **ayrılmalı**, yoksa teknik olmayan okuyucu ayrıntıda boğulur.
- ⭐ **Şekil 6.9 (grid):** ilk cümle **3 tür** karıştırır: (1) ızgara sağla (kavramsal, functional), (2) cm ya da inç (detay, non-functional), (3) açma ve kapama (UI, non-functional). Başlangıç bilgisi de eksik.
- Fazla bilgi, geliştiricinin **yenilikçi çözüm** özgürlüğünü kısıtlar. **Temel özelliklere** odaklan.
- ⭐ **Şekil 6.10 formatı:** gereksinim (kalın) + **Rationale** + **Specification** referansı + **Source**. Örnekte **pasif ızgara** (aktif "snap-to" bilinçli olarak reddedildi).
- ⭐ Rationale: gereksinimin **neden** konduğu. Özellikle **değişiklik** sırasında faydalı.
- ⭐⭐ **Yazım için 4 kural:**
  - 1. **Standart format** (eksikleri azaltır, kontrolü kolaylaştırır).
  - 2. Dili **tutarlı** kullan: **shall = mandatory** (zorunlu), **should = desirable** (istenen).
  - 3. **Vurgula** (kalın, italik, renk).
  - 4. **Jargondan** kaçın.
- **VOLERE** (Robertson): her gereksinim ayrı **kart**ta (gerekçe, bağımlılıklar, kaynak, destek materyali). User ve system req. için.

Sayfa 129–131 — 6.3 System requirements · Tasarım bilgisi · Doğal dilin sorunları · Şekil 6.11

📖 Sayfanın özeti

TANIM

Sistem gereksinimleri, kullanıcı gereksinimlerinin **genişletilmiş** halleridir ve yazılım mühendisleri tarafından sistem tasarımının **başlangıç noktası** olarak kullanılır. Ayrıntı ekler ve kullanıcı gereksinimlerinin sistemde nasıl karşılanacağını açıklar. Gerçekleştirme **sözleşmesinin parçası** olarak kullanılabilirler, bu yüzden bütün sistemin **eksiksiz ve tutarlı** bir tanımlaması olmalıdırlar.

TASARIM BİLGİSİ NEDEN TAMAMEN DIŞLANAMAZ?

İdealde sistem gereksinimleri sadece sistemin **dış davranışını** ve çalışma kısıtlarını anlatmalı, nasıl tasarlanacağı ya da gerçekleştirileceğiyle ilgilenmemelidir. Ama karmaşık bir sistemi tam tanımlamak

için gereken ayrıntı seviyesinde bütün tasarım bilgisini dışlamak pratikte imkânsızdır. **3 neden:** (1) Gereksinim tanımlamasını yapılandırmak için sistemin bir **ilk mimarisi** tasarlanabilir ve gereksinimler alt sistemlere göre düzenlenir. Bu mimari tanımı, bileşenleri yeniden kullanmak istiyorsan şarttır. (2) Çoğu sistem mevcut başka sistemlerle **birlikte çalışmak** zorundadır, bunlar tasarımı kısıtlar ve yeni sisteme gereksinim getirir. (3) Fonksiyonel olmayan gereksinimleri (ör. güvenilirlik için **N-version programming**) karşılamak için belirli bir mimari gerekebilir. Sistemi sertifikalandıracak bir **dış düzenleyici** daha önce onaylanmış bir mimarinin kullanılmasını isteyebilir.

DOĞAL DİLİN SİSTEM GEREKSİNİMLERİNDEKİ 3 SORUNU

Doğal dil sistem gereksinimleri için de sık kullanılır, ama daha ayrıntılı oldukları için kafa karıştırıcı ve anlaşılması zor olabilir: **1)** Doğal dili anlamak, okuyan ve yazanın aynı kavram için **aynı kelimeleri** kullanmasına dayanır. Doğal dilin **belirsizliği (ambiguity)** yüzünden yanlış anlaşılmalar olur. Jackson'ın örneği: yürüyen merdivende görülen "Ayakkabı giyilmelidir" ve "Köpekler taşınmalıdır" levhaları farklı şekillerde yorumlanabilir. **2)** Doğal dil tanımlaması **fazla esnektir (overflexible)**: aynı şey tamamen farklı biçimlerde söylenebilir, gereksinimlerin ne zaman aynı ne zaman farklı olduğunu okuyucu çözmek zorunda kalır. **3)** Doğal dil gereksinimlerini **modülerleştirmenin** kolay yolu yoktur. İlgili bütün gereksinimleri bulmak zordur, bir değişikliğin etkisini anlamak için ilgili grup yerine **her gereksinime** bakmak gerekebilir. Bu yüzden yanlış anlaşılmalar çoğu zaman sürecin son aşamalarına kadar fark edilmez ve çözmesi çok pahalı olabilir.

ŞEKİL 6.11: ALTERNATİF GÖSTERİMLER

Gösterim	Kısaca
Structured natural language	Gereksinimleri ifade etmek için **standart form ya da şablonlar** tanımlanır.
Design description languages	Programlama diline benzeyen ama daha soyut bir dil. Sistemin işlemsel bir modelini tanımlar. Artık pek kullanılmaz, arayüz tanımlamaları için faydalı olabilir.
Graphical notations	Metin açıklamalarıyla desteklenen grafiksel dil. Eski örneği **SADT**. Bugün **use-case** açıklamaları ve **sequence diagram**'lar yaygın.
Mathematical specifications	**Sonlu durum makineleri** ya da **kümeler** gibi matematiksel kavramlara dayalı. Belirsizlik içermez, müşteri ile yüklenici arasındaki tartışmaları azaltır. Ama çoğu müşteri anlamaz ve sözleşme olarak kabul etmek istemez.

SEÇİM

Kullanıcı gereksinimleri uzman olmayanların anlayacağı dilde yazılmalıdır. Sistem gereksinimleri ise daha özel gösterimlerle yazılabilir. Bu bölümde grafiksel modellerle desteklenen yapılandırılmış doğal dil anlatılıyor (grafiksel modeller Bölüm 8'de, formal tanımlama Bölüm 10'da).

👨‍🏫 Hoca anlatımı

System requirements = user requirements'ın **ayrıntılı, teknik versiyonu**. Tasarımcının başlangıç noktasıdır ve sözleşmeye girebilir.

"Gereksinimde tasarım olmasın" kuralı ideal, ama pratikte biraz tasarım kaçınılmaz. 3 nedeni: (1) gereksinimleri düzenlemek için kaba bir mimari lazım, (2) var olan sistemlerle uyum tasarımı kısıtlar, (3) bazı fonksiyonel olmayan gereksinimler (güvenilirlik) belli bir mimariyi zorunlu kılar.

Doğal dilin belirsizliği için Jackson'ın yürüyen merdiven örneği çok komik ve öğretici: "Dogs must be carried" → "Köpeğin varsa onu kucağında taşı" mı, yoksa "Yürüyen merdivene binen herkes bir köpek taşımak zorunda" mı? "Shoes must be worn" → "Herkes ayakkabı giymeli" mi, yoksa "ayakkabı giymek zorundasın" mı? Aynı cümle, iki anlam.

4 gösterim (Şekil 6.11), basitten resmiye doğru: yapılandırılmış doğal dil (form doldurmak), tasarım tanımlama dilleri (programlama diline benzer), grafiksel gösterimler (use-case, sequence diagram), matematiksel tanımlamalar (en kesin ama müşteri anlamaz).

📝 Deftere not

- ⭐ **System requirements:** user requirements'ın **genişletilmiş** hali, **tasarımın başlangıç noktası**. Ayrıntı ekler, nasıl karşılanacağını açıklar. **Sözleşmenin parçası** olabilir, bu yüzden **complete + consistent** olmalı.
- İdealde sadece **dış davranış ve kısıtlar**, tasarım yok.
- ⭐⭐ **Tasarım bilgisi neden tamamen dışlanamaz? (3 neden)**
  - 1. Spec'i yapılandırmak için **ilk mimari** gerekir (alt sistemlere göre). Yeniden kullanım için şart.
  - 2. Sistemler mevcut sistemlerle **birlikte çalışmak (interoperate)** zorunda, bu tasarımı kısıtlar.
  - 3. NFR'ler için belirli **mimari** gerekebilir (ör. **N-version programming**). **Dış düzenleyici** onaylı mimari isteyebilir.
- ⭐⭐ **Doğal dilin sistem gereksinimlerindeki 3 sorunu:**
  - 1. **Ambiguity:** okuyan ve yazan aynı kelimeleri aynı anlamda kullanmalı. Ör (Jackson): "**Shoes must be worn**", "**Dogs must be carried**".
  - 2. **Overflexible:** aynı şey farklı şekillerde söylenebilir, gereksinim aynı mı farklı mı anlaşılmaz.
  - 3. **Modülerleştirmek zor:** ilgili gereksinimleri bulmak zor, değişiklik etkisi için **her gereksinime** bakmak gerekir.

  Sonuç: yanlış anlaşılmalar **geç** fark edilir ve çözmek **çok pahalı** olur.
- ⭐⭐ **Şekil 6.11, gösterimler:**
  - **Structured natural language:** standart **form ya da şablon**.
  - **Design description languages:** programlama diline benzer, soyut. Artık az kullanılır (arayüz için faydalı).
  - **Graphical notations:** grafik + metin. **SADT** (eski), **use-cases**, **sequence diagrams**.
  - **Mathematical specifications: finite-state machines**, **sets**. Belirsizlik yok, ama müşteri **anlamaz** ve sözleşme olarak kabul etmek istemez.
- User req. → uzman olmayanların dilinde. System req. → daha özel gösterimler olabilir.

Sayfa 131–134 6.3.1 Structured language specifications · Şekil 6.12, 6.13, 6.14

📖 Sayfanın özeti

YAPILANDIRILMIŞ DOĞAL DİL

Gereksinim yazarının özgürlüğünün **sınırlandığı** ve bütün gereksinimlerin **standart bir şekilde** yazıldığı bir yöntemdir. Avantajı, doğal dilin ifade gücünü ve anlaşılırlığını büyük ölçüde korurken tanımlamaya bir **tekdüzelik (uniformity)** kazandırmasıdır. Yapılandırılmış dil gösterimleri

kullanılabilecek terminolojiyi sınırlar ve gereksinimleri **şablonlarla** tanımlar. Programlama dillerinden alınan kontrol yapıları ve tanımlamayı bölümlere ayıran grafik vurgulama içerebilir. Erken bir örnek **Heninger**'in projesidir: bir uçak yazılım sisteminin girdi, çıktı ve fonksiyonlarını tanımlamak için özel formlar tasarlanmıştır.

FORM TABANLI YAKLAŞIM (ŞEKİL 6.12)

Bir ya da daha fazla standart form veya şablon tanımlanır. Tanımlama, sistemin işlediği **nesneler**, gerçekleştirdiği **fonksiyonlar** ya da işlediği **olaylar** etrafında yapılandırılabilir. Şekil 6.12, Bölüm 3'teki **insülin pompası** için bir örnek gösterir: pompa insülin ihtiyacını kan şekerinin **değişim hızına** göre hesaplar. Form şu alanları içerir: **Function** (insülin dozunu hesapla: güvenli şeker seviyesi), **Description** (ölçülen şeker güvenli bölgedeyken (3 ile 7 birim arası) verilecek dozu hesaplar), **Inputs** (şu anki okuma r2 ve önceki iki okuma r0, r1), **Source** (şu anki okuma sensörden, diğerleri bellekten), **Outputs** (CompDose, verilecek doz), **Destination** (ana kontrol döngüsü), **Action** (şeker sabit ya da düşüyorsa ya da artış hızı azalıyorsa doz sıfır, artış hızı artıyorsa doz, mevcut ve önceki seviye farkının 4'e bölünüp yuvarlanmasıyla bulunur, sonuç sıfırsa verilebilecek en küçük doz kullanılır), **Requires** (değişim hızını hesaplamak için önceki iki okuma), **Pre-condition** (haznede en az izin verilen en büyük tek doz kadar insülin olmalı), **Post-condition** (r0 yerine r1, r1 yerine r2 geçer), **Side effects** (yok).

FORMDA BULUNMASI GEREKEN 7 BİLGİ

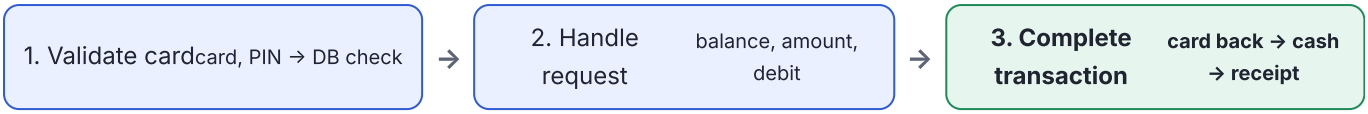
Fonksiyonel gereksinimler standart formla yazılınca şunlar olmalıdır: (1) tanımlanan fonksiyonun ya da varlığın **açıklaması**, (2) **girdileri** ve nereden geldikleri, (3) **çıktıları** ve nereye gittikleri, (4) kullanılan diğer varlıklar (**requires** kısmı), (5) yapılacak **eylemin** açıklaması, (6) fonksiyonel yaklaşım kullanılıyorsa fonksiyon çağrılmadan önce doğru olması gerekenleri belirten **pre-condition** ve çağrıldıktan sonra doğru olanı belirten **post-condition**, (7) varsa işlemin **yan etkileri (side effects)**.

SINIRLARI VE TABLOLAR (ŞEKİL 6.13)

Biçimlendirilmiş tanımlamalar doğal dilin bazı sorunlarını giderir: tanımlamadaki **değişkenlik azalır** ve gereksinimler daha iyi düzenlenir. Ama özellikle karmaşık hesaplamalarda gereksinimleri belirsizlik olmadan yazmak yine zordur. Ör: Şekil 6.12'de pre-condition sağlanmazsa ne olacağı açık değildir. Bunu çözmek için doğal dil gereksinimlerine **tablolar** ya da **grafiksel modeller** eklenebilir. Bunlar hesaplamaların nasıl ilerlediğini, durumun nasıl değiştiğini, kullanıcıların nasıl etkileştiğini ve eylem dizilerinin nasıl yapıldığını gösterebilir. **Tablolar**, birçok olası alternatif durum varsa ve her biri için yapılacak eylemi anlatmak gerekiyorsa özellikle faydalıdır. Şekil 6.13, insülin dozu hesabının tablo halidir: **şeker düşüyorsa** doz 0, **sabitse** doz 0, **artıyor ama artış hızı azalıyorsa** doz 0, **artıyor ve artış hızı sabit ya da artıyorsa** doz = (r2 − r1)/4 yuvarlanmış, sonuç 0 ise en küçük doz.

SEQUENCE DİAGRAM (ŞEKİL 6.14)

Grafiksel modeller, durumun nasıl değiştiğini göstermek ya da bir **eylem dizisini** anlatmak için en faydalısıdır. Şekil 6.14, bir kullanıcının **ATM**'den para çekmesindeki eylem sırasını gösterir. Sequence diagram, eylemlerin sırasını görmek için **yukarıdan aşağı** okunur. Katılımcılar kullanıcı, ATM ve veritabanıdır. Üç temel alt dizi vardır: **1) Validate card:** kart numarası ve kullanıcının PIN'i kontrol edilerek kart doğrulanır (geçersiz kart istisnası). **2) Handle request:** kullanıcının isteği işlenir. Para çekmede bakiyeyi kontrol etmek ve çekilen tutarı hesaptan düşmek için veritabanı sorgulanır. Hesapta yeterli para yoksa bir **istisna** (yetersiz bakiye) vardır. **3) Complete transaction:** kart iade edilir, kart alınınca para ve fiş verilir. Sequence diagram'lar Bölüm 8 ve 14'te tekrar geliyor.



Şekil 6.14 (sadeleştirilmiş): ATM'den para çekme sequence diagram'ının 3 alt dizisi

👩‍🏫 Hoca anlatımı

Structured natural language = boşlukları doldurulan bir **form**. Hâlâ normal cümleler kullanırsın ama her gereksinim aynı alanlara sahiptir: fonksiyon, açıklama, girdi, kaynak, çıktı, hedef, eylem, ön koşul, son koşul, yan etki. Bir hastane kayıt formu gibi: herkes aynı başlıkları doldurur, eksik bir şey olunca hemen fark edilir.

Pre-condition ve post-condition farkını iyi anla: **Pre** = fonksiyon çalışmadan **önce** doğru olması gereken (hazne boş olmamalı). **Post** = fonksiyon çalıştıktan **sonra** doğru olan (okumalar bir kaydırıldı).

Tablolar "eğer şöyleyse böyle yap" türü çok durumlu kurallarda harikadır. İnsülin örneğinde 4 durum ve 4 eylem tek tabloda net görünüyor.

Sequence diagram sınavda çizdirilebilir (alıştırma 6.5: bilet makinesi). Kural: en üstte katılımcılar (kullanıcı, makine, veritabanı), aşağı doğru zaman akar, oklar mesajlardır. ATM örneğini şablon olarak kullan: **doğrula → isteği işle → işlemi tamamla**, arada da hata durumları (exception).

📝 Deftere not

- ⭐ **Structured natural language:** yazarın özgürlüğü **sınırlı**, tüm gereksinimler **standart şekilde**. Avantaj: doğal dilin anlaşılırlığı korunur + **tekdüzelik (uniformity)**. Terminoloji sınırlı, **şablonlar**, programlama kontrol yapıları, grafik vurgulama. Erken örnek: **Heninger** (uçak yazılımı formları).
- Form tabanlı: tanımlama **nesneler**, **fonksiyonlar** ya da **olaylar** etrafında yapılandırılır.
- ⭐ **Şekil 6.12 (insülin pompası formu) alanları:** Function · Description · Inputs · Source · Outputs · Destination · Action · Requires · **Pre-condition** · **Post-condition** · Side effects. (Güvenli bölge: 3–7 birim. Doz değişim hızına göre.)
- ⭐⭐ **Standart formdaki 7 bilgi:** (1) fonksiyon ya da varlığın **açıklaması**, (2) **girdiler** + kaynağı, (3) **çıktılar** + hedefi, (4) kullanılan diğer varlıklar (**requires**), (5) **eylem**, (6) **pre-condition** (önce doğru olan) ve **post-condition** (sonra doğru olan), (7) **side effects**.
- Avantaj: **değişkenlik azalır**, düzen artar. Ama karmaşık hesaplamada belirsizlik kalabilir (ör. pre-condition sağlanmazsa ne olur?).
- ⭐ Çözüm: **tablolar** + **grafiksel modeller** ekle.
  - **Tablolar:** çok sayıda **alternatif durum** ve her birinin eylemi. **Şekil 6.13:** şeker düşüyor / sabit / artıyor ama hız azalıyor → doz 0. Artıyor ve hız sabit ya da artıyor → doz = round((r2−r1)/4), 0 ise minimum doz.
  - **Grafiksel modeller: durum değişimi** ya da **eylem dizisi** göstermek için.
- ⭐⭐ **Sequence diagram (Şekil 6.14, ATM): yukarıdan aşağı** okunur. Katılımcılar: kullanıcı, ATM, veritabanı. **3 alt dizi:**
  - **Validate card:** kart numarası + **PIN** kontrolü (exception: invalid card).
  - **Handle request:** veritabanında **bakiye** kontrolü, tutarı **düş (debit)** (exception: insufficient cash).

- **Complete transaction:** kart iade → kart alınınca **para** ve **fiş**.

Sayfa 135–136 6.4 Interface specification (Arayüz tanımlama) · Şekil 6.15

📖 Sayfanın özeti

NEDEN GEREKLİ?

Neredeyse bütün yazılım sistemleri, ortamda zaten kurulu olan **mevcut sistemlerle** birlikte çalışmak zorundadır. Yeni sistem ile mevcut sistemler birlikte çalışacaksa mevcut sistemlerin arayüzleri **kesin olarak** tanımlanmalıdır. Bu tanımlamalar sürecin **başında** yapılmalı ve gereksinim belgesine (belki ek olarak) konmalıdır.

3 TÜR ARAYÜZ

1) Procedural interfaces: mevcut programların ya da alt sistemlerin, arayüz prosedürleri çağrılarak erişilen hizmetler sunduğu arayüzlerdir. Bunlara bazen **API (Application Programming Interface)** denir. **2) Data structures:** bir alt sistemden diğerine aktarılan veri yapılarıdır. Bunları anlatmak için en iyi gösterim **grafiksel veri modelleridir** (Bölüm 8). Gerekirse bu tanımlardan Java ya da C++ program tanımları otomatik üretilebilir. **3) Representations of data:** mevcut bir alt sistem için kurulmuş veri gösterimleri, ör. **bitlerin sıralaması**. En çok **gömülü, gerçek zamanlı** sistemlerde görülür. Ada gibi bazı diller (Java değil) bu seviyede tanımlamayı destekler, ama en iyi yol muhtemelen yapının diyagramını çizip her bit grubunun işlevini açıklayan notlar eklemektir.

FORMAL GÖSTERİM VE JAVA

Formal gösterimler (Bölüm 10) arayüzlerin **belirsizlik olmadan** tanımlanmasını sağlar, ama özel eğitim olmadan anlaşılmaz. Yazara göre bu amaca çok uygun olsalar da pratikte nadiren kullanılırlar. Arayüzün sözdizimini (syntax) anlatmak için **Java** gibi bir programlama dili kullanılabilir, ama tanımlanan her işlemin anlamını (**semantics**) açıklayan ek bir açıklama gerekir.

ÖRNEK (ŞEKİL 6.15)

Java ile tanımlanmış bir **yazdırma sunucusu (print server)** işlemsel arayüzüdür. Sunucu, farklı yazıcılarda yazdırılacak dosyaların istek **kuyruğunu** yönetir. Kullanıcılar bir yazıcının kuyruğunu görebilir, kendi işlerini kuyruktan kaldırabilir ve işleri bir yazıcıdan diğerine aktarabilir. Arayüzde başlatma (initialize), yazdırma (print), kuyruğu görüntüleme (displayPrintQueue), iş iptali (cancelPrintJob) ve yazıcı değiştirme (switchPrinter) işlemleri tanımlıdır. Arayüz yazıcı ve belge arayüzlerine ihtiyaç duyar. Bu, arayüz ayrıntılarını göstermeyen **soyut bir modeldir**. İşlemlerin işlevselliği yapılandırılmış doğal dille ya da tablolarla tanımlanabilir.

👨‍🏫 Hoca anlatımı

Yeni bir yazılım genelde boşlukta yaşamaz: bankanın eski sistemiyle, yazıcıyla, başka bir şirketin servisiyle konuşması gerekir. Nasıl konuşacaklarını **en baştan** net yazmazsan sonra "bizim sistem senin gönderdiğin veriyi anlamıyor" kavgası çıkar.

3 arayüz türü: (1) **Procedural** (API): "şu fonksiyonu çağır, şu işi yapsın". Ör: Google Maps API'sini çağırıp harita almak. (2) **Data structures**: sistemler arası taşınan veri paketleri (müşteri kaydının yapısı gibi). (3) **Representations of data**: en alt seviye, bitlerin dizilişi. Bir sensörün gönderdiği 8 bitten ilk 3'ü sıcaklık, kalan 5'i nem gibi. Gömülü sistemlerde yaygın.

Java ile yazılan arayüz (Şekil 6.15) sadece **"ne çağrılabilir"** (sözdizimi) gösterir. **"Ne işe yarar"** (anlam) için ayrıca açıklama yazman gerekir.

📝 Deftere not

- ⭐ Neredeyse her sistem **mevcut sistemlerle** çalışmak zorunda, bu yüzden arayüzler **kesin** tanımlanmalı, **süreç başında**, gereksinim belgesine (ek olarak) konmalı.
- ⭐⭐ **3 arayüz türü:**
  - Procedural interfaces: hizmetlere **arayüz prosedürleri çağrılarak** erişilir. = **API**.
  - Data structures: alt sistemler arası aktarılan veri yapıları. En iyi gösterim: **grafiksel veri modelleri** (Java ya da C++ kodu otomatik üretilebilir).
  - Representations of data: ör. **bit sıralaması**. En çok **gömülü, real-time** sistemlerde. Ada destekler (Java değil). En iyisi: diyagram + bit gruplarının açıklaması.
- Formal gösterimler: **belirsizliksiz** ama özel eğitim ister, pratikte **nadir**.
- ⭐ Java ile arayüz: **syntax**'ı gösterir. **Semantics** (anlam) için **ek açıklama** gerekir.
- **Şekil 6.15:** Java ile **PrintServer** arayüzü (initialize, print, displayPrintQueue, cancelPrintJob, switchPrinter). Yazıcı kuyruklarını yönetir. **Soyut model**, işlevler yapılandırılmış dil ya da tabloyla tanımlanabilir.

Sayfa 136–139 — 6.5 The software requirements document (SRS) · Şekil 6.16 · IEEE 830 · Şekil 6.17

📖 Sayfanın özeti

SRS NEDİR?

Yazılım gereksinim belgesi (bazen **software requirements specification, SRS** denir), geliştiricilerin neyi gerçekleştirmesi gerektiğinin **resmi ifadesidir**. Hem kullanıcı gereksinimlerini hem de sistem gereksinimlerinin ayrıntılı tanımlamasını içermelidir. Bazen ikisi tek bir açıklamada birleştirilir, bazen kullanıcı gereksinimleri sistem gereksinimleri tanımlamasının girişinde yer alır. Gereksinim çok fazlaysa ayrıntılı sistem gereksinimleri **ayrı bir belgede** sunulabilir.

KULLANICILARI (ŞEKİL 6.16)

Belgenin çok çeşitli kullanıcıları vardır: **System customers** (gereksinimleri belirtir, ihtiyaçlarını karşılayıp karşılamadığını kontrol etmek için okur, değişiklikleri belirtir), **Managers** (sistem için teklif planlamak ve geliştirme sürecini planlamak için kullanır), **System engineers** (hangi sistemin geliştirileceğini anlamak için kullanır), **System test engineers** (sistem için geçerleme testleri geliştirmek için kullanır), **System maintenance engineers** (sistemi ve parçaları arasındaki ilişkileri anlamak için kullanır).

BİR UZLAŞMA BELGESİ

Bu çeşitlilik yüzünden belge, gereksinimleri müşterilere **iletmek**, geliştiriciler ve test edenler için **kesin ayrıntıda** tanımlamak ve sistemin olası **evrimi** hakkında bilgi vermek arasında bir **uzlaşmadır**. Beklenen değişiklikler hakkındaki bilgi, tasarımcıların kısıtlayıcı tasarım kararlarından kaçınmasına ve bakım mühendislerinin sistemi yeni gereksinimlere uyarlamasına yardım eder.

AYRINTI SEVİYESİ

Belgedeki ayrıntı seviyesi sistemin türüne ve kullanılan sürece bağlıdır. Sistem bir **dış yüklenici** tarafından geliştirilecekse kritik sistem tanımlamaları **kesin ve çok ayrıntılı** olmalıdır. Gereksinimlerde daha fazla esneklik varsa ve kurum içi, **yinelemeli** bir süreç kullanılıyorsa belge **çok daha az ayrıntılı** olabilir ve belirsizlikler geliştirme sırasında çözülür.

IEEE/ANSI 830-1998

ABD Savunma Bakanlığı ve IEEE gibi büyük kurumlar gereksinim belgeleri için standartlar tanımlamıştır. En bilineni **IEEE/ANSI 830-1998**'dir ve şu yapıyı önerir: **1. Introduction** (belgenin amacı, ürünün kapsamı, tanımlar ve kısaltmalar, referanslar, belgenin geri kalanına genel bakış), **2. General description** (ürün perspektifi, ürün fonksiyonları, kullanıcı özellikleri, genel kısıtlar, varsayımlar ve bağımlılıklar), **3. Specific requirements** (fonksiyonel, fonksiyonel olmayan ve arayüz gereksinimleri. Belgenin en kapsamlı kısmıdır, ama kurumlar arası farklılık çok büyük olduğu için standart bir yapı tanımlanmamıştır. Dış arayüzler, sistem işlevselliği ve performansı, mantıksal veritabanı gereksinimleri, tasarım kısıtları, emergent özellikler ve kalite özellikleri belgelenebilir), **4. Appendices**, **5. Index**.

DEĞERLENDİRME

IEEE standardı ideal değildir ama gereksinim yazma ve sorunlardan kaçınma konusunda çok iyi tavsiye içerir. Kendi başına bir kurum standardı olamayacak kadar **geneldir**. Bir kurumun ihtiyacına göre uyarlanabilecek **genel bir çerçevedir**. Yazar, Şekil 6.17'de IEEE'ye dayalı bir yapı öneriyor ve buna öngörülen **sistem evrimi** bilgisini ekliyor (ilk kez **Heninger** önermiştir). Bu, bakım yapanlara yardım eder ve tasarımcıların gelecekteki özellikler için destek koymasını sağlayabilir.

Bölüm (Şekil 6.17)	Kısaca içeriği
Preface	Beklenen okuyucular, sürüm geçmişi, yeni sürümün gerekçesi ve değişikliklerin özeti
Introduction	Sistemin neden gerektiği, fonksiyonları, diğer sistemlerle nasıl çalışacağı, kurumun iş ya da stratejik hedeflerine nasıl uyduğu
Glossary	Belgedeki teknik terimlerin tanımları (okuyucunun deneyimi hakkında varsayım yapma)
User requirements definition	Kullanıcıya verilen hizmetler ve fonksiyonel olmayan sistem gereksinimleri. Müşterinin anlayacağı dil ve diyagramlar. Uyulacak ürün ve süreç standartları.
System architecture	Beklenen sistem mimarisine üst seviye bakış, fonksiyonların modüllere dağılımı, yeniden kullanılan mimari bileşenler
System requirements specification	Fonksiyonel ve fonksiyonel olmayan gereksinimlerin ayrıntısı, gerekirse diğer sistemlere arayüzler
System models	Bileşenler ve sistem ile ortamı arasındaki ilişkileri gösteren modeller (nesne, veri akışı, semantik veri modelleri)
System evolution	Sistemin dayandığı temel varsayımlar ve donanım evrimi, değişen kullanıcı ihtiyaçları gibi nedenlerle beklenen değişiklikler
Appendices	Uygulamaya özel ayrıntılı bilgi: donanım (minimum ve optimum konfigürasyon) ve veritabanı (verinin mantıksal düzeni ve ilişkileri) tanımları

Index	Alfabetik dizin, diyagram dizini, fonksiyon dizini vb.

BELGENİN İÇERİĞİ SÜRECE BAĞLIDIR

Bir yazılım ürünü için **evrimsel** yaklaşım kullanılıyorsa belge ayrıntılı bölümlerin çoğunu dışarıda bırakır ve **kullanıcı gereksinimlerine** ve üst seviye fonksiyonel olmayan gereksinimlere odaklanır. Tasarımcılar ve programcılar ana hatları çizilmiş gereksinimleri nasıl karşılayacaklarına kendi muhakemeleriyle karar verir. Yazılım, etkileşen donanım ve yazılım içeren **büyük bir sistem mühendisliği** projesinin parçasıysa gereksinimler çok ince ayrıntıyla tanımlanmalıdır. Bu belgeler çok uzun olur ve Şekil 6.17'deki bölümlerin çoğunu içerir. Uzun belgelerde okuyucunun bilgiyi bulabilmesi için kapsamlı bir **içindekiler** ve **dizin** önemlidir.

AGİLE YAKLAŞIM

Gereksinim belgeleri, yazılımı **dış bir yüklenici** geliştirirken şarttır. Ama **agile** yöntemler, gereksinimlerin o kadar hızlı değiştiğini savunur ki belge daha yazılırken eskir ve emek boşa gider. **Extreme programming** gibi yaklaşımlar resmi belge yerine kullanıcı gereksinimlerinin artımlı olarak toplanıp **kartlara** yazılmasını önerir. Kullanıcı bir sonraki artımda gerçekleştirilecek gereksinimleri önceliklendirir. Yazara göre gereksinimleri kararsız olan iş sistemleri için bu iyi bir yaklaşımdır. Ama sistemin iş ve güvenilebilirlik gereksinimlerini tanımlayan **kısa destekleyici bir belge** yazmak yine faydalıdır, çünkü bir sonraki sürümün fonksiyonel gereksinimlerine odaklanırken bütün sisteme uygulanan gereksinimler unutulabilir.

👨‍🏫 Hoca anlatımı

SRS = sistemin **"resmi sözleşmesi"**. "Geliştiriciler ne yapacak?" sorusunun resmi cevabı.

Şekil 6.16'daki 5 kullanıcı sınavda sorulabilir: **müşteri** (istiyor ve kontrol ediyor), **yönetici** (teklif ve planlama), **sistem mühendisi** (ne yapacağını anlıyor), **test mühendisi** (testleri hazırlıyor), **bakım mühendisi** (sonradan sistemi anlıyor). Herkes aynı belgeyi farklı amaçla okur, bu yüzden belge bir **uzlaşmadır**.

IEEE 830 yapısı ezberlenmeli: **Introduction → General description → Specific requirements → Appendices → Index.** En büyük kısım "Specific requirements" ama onun yapısı standartta tanımlanmamış (şirketlere göre çok değişiyor).

Ayrıntı seviyesi mantığı: dışarıdaki bir firmaya yaptırıyorsan her şeyi çok net yazarsın (sözleşme). Kendi ekibinle, adım adım yapıyorsan daha kısa yazıp belirsizlikleri yol üstünde çözersin.

Agile tartışması: XP "belge yazmak boşa zaman, çünkü gereksinimler sürekli değişiyor, kartlara yaz" der. Yazar "iş sistemlerinde haklılar ama bütün sistemi kapsayan kısa bir belge yine yazılmalı" diyor.

📝 Deftere not

- ⭐ **SRS (software requirements specification):** geliştiricilerin ne gerçekleştireceğinin **resmi ifadesi**. **User req. + ayrıntılı system req.** içerir (birleşik, giriş + spec ya da ayrı belgeler).
- ⭐⭐ **Şekil 6.16, belgenin kullanıcıları:**
  - **System customers:** gereksinimleri belirtir, ihtiyacı karşılıyor mu diye okur, **değişiklik** belirtir.
  - **Managers: teklif** ve geliştirme sürecini **planlamak** için.
  - **System engineers:** hangi sistemin geliştirileceğini **anlamak** için.

- **System test engineers: geçerleme testleri** geliştirmek için.
- **System maintenance engineers:** sistemi ve parçaların ilişkilerini anlamak için.

- ⭐ Belge bir **uzlaşma (compromise)**: müşteriye iletmek + geliştirici ve test için kesin ayrıntı + **evrim** bilgisi (kısıtlayıcı tasarımdan kaçınma, bakımı kolaylaştırma).
- ⭐ **Ayrıntı seviyesi** sistem türüne ve sürece bağlı: **dış yüklenici** → çok ayrıntılı ve kesin. **Kurum içi, yinelemeli** → daha az ayrıntı.
- ⭐⭐ **IEEE/ANSI 830-1998 yapısı:**
  - **1. Introduction:** amaç, kapsam, tanımlar ve kısaltmalar, referanslar, genel bakış.
  - **2. General description:** ürün perspektifi, ürün fonksiyonları, kullanıcı özellikleri, genel kısıtlar, varsayımlar ve bağımlılıklar.
  - **3. Specific requirements:** functional, non-functional, interface. **En kapsamlı** kısım, yapısı **standartta tanımlı değil**.
  - **4. Appendices** · **5. Index**.
- IEEE ideal değil, **çok genel**. Kuruma göre uyarlanacak **çerçeve**.
- ⭐ **Şekil 6.17 (yazarın yapısı):** Preface · Introduction · Glossary · User requirements definition · System architecture · System requirements specification · System models · **System evolution** (Heninger'in fikri) · Appendices (donanım, veritabanı) · Index.
- **Evrimsel yaklaşım:** ayrıntılı bölümler atlanır, user req. ve üst seviye NFR'ye odaklanılır. **Büyük sistem mühendisliği projesi:** çok ayrıntılı, uzun, içindekiler ve dizin önemli.
- ⭐ **Agile (XP):** gereksinimler o kadar hızlı değişir ki belge yazılırken **eskir**. Kullanıcı gereksinimleri artımlı olarak **kartlara** yazılır, kullanıcı bir sonraki artım için önceliklendirir. Yazar: kararsız iş sistemleri için iyi, ama iş ve güvenilebilirlik gereksinimleri için **kısa destek belgesi** yine faydalı.

Sayfa 140–141 Key Points · Further Reading · Exercises

📖 Sayfanın özeti

KEY POİNTS (BÖLÜM ÖZETİ)

Bir yazılım sisteminin gereksinimleri sistemin ne yapması gerektiğini ortaya koyar ve çalışması ile gerçekleştirilmesi üzerindeki kısıtları tanımlar. Fonksiyonel gereksinimler sistemin vermesi gereken hizmetlerin ifadeleri ya da bazı hesaplamaların nasıl yapılacağının tanımlarıdır. Alan gereksinimleri, uygulama alanının özelliklerinden türeyen fonksiyonel gereksinimlerdir. Fonksiyonel olmayan gereksinimler geliştirilen sistemi ve kullanılacak geliştirme sürecini kısıtlar. Ürün, kurum ya da dış gereksinimler olabilir. Çoğu zaman sistemin emergent özellikleriyle ilgilidir ve bu yüzden sistemin bütününe uygulanır. Kullanıcı gereksinimleri sistemi kullanan ve satın alan kişiler içindir. Kolay anlaşılan tablolar ve diyagramlarla desteklenen doğal dille yazılmalıdır. Sistem gereksinimleri, sistemin vermesi gereken fonksiyonları kesin bir şekilde iletmeyi amaçlar. Belirsizliği azaltmak için tablolar ve sistem modelleriyle desteklenen yapılandırılmış doğal dille yazılabilir. Yazılım gereksinim belgesi, sistem gereksinimlerinin **üzerinde anlaşılmış** ifadesidir ve hem müşteriler hem geliştiriciler tarafından kullanılabilecek şekilde düzenlenmelidir. Gereksinim belgeleri için IEEE standardı, daha özel gereksinim tanımlama standartları için faydalı bir başlangıç noktasıdır.

FURTHER READİNG

Weigers'in *Software Requirements*, Robertson'ların *Mastering the Requirements Process* (VOLERE), Kotonya ve Sommerville'in *Requirements Engineering* kitapları ve IEEE standardını tartışan makaleler. *Sınav için önemli değil.*

ALIŞTIRMALAR (KONULARI)

#	Konu
6.1	Bilgisayar tabanlı bir sistem için tanımlanabilecek **4 tür gereksinimi** belirleyip kısaca açıklayın.
6.2	Gereksinimleri tanımlamak için **doğal dil** kullanmanın sorunları nelerdir? Yapılandırılmış doğal dil formlarının bunlardan bazılarını nasıl önlediğini küçük örneklerle gösterin.
6.3	Bir **tren bileti satış sistemi** tanımında belirsizlik ve eksikleri bulun.
6.4	Aynı tanımı bölümdeki **yapılandırılmış yaklaşımla** yeniden yazın.
6.5	Bilet sistemindeki eylemleri gösteren bir **sequence diagram** çizin (kullanıcı hatalarına dikkat).
6.6	Standart biçimli doğal dille şu fonksiyonlar için kullanıcı gereksinimleri yazın: ATM'de para verme, kelime işlemcide yazım denetimi, kredi kartlı insansız benzin pompası.
6.7	**4 tür fonksiyonel olmayan gereksinimi** açıklayıp her birine örnek verin.
6.8	Bilet sistemi için güvenilirlik ve yanıt süresini belirten fonksiyonel olmayan gereksinimler yazın.
6.9	Fonksiyonel ve fonksiyonel olmayan gereksinimler arasındaki ilişkiler nasıl takip edilebilir?
6.10	Eski işvereninle yeni işvereninin gereksinim yorumu farklı. Gizlilik yükümlülüğün varken ne yaparsın? (etik)

👨‍🏫 Hoca anlatımı

6.1'deki "4 tür gereksinim": user requirements, system requirements, functional ve non-functional (bazı hocalar domain'i de ister). Cevap kalıbını bölüm sonuna koydum.

6.3 / 6.4 / 6.5 / 6.8 uygulama soruları. Bilet makinesinde aranacak belirsizlikler: kart reddedilirse ne olur? Kullanıcı yanlış PIN girerse? Tek yön mü gidiş-dönüş mü? Bilet fiyatı ne zaman gösterilir? Kaç bilet alınabilir? Bunları yapılandırılmış forma dökersin (Şekil 6.12 alanlarıyla) ve sequence diagram'ı ATM örneği gibi çizersin.

6.8 için: "Sistem güvenilir olmalı" yazma! **Ölçülebilir** yaz: "Sistem ayda en fazla 1 kez arızalanmalı (availability %99.9)", "kart doğrulama 5 saniyeyi geçmemeli", "bilet 10 saniyede basılmalı".

📝 Deftere not

- ⭐ **KEY POINTS:** gereksinimler = ne yapmalı + kısıtlar · functional = hizmetler ya da hesaplamaların nasıl yapılacağı · domain = alandan türeyen functional req. · non-functional = sistemi ve süreci kısıtlar, product / organisational / external, emergent özelliklerle ilgili, **bütüne** uygulanır · user req. = kullanan ve satın alanlar için, **doğal dil + tablo ve diyagram** · system req. = fonksiyonları **kesin** iletir, **yapılandırılmış dil + tablo + model** · SRS = **üzerinde anlaşılmış** ifade, müşteri ve geliştirici kullanır · **IEEE standardı** = faydalı başlangıç noktası.
- ⭐ Sınavda en olası: **6.1, 6.2, 6.7** + uygulama (**6.3–6.5, 6.8**).

- ⭐ NFR yazarken **ölçülebilir** yaz (süre, oran, sayı). "Güvenilir olsun" yetmez.

BÖLÜM 6 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 6'daki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
requirement	gereksinim	Sistemin hizmetleri ve kısıtlarının tanımı	118
requirements engineering (RE)	gereksinim mühendisliği	Gereksinimleri bulma, analiz, belgeleme, kontrol	118
operational constraints	çalışma kısıtları	Sistemin çalışmasına konan sınırlar	118
user requirements	kullanıcı gereksinimleri	Doğal dil + diyagram, soyut	118
system requirements	sistem gereksinimleri	Ayrıntılı, kesin tanımlama	118
functional specification	fonksiyonel tanımlama	Sistem gereksinim belgesinin diğer adı	118
LIBSYS	kütüphane sistemi	Bölümün örnek sistemi	119–120
copyright licensing	telif hakkı lisansı	LIBSYS'in takip ettiği veri	119
functional requirements	fonksiyonel gereksinimler	Sistemin vermesi gereken hizmetler	119–120
non-functional requirements	fonksiyonel olmayan gereksinimler	Hizmetler üzerindeki kısıtlar, sistemin bütünü	119, 121
domain requirements	alan gereksinimleri	Uygulama alanından türeyen gereksinimler	120, 125
authentication	kimlik doğrulama	Güvenlikten türeyen fonksiyonel gereksinim	120
imprecision / ambiguity	belirsizlik	Farklı yorumlanabilen ifade	121, 130
completeness	eksiksizlik	İstenen tüm hizmetlerin tanımlanması	121
consistency	tutarlılık	Gereksinimlerin çelişmemesi	121
stakeholders	paydaşlar	Sistemden etkilenen ve onu etkileyen kişiler	121
store occupancy	bellek kullanımı	NFR örneği	121

product requirements	ürün gereksinimleri	Ürün davranışı (hız, bellek, güvenilirlik...)	122–123
efficiency / performance / space	verimlilik / performans / alan	Product req. alt türleri	122
portability	taşınabilirlik	Farklı platformlarda çalışabilme	122
organisational requirements	kurumsal gereksinimler	Kurum politika ve prosedürlerinden	122–123
delivery / implementation / standards	teslim / gerçekleştirme / standartlar	Organisational req. alt türleri	122
external requirements	dış gereksinimler	Sistem dışı etkenlerden	122–123
interoperability	birlikte çalışabilirlik	Başka sistemlerle etkileşim	122
ethical / legislative	etik / yasal	External req. alt türleri	122
privacy	gizlilik, mahremiyet	Legislative req. alt türü	122
system goal	sistem hedefi	Belirsiz, test edilemeyen istek	124
verifiable / testable	doğrulanabilir / test edilebilir	Nesnel olarak kontrol edilebilen	124
quantitative	nicel	Sayıyla ifade edilen	124
metrics	ölçütler	NFR'yi ölçmek için birimler	124–125
mean time to failure (MTTF)	arızalar arası ortalama süre	Reliability ölçütü	125
robustness	sağlamlık	Arızaya dayanma ve toparlanma	125
trade-off	ödünleşim	Çatışan gereksinimler arası tercih	125
domain expert	alan uzmanı	Uygulama alanını bilen kişi	126
lack of clarity	açıklık eksikliği	Doğal dil sorunu	127
requirements confusion	gereksinim karışıklığı	Gereksinim türlerinin karışması	127
requirements amalgamation	gereksinim birleşmesi	Birçok gereksinimin tek cümlede olması	127
rationale	gerekçe	Gereksinimin neden konduğu	128–129
mandatory / desirable	zorunlu / istenen	"shall" / "should"	129

jargon	teknik jargon	Uzman olmayanın anlamadığı terimler	129
VOLERE	VOLERE yöntemi	Gereksinimleri kartlara yazma	129
N-version programming	N-sürümlü programlama	Güvenilirlik için mimari	130
overflexible	aşırı esnek	Aynı şeyin farklı söylenebilmesi	130
modularise	modülerleştirmek	Parçalara ayırıp gruplamak	130
structured natural language	yapılandırılmış doğal dil	Standart form ve şablonlar	131
design description languages	tasarım tanımlama dilleri	Programlama diline benzer soyut dil	131
graphical notations	grafiksel gösterimler	SADT, use-case, sequence diagram	131
mathematical specifications	matematiksel tanımlamalar	Sonlu durum makineleri, kümeler	131
template / form	şablon / form	Standart yazım kalıbı	131–132
pre-condition / post-condition	ön koşul / son koşul	Fonksiyondan önce ve sonra doğru olanlar	132–133
side effects	yan etkiler	İşlemin ek etkileri	133
tabular specification	tablolu tanımlama	Durum ve eylem tablosu	133–134
sequence diagram	sıralama diyagramı	Eylem sırasını gösteren diyagram (yukarıdan aşağı)	133–134
exception	istisna	Hata durumu (geçersiz kart, yetersiz bakiye)	134
interface specification	arayüz tanımlama	Mevcut sistemlerle arayüzlerin kesin tanımı	135
procedural interface / API	işlemsel arayüz / API	Prosedür çağırarak erişilen hizmetler	135
data structures	veri yapıları	Alt sistemler arası aktarılan veri	135
representations of data	veri gösterimleri	Bit sıralaması gibi düşük seviye	135
syntax / semantics	sözdizimi / anlam	Arayüzün biçimi / işlemlerin anlamı	135

software requirements specification (SRS)	yazılım gereksinim belgesi	Geliştiricilerin ne yapacağının resmi ifadesi	136
IEEE/ANSI 830-1998	IEEE gereksinim standardı	En bilinen SRS yapısı	137
glossary	terimler sözlüğü	Belgedeki teknik terimlerin tanımı	139
system evolution (section)	sistem evrimi bölümü	Beklenen değişiklikler (Heninger)	138–139

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 6'dan sınavda en çok çıkabilecekler

1. **Requirements** = hizmetler + kısıtlar. **RE** = bulma, analiz, belgeleme, kontrol. (s.118)
2. **User ve system requirements** farkı + **okuyucuları** (Şekil 6.2) (s.118–120)
3. **3 tür:** functional, non-functional, domain (s.119–120)
4. **Imprecision** sorunu ("appropriate viewers") · **complete ve consistent** (s.121)
5. **NFR neden daha kritik** (sistem kullanılamaz hale gelir) (s.122)
6. **NFR sınıflandırması (Şekil 6.3):** product, organisational, external + alt türleri (s.122–123)
7. **Goal ve verifiable NFR** (2 saat eğitim, günde ≤ 2 hata) + **ölçütler** (Şekil 6.6) (s.124–125)
8. **NFR çatışması** (4 MB ve Ada) (s.124–125)
9. **Domain req.** ana sorunu: alan dili, uzmanlar "açık" bilgileri atlar (s.126)
10. **Doğal dilin 3 sorunu (user req.):** lack of clarity, confusion, amalgamation (s.127)
11. **4 yazım kuralı:** standart format, **shall / should**, vurgulama, jargondan kaçın · **rationale** (s.128–129)
12. **Tasarım bilgisinin dışlanamamasının 3 nedeni** (s.130)
13. **Doğal dilin 3 sorunu (system req.):** ambiguity, overflexible, modülerleştirme (s.130)
14. **4 gösterim (Şekil 6.11)** (s.131)
15. **Standart formun 7 bilgisi** (pre / post-condition) · tablo · **sequence diagram** (ATM, 3 alt dizi) (s.132–134)
16. **3 arayüz türü:** procedural (API), data structures, representations of data (s.135)
17. **SRS**, **5 kullanıcısı** (Şekil 6.16), **IEEE 830 yapısı**, Şekil 6.17 bölümleri, agile görüşü (s.136–139)

C Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What is the difference between user requirements and system requirements?

User ve system requirements farkı (s.118–120)

> *User requirements are statements in natural language plus diagrams of what services the system should provide and the constraints under which it must operate. They are written for customers and managers. System requirements set out the system's functions, services and operational constraints in detail. They should be precise and may be part of the contract. They are written for system architects and software developers.*

Kullanıcı gereksinimleri, sistemin hangi hizmetleri vereceğinin ve hangi kısıtlar altında çalışacağının doğal dil ve diyagramlarla yazılmış ifadeleridir, müşteriler ve yöneticiler için yazılır. Sistem gereksinimleri fonksiyonları, hizmetleri ve kısıtları ayrıntılı tanımlar, kesin olmalıdır ve sözleşmenin parçası olabilir. Sistem mimarları ve geliştiriciler için yazılır.

Q2. Describe functional, non-functional and domain requirements. (Ex. 6.1)

Gereksinim türlerini açıklayın (s.119–120)

> *Functional requirements are statements of services the system should provide, how it should react to inputs and how it should behave in particular situations. Non-functional requirements are constraints on the services or functions, such as timing constraints, constraints on the development process and standards. They often apply to the system as a whole. Domain requirements come from the application domain and reflect its characteristics and constraints. They may be functional or non-functional.*

Fonksiyonel gereksinimler sistemin vereceği hizmetleri, girdilere tepkisini ve belirli durumlardaki davranışını tanımlar. Fonksiyonel olmayan gereksinimler hizmetler üzerindeki kısıtlardır (zamanlama, süreç, standartlar) ve genelde sistemin bütününe uygulanır. Alan gereksinimleri uygulama alanından gelir, onun özelliklerini ve kısıtlarını yansıtır, fonksiyonel ya da fonksiyonel olmayan olabilir.

Q3. What do completeness and consistency mean? Why are they difficult to achieve?

Eksiksizlik ve tutarlılık (s.121)

> *Completeness means that all services required by the user are defined. Consistency means that requirements do not have contradictory definitions. In large, complex systems they are practically impossible to achieve, because it is easy to make mistakes and omissions, and different stakeholders have different and often inconsistent needs.*

Eksiksizlik, kullanıcının istediği bütün hizmetlerin tanımlanmasıdır. Tutarlılık, gereksinimlerin çelişmemesidir. Büyük sistemlerde bunlar pratikte imkânsızdır, çünkü hata ve eksik yapmak kolaydır ve farklı paydaşların farklı, çoğu zaman tutarsız ihtiyaçları vardır.

Q4. Why are non-functional requirements often more critical than functional requirements?
NFR neden daha kritiktir? (s.122)

> *Users can usually find ways to work around a function that does not meet their needs. But failing to meet a non-functional requirement can make the whole system unusable. For example, if an aircraft system does not meet its reliability requirements, it will not be certified as safe for operation.*

Kullanıcılar ihtiyacını karşılamayan bir fonksiyonun etrafından genelde dolanabilir. Ama fonksiyonel olmayan bir gereksinimi karşılamamak bütün sistemi kullanılamaz hale getirebilir. Örneğin uçak sistemi güvenilirlik gereksinimini karşılamazsa uçuş için güvenli sertifikası alamaz.

Q5. Describe the types of non-functional requirements and give examples. (Ex. 6.7)
NFR türleri ve örnekleri (s.122–123)

> *1) Product requirements specify product behaviour, for example performance, memory, reliability, portability and usability. 2) Organisational requirements come from policies and procedures, for example process standards, implementation language and delivery dates. 3) External requirements come from factors outside the system, for example interoperability, legislative requirements such as privacy, and ethical requirements.*

1) Ürün gereksinimleri ürün davranışını belirtir (performans, bellek, güvenilirlik, taşınabilirlik, kullanılabilirlik). 2) Kurumsal gereksinimler politika ve prosedürlerden gelir (süreç standartları, programlama dili, teslim tarihleri). 3) Dış gereksinimler sistem dışı etkenlerden gelir (birlikte çalışabilirlik, gizlilik gibi yasal gereksinimler, etik gereksinimler).

Q6. Why are non-functional requirements difficult to verify? How can this be solved?
NFR neden zor doğrulanır, çözüm? (s.124)

> *Users often state them as vague general goals, such as "the system should be easy to use". Goals are open to interpretation and cause disputes. Non-functional requirements should be written quantitatively, using metrics, so that they can be objectively tested. For example: "after two hours of training, experienced users shall make no more than two errors per day".*

Kullanıcılar bunları genelde "kolay kullanılmalı" gibi belirsiz hedefler olarak ifade eder. Hedefler yoruma açıktır ve anlaşmazlık çıkarır. NFR'ler ölçütlerle nicel yazılmalıdır ki nesnel test edilebilsin. Örneğin: "iki saatlik eğitimden sonra deneyimli kullanıcılar günde en fazla iki hata yapmalı".

Q7. What is the main problem with domain requirements?
Alan gereksinimlerinin ana sorunu (s.126)

> *They are written in the language of the application domain, for example mathematical equations, so software engineers find them difficult to understand. Domain experts may leave information out because it is obvious to them, but it may not be obvious to developers, who may implement the requirement in the wrong way.*

Uygulama alanının dilinde (ör. matematik denklemleri) yazılırlar, bu yüzden yazılım mühendisleri zor anlar. Alan uzmanları kendilerine çok açık gelen bilgileri yazmayabilir, ama bu geliştiriciye açık olmayabilir ve gereksinim yanlış gerçekleştirilebilir.

Q8. What problems arise when requirements are written in natural language? (Ex. 6.2)
Doğal dille gereksinim yazmanın sorunları (s.127, 130)

> *1) Lack of clarity: it is hard to be precise without making the document wordy. 2) Requirements confusion: functional, non-functional requirements, goals and design information are mixed. 3) Requirements amalgamation: several requirements are expressed as one. Also, natural language is ambiguous, overflexible and difficult to modularise. Structured natural language with standard forms reduces variability and organises requirements better.*

1) Açıklık eksikliği: belgeyi uzatmadan kesin olmak zordur. 2) Karışıklık: fonksiyonel, fonksiyonel olmayan gereksinimler, hedefler ve tasarım karışır. 3) Birleşme: birkaç gereksinim tek gereksinim gibi yazılır. Ayrıca doğal dil belirsiz, aşırı esnek ve modülerleştirmesi zordur. Standart formlu yapılandırılmış doğal dil değişkenliği azaltır ve gereksinimleri daha iyi düzenler.

Q9. What guidelines should be followed when writing user requirements?
Kullanıcı gereksinimi yazma kuralları (s.128–129)

> *1) Use a standard format for all requirements. 2) Use language consistently: "shall" for mandatory requirements and "should" for desirable requirements. 3) Use text highlighting to pick out key parts. 4) Avoid computer jargon. It is also good to include a rationale explaining why each requirement is included.*

1) Tüm gereksinimler için standart format kullan. 2) Dili tutarlı kullan: zorunlu için "shall", istenen için "should". 3) Önemli kısımları vurgula. 4) Bilgisayar jargonundan kaçın. Ayrıca her gereksinimin neden konduğunu açıklayan bir gerekçe eklemek iyidir.

Q10. What information should a standard form for a functional requirement include?
Standart formda hangi bilgiler olmalı? (s.132–133)

> *1) A description of the function. 2) Its inputs and where they come from. 3) Its outputs and where they go. 4) Other entities that are used (requires). 5) The action to be taken. 6) A pre-condition and a post-condition. 7) Any side effects of the operation.*

1) Fonksiyonun açıklaması. 2) Girdileri ve nereden geldikleri. 3) Çıktıları ve nereye gittikleri. 4) Kullanılan diğer varlıklar. 5) Yapılacak eylem. 6) Ön koşul ve son koşul. 7) İşlemin yan etkileri.

Q11. What types of interface may have to be defined?
Hangi arayüz türleri tanımlanmalı? (s.135)

> *1) Procedural interfaces, where existing programs offer services through interface procedures (APIs). 2) Data structures that are passed from one sub-system to another. 3) Representations of data, such as the ordering of bits, which are common in embedded, real-time systems.*

1) İşlemsel arayüzler: mevcut programlar arayüz prosedürleriyle hizmet sunar (API). 2) Alt sistemler arasında aktarılan veri yapıları. 3) Bit sıralaması gibi veri gösterimleri (gömülü, gerçek zamanlı sistemlerde yaygın).

Q12. Who are the users of a requirements document and how do they use it?
Gereksinim belgesinin kullanıcıları (s.136–137)

> *System customers specify the requirements and check that they meet their needs. Managers use it to plan a bid and the development process. System engineers use it to understand the system to be developed. Test engineers use it to develop validation tests. Maintenance engineers use it to understand the system and the relationships between its parts.*

Müşteriler gereksinimleri belirtir ve ihtiyaçlarını karşılayıp karşılamadığını kontrol eder. Yöneticiler teklif ve geliştirme sürecini planlamak için kullanır. Sistem mühendisleri geliştirilecek sistemi anlamak için kullanır. Test mühendisleri geçerleme testleri hazırlamak için kullanır. Bakım mühendisleri sistemi ve parçaları arasındaki ilişkileri anlamak için kullanır.

Q13. Describe the structure of a requirements document according to the IEEE standard.
IEEE standardına göre belge yapısı (s.137–138)

> *1) Introduction: purpose, scope, definitions, references and overview. 2) General description: product perspective, product functions, user characteristics, general constraints, assumptions and dependencies. 3) Specific requirements: functional, non-functional and interface requirements. 4) Appendices. 5) Index.*

1) Giriş: amaç, kapsam, tanımlar, referanslar, genel bakış. 2) Genel açıklama: ürün perspektifi, ürün fonksiyonları, kullanıcı özellikleri, genel kısıtlar, varsayımlar ve bağımlılıklar. 3) Özel gereksinimler: fonksiyonel, fonksiyonel olmayan ve arayüz gereksinimleri. 4) Ekler. 5) Dizin.

D Mini test

cevaplar en altta

1. Statements in natural language plus diagrams for customers are:
   a) system requirements
   b) user requirements
   c) design specifications
   d) test plans
2. Which readers usually read only the system requirements, not the user requirements?
   a) Client managers
   b) Contractor managers
   c) Software developers
   d) Nobody
3. "The user shall be able to search all databases" is a ___ requirement.
   a) functional
   b) non-functional
   c) organisational
   d) external
4. "All services required by the user should be defined" describes:
   a) consistency
   b) completeness
   c) portability
   d) usability
5. Non-functional requirements usually apply to:
   a) one feature only
   b) the system as a whole
   c) only the user interface
   d) only the database
6. Which is a product requirement?
   a) Process standards
   b) Delivery date
   c) Reliability
   d) Privacy law

7. Interoperability, ethical and legislative requirements are:
   a) product requirements
   b) organisational requirements
   c) external requirements
   d) domain requirements

8. "The system should be easy to use" is a:
   a) verifiable requirement
   b) system goal
   c) domain requirement
   d) interface specification

9. Mean time to failure is a metric for:
   a) speed
   b) size
   c) reliability
   d) portability

0. Requirements from the application domain (e.g. the Z39.50 standard) are:
   a) domain requirements
   b) user requirements
   c) product requirements
   d) delivery requirements

1. Several different requirements expressed together as one is called:
   a) lack of clarity
   b) requirements confusion
   c) requirements amalgamation
   d) rationale

2. Mandatory requirements should be written using:
   a) "should"
   b) "shall"
   c) "may"
   d) "could"

3. The explanation of why a requirement is included is the:
   a) source
   b) rationale
   c) side effect
   d) index

4. Use-cases and sequence diagrams are examples of:
   a) mathematical specifications
   b) graphical notations
   c) design description languages
   d) structured natural language

5. "What must be true before the function is called" is the:
   a) post-condition
   b) pre-condition
   c) side effect
   d) destination

6. A sequence diagram is read:
   a) right to left
   b) bottom to top
   c) top to bottom
   d) in any order

7. An API is a type of:
   a) procedural interface
   b) data representation
   c) domain requirement
   d) test plan

8. The most widely known standard for requirements documents is:
   a) ISO 9001
   b) IEEE/ANSI 830-1998
   c) UML 2.0
   d) Z39.50

Cevap anahtarı: 1-b · 2-c · 3-a · 4-b · 5-b · 6-c · 7-c · 8-b · 9-c · 10-a · 11-c · 12-b · 13-b · 14-b · 15-b · 16-c · 17-a · 18-b

YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 7 Requirements engineering processes

Gereksinim Mühendisliği Süreçleri

Kitap s.142–168 (Bölüm 7 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 7, **gereksinim mühendisliği sürecindeki faaliyetleri** anlatıyor. Bölüm 6 gereksinimlerin "ne" olduğunu anlatmıştı, Bölüm 7 ise onların "nasıl" bulunup kontrol edilip yönetildiğini anlatıyor. Dört kısımdan oluşuyor:

- **7.1 Feasibility studies:** Fizibilite çalışması.
- **7.2 Requirements elicitation and analysis:** Gereksinim çıkarma ve analiz: viewpoints, interviewing, scenarios, use-cases, ethnography.
- **7.3 Requirements validation:** Gereksinim geçerleme ve gözden geçirmeler.
- **7.4 Requirements management:** Kalıcı ve değişken gereksinimler, izlenebilirlik (traceability), değişiklik yönetimi.

Bölümün amaçları: (1) temel gereksinim mühendisliği faaliyetlerini ve ilişkilerini anlamak, (2) gereksinim çıkarma ve analiz tekniklerini tanımak, (3) gereksinim geçerlemenin önemini ve gözden geçirmelerin nasıl kullanıldığını anlamak, (4) gereksinim yönetiminin neden gerektiğini ve diğer faaliyetleri nasıl desteklediğini anlamak.

Sayfa 143–146 RE sürecine genel bakış (Şekil 7.1, 7.2) · 7.1 Feasibility studies

📖 Sayfanın özeti

AMAÇ VE 4 ALT SÜREÇ

Gereksinim mühendisliği sürecinin amacı bir **sistem gereksinim belgesi** oluşturmak ve sürdürmektir. Süreç dört üst seviye alt süreçten oluşur: sistemin işe yararlılığını değerlendirmek (**feasibility study**), gereksinimleri keşfetmek (**elicitation and analysis**), bunları standart bir biçime dönüştürmek (**specification**) ve gereksinimlerin müşterinin istediği sistemi tanımladığını kontrol etmek (**validation**). Şekil 7.1 bu faaliyetleri ve her aşamanın ürettiği belgeleri gösterir: fizibilite raporu, sistem modelleri, kullanıcı ve sistem gereksinimleri, gereksinim belgesi. Tanımlama ve belgeleme Bölüm 6'da anlatıldı, bu bölüm diğer faaliyetlere odaklanıyor.

GEREKSİNİMLER DEĞİŞİR

Neredeyse bütün sistemlerde gereksinimler değişir: insanlar yazılımdan ne istediklerini daha iyi anlar, sistemi satın alan kurum değişir, donanım, yazılım ve kurumsal ortamda değişiklikler olur. Değişen gereksinimleri yönetme sürecine **requirements management** denir (bölümün son kısmı).

SPİRAL GÖRÜNÜM (ŞEKİL 7.2)

Alternatif bir bakış: süreç, bir spiral etrafında yinelemeli olarak düzenlenmiş **üç aşamalı** bir faaliyettir: **requirements specification**, **requirements validation** ve **requirements elicitation**. Her yinelemede her faaliyete ayrılan süre, sürecin aşamasına ve sistemin türüne bağlıdır. Başta üst seviye **iş gereksinimleri**, fonksiyonel olmayan gereksinimler ve **kullanıcı gereksinimleri** üzerinde durulur (iç halkalar: business requirements specification, feasibility study). Sonraki dış halkalarda **sistem gereksinimleri** ve sistem modellemeye daha çok emek harcanır. Doğrulama tarafında prototipleme ve gözden geçirmeler yer alır, en dışta sistem gereksinim belgesi çıkar. Bu model gereksinimlerin farklı ayrıntı seviyelerinde geliştirilmesine izin verir. Spiralden bazı ya da bütün kullanıcı gereksinimleri çıkarıldıktan sonra çıkılabilir. Prototipleme yinelemeli geliştirmeye genişletilirse gereksinimler ve gerçekleştirme birlikte geliştirilebilir.

YAPILANDIRILMIŞ ANALİZ YETERLİ DEĞİL

Bazıları gereksinim mühendisliğini nesne yönelimli analiz gibi yapılandırılmış bir analiz yönteminin uygulanması olarak görür: use-case modelleri gibi grafiksel sistem modelleri geliştirilip tanımlama olarak kullanılır. Bu yöntemlerin bir rolü vardır, ama gereksinim mühendisliği bunlardan çok daha fazlasıdır. Özellikle **gereksinim çıkarma insan merkezli** bir faaliyettir ve insanlar katı sistem modellerinin getirdiği kısıtları sevmez.

7.1 FİZİBİLİTE ÇALIŞMASI

Bütün yeni sistemlerde süreç bir **fizibilite çalışmasıyla** başlamalıdır. Girdisi: ön iş gereksinimleri, sistemin ana hatları ve iş süreçlerini nasıl destekleyeceği. Çıktısı: gereksinim mühendisliğine ve sistem geliştirmeye **devam etmeye değip değmeyeceğini** öneren bir **rapor**. Kısa, odaklı bir çalışmadır ve üç soruyu cevaplamaya çalışır: (1) Sistem kurumun genel **hedeflerine katkı** sağlıyor mu? (2) Sistem mevcut **teknolojiyle**, belirlenen **maliyet ve takvim** kısıtları içinde gerçekleştirilebilir mi? (3) Sistem zaten var olan diğer sistemlerle **entegre** edilebilir mi?

HEDEFLERE KATKI KRİTİK

Sistem iş hedeflerini desteklemiyorsa işe gerçek bir değeri yoktur. Bu çok açık görünse de birçok kurum hedeflerine katkı sağlamayan sistemler geliştirir, çünkü hedefleri açıkça belirtilmemiştir, iş gereksinimlerini tanımlayamamışlardır ya da politik veya kurumsal faktörler satın almayı etkilemiştir. Fizibilite çalışması RUP'un **Inception** fazının parçası olmalıdır.

NASIL YAPILIR?

Bilgi değerlendirme, bilgi toplama ve rapor yazmayı içerir. Bilgi değerlendirme aşamasında yukarıdaki üç soru için gereken bilgi belirlenir, sonra bilgi kaynaklarıyla konuşulur. Sorulabilecek sorular: (1) Bu sistem uygulanmasaydı kurum nasıl idare ederdi? (2) Mevcut süreçlerde ne sorunlar var, yeni sistem bunları nasıl hafifletir? (3) Sistem iş hedeflerine ve gereksinimlerine doğrudan ne katkı sağlar? (4) Bilgi diğer kurumsal sistemlere aktarılıp onlardan alınabilir mi? (5) Sistem kurumda daha önce kullanılmamış bir teknoloji gerektiriyor mu? (6) Sistem neyi desteklemeli, neyi desteklemek zorunda değil? Bilgi kaynakları: sistemin kullanılacağı bölümlerin yöneticileri, benzer sistemleri bilen yazılım mühendisleri, teknoloji uzmanları ve son kullanıcılar. Fizibilite çalışması normalde **2–3 haftada** bitirilmelidir.

SAYFA 146: RAPOR

Bilgiler toplanınca rapor yazılır ve geliştirmeye **devam edilip edilmeyeceği** konusunda bir öneri yapılır. Raporda sistemin kapsamında, bütçesinde ve takviminde değişiklikler önerilebilir ve ek üst seviye gereksinimler önerilebilir.

👨‍🏫 Hoca anlatımı

Bu bölüm Bölüm 4'teki gereksinim mühendisliği sürecinin **ayrıntılı** halidir. Şekil 7.1, Şekil 4.6'yla aynıdır: **Feasibility → Elicitation & analysis → Specification → Validation → Requirements document**. Üstüne bir de **Requirements management** eklenir.

Fizibilite = "Bu işe girmeye değer mi?" sorusu. Bir kafe açmadan önce düşündüğün şeyler gibi: **(1)** Amacıma uygun mu? (para kazandırır mı?) **(2)** Param ve zamanım yeter mi, gerekli ekipman var mı? **(3)** Mevcut işlerimle uyumlu mu? İşte fizibilitenin **3 sorusu** bunlar: **hedeflere katkı, teknoloji / maliyet / takvim, entegrasyon**.

Fizibilite **kısa** olmalı: **2–3 hafta**. Sonunda bir **rapor** çıkar: "devam et" ya da "etme" (belki kapsamı, bütçeyi değiştirerek).

Bir de şu önemli: **"Hedeflere katkı"** sorusu en kritik olanı. Şirketin işine yaramayan sistem ne kadar iyi yapılırsa yapılsın değersizdir.

📝 Deftere not

- ⭐ RE sürecinin amacı: **sistem gereksinim belgesi** oluşturmak ve sürdürmek.
- ⭐⭐ **4 üst seviye alt süreç (Şekil 7.1): Feasibility study** (işe yarar mı?) → **Elicitation & analysis** (keşfet) → **Specification** (standart biçime dönüştür) → **Validation** (müşterinin istediği mi?). Ürünler: feasibility report, system models, user & system requirements, requirements document.
- Gereksinimler **her zaman değişir** → **requirements management**.
- ⭐ **Şekil 7.2, spiral görünüm:** 3 aşama (specification, validation, elicitation) spiral etrafında. Başta (iç halka) **iş gereksinimleri**, NFR ve **user requirements**. Sonra (dış halka) **system requirements** ve modelleme. Validation: **prototyping, reviews**. Spiralden istenen noktada çıkılabilir.
- RE, yapılandırılmış analiz yöntemlerinden (OO analiz, use-case) **daha fazlası**. Elicitation **insan merkezli**, insanlar katı modelleri sevmez.
- ⭐ **7.1 Feasibility study:** tüm yeni sistemlerde **ilk adım**. Girdi: ön iş gereksinimleri, sistemin ana hatları. Çıktı: **devam edilmeli mi?** önerisi içeren **rapor**.
- ⭐⭐ **Fizibilitenin 3 sorusu:**
  - 1. Sistem kurumun **genel hedeflerine katkı** sağlıyor mu?
  - 2. Mevcut **teknolojiyle**, **maliyet ve takvim** içinde gerçekleştirilebilir mi?
  - 3. Mevcut diğer sistemlerle **entegre** edilebilir mi?
- ⭐ Hedeflere katkı **kritik** (yoksa gerçek değeri yok). Kurumlar yine de yapar: hedefler belirsiz, iş gereksinimleri tanımsız, **politik** faktörler. Fizibilite = RUP **Inception** fazının parçası.
- Yapılışı: **bilgi değerlendirme → bilgi toplama → rapor yazma**.
- **Örnek sorular:** sistem olmasa ne olur? mevcut süreç sorunları? iş hedeflerine doğrudan katkı? diğer sistemlerle bilgi aktarımı? yeni teknoloji gerekiyor mu? neyi desteklemeli, neyi desteklemek zorunda değil?
- Kaynaklar: bölüm **yöneticileri**, benzer sistemleri bilen **mühendisler**, **teknoloji uzmanları**, **son kullanıcılar**.
- ⭐ Süre: **2–3 hafta**. Rapor: devam / dur önerisi + kapsam, bütçe, takvim değişikliği önerisi + ek üst seviye gereksinimler.

Sayfa 146–149 — 7.2 Requirements elicitation and analysis · Şekil 7.3 · 7.2.1 Requirements discovery

📖 Sayfanın özeti

TANIM VE PAYDAŞLAR

Bu aşamada yazılım mühendisleri müşteriler ve son kullanıcılarla birlikte çalışarak **uygulama alanını**, sistemin vermesi gereken **hizmetleri**, gereken **performansı**, **donanım kısıtlarını** vb. öğrenir. Kurumdaki birçok farklı kişiyi içerebilir. **Stakeholder (paydaş)**, sistemden doğrudan ya da dolaylı

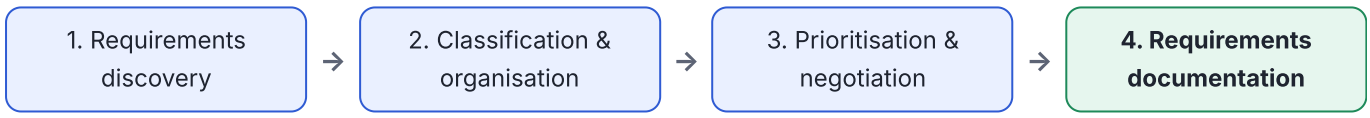
olarak etkilenecek herhangi bir kişi ya da gruptur: sistemle etkileşen son kullanıcılar, kurulumundan etkilenecek herkes, ilgili sistemleri geliştiren ya da bakımını yapan mühendisler, iş yöneticileri, alan uzmanları ve sendika temsilcileri.

NEDEN ZOR? (5 NEDEN)

1) Paydaşlar çoğu zaman sistemden ne istediklerini çok genel ifadeler dışında **bilmezler**. Ne istediklerini anlatmakta zorlanabilir ya da maliyetini bilmedikleri için **gerçekçi olmayan** isteklerde bulunabilirler. **2)** Paydaşlar gereksinimleri doğal olarak **kendi terimleriyle** ve kendi işlerine dair **örtük bilgiyle** ifade eder. Müşterinin alanında deneyimi olmayan mühendisler bunları anlamak zorundadır. **3) Farklı paydaşların farklı gereksinimleri** vardır ve bunları farklı şekillerde ifade edebilirler. Mühendisler bütün kaynakları dikkate alıp ortak noktaları ve **çatışmaları** bulmalıdır. **4) Politik faktörler** gereksinimleri etkileyebilir: yöneticiler kurumdaki etkilerini artıracak gereksinimler isteyebilir. **5)** Analizin yapıldığı ekonomik ve iş ortamı **dinamiktir** ve analiz sırasında kaçınılmaz olarak değişir. Gereksinimlerin önemi değişebilir, başta danışılmamış yeni paydaşlardan yeni gereksinimler çıkabilir.

SÜREÇ (ŞEKİL 7.3)

Çok genel bir süreç modelidir, her kurum personelin uzmanlığı, sistemin türü ve standartlar gibi yerel faktörlere göre kendi versiyonunu kullanır. Faaliyetler bir **spiral** içinde iç içe geçer ve süreç içten dışa ilerler. **4 faaliyet: 1) Requirements discovery:** paydaşlarla etkileşerek gereksinimlerini toplamak. Paydaşlardan ve belgelerden alan gereksinimleri de bu aşamada keşfedilir. **2) Requirements classification and organisation:** düzensiz gereksinim topluluğunu alıp ilgili gereksinimleri gruplamak ve tutarlı kümelere ayırmak. **3) Requirements prioritisation and negotiation:** birden çok paydaş olduğunda gereksinimler kaçınılmaz olarak çatışır. Bu faaliyet gereksinimleri önceliklendirir ve çatışmaları **müzakere** ile bulup çözer. **4) Requirements documentation:** gereksinimler belgelenir ve spiralin bir sonraki turuna girdi olur. Resmi ya da gayriresmi belgeler üretilebilir.



Şekil 7.3: Gereksinim çıkarma ve analiz süreci (spiral, içten dışa)

SAYFA 148: AYRINTILAR

Süreç yinelemelidir, her faaliyetten diğerlerine sürekli geri bildirim olur. Döngü keşifle başlar, belgelemeyle biter, analistin anlayışı her turda gelişir. **Sınıflandırma ve düzenleme**, farklı paydaşların örtüşen gereksinimlerini bulup ilgili olanları gruplamakla ilgilidir. En yaygın gruplama yolu, alt sistemleri belirlemek için bir **sistem mimarisi modeli** kullanıp gereksinimleri alt sistemlerle ilişkilendirmektir. Bu, gereksinim mühendisliği ile mimari tasarımın her zaman ayrılamayacağını gösterir. Paydaşların gereksinimlerin önemi ve önceliği hakkında farklı görüşleri olur ve bazen çatışır. Süreç boyunca **düzenli paydaş müzakereleri** yapılmalı ki uzlaşmaya varılabilsin. Bütün paydaşları tamamen memnun etmek imkânsızdır, ama bazı paydaşlar görüşlerinin dikkate alınmadığını düşünürse süreci bilerek **baltalamaya** çalışabilir. Belgeleme aşamasında çıkarılan gereksinimler, sonraki keşfe yardım edecek şekilde belgelenir. Erken bir sistem gereksinim belgesi üretilebilir ama eksik bölümleri olur. Alternatif olarak gereksinimler bir belgede **tablolar** halinde ya da **kartlarda** (XP yaklaşımı) tutulabilir. Kartlar paydaşların tutması, değiştirmesi ve düzenlemesi kolay olduğu için çok etkilidir.

7.2.1 GEREKSİNİM KEŞFİ

Önerilen ve mevcut sistemler hakkında bilgi toplayıp bu bilgiden kullanıcı ve sistem gereksinimlerini **süzme** sürecidir. Bilgi kaynakları: belgeler, sistem paydaşları ve benzer sistemlerin tanımlamaları.

Paydaşlarla **görüşme** ve **gözlem** yoluyla etkileşilir, keşfe yardım için **senaryolar** ve **prototipler** kullanılabilir. Bu bölümde geniş paydaş kapsamı sağlayan bir yaklaşım (**viewpoints**) ve **interviewing**, **scenarios** ve **ethnography** teknikleri anlatılıyor. Diğer teknikler: yapılandırılmış analiz yöntemleri (Bölüm 8) ve sistem prototipleme (Bölüm 17).

BANKA ATM'SİNİN PAYDAŞLARI

Paydaşlar son kullanıcılardan yöneticilere ve sistemin kabul edilebilirliğini onaylayan düzenleyicilere kadar uzanır. ATM için: (1) sistemden hizmet alan mevcut **banka müşterileri**, (2) karşılıklı anlaşmayla birbirinin ATM'sini kullandıran **diğer bankaların temsilcileri**, (3) sistemden yönetim bilgisi alan **şube müdürleri**, (4) sistemin günlük işleyişinde görevli **gişe personeli**, (5) sistemi bankanın müşteri veritabanıyla entegre etmekten sorumlu **veritabanı yöneticileri**, (6) sistemin güvenlik tehlikesi yaratmamasını sağlaması gereken **banka güvenlik yöneticileri**, (7) sistemi pazarlama aracı olarak kullanmak isteyebilecek bankanın **pazarlama bölümü**, (8) donanım ve yazılımın bakımından ve yükseltilmesinden sorumlu **bakım mühendisleri**, (9) sistemin bankacılık yönetmeliklerine uymasını sağlayan **ulusal bankacılık düzenleyicileri**.

KAYNAKLARI BİRLEŞTİRMEK

Paydaşların yanında gereksinimler **uygulama alanından** ve sistemle etkileşen **diğer sistemlerden** de gelebilir. Bu kaynakların hepsi (paydaşlar, alan, sistemler) **viewpoint** olarak temsil edilebilir. Her viewpoint gereksinimlerin bir alt kümesini sunar ve sisteme yeni bir bakış getirir, ama bu bakışlar tamamen bağımsız değildir, genelde örtüşür ve ortak gereksinimleri vardır.

👨‍🏫 Hoca anlatımı

Stakeholder çok önemli bir kelime: sistemden **etkilenen herkes**. Sadece kullanıcı değil: yönetici, bakım mühendisi, devlet düzenleyicisi, hatta sendika.

Gereksinim çıkarmanın 5 zorluğu sınavda çok sorulur. Bir düğün planlıyormuş gibi düşün: (1) Gelin "güzel bir düğün olsun" diyor ama tam ne istediğini bilmiyor (**ne istediğini bilmiyor**). (2) Herkes kendi diliyle konuşuyor (**kendi terimleri**). (3) Gelin bir şey, damat başka, aileler başka istiyor (**farklı gereksinimler**). (4) Kayınvalide "benim akrabalarım ön masaya" diyor (**politik faktörler**). (5) Hazırlık sürerken bütçe değişiyor, yeni misafirler ekleniyor (**dinamik ortam**).

Şekil 7.3'ün 4 faaliyeti: (1) **Keşfet** (topla), (2) **Sınıflandır ve düzenle** (grupla), (3) **Önceliklendir ve müzakere et** (çatışmaları çöz), (4) **Belgele**. Sonra tekrar başa dön (spiral).

ATM'nin 9 paydaşını ezberlemene gerek yok, ama alıştırma 7.1 tipi soru için mantığı bil: "Kim kullanıyor, kim yönetiyor, kim bakımını yapıyor, kim denetliyor, kim para veriyor?" diye sor.

📝 Deftere not

- ⭐ **Elicitation & analysis:** mühendisler müşteri ve son kullanıcılarla **uygulama alanını**, **hizmetleri**, **performansı**, **donanım kısıtlarını** öğrenir.
- ⭐⭐ Stakeholder: sistemden **doğrudan ya da dolaylı** etkilenen kişi ya da grup. Son kullanıcılar, kurulumdan etkilenenler, ilgili sistemlerin mühendisleri, iş yöneticileri, **alan uzmanları**, **sendika temsilcileri**.
- ⭐⭐ **Gereksinim çıkarmanın 5 zorluğu:**
  - 1. Paydaşlar **ne istediklerini bilmez** (genel ifade), anlatamaz, maliyeti bilmediği için **gerçekçi olmayan** isteklerde bulunur.

- 2. **Kendi terimleriyle** ve **örtük bilgiyle** ifade ederler.
- 3. **Farklı paydaşlar, farklı gereksinimler**, ortak noktaları ve çatışmaları bulmak gerekir.
- 4. **Politik faktörler** (yönetici etkisini artıracak gereksinim ister).
- 5. Ekonomik ve iş ortamı **dinamik**, öncelikler değişir, yeni paydaşlardan yeni gereksinimler.

- ⭐⭐ **Şekil 7.3, 4 faaliyet (spiral):**
  - **1. Requirements discovery:** paydaşlardan topla (+ alan gereksinimleri ve belgeler).
  - **2. Classification & organisation:** ilgili gereksinimleri **grupla**, tutarlı kümeler.
  - **3. Prioritisation & negotiation: önceliklendir**, çatışmaları **müzakere** ile çöz.
  - **4. Requirements documentation:** belgele, sonraki tura girdi (resmi ya da gayriresmi).
- Süreç **yinelemeli**, sürekli geri bildirim. Keşifle başlar, belgelemeyle biter.
- Gruplamanın yaygın yolu: **sistem mimarisi modeli** → alt sistemler. RE ve mimari tasarım **ayrılamaz**.
- ⭐ **Düzenli paydaş müzakereleri** → uzlaşma. Herkesi memnun etmek imkânsız, ama görüşü dikkate alınmayan paydaş süreci **baltalayabilir**.
- Belgeleme: erken taslak (eksik), **tablolar** ya da **kartlar** (XP, tutması ve değiştirmesi kolay).
- ⭐ **7.2.1 Requirements discovery:** bilgi topla, gereksinimleri **süz**. Kaynaklar: **belgeler, paydaşlar, benzer sistemlerin spec'leri**. Yollar: **görüşme, gözlem, senaryolar, prototipler**.
- ⭐ **ATM paydaşları (9):** banka müşterileri · diğer bankaların temsilcileri · şube müdürleri · gişe personeli · veritabanı yöneticileri · güvenlik yöneticileri · pazarlama bölümü · bakım mühendisleri · ulusal bankacılık düzenleyicileri.
- Kaynaklar (paydaşlar, alan, diğer sistemler) = **viewpoints**. Her biri gereksinimlerin alt kümesi, **örtüşürler**.

Sayfa 149–153 Viewpoints (Şekil 7.4) · Interviewing (Görüşme)

📖 Sayfanın özeti

VİEWPOİNT YAKLAŞIMI

Viewpoint odaklı yaklaşımlar hem çıkarma sürecini hem gereksinimlerin kendisini viewpoint'lere göre düzenler. Ana gücü, **birden çok bakış açısını** tanıması ve farklı paydaşların önerdiği gereksinimlerdeki **çatışmaları** bulmak için bir çerçeve sunmasıdır. Viewpoint'ler paydaşları ve diğer gereksinim kaynaklarını sınıflandırmanın bir yoludur.

3 GENEL VİEWPOİNT TÜRÜ

1) Interactor viewpoints: sistemle **doğrudan etkileşen** insanlar ya da diğer sistemler. ATM'de: banka müşterileri ve bankanın hesap veritabanı. **2) Indirect viewpoints:** sistemi **kendileri kullanmayan** ama gereksinimleri bir şekilde etkileyen paydaşlar. ATM'de: banka yönetimi ve güvenlik personeli. **3) Domain viewpoints:** sistem gereksinimlerini etkileyen **alan özellikleri ve kısıtları**. ATM'de: bankalar arası iletişim için geliştirilmiş standartlar. Bu viewpoint'ler genelde farklı tür gereksinimler sağlar: interactor viewpoint'ler sistem özelliklerini ve arayüzleri kapsayan **ayrıntılı sistem gereksinimleri**, indirect viewpoint'ler daha çok **üst seviye kurumsal** gereksinim ve kısıtlar, domain viewpoint'ler ise sisteme uygulanan **alan kısıtlarını** verir.

DAHA ÖZEL VİEWPOİNT TÜRLERİ

İlk belirleme zor olabilir, bu yüzden daha özel türler aranmalıdır: (1) sisteme hizmet **sağlayanlar** ve sistem hizmetlerini **alanlar**, (2) belirlenen sistemle **doğrudan arayüz** kuracak sistemler, (3) sisteme uygulanan **yönetmelik ve standartlar**, (4) iş ve fonksiyonel olmayan gereksinimlerin kaynakları, (5) sistemi geliştirecek, yönetecek ve bakımını yapacak kişilerin gereksinimlerini yansıtan **mühendislik** viewpoint'leri, (6) müşterilerin beklediği ürün özellikleri ve kurumun dış imajına dair gereksinimler üreten **pazarlama** ve diğer viewpoint'ler. Neredeyse bütün kurumsal sistemler diğer sistemlerle birlikte çalışmak zorundadır: bu sistemlerin arayüzleri zaten tasarlanmıştır ve yeni sisteme gereksinim ve kısıt getirebilir. Yeni sistem mevcut yönetmelik ve standartlara da uymak zorunda olabilir.

ÜST SEVİYE GEREKSİNİMLER, MÜHENDİSLİK VE PAZARLAMA

Üst seviye iş ve fonksiyonel olmayan gereksinimler RE sürecinin başında belirlenmeli, kaynakları daha ayrıntılı bir süreçte viewpoint olarak kullanılabilir. **Mühendislik viewpoint'leri** iki nedenle önemlidir: (1) geliştiren mühendislerin benzer sistemlerde deneyimi olabilir ve buna dayanarak gereksinim önerebilir, (2) sistemi yönetecek ve bakımını yapacak teknik personelin, sistem desteğini kolaylaştıracak gereksinimleri olabilir. Sistem yönetimi maliyetleri toplam ömür maliyetinin giderek artan bir kısmı olduğu için bu gereksinimler önem kazanıyor. **Pazarlama** ve dış ilişkiler bölümlerinden gelen gereksinimler özellikle web tabanlı sistemler, e-ticaret ve hazır (shrink-wrapped) yazılım ürünleri için önemlidir: web sistemleri kurumun olumlu bir imajını sunmalı, pazarlama bölümü hangi özelliklerin sistemi satılabilir yapacağını bilmelidir.

HİYERARŞİ (ŞEKİL 7.4)

Önemsiz olmayan her sistemde çok fazla olası viewpoint vardır ve hepsinden gereksinim çıkarmak pratikte imkânsızdır. Bu yüzden viewpoint'leri bir **hiyerarşi** halinde düzenlemek önemlidir: aynı daldaki viewpoint'ler büyük ihtimalle ortak gereksinimleri paylaşır. Şekil 7.4, LIBSYS'in viewpoint hiyerarşisidir: **Indirect** (kütüphane müdürü, finans, makale sağlayıcıları), **Interactor** (kullanıcılar: öğrenciler, personel, dış kullanıcılar ve kütüphane personeli: sistem yöneticileri, katalogçular), **Domain** (UI standartları, sınıflandırma sistemi). Bu sınıflandırma, sistemin doğrudan kullanıcıları dışındaki gereksinim kaynaklarını belirlemeye yardım eder. Viewpoint'ler belirlenip yapılandırıldıktan sonra en önemlileri belirlenmeli ve keşfe onlardan başlanmalıdır.

INTERVİEWİNG (GÖRÜŞME)

Paydaşlarla resmi ya da gayriresmi görüşmeler çoğu RE sürecinin parçasıdır. Gereksinim ekibi paydaşlara kullandıkları ve geliştirilecek sistem hakkında sorular sorar, gereksinimler cevaplardan çıkarılır. **İki tür: closed interviews** (paydaş önceden belirlenmiş bir soru setini cevaplar) ve **open interviews** (önceden belirlenmiş gündem yoktur, ekip paydaşlarla çeşitli konuları keşfeder ve ihtiyaçlarını daha iyi anlar). Pratikte görüşmeler genelde ikisinin **karışımıdır**: bazı cevaplar daha serbest tartışılan başka konulara yol açar. Tamamen açık tartışmalar nadiren iyi sonuç verir, başlamak ve görüşmeyi sisteme odaklı tutmak için bazı sorular gerekir.

NEYE İYİ, NEYE KÖTÜ?

Görüşmeler paydaşların **ne yaptığını**, sistemle nasıl etkileşeceklerini ve mevcut sistemlerle yaşadıkları **zorlukları** genel olarak anlamak için iyidir. İnsanlar işlerinden bahsetmeyi sever ve görüşmelere katılmaktan genelde memnundur. Ama görüşmeler **alan gereksinimlerini** anlamak için o kadar iyi değildir. Görüşmede alan bilgisi çıkarmak iki nedenle zordur: (1) bütün uygulama uzmanları alana özgü **terminoloji ve jargon** kullanır, alan gereksinimlerini bu terminoloji olmadan tartışamaz ve terminolojiyi mühendislerin kolayca yanlış anlayacağı kesin ve ince şekillerde kullanırlar. (2) Bazı alan bilgileri paydaşlara o kadar **tanıdıktır** ki ya açıklamakta zorlanırlar ya da bahsetmeye değmeyecek kadar temel olduğunu düşünürler. Ör: bir kütüphaneci için bütün yeni materyallerin kütüphaneye

eklenmeden önce **kataloglandığı** söylemeye bile gerek olmayan bir şeydir, ama görüşmeci için açık olmayabilir ve gereksinimlere yansımayabilir.

KURUMSAL GEREKSİNİMLER VE İYİ GÖRÜŞMECİ

Görüşmeler **kurumsal** gereksinim ve kısıtları çıkarmak için de etkili değildir, çünkü paydaşlar arasında ince güç ve etki ilişkileri vardır. Yayınlanmış kurum yapıları gerçek karar verme yapısıyla nadiren örtüşür, ama görüşülenler gerçek yapıyı bir yabancıya açmak istemeyebilir. Genelde insanlar gereksinimleri etkileyebilecek politik ve kurumsal konuları konuşmak istemez. **Etkili görüşmecilerin iki özelliği** vardır: (1) **açık fikirlidirler**, gereksinimler hakkında önyargıdan kaçınır ve paydaşları dinlemeye isteklidirler. Paydaş şaşırtıcı bir gereksinimle gelirse sistem hakkındaki fikirlerini değiştirmeye hazırdırlar. (2) Görüşülen kişiyi bir soruyla, bir gereksinim önerisiyle ya da bir prototip üzerinde birlikte çalışmayı önererek **tartışmaya yönlendirirler**. İnsanlara "bana ne istediğini söyle" demek faydalı bilgi getirmez, çoğu insan genel terimler yerine **belirli bir bağlamda** konuşmayı çok daha kolay bulur. Görüşmelerden gelen bilgi belgeler ve kullanıcı gözlemleri gibi diğer bilgileri tamamlar. Bazen belgeler dışında tek bilgi kaynağı görüşmeler olabilir. Ama görüşme tek başına temel bilgileri kaçırabilir, bu yüzden **diğer tekniklerle birlikte** kullanılmalıdır.

👨‍🏫 Hoca anlatımı

Viewpoint = "sisteme hangi açıdan bakıyorsun?" **3 tür** çok önemli: **Interactor** = sistemi **doğrudan kullanan** (ATM'de para çeken müşteri). **Indirect** = kullanmayan ama **etkileyen** (banka müdürü, güvenlik şefi). **Domain** = **alanın kuralları** (bankalar arası iletişim standartları).

Her biri farklı tür gereksinim verir: interactor → **ayrıntılı** sistem gereksinimleri, indirect → **üst seviye kurumsal** gereksinimler, domain → **alan kısıtları**.

Alıştırma 7.2'de bir klinik kayıt sistemi için **viewpoint hiyerarşisi** çizmen isteniyor. Şablon: üç dal (Interactor, Indirect, Domain), altına somut kişi ve sistemler. Klinik için: Interactor → hemşireler, doktorlar, izleme cihazları. Indirect → klinik müdürü, hastanın aile doktoru. Domain → tıbbi kayıt standartları, gizlilik yasası.

Interview'da iki tür: **closed** (hazır sorular, anket gibi) ve **open** (serbest sohbet). Gerçekte **karışık** kullanılır.

Görüşmenin zayıf yanları: (1) **alan bilgisi** zor çıkar (jargon + "bu çok açık" diye söylenmeyen şeyler, kütüphaneci "her kitap kataloglanır" demez çünkü ona göre bariz), (2) **kurumsal / politik** bilgi zor çıkar (insanlar şirketin gerçek güç ilişkilerini yabancıya anlatmaz).

İyi görüşmeci: açık fikirli olur ve "ne istiyorsun?" diye sormak yerine somut bir soru ya da prototiple sohbeti başlatır.

📝 Deftere not

- ⭐ **Viewpoint-oriented** yaklaşım: çıkarma ve gereksinimleri viewpoint'lere göre düzenler. Gücü: **birden çok bakış açısını** tanır, **çatışmaları** bulmak için çerçeve sunar.
- ⭐⭐ **3 genel viewpoint türü:**
  - Interactor viewpoints: sistemle **doğrudan etkileşen** insan ya da sistemler. ATM: müşteriler, hesap veritabanı. → **ayrıntılı** sistem gereksinimleri.
  - Indirect viewpoints: sistemi **kullanmayan ama etkileyen** paydaşlar. ATM: banka yönetimi, güvenlik personeli. → **üst seviye kurumsal** gereksinimler.

- Domain viewpoints: **alan özellikleri ve kısıtları**. ATM: bankalar arası iletişim standartları. → **alan kısıtları**.

- **Daha özel türler:** hizmet sağlayan ve alanlar · doğrudan arayüz kuran sistemler · yönetmelik ve standartlar · iş ve NFR kaynakları · **mühendislik** viewpoint'leri · **pazarlama** viewpoint'leri.
- **Mühendislik viewpoint'i** neden önemli: (1) benzer sistem deneyimi, (2) yönetim ve bakımı kolaylaştıracak gereksinimler (sistem yönetimi maliyeti artıyor).
- **Pazarlama viewpoint'i:** web, e-ticaret, hazır ürünler için önemli (kurum imajı, satılabilirlik).
- ⭐ Çok fazla viewpoint var, hepsinden çıkarmak imkânsız → **hiyerarşi** kur (aynı dal = ortak gereksinim). En önemlilerden başla.
- ⭐ **Şekil 7.4 (LIBSYS):** Indirect (kütüphane müdürü, finans, makale sağlayıcıları) · Interactor (kullanıcılar: öğrenci, personel, dış kullanıcı · kütüphane personeli: sistem yöneticisi, katalogçu) · Domain (UI standartları, sınıflandırma sistemi).
- ⭐⭐ **Interviewing, 2 tür:**
  - Closed interviews: önceden belirlenmiş **soru seti**.
  - Open interviews: **gündem yok**, konular serbestçe keşfedilir.

  Pratikte **karışım**. Tamamen açık tartışma nadiren işe yarar.
- ⭐ Görüşme **iyi**: paydaşın ne yaptığını, sistemle etkileşimini, mevcut sistem **zorluklarını** anlamak (insanlar işini anlatmayı sever).
- ⭐⭐ Görüşme **kötü**: **alan gereksinimleri**, çünkü (1) alan **jargonu** kolay yanlış anlaşılır, (2) bazı bilgiler o kadar **tanıdık** ki söylenmez (ör. kütüphaneci: "yeni materyal önce **kataloglanır**"). Ayrıca **kurumsal ve politik** gereksinimler (güç ilişkileri, gerçek karar yapısı yabancıya açılmaz).
- ⭐ **Etkili görüşmecinin 2 özelliği:** (1) **açık fikirli**, önyargısız, dinler, fikir değiştirmeye hazır, (2) tartışmayı bir soru, gereksinim önerisi ya da **prototiple** başlatır ("ne istiyorsun?" diye sormaz, **belirli bağlam**).
- Görüşme diğer bilgileri **tamamlar**. Tek başına temel bilgiyi kaçırabilir, **diğer tekniklerle birlikte** kullan.

Sayfa 153–156 Scenarios (Şekil 7.5) · Use-cases (Şekil 7.6, 7.7, 7.8)

📖 Sayfanın özeti

SENARYOLAR

İnsanlar soyut tanımlardan çok **gerçek hayattan örneklerle** ilişki kurmayı kolay bulur. Bir yazılım sistemiyle nasıl etkileşeceklerini anlatan bir senaryoyu anlayıp eleştirebilirler. Mühendisler bu tartışmadan elde ettikleri bilgiyle gerçek sistem gereksinimlerini yazar. Senaryolar özellikle **ana hatlarıyla** yazılmış bir gereksinim tanımına **ayrıntı eklemek** için faydalıdır. Örnek etkileşim oturumlarının tanımlarıdır, her senaryo bir ya da birkaç olası etkileşimi kapsar. Farklı ayrıntı seviyesinde farklı bilgi veren çeşitli senaryo biçimleri vardır. Senaryo kullanmak, **extreme programming** gibi agile yöntemlerin ayrılmaz bir parçasıdır.

SENARYONUN 5 ÖĞESİ

Senaryo etkileşimin ana hatlarıyla başlar, çıkarma sırasında ayrıntı eklenerek tam bir tanım oluşturulur. En genel haliyle şunları içerebilir: (1) senaryo başladığında sistemin ve kullanıcıların **beklentisi** (başlangıç varsayımı), (2) senaryodaki **normal olay akışı**, (3) **neyin ters gidebileceği** ve

bunun nasıl ele alınacağı, (4) aynı anda olabilecek **diğer faaliyetler** hakkında bilgi, (5) senaryo bittiğinde **sistemin durumu**.

LIBSYS MAKALE SENARYOSU (ŞEKİL 7.5)

Bir kullanıcı tıbbi bir dergideki makalenin kişisel kopyasını yazdırmak istiyor. Dergi aboneye ücretsiz, abone olmayana makale başı ücretli. Kullanıcı makaleyi, başlığı ve yayın tarihini biliyor. **Başlangıç varsayımı:** kullanıcı LIBSYS'e giriş yapmış ve dergiyi bulmuş. **Normal akış:** kullanıcı makaleyi seçer, sistem abone bilgisi ya da ödeme yöntemi (kredi kartı ya da kurum hesap numarası) ister, kullanıcı bir telif hakkı formu doldurur, form onaylanınca makalenin PDF'i kullanıcının bilgisayarındaki çalışma alanına indirilir, kullanıcı bir yazıcı seçer ve makale yazdırılır. Makale "sadece yazdırma" olarak işaretliyse yazdırma onaylanınca sistemden silinir. **Ne ters gidebilir:** form yanlış doldurulursa düzeltmek için tekrar gösterilir, yine yanlışsa istek reddedilir. Ödeme reddedilirse istek reddedilir. İndirme başarısız olursa sistem başarılı olana ya da kullanıcı oturumu kapatana kadar tekrar dener. Yazdırılamazsa ve "sadece yazdırma" değilse çalışma alanında tutulur, değilse silinir ve ücret hesaba iade edilir. **Diğer faaliyetler:** başka makalelerin eşzamanlı indirilmesi. **Bitişteki durum:** kullanıcı oturum açık, "sadece yazdırma" makale silinmiş.

GAYRİRESMİ VE YAPILANDIRILMIŞ

Senaryo tabanlı çıkarma gayriresmi yapılabilir: mühendis paydaşlarla birlikte senaryoları belirleyip ayrıntılarını toplar. Senaryolar diyagramlar ve ekran görüntüleriyle desteklenen metin olarak yazılabilir. Ya da **olay senaryoları** veya **use-case**'ler gibi daha yapılandırılmış bir yaklaşım kullanılabilir.

USE-CASES

Use-case'ler, ilk kez **Objectory** yönteminde (Jacobson) tanıtılan senaryo tabanlı bir gereksinim çıkarma tekniğidir. Artık nesne yönelimli sistem modellerini tanımlamak için kullanılan **UML** gösteriminin temel bir özelliğidir. En basit halinde bir use-case **etkileşimin türünü** ve ilgili **aktörleri** belirler. Şekil 7.6, makale yazdırma için üst seviye bir use-case gösterir. Gösterimin temeli: süreçteki aktörler **çöp adam (stick figure)**, her etkileşim sınıfı **adlandırılmış bir elips** ile gösterilir. Use-case kümesi sistem gereksinimlerinde temsil edilecek bütün olası etkileşimleri gösterir. Şekil 7.7 kütüphane sistemi için daha fazla use-case gösterir: **Library User** → Article search, Article printing, User administration. **Library Staff** → User administration, Catalogue services. **Supplier** → Catalogue services.

USE-CASE VE SENARYO İLİŞKİSİ

Bir use-case'in kendi başına bir senaryo mu olduğu konusunda bazen karışıklık olur. **Fowler**'ın önerdiği gibi bir use-case bir **senaryolar kümesini** kapsar ve her senaryo use-case içinden geçen **tek bir yoldur**. Normal etkileşim için bir senaryo ve her olası istisna için ayrı senaryolar olur.

SEQUENCE DİAGRAM İLE AYRINTI (ŞEKİL 7.8)

Use-case'ler sistemle bireysel etkileşimleri belirler. Metinle belgelenebilir ya da senaryoyu daha ayrıntılı geliştiren UML modellerine bağlanabilir. Bir use-case'e bilgi eklemek için sık sık **sequence diagram** kullanılır: etkileşimdeki aktörleri, etkileştikleri nesneleri ve bu nesnelere ait işlemleri gösterir. Şekil 7.8, LIBSYS'te makale indirip yazdırmadaki etkileşimleri gösterir: dört nesne sınıfı vardır (**Article, Form, Workspace, Printer**). Eylem sırası yukarıdan aşağıdır, oklardaki etiketler işlem adlarıdır. Özetle: kullanıcının makale isteği bir telif formu isteğini tetikler, kullanıcı formu doldurunca makale indirilip yazıcıya gönderilir, yazdırma bitince makale LIBSYS çalışma alanından silinir.

NE ZAMAN ETKİLİ?

> UML nesne yönelimli modellemede **fiili (de facto) standarttır**, bu yüzden use-case'ler ve use-case tabanlı çıkarma giderek daha çok kullanılıyor. Senaryolar ve use-case'ler **interactor viewpoint'lerinden** gereksinim çıkarmak için etkilidir, çünkü her etkileşim türü bir use-case olarak gösterilebilir. Sistemden sonuç (ör. yönetim raporu) alan bazı indirect viewpoint'lerle birlikte de kullanılabilir. Ama **etkileşimlere odaklandıkları** için indirect viewpoint'lerden **kısıtları** ya da üst seviye iş ve **fonksiyonel olmayan** gereksinimleri çıkarmakta ya da **alan gereksinimlerini** keşfetmekte o kadar etkili değildir.

👩‍🏫 Hoca anlatımı

Senaryo = bir kullanım **hikâyesi**. İnsanlar "sistem şunu yapmalı" gibi soyut cümlelerden çok "Ayşe giriş yapar, makaleyi seçer, öder, yazdırır..." gibi hikâyeleri daha kolay anlar ve "aaa bir de şöyle olursa ne olacak?" diye eksikleri fark eder.

Senaryonun 5 parçası ezberlenmeli: (1) **başlangıç** durumu, (2) **normal akış**, (3) **ne ters gidebilir** ve nasıl çözülür, (4) **eşzamanlı** diğer faaliyetler, (5) **bitiş** durumu. Yani: başı, ortası, sorunları, yan işler, sonu.

Use-case = UML'de çöp adam + elips. Çöp adam **aktör** (kullanan kişi ya da sistem), elips **etkileşim türü** (ne yapıyor). Alıştırma 7.5 ATM için use-case çizmeni istiyor: aktör "Customer", elipsler "Withdraw cash", "Check balance", "Deposit", "Change PIN". Bir de "Bank" sistemi aktör olabilir.

Use-case ve senaryo farkı (Fowler): Use-case "para çekme" gibi bir **başlık**. Altındaki senaryolar: "her şey yolunda gitti", "PIN yanlış", "bakiye yetersiz"... Yani **bir use-case = birden çok senaryo**.

Zayıf yanı: use-case'ler etkileşime odaklandığı için "sistem 2 saniyede cevap versin" (NFR) ya da alan kuralları gibi şeyleri yakalamakta iyi değildir.

📝 Deftere not

- ⭐ **Scenarios:** insanlar **gerçek örneklerle** daha kolay ilişki kurar. Senaryo = **örnek etkileşim oturumu** tanımı. Ana hatlı gereksinimlere **ayrıntı eklemek** için faydalı. **XP / agile** yöntemlerin ayrılmaz parçası.
- ⭐⭐ **Senaryonun 5 öğesi:**
  - 1. Başlangıçta sistemin ve kullanıcının **beklentisi** (initial assumption).
  - 2. **Normal olay akışı**.
  - 3. **Ne ters gidebilir** ve nasıl ele alınır.
  - 4. Aynı anda olan **diğer faaliyetler**.
  - 5. Bitişte **sistemin durumu**.
- **Şekil 7.5 (LIBSYS makale indirme):** başlangıç (giriş yapılmış, dergi bulunmuş) → normal (seç, abone ya da ödeme bilgisi, **telif formu**, PDF indir, yazdır, "print-only" ise sil) → hatalar (form yanlış → tekrar, sonra red · ödeme red · indirme başarısız → tekrar dene · yazdırılamazsa tut ya da sil ve iade) → diğer faaliyetler (eşzamanlı indirmeler) → bitiş durumu.
- Senaryolar **gayriresmi** (metin + diyagram + ekran görüntüsü) ya da **yapılandırılmış** (event scenarios, use-cases).
- ⭐ **Use-cases:** senaryo tabanlı teknik. İlk kez **Objectory** yönteminde (**Jacobson**). Artık **UML**'nin temel özelliği. Etkileşimin **türünü** ve **aktörleri** belirler.

- ⭐⭐ **Gösterim:** aktör = **çöp adam (stick figure)**, etkileşim = **adlandırılmış elips**. Use-case kümesi = tüm olası etkileşimler.
- **Şekil 7.7 (LIBSYS):** Library User (article search, article printing, user administration) · Library Staff (user administration, catalogue services) · Supplier (catalogue services).
- ⭐ **Use-case ve senaryo (Fowler):** use-case = **senaryolar kümesi**, her senaryo = use-case içinde **tek bir yol** (normal akış + her istisna için ayrı senaryo).
- ⭐ Use-case'e ayrıntı eklemek için **sequence diagram**: aktörler, nesneler, işlemler. **Şekil 7.8:** nesneler Article, Form, Workspace, Printer. Yukarıdan aşağı: istek → telif formu → indir → yazdır → **sil**.
- UML = OO modellemede **de facto standart**, bu yüzden use-case kullanımı artıyor.
- ⭐⭐ Senaryolar ve use-case'ler **interactor viewpoint**'leri için **etkili**. Ama etkileşime odaklandıkları için **kısıtları**, **üst seviye iş ve NFR**'yi ve **alan gereksinimlerini** bulmakta **az etkili**.

Sayfa 157–160 — 7.2.2 Ethnography (Şekil 7.9) · 7.3 Requirements validation · 7.3.1 Requirements reviews

📖 Sayfanın özeti

ETNOGRAFİ NEDİR?

Yazılım sistemleri boşlukta var olmaz: sosyal ve kurumsal bir bağlamda kullanılır ve gereksinimler bu bağlamdan türeyebilir ya da onunla kısıtlanabilir. Bu sosyal ve kurumsal gereksinimleri karşılamak sistemin başarısı için çoğu zaman kritiktir. Teslim edilen ama hiç kullanılmayan birçok sistemin bir nedeni bu gereksinimlerin önemini hesaba katmamasıdır. **Etnografi**, sosyal ve kurumsal gereksinimleri anlamak için kullanılan bir **gözlem tekniğidir**. Analist kendini sistemin kullanılacağı **çalışma ortamına yerleştirir**, günlük işi gözler ve katılımcıların yaptığı **gerçek işleri** not eder. Değeri, analistlerin insanların içinde bulunduğu resmi süreçleri değil **gerçek süreçleri** yansıtan **örtük** sistem gereksinimlerini keşfetmesine yardım etmesidir.

NEDEN İŞE YARAR?

İnsanlar işlerinin ayrıntılarını anlatmakta çok zorlanır, çünkü bu onlar için ikinci doğa haline gelmiştir. Kendi işlerini anlarlar ama kurumdaki diğer işlerle ilişkisini anlamayabilirler. İşi etkileyen ama bireylere açık olmayan sosyal ve kurumsal faktörler ancak **tarafsız bir gözlemci** fark edince açıklık kazanabilir. **Suchman** ofis işini incelemek için etnografi kullandı ve gerçek çalışma pratiklerinin ofis otomasyon sistemlerinin varsaydığı basit modellerden çok daha zengin, karmaşık ve dinamik olduğunu buldu. Varsayılan ile gerçek iş arasındaki fark, bu sistemlerin verimliliğe önemli bir etkisinin olmamasının en önemli nedeniydi. Diğer etnografik çalışmalar hava trafik kontrolü, metro kontrol odaları, finansal sistemler ve tasarım faaliyetleri üzerine yapıldı. Yazarın kendi araştırmaları da etnografiyi gereksinim mühendisliği yöntemlerine bağlamayı ve işbirlikçi sistemlerdeki etkileşim kalıplarını belgelemeyi içeriyor.

ETNOGRAFİNİN BULDUĞU 2 GEREKSİNİM TÜRÜ

1) İnsanların gerçekte nasıl çalıştığından türeyen gereksinimler (süreç tanımlarının söylediğinden değil). Ör: hava trafik kontrolörleri, kesişen uçuş yollarındaki uçakları algılayan **çatışma uyarı sistemini kapatabilir**, normal prosedürler kullanılmasını söylese bile. Kontrol stratejileri uçakların sorun çıkmadan ayrılmasını sağlamak üzerine kuruludur ve alarmın dikkatlerini dağıttığını düşünürler.
2) Başka insanların faaliyetleriyle işbirliğinden ve onların farkında olmaktan türeyen

gereksinimler. Ör: kontrolörler kendi sektörlerine girecek uçak sayısını tahmin etmek için diğer kontrolörlerin işinin farkında olurlar ve kontrol stratejilerini bu tahmin edilen iş yüküne göre değiştirirler. Bu yüzden otomatik bir hava trafik kontrol sistemi, bir sektördeki kontrolörlerin **komşu sektörlerdeki işi görebilmesine** izin vermelidir.

PROTOTİPLEMEYLE BİRLİKTE (ŞEKİL 7.9)

Etnografi prototiplemeyle birleştirilebilir. Etnografi prototipin geliştirilmesine bilgi verir, böylece daha az prototip iyileştirme döngüsü gerekir. Prototipleme de etnografiye **odak** kazandırır: etnografla tartışılabilecek sorunları ve soruları belirler, o da sistem çalışmasının sonraki aşamasında bunların cevaplarını arar. Şekil 7.9 akışı: Ethnographic analysis → Debriefing meetings → Focused ethnography ⇄ System prototyping (Generic system development'tan beslenir) → Prototype evaluation.

SINIRLARI

Etnografik çalışmalar diğer çıkarma tekniklerinin kaçırdığı **kritik süreç ayrıntılarını** ortaya çıkarabilir. Ama **son kullanıcıya** odaklandığı için **kurumsal ya da alan** gereksinimlerini keşfetmeye uygun değildir. Ayrıca sisteme eklenmesi gereken **yeni özellikleri** her zaman belirleyemez. Bu yüzden etnografi tek başına tam bir çıkarma yaklaşımı değildir, **use-case analizi** gibi diğer yaklaşımları tamamlamak için kullanılmalıdır.

7.3 REQUİREMENTS VALİDATİON

Gereksinim geçerleme, gereksinimlerin müşterinin **gerçekten istediği** sistemi tanımladığını göstermekle ilgilidir. Gereksinimlerdeki sorunları bulmakla ilgili olduğu için **analizle örtüşür**. Önemlidir, çünkü gereksinim belgesindeki hatalar geliştirme sırasında ya da sistem kullanımdayken bulunursa **çok büyük yeniden çalışma maliyetine** yol açar. Bir gereksinim sorununu sistemi değiştirerek düzeltmenin maliyeti, tasarım ya da kodlama hatalarını düzeltmekten **çok daha fazladır**, çünkü gereksinim değişikliği genelde tasarımın ve gerçekleştirmenin de değişmesi ve sistemin tekrar test edilmesi demektir.

5 KONTROL TÜRÜ

1) Validity checks (geçerlilik): kullanıcı sistemin belirli fonksiyonları yapması gerektiğini düşünebilir, ama daha fazla düşünce ve analiz ek ya da farklı fonksiyonlar gerektiğini gösterebilir. Sistemlerin farklı ihtiyaçları olan çeşitli paydaşları vardır ve her gereksinim kümesi kaçınılmaz olarak paydaşlar arasında bir **uzlaşmadır**. **2) Consistency checks (tutarlılık):** gereksinimler **çelişmemelidir**, aynı sistem fonksiyonunun çelişen kısıtları ya da tanımları olmamalıdır. **3) Completeness checks (eksiksizlik):** belge kullanıcının istediği **bütün fonksiyonları ve kısıtları** tanımlamalıdır. **4) Realism checks (gerçekçilik):** mevcut teknoloji bilgisiyle gereksinimlerin gerçekten **gerçekleştirilebilir** olduğu kontrol edilmelidir. **Bütçe ve takvim** de hesaba katılmalıdır. **5) Verifiability (doğrulanabilirlik):** müşteri ile yüklenici arasında anlaşmazlık olasılığını azaltmak için sistem gereksinimleri her zaman **doğrulanabilir** yazılmalıdır, yani teslim edilen sistemin her gereksinimi karşıladığını gösterecek bir **test kümesi** yazılabilmelidir.

3 GEÇERLEME TEKNİĞİ

Birlikte ya da tek tek kullanılabilir: **1) Requirements reviews:** gereksinimler bir **gözden geçirme ekibi** tarafından sistematik olarak analiz edilir. **2) Prototyping:** sistemin **çalıştırılabilir bir modeli** son kullanıcılara ve müşterilere gösterilir, gerçek ihtiyaçlarını karşılayıp karşılamadığını görmek için onunla deney yapabilirler (Bölüm 17). **3) Test-case generation:** gereksinimler **test edilebilir** olmalıdır. Testler geçerleme sürecinin parçası olarak tasarlanırsa bu çoğu zaman gereksinim sorunlarını ortaya çıkarır. Bir testi tasarlamak **zor ya da imkânsızsa** bu genelde gereksinimin gerçekleştirilmesinin zor

olacağı ve **yeniden düşünülmesi** gerektiği anlamına gelir. Kod yazılmadan önce kullanıcı gereksinimlerinden testler geliştirmek **XP**'nin ayrılmaz parçasıdır.

GEÇERLEME ZORDUR

Geçerlemenin sorunları hafife alınmamalıdır. Bir gereksinim kümesinin kullanıcı ihtiyaçlarını karşıladığını göstermek zordur: kullanıcıların sistemi çalışırken hayal etmesi ve işlerine nasıl uyacağını düşünmesi gerekir. Bu tür soyut analiz deneyimli bilişimciler için bile zor, kullanıcılar için daha da zordur. Sonuç olarak geçerleme sürecinde bütün gereksinim sorunları nadiren bulunur, gereksinim belgesi üzerinde anlaşıldıktan sonra bile eksikleri ve yanlış anlamaları düzeltmek için ek değişiklikler **kaçınılmazdır**.

7.3.1 GEREKSİNİM GÖZDEN GEÇİRMELERİ

Hem müşteri hem yüklenici kurumdan kişilerin katıldığı **elle (manual)** bir süreçtir. Belgeyi **anormallikler ve eksikler** için kontrol ederler. Program denetimleri (Bölüm 22) gibi yönetilebilir ya da farklı kişilerin belgenin farklı kısımlarını kontrol ettiği daha geniş bir faaliyet olarak düzenlenebilir. **Gayriresmi** gözden geçirmede yükleniciler gereksinimleri mümkün olduğunca çok paydaşla tartışır. Geliştiricilerle paydaşlar arasındaki iletişimin çıkarmadan sonra ne sıklıkla kesildiği ve belgelenen gereksinimlerin paydaşların gerçekten istediği şey olduğunun onaylanmadığı şaşırtıcıdır. Resmi bir gözden geçirmeye geçmeden sistemi paydaşlarla konuşarak birçok sorun bulunabilir. **Resmi** gözden geçirmede geliştirme ekibi müşteriyi sistem gereksinimleri boyunca "yürütür", her gereksinimin sonuçlarını açıklar. Ekip her gereksinimi **tutarlılık** ve gereksinimlerin bütününü **eksiksizlik** açısından kontrol eder. Ayrıca şunlar da kontrol edilebilir: **1) Verifiability:** gereksinim gerçekçi şekilde test edilebilir mi? **2) Comprehensibility:** alıcılar ya da son kullanıcılar gereksinimi doğru anlıyor mu? **3) Traceability:** gereksinimin **kaynağı** açıkça belirtilmiş mi? Bir değişikliğin etkisini değerlendirmek için kaynağa geri dönmek gerekebilir. İzlenebilirlik, değişikliğin sistemin geri kalanına etkisini değerlendirmeyi sağladığı için önemlidir. **4) Adaptability:** gereksinim uyarlanabilir mi, yani diğer gereksinimlere büyük ölçekli etki yapmadan değiştirilebilir mi? Çatışmalar, çelişkiler, hatalar ve eksikler gözden geçirenler tarafından belirtilmeli ve **gözden geçirme raporuna** resmi olarak kaydedilmelidir. Sonra kullanıcılar, alıcı ve geliştirici bu sorunlara bir çözüm **müzakere** eder.

👨‍🏫 Hoca anlatımı

Etnografi aslında antropolojiden gelen bir yöntem: araştırmacı bir topluluğun içinde yaşayıp onları gözlemler. Burada analist, sistemi kullanacak ofiste haftalarca oturup insanların **gerçekte** nasıl çalıştığını izler. Çünkü insanların "nasıl çalışıyorsun?" sorusuna verdiği cevap ile gerçekte yaptıkları farklıdır!

Hava trafik kontrolörü örneği çok güzel ve sınavda sorulabilir: kurallar "çatışma alarmını kullan" der, ama kontrolörler alarmı **kapatır**, çünkü uçakları zaten erkenden ayırırlar ve alarm dikkatlerini dağıtır. Görüşmede bunu kimse söylemez! Ama etnograf gözleyince görür. Bu, **"gerçek çalışma şekli"** gereksinimi.

İkinci tür: **"başkalarının farkında olmak"**. Kontrolörler komşu sektöre bakıp "birazdan bana 5 uçak gelecek" diye hazırlanır. Yeni sistem buna izin vermeli.

Etnografinin sınırı: sadece son kullanıcıya bakar, **kurumsal ve alan** gereksinimlerini bulamaz, **yeni özellik** öneremez. Bu yüzden use-case gibi tekniklerle **birlikte** kullanılır.

Validation'ın 5 kontrolü çok önemli: **Validity, Consistency, Completeness, Realism, Verifiability.** Kısaltma: **"V-C-C-R-V"**. Bölüm 4'te 3'ünü görmüştük (realism, consistency, completeness),

burada 5'e çıktı.

Neden önemli? Gereksinim hatası en pahalı hatadır: gereksinim değişirse tasarım, kod ve testler de değişir. Bir evin planını yanlış çizip bina bitince fark etmek gibi.

3 teknik: reviews (insanlar okur), prototyping (çalışan model gösterilir), test-case generation (test yazılamıyorsa gereksinim sorunludur).

Review'da ek 4 kontrol: Verifiability, Comprehensibility, **Traceability**, Adaptability.

📝 Deftere not

- Sistemler **sosyal ve kurumsal bağlamda** kullanılır. Bu gereksinimler hesaba katılmazsa sistem teslim edilir ama **kullanılmaz**.
- ⭐⭐ Ethnography: sosyal ve kurumsal gereksinimleri anlamak için **gözlem tekniği**. Analist kendini **çalışma ortamına yerleştirir**, günlük **gerçek işi** gözler. Resmi değil **gerçek** süreçleri yansıtan **örtük** gereksinimleri bulur.
- İnsanlar işlerinin ayrıntısını anlatamaz (**ikinci doğa**). Sosyal faktörleri **tarafsız gözlemci** fark eder. **Suchman:** gerçek ofis işi, otomasyon sistemlerinin varsaydığından çok daha **zengin, karmaşık, dinamik**. Bu fark verimlilik artmamasının ana nedeni.
- ⭐⭐ **Etnografinin bulduğu 2 tür gereksinim:**
  - 1. İnsanların **gerçekte nasıl çalıştığından** türeyen. Ör: ATC kontrolörleri **çatışma alarmını kapatır** (dikkat dağıtıyor, uçakları zaten erken ayırıyorlar).
  - 2. **İşbirliğinden ve başkalarının faaliyetlerinin farkında olmaktan** türeyen. Ör: kontrolörler **komşu sektörlerin** işini görüp iş yükünü tahmin eder → sistem buna izin vermeli.
- ⭐ **Şekil 7.9, etnografi + prototipleme:** etnografi prototipi besler (daha az döngü), prototip etnografiye **odak** verir. Akış: ethnographic analysis → debriefing meetings → focused ethnography ⇄ system prototyping → prototype evaluation.
- ⭐⭐ **Etnografinin sınırları:** kritik süreç ayrıntılarını bulur, ama **son kullanıcıya** odaklı, bu yüzden **kurumsal ve alan** gereksinimlerine uygun değil. **Yeni özellikleri** her zaman bulamaz. Tek başına yetmez, **use-case** gibi tekniklerle birlikte kullan.
- ⭐ **7.3 Requirements validation:** gereksinimler müşterinin **gerçekten istediği** sistemi tanımlıyor mu? **Analizle örtüşür**.
- ⭐⭐ Neden önemli? Gereksinim hatası geç bulunursa **çok pahalı**: gereksinim değişikliği → tasarım + gerçekleştirme değişir + **tekrar test**. Tasarım ya da kod hatasından **çok daha pahalı**.
- ⭐⭐ **5 kontrol:**
  - **Validity:** gerçekten gereken fonksiyonlar mı? (ek ya da farklı fonksiyon gerekebilir, gereksinimler paydaşlar arası **uzlaşma**).
  - **Consistency:** gereksinimler **çelişmemeli**.
  - **Completeness: tüm** fonksiyon ve kısıtlar tanımlanmalı.
  - **Realism:** mevcut teknolojiyle **gerçekleştirilebilir** mi? + **bütçe ve takvim**.
  - **Verifiability: test yazılabilmeli** (anlaşmazlığı azaltır).
- ⭐⭐ **3 geçerleme tekniği:**
  - **Requirements reviews:** ekip sistematik olarak analiz eder.
  - **Prototyping: çalıştırılabilir model** gösterilir, kullanıcı deneme yapar.

- **Test-case generation:** test tasarlamak **zor ya da imkânsızsa** gereksinim sorunlu, yeniden düşün. (XP: kod öncesi test.)

- Geçerleme **zor**: kullanıcı sistemi çalışırken hayal etmeli (soyut analiz). Tüm sorunlar nadiren bulunur, sonradan değişiklik **kaçınılmaz**.
- ⭐ **7.3.1 Requirements reviews:** müşteri + yüklenici kişilerinin katıldığı **elle (manual)** süreç. **Anormallik ve eksik** arar.
  - **Informal:** yükleniciler çok sayıda paydaşla konuşur (çıkarmadan sonra iletişim çoğu zaman kesilir!).
  - **Formal:** ekip müşteriyi gereksinimler boyunca "**yürütür**", her gereksinim **tutarlılık**, bütün **eksiksizlik** için kontrol edilir.
- ⭐⭐ **Review'da ek 4 kontrol: Verifiability** (gerçekçi test edilebilir mi?) · **Comprehensibility** (alıcı ve kullanıcı anlıyor mu?) · **Traceability** (**kaynağı** belli mi? değişiklik etkisi için) · **Adaptability** (diğerlerini çok etkilemeden **değiştirilebilir** mi?).
- Sorunlar **review raporuna** resmi kaydedilir, sonra kullanıcı, alıcı ve geliştirici çözümü **müzakere** eder.

Sayfa 161–167 — 7.4 Requirements management · Enduring / volatile · Planning · Traceability · Change management

📖 Sayfanın özeti

GEREKSİNİMLER NEDEN DEĞİŞİR?

Büyük sistemlerin gereksinimleri **sürekli değişir**. Bir nedeni, bu sistemlerin genelde "**wicked**" problemleri (Bölüm 2) çözmek için geliştirilmesidir: problem tam tanımlanamadığı için gereksinimler eksik kalmaya mahkûmdur ve süreç boyunca paydaşların problemi anlayışı değişir. Ayrıca sistem kurulduktan sonra yeni gereksinimler **kaçınılmaz olarak** ortaya çıkar: kullanıcılar sistemin kuruma etkisini önceden kestiremez, deneyim kazandıkça yeni ihtiyaçlar ve öncelikler keşfederler. **3 neden:** (1) Büyük sistemlerin farklı gereksinim ve önceliklere sahip **çeşitli kullanıcı toplulukları** vardır. Bunlar çatışabilir, son gereksinimler kaçınılmaz olarak bir uzlaşmadır ve deneyimle farklı kullanıcılara verilen desteğin dengesinin değişmesi gerektiği anlaşılır. (2) Sistemin **parasını ödeyenler** ile **kullananlar** nadiren aynı kişilerdir. Müşteriler kurumsal ve bütçe kısıtları yüzünden gereksinimler koyar, bunlar son kullanıcı gereksinimleriyle çatışabilir ve teslimden sonra kullanıcı desteği için yeni özellikler eklenmesi gerekebilir. (3) Sistemin **iş ve teknik ortamı** kurulumdan sonra değişir ve bu sisteme yansımalıdır: yeni donanım gelebilir, başka sistemlerle arayüz gerekebilir, iş öncelikleri değişebilir, uygulanması gereken yeni **yasa ve yönetmelikler** çıkabilir.

TANIM

Requirements management, sistem gereksinimlerindeki değişiklikleri **anlama ve kontrol etme** sürecidir. Bireysel gereksinimleri takip etmek ve bağımlı gereksinimler arasındaki bağlantıları korumak gerekir ki değişikliklerin etkisi değerlendirilebilsin. Değişiklik önerileri yapmak ve bunları sistem gereksinimlerine bağlamak için **resmi bir süreç** kurulmalıdır. Gereksinim yönetimi, gereksinim belgesinin **taslağı** hazır olur olmaz başlamalıdır, ama değişen gereksinimleri nasıl yöneteceğini planlamaya **çıkarma sırasında** başlanmalıdır.

7.4.1 KALICI VE DEĞİŞKEN GEREKSİNİMLER (ŞEKİL 7.10)

Gereksinimlerin RE süreci sırasında ve sistem hizmete girdikten sonra evrilmesi kaçınılmazdır. Gereksinim geliştirmek dikkati yazılım yeteneklerine, iş hedeflerine ve diğer iş sistemlerine odaklar. Gereksinim tanımı geliştikçe kullanıcı ihtiyaçları daha iyi anlaşılır, bu da kullanıcıya geri bildirilir ve kullanıcı bir değişiklik önerebilir (Şekil 7.10: problemin ilk anlayışı → ilk gereksinimler, zamanla problemin değişen anlayışı → değişen gereksinimler). Büyük bir sistemi tanımlayıp geliştirmek yıllar alabilir, bu sürede ortam ve iş hedefleri değişir. Evrim açısından gereksinimler iki sınıfa ayrılır: **1) Enduring requirements (kalıcı):** kurumun **temel faaliyetinden** türeyen ve doğrudan sistemin alanıyla ilgili **nispeten kararlı** gereksinimler. Ör: bir hastanede her zaman hastalar, doktorlar, hemşireler ve tedavilerle ilgili gereksinimler olacaktır. Bu gereksinimler bir uygulama alanını tanımlayan varlık ve ilişkileri gösteren **alan modellerinden** türetilebilir. **2) Volatile requirements (değişken):** sistem geliştirme sırasında ya da sistem çalışmaya başladıktan sonra **değişmesi muhtemel** gereksinimler. Ör: devletin sağlık politikalarından kaynaklanan gereksinimler.

ŞEKİL 7.11: DEĞİŞKEN GEREKSİNİMLERİN SINIFLANDIRMASI

Harker ve arkadaşları değişken gereksinimlerin beş sınıfa ayrıldığını önerdi, yazar buna dayanarak şu sınıflandırmayı geliştirdi:

Tür	Kısaca
Mutable requirements	Kurumun çalıştığı **ortam** değiştiği için değişen gereksinimler. Ör: hastanede hasta bakımının finansmanı değişir ve farklı tedavi bilgileri toplanması gerekir.
Emergent requirements	Geliştirme sırasında müşterinin sistemi **anlayışı geliştikçe** ortaya çıkan gereksinimler. Tasarım süreci yeni emergent gereksinimleri ortaya çıkarabilir.
Consequential requirements	Bilgisayar sisteminin **kurulmasının sonucu** olarak ortaya çıkan gereksinimler. Sistem kurumun süreçlerini değiştirip yeni çalışma yolları açabilir, bunlar yeni gereksinimler doğurur.
Compatibility requirements	Kurumdaki belirli **diğer sistemlere** ya da iş süreçlerine bağlı gereksinimler. Onlar değiştikçe sipariş edilen ya da teslim edilen sistemin uyumluluk gereksinimleri de değişebilir.

7.4.2 GEREKSİNİM YÖNETİMİ PLANLAMASI

Planlama gereksinim yönetimi sürecinin **ilk temel aşamasıdır**. Gereksinim yönetimi **çok pahalıdır**. Her proje için planlama aşaması gereken yönetim ayrıntı seviyesini belirler. Şunlara karar verilir: **1) Requirements identification:** her gereksinim **benzersiz şekilde tanımlanmalı** ki diğer gereksinimlerden çapraz referans verilebilsin ve izlenebilirlik değerlendirmelerinde kullanılabilsin. **2) A change management process:** değişikliklerin **etkisini ve maliyetini** değerlendiren faaliyetler kümesi. **3) Traceability policies:** gereksinimler arasındaki ve gereksinimler ile sistem tasarımı arasındaki hangi ilişkilerin kaydedileceğini ve bu kayıtların nasıl tutulacağını tanımlayan politikalar. **4) CASE tool support:** gereksinim yönetimi büyük miktarda bilginin işlenmesini içerir. Kullanılabilecek araçlar özel gereksinim yönetim sistemlerinden tablolama programları ve basit veritabanı sistemlerine kadar uzanır.

TRACEABİLİTY (İZLENEBİLİRLİK)

Gereksinimler arasında, gereksinimler ile sistem tasarımı arasında ve gereksinimler ile onların önerilme nedenleri arasında birçok ilişki vardır. Değişiklik önerildiğinde bu değişikliğin diğer gereksinimlere ve tasarıma etkisi izlenmelidir. **Traceability**, bir gereksinim tanımlamasının **ilgili gereksinimleri bulma kolaylığını** yansıtan özelliğidir. **3 tür izlenebilirlik bilgisi: 1) Source**

traceability: gereksinimleri onları öneren **paydaşlara** ve **gerekçelerine** bağlar. Değişiklik önerildiğinde değişiklik hakkında danışılacak paydaşları bulmak için kullanılır. **2) Requirements traceability:** belgedeki **bağımlı gereksinimleri** birbirine bağlar. Bir değişiklikten kaç gereksinimin etkileneceğini ve gereken ek değişikliklerin boyutunu değerlendirmek için kullanılır. **3) Design traceability:** gereksinimleri onları gerçekleştiren **tasarım modüllerine** bağlar. Önerilen değişikliğin sistem tasarımına ve gerçekleştirmesine etkisini değerlendirmek için kullanılır.

TRACEABİLİTY MATRİX (ŞEKİL 7.12)

İzlenebilirlik bilgisi genelde gereksinimleri paydaşlarla, birbirleriyle ya da tasarım modülleriyle ilişkilendiren **izlenebilirlik matrisleriyle** gösterilir. Gereksinim izlenebilirlik matrisinde her gereksinim hem bir **satıra** hem bir **sütuna** yazılır, bağımlılıklar satır ve sütunun kesiştiği hücreye kaydedilir. **"D"**, satırdaki gereksinimin sütundaki gereksinime **bağımlı (depends)** olduğunu, **"R"** ise başka, daha **zayıf bir ilişki (related)** olduğunu gösterir (ör. ikisi de aynı alt sistemin parçalarının gereksinimlerini tanımlıyor olabilir). Matrisler az sayıda gereksinim için kullanılabilir ama çok gereksinimli büyük sistemlerde **hantal ve bakımı pahalı** hale gelir. Bu sistemlerde izlenebilirlik bilgisi, her gereksinimin ilgili gereksinimlere açıkça bağlandığı bir **gereksinim veritabanında** tutulmalıdır. Değişikliklerin etkisi veritabanının gezinme özellikleriyle değerlendirilir ve matrisler veritabanından otomatik üretilebilir.

ARAÇ DESTEĞİ

Gereksinim yönetimi otomatik destek gerektirir, CASE araçları planlama aşamasında seçilmelidir. Araç desteği gerekenler: **1) Requirements storage:** gereksinimler RE sürecindeki herkesin erişebileceği **güvenli, yönetilen** bir veri deposunda tutulmalıdır. **2) Change management:** aktif araç desteği varsa değişiklik yönetim süreci (Şekil 7.13) kolaylaşır. **3) Traceability management:** araç desteği ilgili gereksinimlerin bulunmasını sağlar, bazı araçlar gereksinimler arasındaki olası ilişkileri bulmak için doğal dil işleme teknikleri kullanır. Küçük sistemlerde özel araçlar gerekmeyebilir (kelime işlemci, tablolama programı, PC veritabanları yeterli). Büyük sistemlerde daha özel araçlar gerekir. Örnek araçlar: **DOORS** ve **RequisitePro**.

7.4.3 DEĞİŞİKLİK YÖNETİMİ (ŞEKİL 7.13)

Gereksinimlere önerilen **bütün değişikliklere** uygulanmalıdır. Resmi bir süreç kullanmanın avantajı, bütün değişiklik önerilerinin **tutarlı** şekilde ele alınması ve gereksinim belgesindeki değişikliklerin **kontrollü** yapılmasıdır. **3 aşama: 1) Problem analysis and change specification:** süreç belirlenmiş bir gereksinim sorunuyla ya da bazen belirli bir değişiklik önerisiyle başlar. Bu aşamada sorun ya da öneri **geçerli olup olmadığını** kontrol etmek için analiz edilir. Sonuçlar değişikliği isteyen kişiye geri bildirilir, bazen daha belirli bir değişiklik önerisi yapılır. **2) Change analysis and costing:** önerilen değişikliğin etkisi **izlenebilirlik bilgisi** ve sistem gereksinimleri hakkındaki genel bilgi kullanılarak değerlendirilir. Maliyeti gereksinim belgesinde ve gerekirse tasarım ile gerçekleştirmede yapılacak değişiklikler cinsinden tahmin edilir. Analiz bitince değişikliğe **devam edilip edilmeyeceğine** karar verilir. **3) Change implementation:** gereksinim belgesi ve gerekirse tasarım ile gerçekleştirme değiştirilir. Belge, kapsamlı yeniden yazım ya da yeniden düzenleme gerekmeden değiştirilebilecek şekilde düzenlenmelidir: dış referanslar en aza indirilmeli ve bölümler mümkün olduğunca **modüler** olmalıdır.



Şekil 7.13: Gereksinim değişiklik yönetimi

ACİL DEĞİŞİKLİK TUZAĞI VE AGİLE

> Bir değişiklik acilen gerekiyorsa, değişikliği önce sisteme yapıp gereksinim belgesini **sonradan** güncelleme isteği her zaman vardır. Bu neredeyse kaçınılmaz olarak gereksinim tanımlaması ile sistem gerçekleştirmesinin **uyumsuz** hale gelmesine yol açar: sistem değişikliği yapıldıktan sonra belge değişiklikleri unutulabilir ya da sistemle tutarsız yapılabilir. Extreme programming gibi yinelemeli süreçler geliştirme sırasında değişen gereksinimlerle başa çıkmak için tasarlanmıştır. Bu süreçlerde kullanıcı bir değişiklik önerdiğinde resmi bir değişiklik yönetim sürecinden geçmez. Kullanıcı o değişikliği **önceliklendirir** ve önceliği yüksekse bir sonraki yineleme için planlanan hangi özelliklerin **çıkarılacağına** karar verir.

👨‍🏫 Hoca anlatımı

Gereksinim yönetimi = değişen gereksinimleri kontrol altında tutmak. Çünkü gereksinimler **her zaman** değişir.

Değişimin 3 nedeni: (1) farklı kullanıcı grupları farklı şey istiyor ve denge değişiyor, (2) **parayı veren ile kullanan farklı** kişiler (patron "ucuz olsun" der, çalışan "şu özellik lazım" der), (3) iş ve teknik ortam değişiyor (yeni yasa, yeni donanım).

Enduring ve volatile farkı sınavın favorisi: **Enduring** = kalıcı, işin özünden gelir. Hastanede "hasta, doktor, tedavi" kavramları 50 yıl sonra da olacak. **Volatile** = değişken. "Devletin sağlık politikasına göre şu belge istensin" kuralı yarın değişebilir.

Volatile'ın 4 türü (Şekil 7.11): **Mutable** (ortam değişti), **Emergent** (müşteri daha iyi anladı), **Consequential** (sistemin kendisi çalışma şeklini değiştirdi), **Compatibility** (bağlı olduğu diğer sistemler değişti). Hatırlatıcı: "**MECC**".

Traceability = "bu gereksinim nereden geldi ve neyi etkiliyor?" **3 tür**: **Source** (kim önerdi, neden?), **Requirements** (hangi gereksinimler birbirine bağlı?), **Design** (hangi kod modülünde gerçekleştirildi?). Bir domino taşı gibi: birini değiştirince hangilerinin düşeceğini bilmen lazım.

Traceability matrix: satır ve sütunlarda gereksinimler. **D** = depends (güçlü bağımlılık), **R** = related (zayıf ilişki). Büyük sistemlerde matris çok büyür, veritabanı kullanılır (alıştırma 7.8 bunu soruyor).

Değişiklik yönetimi 3 adım: **analiz et → maliyetini hesapla → uygula**. Ve önemli uyarı: acele edip önce kodu değiştirip belgeyi sonraya bırakırsan, belge ile sistem birbirini tutmaz hale gelir (alıştırma 7.9).

📝 Deftere not

- ⭐ Gereksinimler **sürekli değişir**: sistemler **wicked problems** için yapılır (gereksinimler eksik kalır, anlayış değişir) + kurulumdan sonra **yeni gereksinimler** kaçınılmaz.
- ⭐⭐ **Değişimin 3 nedeni:**
  - 1. **Çeşitli kullanıcı toplulukları**, farklı ve çatışan öncelikler, gereksinimler uzlaşma, deneyimle denge değişir.
  - 2. **Parayı ödeyenler ≠ kullananlar**: müşteri kurumsal ve bütçe kısıtları koyar, son kullanıcıyla çatışır.
  - 3. **İş ve teknik ortam** değişir (yeni donanım, arayüzler, iş öncelikleri, yeni **yasalar**).
- ⭐ **Requirements management:** gereksinim değişikliklerini **anlama ve kontrol etme** süreci. Bireysel gereksinimleri takip et, bağımlılık bağlantılarını koru, **resmi değişiklik süreci** kur. **Taslak**

belge hazır olunca başla, planlamaya **çıkarma sırasında** başla.

- **Şekil 7.10:** ilk anlayış → ilk gereksinimler. Zamanla değişen anlayış → değişen gereksinimler.
- ⭐⭐ **7.4.1, evrim açısından 2 sınıf:**
  - Enduring requirements: **kararlı**, kurumun **temel faaliyetinden** ve alandan. Ör: hastanede hasta, doktor, hemşire, tedavi. **Alan modellerinden** türetilebilir.
  - Volatile requirements: geliştirme sırasında ya da sonra **değişmesi muhtemel**. Ör: devletin **sağlık politikalarından** gelenler.
- ⭐⭐ **Şekil 7.11, volatile gereksinim türleri (Harker'a dayalı):**
  - **Mutable: ortam** değiştiği için (ör. hasta bakımı finansmanı değişir).
  - **Emergent:** müşterinin **anlayışı geliştikçe** ortaya çıkan (tasarım yeni gereksinim ortaya çıkarabilir).
  - **Consequential:** sistemin **kurulmasının sonucu** (süreçler değişir, yeni çalışma yolları).
  - **Compatibility:** kurumdaki **diğer sistem ve süreçlere** bağlı, onlar değişince değişir.
- ⭐ **7.4.2 Planning:** gereksinim yönetiminin **ilk** aşaması. Gereksinim yönetimi **çok pahalı**, ayrıntı seviyesi belirlenir. **4 karar:**
  - **Requirements identification:** her gereksinim **benzersiz kimlikli** (çapraz referans, izlenebilirlik).
  - **Change management process:** değişikliğin **etkisini ve maliyetini** değerlendirme.
  - **Traceability policies:** hangi ilişkiler kaydedilecek, nasıl tutulacak.
  - **CASE tool support:** özel araçlardan tablolama ve basit veritabanına.
- ⭐ Traceability: gereksinim tanımlamasının **ilgili gereksinimleri bulma kolaylığı**.
- ⭐⭐ **3 tür izlenebilirlik:**
  - **Source traceability:** gereksinim → **öneren paydaş** + **gerekçe** (değişiklikte kime danışılacak).
  - **Requirements traceability:** gereksinim → **bağımlı gereksinimler** (kaç gereksinim etkilenir).
  - **Design traceability:** gereksinim → **tasarım modülleri** (tasarım ve gerçekleştirmeye etki).
- ⭐⭐ **Traceability matrix (Şekil 7.12):** gereksinimler hem satır hem sütunda. **D** = satırdaki, sütundakine **bağımlı (depends)**. **R** = **zayıf ilişki (related)**, ör. aynı alt sistem. Az gereksinimde iyi, büyük sistemde **hantal ve pahalı** → **gereksinim veritabanı** (matris otomatik üretilir).
- ⭐ **Araç desteği gerekenler: Requirements storage** (güvenli, yönetilen, herkes erişebilir) · **Change management** · **Traceability management** (doğal dil işleme ile ilişki bulma). Küçük sistem: kelime işlemci, tablolama yeter. Büyük: **DOORS, RequisitePro**.
- ⭐⭐ **7.4.3 Change management (Şekil 7.13): tüm** değişikliklere uygulanır (tutarlı ve kontrollü). **3 aşama:**
  - **1. Problem analysis & change specification:** sorun ya da öneri **geçerli mi?** İsteyene geri bildir.
  - **2. Change analysis & costing: izlenebilirlikle** etkiyi değerlendir, **maliyeti** tahmin et, **devam mı?** kararı.
  - **3. Change implementation:** belgeyi (ve gerekirse tasarım ve kodu) değiştir. Belge **modüler**, dış referanslar az olmalı.
- ⭐ **Tuzak:** acil değişikliği önce sisteme yapıp belgeyi **sonra** güncellemek → spec ve sistem **uyumsuz** hale gelir.

- **XP:** resmi değişiklik süreci yok. Kullanıcı değişikliği **önceliklendirir**, yüksekse sonraki yinelemeden hangi özelliğin **çıkarılacağına** karar verir.

Sayfa 166–168 Key Points · Further Reading · Exercises

📖 Sayfanın özeti

KEY POİNTS (BÖLÜM ÖZETİ)

Gereksinim mühendisliği süreci fizibilite çalışmasını, gereksinim çıkarma ve analizini, tanımlamayı, geçerlemeyi ve yönetimi içerir. Gereksinim çıkarma ve analizi, keşif, sınıflandırma ve düzenleme, müzakere ve belgeleme faaliyetlerinden oluşan bir spiral olarak gösterilebilen yinelemeli bir süreçtir. Farklı paydaşların farklı gereksinimleri vardır, bu yüzden bütün karmaşık sistemler birkaç viewpoint'ten analiz edilmelidir. Viewpoint'ler sistemle etkileşen insanlar ya da sistemler, sistemden etkilenen paydaşlar ya da gereksinimleri kısıtlayan alan viewpoint'leri olabilir. Sosyal ve kurumsal faktörler sistem gereksinimlerini güçlü şekilde etkiler ve yazılımın gerçekten kullanılıp kullanılmayacağını belirleyebilir. Gereksinim geçerleme, gereksinimleri geçerlilik, tutarlılık, eksiksizlik, gerçekçilik ve doğrulanabilirlik açısından kontrol etme sürecidir. Gözden geçirmeler ve prototipleme temel geçerleme teknikleridir. İş, kurum ve teknik değişiklikler kaçınılmaz olarak gereksinimlerde değişikliğe yol açar, gereksinim yönetimi bu değişiklikleri yönetme ve kontrol etme sürecidir. Gereksinim yönetimi süreci, politika ve prosedürlerin tasarlandığı **yönetim planlamasını** ve önerilen değişikliklerin analiz edilip etkilerinin değerlendirildiği **değişiklik yönetimini** içerir.

FURTHER READİNG

IEEE Software'in araba ve tıbbi cihaz alanlarına odaklı gereksinim mühendisliği özel sayısı, Robertson'ların *Mastering the Requirements Process* kitabı ve Kotonya ile Sommerville'in VORD yöntemini anlatan kitabı. *Sınav için önemli değil.*

ALIŞTIRMALAR (KONULARI)

#	Konu
7.1	Üniversite öğrenci kayıt sisteminin **paydaşları** kimler? Gereksinimleri neden çatışır?
7.2	Klinik hasta kayıt sistemi için temel **viewpoint'leri** belirleyip bir **viewpoint hiyerarşisi** çizin.
7.3	LIBSYS'teki üç viewpoint için her birinden gelebilecek üç gereksinim önerin.
7.4	Birden çok makineye dağıtılmış kataloglama için en önemli **fonksiyonel olmayan** gereksinim türleri nelerdir?
7.5	Bir **ATM** için gereksinimleri anlamaya temel olacak bir **use-case** kümesi geliştirin.
7.6	Sosyal ve politik faktörlerin gereksinimleri güçlü şekilde etkileyebileceği bir sistem örneği tartışın.
7.7	Gereksinim **gözden geçirmesine** kimler katılmalı? Süreç modelini çizin.
7.8	Çok gereksinim olunca **izlenebilirlik matrisleri** neden yönetilmesi zor hale gelir? Viewpoint'lere dayalı bir yapılandırma mekanizması önerin.
7.9	Acil değişikliklerde gereksinim belgesi ile sistemin **tutarsızlaşmamasını** sağlayacak bir süreç modeli önerin.

7.10	Şirketin standart analiz yöntemi sosyal faktörleri gösteremiyor ve yönetici standarda uyulmasını istiyor. Ne yaparsın? (etik)

👩‍🏫 Hoca anlatımı

7.1 (öğrenci kayıt sistemi paydaşları): öğrenciler, öğretim üyeleri, öğrenci işleri personeli, bölüm başkanları, rektörlük, finans bölümü, bilgi işlem, YÖK gibi düzenleyiciler. Neden çatışır? Örneğin öğrenci notlarını hemen görmek ister, öğretim üyesi notları ilan etmeden önce düzeltme süresi ister. Güvenlik birimi erişimi kısıtlamak ister, kullanıcılar kolay erişim ister.

7.7 (review'a kimler katılır?): hem müşteri tarafı (kullanıcı temsilcileri, alıcı yöneticiler) hem yüklenici tarafı (gereksinim mühendisleri, tasarımcılar, test mühendisleri). Süreç: belgeyi dağıt → herkes ayrı okur → toplantı → sorunları rapora yaz → müzakere ile çöz → belgeyi güncelle.

7.9 için fikir: acil değişiklik yapılsa bile bir "acil değişiklik kaydı" açılsın ve belgenin güncellenmesi değişikliğin **tamamlanma şartı** olsun.

📝 Deftere not

- ⭐ **KEY POINTS:** RE = fizibilite, çıkarma ve analiz, tanımlama, geçerleme, yönetim · çıkarma = yinelemeli **spiral** (discovery, classification, negotiation, documentation) · karmaşık sistemler **birden çok viewpoint**'ten analiz edilir (interactor, indirect, domain) · **sosyal ve kurumsal faktörler** yazılımın kullanılıp kullanılmayacağını belirleyebilir · geçerleme = validity, consistency, completeness, realism, verifiability. Temel teknikler: **reviews + prototyping** · değişim kaçınılmaz, **requirements management** = değişikliği yönet · yönetim = **management planning** + **change management**.
- ⭐ Sınavda en olası: **7.1** (paydaşlar), **7.2** (viewpoint hiyerarşisi), **7.5** (ATM use-case), **7.7** (review), **7.8** (traceability matrix), **7.9** (acil değişiklik).

BÖLÜM 7 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 7'deki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
requirements engineering process	gereksinim mühendisliği süreci	Gereksinim belgesini oluşturma ve sürdürme	143
feasibility study	fizibilite çalışması	Sisteme değer mi? kısa, odaklı çalışma	144–146
feasibility report	fizibilite raporu	Devam edilmeli mi önerisi	146
requirements elicitation and analysis	gereksinim çıkarma ve analiz	Paydaşlarla gereksinimleri bulma	146
requirements management	gereksinim yönetimi	Değişiklikleri anlama ve kontrol	143, 161
business requirements	iş gereksinimleri	Üst seviye iş ihtiyaçları	144
stakeholder	paydaş	Sistemden doğrudan ya da dolaylı etkilenen	146
trade union representatives	sendika temsilcileri	Paydaş örneği	146
requirements discovery	gereksinim keşfi	Paydaşlardan gereksinim toplama	147–148
classification and organisation	sınıflandırma ve düzenleme	Gereksinimleri gruplama	147
prioritisation and negotiation	önceliklendirme ve müzakere	Çatışmaları çözme	147–148
requirements documentation	gereksinim belgeleme	Gereksinimleri yazma	148
compromise	uzlaşma	Farklı isteklerin ortası	148
viewpoint	bakış açısı	Gereksinim kaynağı ve onun bakışı	149
interactor viewpoints	etkileşimci bakış açıları	Sistemle doğrudan etkileşenler	150
indirect viewpoints	dolaylı bakış açıları	Kullanmayan ama etkileyen paydaşlar	150
domain viewpoints	alan bakış açıları	Alan özellikleri ve kısıtları	150
viewpoint hierarchy	bakış açısı hiyerarşisi	Viewpoint'lerin ağaç düzeni	151
interviewing	görüşme	Paydaşlara soru sorma	152

closed / open interviews	kapalı / açık görüşmeler	Hazır sorular / serbest gündem	152
open-minded	açık fikirli	İyi görüşmecinin özelliği	153
scenario	senaryo	Örnek etkileşim oturumu tanımı	153
initial assumption	başlangıç varsayımı	Senaryonun başlangıç durumu	154
normal flow of events	normal olay akışı	Her şey yolunda giderse	153–154
what can go wrong	ne ters gidebilir	Senaryodaki hata durumları	154
use-case	kullanım durumu	Etkileşim türü + aktörler (UML)	154–155
actor	aktör	Sistemle etkileşen kişi ya da sistem (çöp adam)	155
Objectory method	Objectory yöntemi	Use-case'lerin ilk tanıtıldığı yöntem (Jacobson)	154
de facto standard	fiili standart	UML'nin OO modellemedeki yeri	155
ethnography	etnografi	Çalışma ortamında gözlem tekniği	157
implicit requirements	örtük gereksinimler	Söylenmeyen ama var olan gereksinimler	157
conflict alert system	çatışma uyarı sistemi	ATC'de kontrolörlerin kapattığı alarm	157
focused ethnography	odaklı etnografi	Prototiple yönlendirilen gözlem	158
debriefing meetings	değerlendirme toplantıları	Etnografi sonrası toplantılar	158
requirements validation	gereksinim geçerleme	Müşterinin istediği sistem mi?	158
rework	yeniden çalışma	Geç bulunan hatanın düzeltme maliyeti	158
validity checks	geçerlilik kontrolleri	Gerçekten gereken fonksiyonlar mı?	159
consistency checks	tutarlılık kontrolleri	Çelişki var mı?	159
completeness checks	eksiksizlik kontrolleri	Her şey tanımlı mı?	159
realism checks	gerçekçilik kontrolleri	Gerçekleştirilebilir mi?	159

verifiability	doğrulanabilirlik	Test yazılabilir mi?	159–160
requirements reviews	gereksinim gözden geçirmeleri	Ekiple sistematik analiz	159–160
prototyping	prototipleme	Çalıştırılabilir model gösterme	159
test-case generation	test durumu üretimi	Gereksinimlerden test yazma	159
comprehensibility	anlaşılabilirlik	Kullanıcı gereksinimi anlıyor mu?	160
traceability	izlenebilirlik	İlgili gereksinimleri ve kaynağı bulma kolaylığı	160, 163
adaptability	uyarlanabilirlik	Diğerlerini etkilemeden değiştirilebilme	160
review report	gözden geçirme raporu	Sorunların resmi kaydı	160
enduring requirements	kalıcı gereksinimler	Kararlı, temel faaliyetten gelen	162
volatile requirements	değişken gereksinimler	Değişmesi muhtemel	162
mutable requirements	değişebilir gereksinimler	Ortam değişikliğinden	163
emergent requirements	ortaya çıkan gereksinimler	Anlayış geliştikçe	163
consequential requirements	sonuçsal gereksinimler	Sistemin kurulmasının sonucu	163
compatibility requirements	uyumluluk gereksinimleri	Diğer sistem ve süreçlere bağlı	163
requirements identification	gereksinim kimliklendirme	Her gereksinime benzersiz kimlik	162
traceability policies	izlenebilirlik politikaları	Hangi ilişkiler kaydedilecek	163
source / requirements / design traceability	kaynak / gereksinim / tasarım izlenebilirliği	3 izlenebilirlik türü	163–164
traceability matrix	izlenebilirlik matrisi	D (depends) ve R (related) tablosu	164
requirements database	gereksinim veritabanı	Büyük sistemlerde izlenebilirlik	164

requirements storage	gereksinim depolama	Güvenli, yönetilen veri deposu	165
DOORS / RequisitePro	gereksinim yönetim araçları	Örnek CASE araçları	165
change management	değişiklik yönetimi	Değişiklikleri tutarlı ve kontrollü işleme	165
problem analysis and change specification	sorun analizi ve değişiklik tanımı	Değişiklik yönetimi 1. aşama	165
change analysis and costing	değişiklik analizi ve maliyetlendirme	Değişiklik yönetimi 2. aşama	165
change implementation	değişiklik uygulama	Değişiklik yönetimi 3. aşama	166

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 7'den sınavda en çok çıkabilecekler

1. **RE'nin 4 alt süreci** (feasibility, elicitation & analysis, specification, validation) + **management** · spiral görünüm (s.143–144)
2. **Fizibilitenin 3 sorusu:** hedeflere katkı, teknoloji / maliyet / takvim, entegrasyon. **2–3 hafta**, sonunda rapor. (s.145–146)
3. **Stakeholder** tanımı (s.146)
4. **Gereksinim çıkarmanın 5 zorluğu** (s.146)
5. **Çıkarma ve analiz süreci 4 faaliyet:** discovery, classification & organisation, prioritisation & negotiation, documentation (s.147–148)
6. **3 viewpoint türü:** interactor, indirect, domain (ATM örnekleri) + **hiyerarşi** (s.150–151)
7. **Closed ve open interview** · görüşmenin **zayıf** yanları (alan bilgisi, kurumsal bilgi) · **iyi görüşmecinin 2 özelliği** (s.152–153)
8. **Senaryonun 5 öğesi** (s.153)
9. **Use-case** = aktör (çöp adam) + elips · use-case = senaryolar kümesi · interactor için iyi, NFR ve alan için zayıf (s.154–155)
10. **Ethnography** + **2 tür gereksinim** (gerçek çalışma, işbirliği) + sınırları (s.157–158)
11. **Validation neden önemli** (gereksinim hatası en pahalı) · **5 kontrol** (V-C-C-R-V) · **3 teknik** (reviews, prototyping, test-case generation) (s.158–159)
12. **Review'da 4 ek kontrol:** verifiability, comprehensibility, traceability, adaptability (s.160)
13. **Gereksinimlerin değişmesinin 3 nedeni** (s.161)
14. **Enduring ve volatile** · **volatile 4 türü** (mutable, emergent, consequential, compatibility) (s.162–163)
15. **Yönetim planlaması 4 kararı** · **3 izlenebilirlik türü** (source, requirements, design) · **matris** (D / R) (s.162–164)
16. **Değişiklik yönetimi 3 aşaması** · acil değişiklik tuzağı (s.165–166)

C Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What are the main activities of the requirements engineering process?

RE sürecinin ana faaliyetleri (s.143)

> *1) Feasibility study: is the system useful to the business? 2) Requirements elicitation and analysis: discovering requirements. 3) Requirements specification: converting requirements into a standard form. 4) Requirements validation: checking that the requirements define the system the customer wants. In addition, requirements management handles changing requirements.*

1) Fizibilite: sistem işe yarar mı? 2) Çıkarma ve analiz: gereksinimleri keşfetme. 3) Tanımlama: standart biçime dönüştürme. 4) Geçerleme: müşterinin istediği sistemi tanımlıyor mu? Ayrıca gereksinim yönetimi değişen gereksinimleri ele alır.

Q2. What questions does a feasibility study try to answer?

Fizibilite hangi soruları cevaplar? (s.145)

> *1) Does the system contribute to the overall objectives of the organisation? 2) Can the system be implemented using current technology and within cost and schedule constraints? 3) Can the system be integrated with other systems that are already in place? The result is a report that recommends whether or not to continue. It should take about two or three weeks.*

1) Sistem kurumun genel hedeflerine katkı sağlıyor mu? 2) Mevcut teknolojiyle, maliyet ve takvim içinde gerçekleştirilebilir mi? 3) Mevcut sistemlerle entegre edilebilir mi? Sonuç, devam edilip edilmeyeceğini öneren bir rapordur. Yaklaşık iki üç hafta sürmelidir.

Q3. Why is it difficult to elicit and understand stakeholder requirements?

Gereksinim çıkarmak neden zordur? (s.146)

> *1) Stakeholders often don't know what they want, or make unrealistic demands. 2) They express requirements in their own terms with implicit knowledge. 3) Different stakeholders have different, often conflicting, requirements. 4) Political factors may influence requirements. 5) The economic and business environment is dynamic, so requirements change during analysis.*

1) Paydaşlar çoğu zaman ne istediklerini bilmez ya da gerçekçi olmayan isteklerde bulunur. 2) Gereksinimleri kendi terimleri ve örtük bilgileriyle ifade ederler. 3) Farklı paydaşların farklı, çoğu zaman çatışan gereksinimleri vardır. 4) Politik faktörler etkileyebilir. 5) Ekonomik ve iş ortamı dinamiktir, analiz sırasında gereksinimler değişir.

Q4. Describe the requirements elicitation and analysis process.

Çıkarma ve analiz süreci (s.147–148)

> *It is an iterative process shown as a spiral with four activities: 1) requirements discovery: collecting requirements from stakeholders, 2) classification and organisation: grouping related requirements into clusters, 3) prioritisation and negotiation: prioritising requirements and resolving conflicts, 4) requirements documentation: documenting requirements as input to the next round.*

Dört faaliyetli bir spiral olarak gösterilen yinelemeli bir süreçtir: 1) keşif: paydaşlardan gereksinim toplama, 2) sınıflandırma ve düzenleme: ilgili gereksinimleri gruplama, 3) önceliklendirme ve müzakere: öncelik belirleme ve çatışmaları çözme, 4) belgeleme: sonraki tura girdi olarak gereksinimleri yazma.

Q5. What are the three generic types of viewpoint? Give examples for a bank ATM.

3 viewpoint türü ve ATM örnekleri (s.150)

> *1) Interactor viewpoints: people or systems that interact directly with the system, for example bank customers and the account database. 2) Indirect viewpoints: stakeholders who do not use the system but influence the requirements, for example bank management and security staff. 3) Domain viewpoints: domain characteristics and constraints, for example interbank communication standards.*

1) Etkileşimci: sistemle doğrudan etkileşen insan ya da sistemler (banka müşterileri, hesap veritabanı). 2) Dolaylı: sistemi kullanmayan ama gereksinimleri etkileyen paydaşlar (banka yönetimi, güvenlik personeli). 3) Alan: alan özellikleri ve kısıtları (bankalar arası iletişim standartları).

Q6. What are the two types of interview? Why are interviews not good for domain requirements?

Görüşme türleri ve alan bilgisi sorunu (s.152)

> *Closed interviews use a predefined set of questions. Open interviews have no predefined agenda. In practice they are mixed. Interviews are not good for domain requirements because domain specialists use specific terminology that is easy to misunderstand, and some domain knowledge is so familiar to them that they do not mention it.*

Kapalı görüşmeler önceden belirlenmiş sorular kullanır, açık görüşmelerin gündemi yoktur. Pratikte karışık kullanılır. Görüşmeler alan gereksinimleri için iyi değildir, çünkü uzmanlar kolay yanlış anlaşılan özel terminoloji kullanır ve bazı alan bilgileri onlara o kadar tanıdıktır ki bahsetmezler.

Q7. What information may a scenario include?

Senaryo neleri içerir? (s.153)

> *1) What the system and users expect when the scenario starts. 2) The normal flow of events. 3) What can go wrong and how this is handled. 4) Other activities going on at the same time. 5) The system state when the scenario finishes.*

1) Senaryo başladığında sistem ve kullanıcıların beklentisi. 2) Normal olay akışı. 3) Ne ters gidebilir ve nasıl ele alınır. 4) Aynı anda olan diğer faaliyetler. 5) Senaryo bittiğinde sistemin durumu.

Q8. What is a use-case? How is it related to scenarios?

Use-case nedir, senaryoyla ilişkisi? (s.154–155)

> *A use-case is a scenario-based technique, now part of UML. It identifies the type of interaction and the actors involved. Actors are shown as stick figures and interactions as named ellipses. A use-case encapsulates a set of scenarios, and each scenario is a single thread through the use-case. Use-cases are good for interactor viewpoints but less effective for constraints, non-functional and domain requirements.*

Use-case, artık UML'nin parçası olan senaryo tabanlı bir tekniktir. Etkileşim türünü ve aktörleri belirler. Aktörler çöp adam, etkileşimler adlandırılmış elips olarak gösterilir. Bir use-case bir senaryolar kümesini kapsar, her senaryo use-case içinden tek bir yoldur. Interactor viewpoint'ler için iyidir, kısıtlar, NFR ve alan gereksinimleri için daha az etkilidir.

Q9. What is ethnography? What types of requirements can it discover?

Etnografi nedir, hangi gereksinimleri bulur? (s.157–158)

> *Ethnography is an observational technique: the analyst immerses himself in the working environment and observes the actual day-to-day work. It discovers 1) requirements derived from the way people actually work, not how process definitions say they should work, and 2) requirements derived from cooperation and awareness of other people's activities. It is not good for organisational or domain requirements, so it should be used with other techniques such as use-cases.*

Etnografi bir gözlem tekniğidir: analist kendini çalışma ortamına yerleştirir ve gerçek günlük işi gözler. Şunları bulur: 1) süreç tanımlarına göre değil insanların gerçekte nasıl çalıştığından türeyen gereksinimler, 2) işbirliğinden ve başkalarının faaliyetlerinin farkında olmaktan türeyen gereksinimler. Kurumsal ya da alan gereksinimleri için iyi değildir, use-case gibi tekniklerle birlikte kullanılmalıdır.

Q10. Why is requirements validation important? What checks are carried out?

Geçerleme neden önemli, hangi kontroller? (s.158–159)

> *Errors in the requirements document can lead to extensive rework costs if they are found during development or after the system is in service. Fixing a requirements error is much more expensive than fixing a design or coding error. Checks: validity, consistency, completeness, realism and verifiability.*

Gereksinim belgesindeki hatalar geliştirme sırasında ya da kullanımda bulunursa çok büyük yeniden çalışma maliyetine yol açar. Gereksinim hatasını düzeltmek tasarım ya da kod hatasından çok daha pahalıdır. Kontroller: geçerlilik, tutarlılık, eksiksizlik, gerçekçilik ve doğrulanabilirlik.

Q11. What techniques can be used for requirements validation?

Geçerleme teknikleri (s.159–160)

> *1) Requirements reviews: requirements are analysed systematically by a team of reviewers, checking also verifiability, comprehensibility, traceability and adaptability. 2) Prototyping: an executable model is shown to users so they can experiment with it. 3) Test-case generation: if a test is difficult to design, the requirement is probably difficult to implement.*

1) Gözden geçirmeler: ekip gereksinimleri sistematik analiz eder, doğrulanabilirlik, anlaşılabilirlik, izlenebilirlik ve uyarlanabilirliği de kontrol eder. 2) Prototipleme: çalıştırılabilir model kullanıcılara gösterilir. 3) Test durumu üretimi: test tasarlamak zorsa gereksinim de muhtemelen zor gerçekleştirilir.

Q12. Why do requirements change? What is the difference between enduring and volatile requirements?

Gereksinimler neden değişir? Enduring ve volatile farkı (s.161–162)

> *Requirements change because large systems have diverse users with conflicting priorities, the people who pay for a system are not the users, and the business and technical environment changes. Enduring requirements are stable requirements derived from the core activity of the organisation, for example patients and doctors in a hospital. Volatile requirements are likely to change, for example requirements from government healthcare policies.*

Gereksinimler değişir, çünkü büyük sistemlerin çatışan öncelikleri olan çeşitli kullanıcıları vardır, parayı ödeyenler kullanıcılar değildir ve iş ile teknik ortam değişir. Kalıcı gereksinimler kurumun temel faaliyetinden gelen kararlı gereksinimlerdir (hastanede hastalar ve doktorlar). Değişken gereksinimler değişmesi muhtemel olanlardır (devlet sağlık politikalarından gelenler).

Q13. What are the types of traceability information?

İzlenebilirlik türleri (s.163–164)

> *1) Source traceability links requirements to the stakeholders who proposed them and their rationale. 2) Requirements traceability links dependent requirements in the document. 3) Design traceability links requirements to the design modules that implement them. Traceability is often shown in a traceability matrix, where D means "depends on" and R means a weaker relationship.*

1) Kaynak izlenebilirliği gereksinimleri öneren paydaşlara ve gerekçelerine bağlar. 2) Gereksinim izlenebilirliği belgedeki bağımlı gereksinimleri bağlar. 3) Tasarım izlenebilirliği gereksinimleri onları gerçekleştiren tasarım modüllerine bağlar. Genelde bir izlenebilirlik matrisiyle gösterilir: D "bağımlı", R daha zayıf bir ilişki demektir.

Q14. Describe the requirements change management process.

Değişiklik yönetim süreci (s.165–166)

> *1) Problem analysis and change specification: the problem or change proposal is analysed to check that it is valid. 2) Change analysis and costing: the effect of the change is assessed using traceability information and the cost is estimated, then a decision is made. 3) Change implementation: the requirements document and, if needed, the design and implementation are changed. Using a formal process means all changes are treated consistently and in a controlled way.*

1) Sorun analizi ve değişiklik tanımı: sorun ya da öneri geçerli mi diye analiz edilir. 2) Değişiklik analizi ve maliyetlendirme: etkisi izlenebilirlik bilgisiyle değerlendirilir, maliyet tahmin edilir, karar verilir. 3) Değişiklik uygulama: gereksinim belgesi ve gerekirse tasarım ve kod değiştirilir. Resmi süreç, bütün değişikliklerin tutarlı ve kontrollü ele alınmasını sağlar.

D Mini test

cevaplar en altta

1. The requirements engineering process should start with a:
   a) requirements review
   b) feasibility study
   c) traceability matrix
   d) prototype
2. A feasibility study should normally take about:
   a) two or three days
   b) two or three weeks
   c) two or three months
   d) one year
3. Any person or group affected by the system directly or indirectly is a:
   a) stakeholder
   b) actor
   c) viewpoint
   d) contractor
4. Which activity resolves conflicts between requirements?
   a) Requirements discovery
   b) Classification and organisation
   c) Prioritisation and negotiation
   d) Documentation
5. Bank management and security staff in an ATM system are ___ viewpoints.
   a) interactor
   b) indirect
   c) domain
   d) engineering
6. Interbank communication standards are an example of a ___ viewpoint.
   a) interactor
   b) indirect
   c) domain
   d) marketing
7. An interview with a predefined set of questions is a(n):
   a) open interview
   b) closed interview
   c) ethnography
   d) review
8. Interviews are NOT very good for discovering:
   a) what stakeholders do
   b) problems with current systems
   c) domain requirements
   d) general understanding

9. In use-case notation, actors are shown as:
   a) rectangles
   b) stick figures
   c) ellipses
   d) arrows

0. According to Fowler, a use-case:
   a) is a single scenario
   b) encapsulates a set of scenarios
   c) is a test case
   d) is a data model

1. An observational technique where the analyst works in the user environment is:
   a) interviewing
   b) ethnography
   c) prototyping
   d) traceability

2. Ethnography is NOT appropriate for discovering:
   a) actual work practices
   b) cooperation requirements
   c) organisational or domain requirements
   d) implicit requirements

3. "Can the requirements actually be implemented with current technology?" is a:
   a) validity check
   b) realism check
   c) consistency check
   d) completeness check

4. "Is the origin of the requirement clearly stated?" checks:
   a) adaptability
   b) comprehensibility
   c) traceability
   d) verifiability

5. Stable requirements derived from the core activity of the organisation are:
   a) volatile
   b) enduring
   c) mutable
   d) consequential

6. Requirements that emerge because the computer system is introduced are:
   a) mutable
   b) emergent
   c) consequential
   d) compatibility

7. Linking requirements to the design modules that implement them is:
   a) source traceability
   b) requirements traceability
   c) design traceability
   d) change costing

8. In a traceability matrix, "D" means:
   a) deleted
   b) the row requirement depends on the column requirement
   c) design
   d) domain

Cevap anahtarı: 1-b · 2-b · 3-a · 4-c · 5-b · 6-c · 7-b · 8-c · 9-b · 10-b · 11-b · 12-c · 13-b · 14-c · 15-b · 16-c · 17-c · 18-b

YAZILIM MÜHENDİSLİĞİ · VİZE ÇALIŞMA NOTLARI

Bölüm 11
Architectural design

Mimari Tasarım

Kitap s.241–265 (Bölüm 11 tamamı)

Kaynak: Ian Sommerville, *Software Engineering*, 8. baskı
İçerik: Giriş · Sayfa sayfa açıklama, hoca anlatımı ve deftere not · Bölüm sonu: Terim sözlüğü, ⭐ Mutlaka bil, Sınav soruları, Mini test

G Giriş: Bu bölüm ne anlatıyor?

Bölüm 11, kitabın **Part 3: Design** kısmının ilk bölümü. Bu kısmın genel fikri şu: yazılım tasarımının özü, yazılımın **mantıksal organizasyonu** hakkında karar vermektir. Yazara göre tasarım **yaratıcı** bir süreçtir, herkese uyan bir "tarif" yoktur. Tasarım, mevcut tasarım örneklerine bakarak ve tasarımı başkalarıyla tartışarak öğrenilir. Bölüm 11–13 yazılımın soyut yapılarını anlatır, Bölüm 11 bunların ilkidir.

Bölüm 11 **yazılım mimarisi** ve **mimari tasarım** kavramlarını tanıtıyor. Beş kısımdan oluşuyor:

- **11.1 Architectural design decisions:** Mimari tasarımda verilen temel kararlar.
- **11.2 System organisation:** Repository, client-server ve layered (katmanlı) modeller.
- **11.3 Modular decomposition styles:** Nesne yönelimli ayrıştırma ve fonksiyon odaklı boru hattı (pipeline).
- **11.4 Control styles:** Merkezi kontrol (call-return, manager) ve olay güdümlü kontrol (broadcast, interrupt).
- **11.5 Reference architectures:** OSI modeli ve CASE referans mimarisi (ECMA).

Bölümün amaçları: (1) yazılımın mimari tasarımının neden önemli olduğunu anlamak, (2) mimari tasarım sürecinde sistem mimarisi hakkında verilmesi gereken kararları anlamak, (3) genel sistem organizasyonu, modüler ayrıştırma ve kontrolü kapsayan birbirini tamamlayan **üç mimari stil** grubunu tanımak, (4) referans mimarilerin mimari kavramları iletmek ve sistem mimarilerini değerlendirmek için nasıl kullanıldığını anlamak.

Sayfa 242–244 — Mimari tasarım nedir? · Avantajları · NFR'lere göre mimari · Blok diyagramları (Şekil 11.1)

📖 Sayfanın özeti

TANIM

Büyük sistemler her zaman ilişkili hizmetler sunan **alt sistemlere** ayrılır. Bu alt sistemleri belirleme ve alt sistemlerin **kontrolü ve iletişimi** için bir çerçeve kurma sürecine **mimari tasarım (architectural design)** denir. Bu sürecin çıktısı **yazılım mimarisinin** tanımıdır. Bölüm 4'teki modelde mimari tasarım, tasarım sürecinin **ilk aşamasıdır** ve tasarım ile gereksinim mühendisliği arasında **kritik bir bağdır**. Sistemin ana bileşenlerini ve aralarındaki iletişimi belirleyen temel yapısal çerçeveyi kurar.

MİMARİYİ AÇIKÇA TASARLAMANIN 3 AVANTAJI (BASS)

1) Stakeholder communication: mimari, sistemin farklı paydaşlar arasında tartışmaya odak olabilecek **üst seviye** bir sunumudur. **2) System analysis:** mimariyi erken aşamada açık hale getirmek analiz gerektirir. Mimari kararlar sistemin **performans, güvenilirlik ve bakım kolaylığı** gibi kritik gereksinimleri karşılayıp karşılamayacağını derinden etkiler. **3) Large-scale reuse:** mimari modeli, sistemin nasıl düzenlendiğinin ve bileşenlerin nasıl birlikte çalıştığının kısa, yönetilebilir bir tanımıdır. Benzer gereksinimli sistemlerin mimarisi çoğu zaman aynıdır, bu yüzden **büyük ölçekli yeniden kullanımı** destekleyebilir (aynı mimarinin ilişkili sistemlerde kullanıldığı ürün hattı mimarileri, Bölüm 18).

HOFMEİSTER'E GÖRE

Mimari tasarım aşaması tasarımcıları önemli tasarım konularını sürecin **erken** aşamasında düşünmeye zorlar. Yazılım mimarisi, sistem gereksinimlerini **müzakere etmek** ve müşteriler, geliştiriciler ve

yöneticilerle tartışmaları yapılandırmak için bir **tasarım planı** olarak kullanılabilir. Ayrıca **karmaşıklık yönetimi** için temel bir araçtır: ayrıntıları gizler ve tasarımcıların ana sistem soyutlamalarına odaklanmasını sağlar.

NFR'LER MİMARİYİ BELİRLER (BOSCH)

Sistem mimarisi performansı, sağlamlığı, dağıtılabilirliği ve bakım kolaylığını etkiler. Seçilen stil ve yapı bu yüzden **fonksiyonel olmayan gereksinimlere** bağlı olabilir: **1) Performance:** kritik işlemler **az sayıda alt sistemde** toplanmalı, alt sistemler arası iletişim mümkün olduğunca az olmalı. Bu, iletişimi azaltmak için ince taneli (fine-grain) değil **büyük taneli (large-grain)** bileşenler kullanmak demek olabilir. **2) Security: katmanlı** bir yapı kullanılmalı, en kritik varlıklar **en içteki** katmanlarda korunmalı ve bu katmanlara yüksek seviyede güvenlik doğrulaması uygulanmalı. **3) Safety:** emniyetle ilgili işlemlerin hepsi **tek bir alt sistemde** ya da az sayıda alt sistemde toplanmalı. Bu, emniyet doğrulamasının maliyetini ve sorunlarını azaltır ve ilgili koruma sistemleri sağlamayı mümkün kılar. **4) Availability: yedekli (redundant)** bileşenler içermeli ki sistem durdurulmadan bileşenler değiştirilip güncellenebilsin (hata toleranslı mimariler Bölüm 20). **5) Maintainability:** kolayca değiştirilebilen **ince taneli (fine-grain), kendi kendine yeten** bileşenler kullanılmalı. Veri üreticileri tüketicilerden ayrılmalı ve **paylaşılan veri yapılarından kaçınılmalı**.

ÇATIŞMA

Bu mimariler arasında çatışma olabilir: büyük taneli bileşenler **performansı**, ince taneli bileşenler **bakım kolaylığını** artırır. İkisi de önemliyse bir uzlaşma bulunmalıdır. Bu bazen sistemin **farklı kısımları için farklı mimari stiller** kullanarak sağlanabilir.

GEREKSİNİMLERLE ÖRTÜŞME

Gereksinim mühendisliği ile mimari tasarım arasında önemli bir örtüşme vardır. İdealde sistem tanımlaması tasarım bilgisi içermemeli, ama çok küçük sistemler dışında bu gerçekçi değildir. Tanımlamayı yapılandırmak ve düzenlemek için mimari ayrıştırma gerekir (ör. Bölüm 2'deki hava trafik kontrol mimarisi, alt sistem tanımlamasının başlangıç noktası olarak kullanılabilir).

BLOK DİYAGRAMLARI VE PAKETLEME ROBOTU (ŞEKİL 11.1)

Alt sistem tasarımı, sistemin her biri kendi başına büyük bir sistem olabilecek **büyük taneli bileşenlere** soyut olarak ayrılmasıdır. Genelde **blok diyagramlarıyla** anlatılır: her kutu bir alt sistemdir, kutu içindeki kutular alt sistemin kendisinin de alt sistemlere ayrıldığını gösterir, oklar veri ya da kontrol sinyallerinin ok yönünde geçtiğini gösterir. Blok diyagramları farklı disiplinlerden insanların kolayca anlayabileceği üst seviye bir resim sunar. Şekil 11.1 bir **paketleme robotu** sisteminin soyut mimarisidir: farklı türde nesneleri paketleyebilir. Bir **görüntü (vision)** alt sistemiyle konveyördeki nesneleri seçer, nesnenin türünü belirler (object identification) ve doğru paketleme türünü seçer (packaging selection). Sonra nesneleri teslim konveyöründen paketlenmek üzere alır ve paketlenmiş nesneleri başka bir konveyöre koyar. Alt sistemler: vision system, object identification system, arm controller, gripper controller, packaging selection system, packing system, conveyor controller.



Şekil 11.1 (sadeleştirilmiş): Paketleme robotu kontrol sisteminin alt sistemleri

KUTU-ÇİZGİ DİYAGRAMLARININ DEĞERİ

Bass ve arkadaşları basit kutu-çizgi diyagramlarının faydalı mimari gösterimler olmadığını savunur, çünkü bileşenler arası ilişkilerin doğasını ya da bileşenlerin dışarıdan görünen özelliklerini göstermezler. Yazara göre yazılım tasarımcısı açısından bu **tamamen doğrudur**. Ama bu tür model

paydaşlarla **iletişim** ve **proje planlaması** için etkilidir, çünkü ayrıntıyla dolu değildir. Paydaşlar sistemin soyut görünümünü anlayabilir. Model bağımsız geliştirilecek ana alt sistemleri gösterir, böylece yöneticiler bu sistemlerin geliştirilmesine insan atamaya başlayabilir. Kutu-çizgi diyagramları tek mimari gösterim olmamalı, ama birçok faydalı modelden biridir.

ALT SİSTEMLERE AYIRMA ZORDUR

Sistemi alt sistemlere nasıl ayıracağına karar vermek zor bir problemdir. Sistem gereksinimleri ana faktördür: gereksinimler ile alt sistemler arasında **yakın bir eşleşme** olan bir tasarım oluşturmaya çalışılmalıdır. Böylece gereksinimler değişirse değişiklik birkaç alt sisteme dağılmak yerine büyük ihtimalle **tek bir yerde (localised)** kalır. Bölüm 13'te alt sistem belirlemede başlangıç noktası olarak kullanılabilecek genel uygulama mimarileri anlatılıyor.

👩‍🏫 Hoca anlatımı

Mimari tasarım = sistemin **iskeletini** çizmek. Bir bina projesinde önce "kaç kat, hangi oda nerede, merdiven nerede, tesisat nereden geçecek" kararlarını verirsin, tuğlanın rengine sonra bakarsın. Yazılımda da önce "hangi büyük parçalar (alt sistemler) var ve birbirleriyle nasıl konuşacaklar" belirlenir.

3 avantaj (Bass): (1) **iletişim**, herkes aynı resme bakıp konuşur, (2) **analiz**, erken aşamada "bu mimari hızlı olur mu, güvenilir olur mu?" diye düşünürsün, (3) **yeniden kullanım**, benzer sistemler aynı mimariyi kullanabilir.

NFR'lere göre mimari bu bölümün en çok sorulan kısmı. Kısa hatırlatıcı:

Performance → **az ve büyük** parçalar (az iletişim, az bekleme). **Security** → **katmanlar**, değerli şeyler **en içte** (soğan gibi, ya da bankanın kasası gibi: dış kapı, iç kapı, kasa). **Safety** → emniyetle ilgili her şey **tek yerde** (kontrol etmesi kolay). **Availability** → **yedek parçalar** (biri bozulunca diğeri devreye girer, sistem durmaz). **Maintainability** → **küçük ve bağımsız** parçalar (LEGO gibi, kolayca değiştirilir), paylaşılan veri yok.

Çatışma: performans "büyük parça" der, bakım "küçük parça" der. Çözüm: sistemin farklı yerlerinde farklı stil kullan. Alıştırma 11.2 de böyle bir çatışmayı soruyor (availability ve security).

Blok diyagramı tartışması: Bass "kutu ve oklar yetersiz, ilişkinin doğasını göstermiyor" der. Yazar "doğru, ama yöneticiler ve müşterilerle konuşmak ve proje planlamak için çok iyi" der.

📝 Deftere not

- ⭐⭐ **Architectural design:** alt sistemleri belirleme ve alt sistemlerin **kontrolü ve iletişimi** için çerçeve kurma. Çıktı: **yazılım mimarisinin tanımı**.
- ⭐ Tasarım sürecinin **ilk aşaması**, tasarım ile RE arasında **kritik bağ**. Ana bileşenler + aralarındaki iletişim.
- ⭐⭐ **Mimariyi açıkça tasarlamanın 3 avantajı (Bass):**
  - Stakeholder communication: paydaşlar arası tartışmaya odak olan **üst seviye** sunum.
  - System analysis: erken analiz. Mimari kararlar **performans, güvenilirlik, bakım** gereksinimlerinin karşılanmasını etkiler.
  - Large-scale reuse: benzer sistemler aynı mimariyi kullanır (**product-line** mimarileri).

- **Hofmeister:** önemli tasarım konuları **erken** düşünülür. Mimari = gereksinimleri **müzakere** etmek için tasarım planı + **karmaşıklık yönetimi** aracı (ayrıntıyı gizler).
- ⭐⭐ **NFR'lere göre mimari (Bosch):**
  - **Performance:** kritik işlemler **az sayıda alt sistemde**, az iletişim, **large-grain** bileşenler.
  - **Security: katmanlı** yapı, kritik varlıklar **en iç** katmanda, yüksek güvenlik doğrulaması.
  - **Safety:** emniyet işlemleri **tek bir** ya da az sayıda alt sistemde (doğrulama ucuzlar, koruma sistemi eklenebilir).
  - **Availability: redundant** bileşenler, sistem durmadan değiştirme ve güncelleme.
  - **Maintainability: fine-grain, self-contained** bileşenler. Üretici ve tüketici ayrı, **paylaşılan veri yapısı yok**.
- ⭐ **Çatışma:** large-grain → performans, fine-grain → bakım. Çözüm: sistemin **farklı kısımlarında farklı stiller**.
- RE ve mimari tasarım **örtüşür**: spec'i yapılandırmak için mimari ayrıştırma gerekir (ör. ATC mimarisi).
- ⭐ **Alt sistem tasarımı: large-grain** bileşenlere soyut ayrıştırma. **Blok diyagramı:** kutu = alt sistem, kutu içinde kutu = alt sistemin alt sistemleri, ok = veri ya da kontrol sinyali yönü.
- **Şekil 11.1, paketleme robotu:** vision system, object identification, arm controller, gripper controller, packaging selection, packing system, conveyor controller.
- ⭐ **Bass:** kutu-çizgi diyagramları ilişkinin doğasını ve bileşenlerin dışarıdan görünen özelliklerini göstermez. **Yazar:** doğru, ama **paydaş iletişimi** ve **proje planlaması** için etkili (ayrıntısız, yöneticiler insan atayabilir). Tek gösterim olmamalı.
- ⭐ Alt sistemlere ayırma **zor**. Gereksinimler ile alt sistemler **yakın eşleşmeli**, böylece gereksinim değişikliği **tek yerde** (localised) kalır.

Sayfa 245–247 11.1 Architectural design decisions (Mimari tasarım kararları)

📖 Sayfanın özeti

KARAR BAKIŞ AÇISI

Mimari tasarım, fonksiyonel ve fonksiyonel olmayan gereksinimleri karşılayacak bir sistem organizasyonu kurmaya çalışılan **yaratıcı** bir süreçtir. Yaratıcı olduğu için süreçteki faaliyetler sistemin türüne, mimarın geçmişine ve deneyimine ve sistemin özel gereksinimlerine göre çok farklıdır. Bu yüzden süreci faaliyetler yerine **kararlar** açısından düşünmek daha faydalıdır. Mimarlar bilgi ve deneyimlerine dayanarak sistemi ve geliştirme sürecini derinden etkileyen şu **8 temel soruyu** cevaplamalıdır: (1) Tasarlanan sistem için şablon olabilecek **genel bir uygulama mimarisi** var mı? (2) Sistem birden çok işlemciye nasıl **dağıtılacak**? (3) Sistem için hangi **mimari stil(ler)** uygun? (4) Sistemi yapılandırmak için temel **yaklaşım** ne olacak? (5) Sistemdeki yapısal birimler **modüllere** nasıl ayrıştırılacak? (6) Birimlerin çalışmasını **kontrol** etmek için hangi strateji kullanılacak? (7) Mimari tasarım nasıl **değerlendirilecek**? (8) Sistemin mimarisi nasıl **belgelenecek**?

GENEL MİMARİLER VE DAĞITIM

Her yazılım sistemi benzersiz olsa da aynı uygulama alanındaki sistemler, alanın temel kavramlarını yansıtan **benzer mimarilere** sahiptir. Bunlar bilgi yönetim sistemleri mimarisi gibi oldukça genel ya da çok daha özel olabilir. Örneğin **uygulama ürün hatları**, belirli müşteri gereksinimlerini karşılayan

varyantlarıyla bir **çekirdek mimari** etrafında kurulan uygulamalardır. Mimariyi tasarlarken sistemin ve daha geniş uygulama sınıflarının ortak noktalarına ve bu mimarilerden ne kadar bilginin yeniden kullanılabileceğine karar verilir (Bölüm 13 ve 18). Gömülü sistemlerde ve kişisel bilgisayarlar için tasarlanan sistemlerde genelde tek işlemci vardır ve dağıtık mimari tasarlamak gerekmez. Ama büyük sistemlerin çoğu artık yazılımın birçok bilgisayara dağıtıldığı **dağıtık sistemlerdir**. Dağıtım mimarisinin seçimi sistemin **performansını ve güvenilirliğini** etkileyen temel bir karardır (Bölüm 12).

MİMARİ STİLLER

Bir yazılım sisteminin mimarisi belirli bir **mimari modele ya da stile** dayanabilir. Mimari stil, **client-server** ya da **katmanlı** mimari gibi bir **sistem organizasyonu kalıbıdır** (Garlan ve Shaw). Bu stillerin, uygulamalarının ve güçlü ve zayıf yanlarının farkında olmak önemlidir. Ama büyük sistemlerin çoğu tek bir stile uymaz: sistemin farklı kısımları farklı stillerle tasarlanabilir ve bazen genel mimari, farklı stillerin birleştirildiği **bileşik (composite)** bir mimaridir. Garlan ve Shaw'un stil kavramı sonraki üç tasarım sorusunu kapsar: gereksinimleri karşılayacak en uygun **yapı** (client-server ya da katmanlı gibi) seçilir, yapısal birimleri **modüllere ayırma** stratejisine karar verilir, son olarak **kontrol modellemede** alt sistemlerin çalışmasının nasıl kontrol edileceğine karar verilir. Bu üç konu Bölüm 11.2–11.4'te.

DEĞERLENDİRME VE BELGELEME

Mimari tasarımı değerlendirmek **zordur**, çünkü bir mimarinin gerçek testi, sistem kurulduktan sonra fonksiyonel ve fonksiyonel olmayan gereksinimleri ne kadar iyi karşıladığıdır. Ama bazı durumlarda tasarım **referans ya da genel mimari modellerle** karşılaştırılarak bir miktar değerlendirme yapılabilir (Bölüm 11.5 ve 13). Mimari tasarım sürecinin ürünü bir **mimari tasarım belgesidir**. Sistemin grafiksel gösterimlerini ve açıklayıcı metinleri içerebilir. Sistemin alt sistemlere nasıl ayrıldığını, benimsenen yaklaşımı ve her alt sistemin modüllere nasıl ayrıldığını anlatmalıdır.

5 MİMARİ MODEL TÜRÜ

Grafiksel modeller mimariye farklı bakış açıları sunar: **1) Static structural model:** ayrı birimler olarak geliştirilecek alt sistemleri ya da bileşenleri gösterir. **2) Dynamic process model:** sistemin çalışma zamanında **süreçlere** nasıl düzenlendiğini gösterir (statik modelden farklı olabilir). **3) Interface model:** her alt sistemin **genel arayüzü** üzerinden sunduğu hizmetleri tanımlar. **4) Relationship models:** alt sistemler arası **veri akışı** gibi ilişkileri gösterir. **5) Distribution model:** alt sistemlerin bilgisayarlara nasıl **dağıtılabileceğini** gösterir.

SAYFA 247: ADL'LER

Bazı araştırmacılar sistem mimarilerini tanımlamak için **mimari tanımlama dilleri (ADL, Architectural Description Languages)** kullanılmasını önermiştir. ADL'lerin temel öğeleri **bileşenler (components)** ve **bağlayıcılardır (connectors)**, ayrıca iyi biçimli mimariler için kurallar ve yönergeler içerirler. Ama bütün özel diller gibi ADL'ler de sadece **dil uzmanlarının** anlayabildiği, alan ve uygulama uzmanlarının erişemediği dillerdir. Bu yüzden pratik açıdan analizleri zordur. Yazara göre az sayıda uygulamada kullanılacaklar. **UML** gibi gayriresmi modeller ve gösterimler mimari tanım için **en yaygın** kullanılan gösterim olmaya devam edecek.

👨‍🏫 Hoca anlatımı

Mimari tasarım **yaratıcı** bir iş, bu yüzden "şu adımları izle" demek yerine "şu soruları cevapla" demek daha mantıklı. **8 soru**yu kendi cümlelerinle hatırla: **(1)** Hazır bir şablon var mı? **(2)** Kaç bilgisayara dağıtılacak? **(3)** Hangi stil? **(4)** Temel yapı yaklaşımı ne? **(5)** Modüllere nasıl bölünecek? **(6)** Kontrol nasıl olacak? **(7)** Nasıl değerlendirilecek? **(8)** Nasıl belgelenecek?

Mimari stil = bir "kalıp", hazır bir yapı tarzı. Mimarlıkta "Gotik", "modern" gibi stiller vardır. Yazılımda da "client-server", "katmanlı", "repository" gibi stiller vardır. Bu bölümün geri kalanı bu stilleri anlatıyor. Ama gerçek sistemler genelde **karışık** stil kullanır.

Bölümün yapısı burada belli oluyor, iyi kavra: **3 karar grubu = 3 stil grubu**: (1) **System organisation** (11.2): genel yapı (repository, client-server, layered). (2) **Modular decomposition** (11.3): alt sistemleri modüllere bölme (OO, pipeline). (3) **Control** (11.4): kim kimi çalıştırıyor (centralised, event-based).

5 mimari model: **static** (parçalar neler), **dynamic** (çalışırken süreçler), **interface** (hizmetler), **relationship** (veri akışı), **distribution** (hangi bilgisayarda).

ADL konusunda yazarın görüşü: çok teknik, sadece uzmanlar anlar, pek kullanılmayacak. **UML** en yaygın kalacak.

📝 Deftere not

- ⭐ Mimari tasarım = **yaratıcı** süreç, faaliyetler sistem türüne, mimarın deneyimine ve gereksinimlere göre değişir. Bu yüzden **kararlar** açısından düşünülür.
- ⭐⭐ **8 temel karar sorusu:**
  - 1. Şablon olabilecek **genel uygulama mimarisi** var mı?
  - 2. Sistem işlemcilere nasıl **dağıtılacak**?
  - 3. Hangi **mimari stil(ler)**?
  - 4. Sistemi yapılandırmanın temel **yaklaşımı**?
  - 5. Yapısal birimler **modüllere** nasıl ayrışacak?
  - 6. Birimlerin çalışmasını **kontrol** stratejisi?
  - 7. Mimari nasıl **değerlendirilecek**?
  - 8. Mimari nasıl **belgelenecek**?
- Aynı alandaki sistemler **benzer mimariler** taşır. **Product lines** = çekirdek mimari + müşteriye özel varyantlar (Bölüm 13, 18).
- Gömülü ve PC sistemleri: genelde tek işlemci. Büyük sistemler: **dağıtık**. Dağıtım mimarisi **performans ve güvenilirliği** etkiler (Bölüm 12).
- ⭐⭐ Architectural style (Garlan & Shaw): **sistem organizasyonu kalıbı** (ör. client-server, layered). Büyük sistemler genelde tek stile uymaz, **composite** mimari olabilir.
- ⭐ Stil kavramı **3 kararı** kapsar: **yapı** (11.2) → **modüllere ayrıştırma** (11.3) → **kontrol modeli** (11.4).
- ⭐ Değerlendirme **zor**: gerçek test, kurulumdan sonra gereksinimleri karşılama. Kısmen **referans ve genel mimarilerle karşılaştırma**.
- Ürün: **mimari tasarım belgesi** (grafikler + metin): alt sistemlere ayrılma, yaklaşım, modüllere ayrılma.
- ⭐⭐ **5 mimari model:**
  - **Static structural model:** ayrı geliştirilecek alt sistemler ve bileşenler.
  - **Dynamic process model:** çalışma zamanında **süreçler**.
  - **Interface model:** her alt sistemin genel arayüzündeki **hizmetler**.
  - **Relationship models:** alt sistemler arası ilişkiler (**veri akışı**).

- **Distribution model:** alt sistemlerin **bilgisayarlara dağılımı**.
- ⭐ **ADL (Architectural Description Languages):** temel öğeler **components + connectors** + kurallar. Sadece **dil uzmanları** anlar, pratikte analizi zor, az kullanılacak. **UML** gibi gayriresmi gösterimler **en yaygın**.

Sayfa 247–252 — 11.2 System organisation · Repository (Şekil 11.2) · Client-server (Şekil 11.3) · Layered (Şekil 11.4)

📖 Sayfanın özeti

SİSTEM ORGANİZASYONU

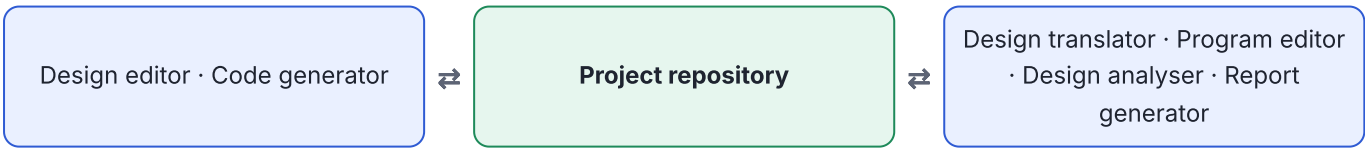
Sistem organizasyonu, sistemi yapılandırmak için kullanılan temel stratejiyi yansıtır ve mimari tasarım sürecinin **başında** kararlaştırılmalıdır. Alt sistem yapısına doğrudan yansıyabilir, ama alt sistem modeli çoğu zaman organizasyon modelinden daha fazla ayrıntı içerir. Bu bölümde çok yaygın **üç organizasyon stili** anlatılıyor: **paylaşılan veri deposu (repository)**, **paylaşılan hizmetler ve sunucular (client-server)** ve sistemin fonksiyonel katmanlar halinde düzenlendiği **soyut makine ya da katmanlı** stil. Bu stiller ayrı ayrı ya da **birlikte** kullanılabilir (ör. paylaşılan bir veri deposu etrafında düzenlenen bir sistem, verinin daha soyut bir görünümünü sunmak için üstüne katmanlar kurabilir).

11.2.1 REPOSİTORY MODEL

Alt sistemler etkili çalışmak için bilgi alışverişi yapmalıdır. Bunun **iki yolu** vardır: (1) bütün paylaşılan veri, bütün alt sistemlerin erişebildiği **merkezi bir veritabanında** tutulur. Paylaşılan veritabanına dayalı bir sisteme **repository model** denir. (2) Her alt sistem **kendi veritabanını** tutar, veri diğer alt sistemlerle **mesaj** göndererek değiş tokuş edilir. Büyük miktarda veri kullanan sistemlerin çoğu paylaşılan bir veritabanı etrafında düzenlenir. Bu model, verinin bir alt sistemde üretilip başka bir alt sistemde kullanıldığı uygulamalara uygundur. Örnekler: komuta-kontrol sistemleri, yönetim bilgi sistemleri, **CAD** sistemleri ve **CASE araç takımları**.

ŞEKİL 11.2: CASE ARAÇ TAKIMI

Paylaşılan bir **proje deposu (project repository)** etrafında: design editor, code generator, design translator, program editor, design analyser ve report generator. CASE araçları için ilk paylaşılan depo muhtemelen 1970'lerin başında İngiliz **ICL** şirketi tarafından işletim sistemi geliştirmeyi desteklemek için yapıldı. Model, **Buxton**'ın Ada ile yazılan sistemlerin geliştirilmesini destekleyen **Stoneman** ortamı önerisiyle daha yaygın tanındı. O zamandan beri birçok CASE araç takımı paylaşılan depo etrafında geliştirildi.



Şekil 11.2: Repository tabanlı CASE araç takımı mimarisi

REPOSİTORY: AVANTAJ VE DEZAVANTAJLAR (4 ÇİFT)

1) + Büyük miktarda veriyi paylaşmanın **verimli** yoludur, veriyi alt sistemler arasında açıkça aktarmaya gerek yoktur. **–** Alt sistemler deponun **veri modeli üzerinde anlaşmak** zorundadır. Bu kaçınılmaz olarak her aracın özel ihtiyaçları arasında bir uzlaşmadır ve performansı olumsuz etkileyebilir. Veri modeli anlaşılan şemaya uymayan yeni alt sistemleri entegre etmek zor ya da imkânsız olabilir. **2) +** Veri üreten alt sistemler o verinin diğer alt sistemlerce nasıl kullanıldığıyla **ilgilenmek zorunda değildir**. **–** Anlaşılmış bir veri modeline göre büyük miktarda bilgi üretildiği için

evrim zor olabilir. Bunu yeni bir modele çevirmek kesinlikle pahalı, zor ya da imkânsız olabilir. **3) +** Yedekleme, güvenlik, erişim kontrolü ve hatadan kurtarma gibi faaliyetler **merkezileşir**. Bunlar depo yöneticisinin sorumluluğudur, araçlar asıl işlevlerine odaklanabilir. **–** Farklı alt sistemlerin güvenlik, kurtarma ve yedekleme için **farklı ihtiyaçları** olabilir, ama repository modeli bütün alt sistemlere **aynı politikayı** dayatır. **4) +** Paylaşım modeli depo şeması üzerinden görünürdür. Anlaşılmış veri modeline uyumlu oldukları sürece **yeni araçları entegre etmek** kolaydır. **–** Depoyu birden çok makineye **dağıtmak zor** olabilir. Mantıksal olarak merkezi bir depoyu dağıtmak mümkün olsa da veri tekrarı ve tutarsızlık sorunları olabilir.

BLACKBOARD MODELİ

Yukarıdaki modelde depo **pasiftir**, kontrol depoyu kullanan alt sistemlerin sorumluluğundadır. Alternatif olarak yapay zekâ sistemlerinden türeyen **"blackboard" (kara tahta)** modeli vardır: belirli veriler hazır olunca alt sistemleri **tetikler**. Depodaki verinin biçimi daha az yapılandırılmışsa uygundur, çünkü hangi aracın çalıştırılacağına ancak veri analiz edildikten sonra karar verilebilir (Nii, Bosch).

11.2.2 CLİENT-SERVER MODEL

Sistemin bir **hizmetler** kümesi, ilgili **sunucular** ve bu hizmetlere erişip kullanan **istemciler** olarak düzenlendiği modeldir. **3 ana bileşen:** (1) diğer alt sistemlere hizmet sunan **sunucular** kümesi. Ör: yazdırma hizmeti sunan print server, dosya yönetim hizmeti sunan file server, derleme hizmeti sunan compile server. (2) Sunucuların hizmetlerini çağıran **istemciler** kümesi. Bunlar genelde kendi başına alt sistemlerdir ve bir istemci programın birçok örneği eşzamanlı çalışabilir. (3) İstemcilerin bu hizmetlere erişmesini sağlayan bir **ağ**. Bu kesin şart değildir (istemci ve sunucu tek makinede çalışabilir), ama pratikte çoğu client-server sistem dağıtık olarak gerçekleştirilir. İstemcilerin sunucuların ve sundukları hizmetlerin adlarını bilmesi gerekebilir, ama sunucuların istemcilerin kimliğini ya da sayısını bilmesi gerekmez. İstemciler hizmetlere **uzak prosedür çağrılarıyla** ya da WWW'deki **http** gibi bir **istek-cevap (request-reply)** protokolüyle erişir: istemci sunucuya istek gönderir ve cevap gelene kadar bekler.

ŞEKİL 11.3: FİLM VE FOTOĞRAF KÜTÜPHANESİ

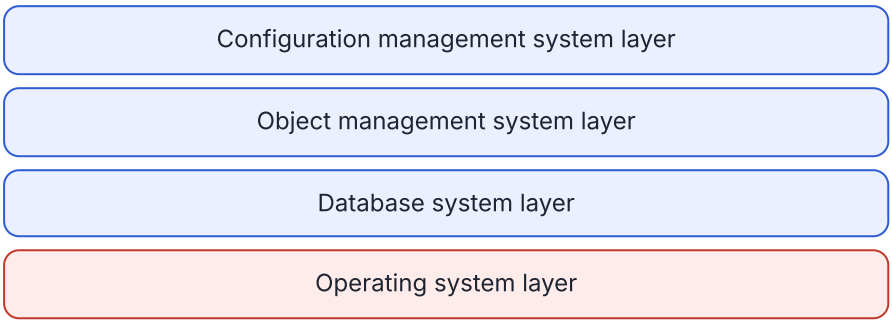
Çok kullanıcılı, web tabanlı bir film ve fotoğraf kütüphanesi sistemi. İstemciler (client 1–4) internet üzerinden şu sunuculara erişir: **katalog sunucusu** (kütüphane kataloğu), **video sunucusu** (film klipleri), **resim sunucusu** (dijital fotoğraflar) ve **web sunucusu** (film ve fotoğraf bilgisi). Farklı medya türlerini farklı sunucular yönetir. Video kareleri hızlı ve senkron iletilmeli ama nispeten düşük çözünürlükte olabilir, sıkıştırılmış saklanabilir, bu yüzden video sunucusu farklı formatlara sıkıştırma ve açma işini yapabilir. Fotoğraflar ise **yüksek çözünürlükte** tutulmalıdır, bu yüzden ayrı bir sunucuda olmaları uygundur. Katalog çeşitli sorguları yanıtlamalı ve film ve video bilgisi içeren web bilgi sistemine ve film ve video klip satışını destekleyen bir e-ticaret sistemine bağlantılar vermelidir. İstemci programı bu hizmetlere web tarayıcısıyla kurulmuş entegre bir kullanıcı arayüzüdür.

CLİENT-SERVER: AVANTAJ VE DEZAVANTAJ

En önemli avantajı dağıtık bir mimari olmasıdır. Birçok dağıtık işlemcili ağ bağlantılı sistemler etkili kullanılabilir. Yeni bir sunucu eklemek ve sistemin geri kalanıyla entegre etmek ya da sunucuları sistemin diğer kısımlarını etkilemeden **şeffaf şekilde yükseltmek** kolaydır (Bölüm 12). **Dezavantajı:** yeni bir sunucunun bütün faydasını almak için mevcut istemci ve sunucularda değişiklik gerekebilir. Sunucular arasında **paylaşılan bir veri modeli olmayabilir** ve alt sistemler verilerini farklı şekillerde düzenleyebilir. Bu, performansı optimize etmek için her sunucuda özel veri modelleri kurulabileceği anlamına gelir. XML tabanlı veri gösterimi kullanılırsa bir şemadan diğerine dönüştürmek nispeten kolay olabilir, ama XML veriyi göstermenin **verimsiz** bir yoludur ve performans sorunları çıkabilir.

11.2.3 LAYERED MODEL

Katmanlı model (bazen **abstract machine model** denir) sistemi her biri bir hizmet kümesi sunan **katmanlar** halinde düzenler. Her katman, makine dili o katmanın sunduğu hizmetlerle tanımlanan bir **soyut makine** olarak düşünülebilir. Bu "dil" bir sonraki soyut makine seviyesini gerçekleştirmek için kullanılır. Ör: bir dili gerçekleştirmenin yaygın yolu ideal bir "dil makinesi" tanımlayıp dili bu makinenin koduna derlemektir. Sonra bir çeviri adımı bu soyut makine kodunu gerçek makine koduna çevirir. Katmanlı modele örnekler: ağ protokollerinin **OSI referans modeli** (11.5) ve **Buxton**'ın Ada programlama destek ortamı (**APSE**) için önerdiği üç katmanlı model.



Şekil 11.4: Sürüm yönetim sisteminin katmanlı modeli (APSE yapısını yansıtır)

ŞEKİL 11.4 AÇIKLAMASI

En üstteki **konfigürasyon yönetim sistemi** nesnelerin sürümlerini yönetir ve genel konfigürasyon yönetimi olanakları sağlar (Bölüm 29). Bunu desteklemek için konfigürasyon öğeleri ya da nesneler için bilgi saklama ve yönetim hizmetleri sunan bir **nesne yönetim sistemi** kullanır. Bu sistem, temel veri saklama ve işlem yönetimi, geri alma (rollback), kurtarma ve erişim kontrolü gibi hizmetler sunan bir **veritabanı sistemi** üzerine kurulmuştur. Veritabanı yönetimi de gerçekleştirmesinde alttaki **işletim sistemi** olanaklarını ve dosya sistemini kullanır (diğer örnekler Bölüm 13'te).

KATMANLI MODEL: AVANTAJLAR

Katmanlı yaklaşım sistemlerin **artımlı geliştirilmesini** destekler: bir katman geliştirilince sunduğu hizmetlerin bir kısmı kullanıcılara açılabilir. Bu mimari **değiştirilebilir ve taşınabilirdir**. Arayüzü değişmediği sürece bir katman **eşdeğer başka bir katmanla değiştirilebilir**. Ayrıca katman arayüzleri değiştiğinde ya da bir katmana yeni olanaklar eklendiğinde sadece **komşu katman** etkilenir. Katmanlı sistemler makineye bağımlılıkları iç katmanlarda topladığı için uygulamanın **çok platformlu** gerçekleştirmelerini sağlamayı kolaylaştırır: farklı bir işletim sistemi ya da veritabanına uyum için sadece iç, makineye bağımlı katmanların yeniden gerçekleştirilmesi gerekir.

SAYFA 252: DEZAVANTAJLAR

Sistemi bu şekilde yapılandırmak **zor** olabilir. İç katmanlar her seviyede gereken dosya yönetimi gibi temel olanaklar sağlayabilir. Üst seviyedeki bir kullanıcının ihtiyaç duyduğu hizmetler bu yüzden birkaç seviye aşağıdaki hizmetlere erişmek için komşu katmanları **"delip geçmek" (punch through)** zorunda kalabilir. Bu modeli bozar, çünkü dış katman artık sadece hemen altındakine bağlı değildir. **Performans** da sorun olabilir, çünkü bazen birden çok seviyede komut yorumlanması gerekir. Çok katman varsa üst katmandan gelen bir hizmet isteği işlenmeden önce farklı katmanlarda birkaç kez yorumlanmak zorunda kalabilir. Bu sorunlardan kaçınmak için uygulamalar komşu katmanın hizmetlerini kullanmak yerine doğrudan iç katmanlarla iletişim kurmak zorunda kalabilir.

👨‍🏫 Hoca anlatımı

Bu bölümün **en önemli kısmı** bu 3 model! Sınavda neredeyse kesin çıkar. Her birinin **tanımını, örneğini, artısını ve eksisini** bil.

1) Repository (depo) modeli = herkesin kullandığı **ortak bir dolap**. Bir evde herkesin yemeklerini koyduğu ortak bir buzdolabı gibi. Kimse kimseye yemek taşımaz, herkes buzdolabına koyar ve alır. **Artı:** veri taşımaya gerek yok, yedekleme ve güvenlik tek merkezden, yeni araç eklemek kolay. **Eksi:** herkes buzdolabının düzenine uymak zorunda (ortak veri modeli), düzeni değiştirmek çok zor (evrim), herkese aynı kurallar (aynı yedekleme ve güvenlik politikası), buzdolabını birden çok eve dağıtmak zor. Örnek: **CASE araç takımları**.

Blackboard farkı: normal depo **pasif** (sadece bekler). Blackboard **aktif**: tahtaya yeni bir şey yazılınca ilgili uzmanı çağırır.

2) Client-server = **restoran** modeli. Garsonlar ve mutfaklar (sunucular) hizmet verir, müşteriler (istemciler) sipariş verir. Mutfak kimin sipariş verdiğini bilmek zorunda değil. Müşteri istek gönderir ve cevabı bekler (**request-reply**, ör. http). **3 bileşen:** sunucular, istemciler, ağ. **Artı: dağıtık**, yeni sunucu eklemek ve yükseltmek kolay. **Eksi:** sunucular arasında **ortak veri modeli yok**, değişiklik istemcileri de etkileyebilir. Örnek: **film ve fotoğraf kütüphanesi** (her medya türü kendi sunucusunda, çünkü video hızlı ve düşük çözünürlük, fotoğraf yüksek çözünürlük ister).

3) Layered (katmanlı) model = **apartman katları** ya da soğan. Her katman sadece **hemen altındakini** kullanır. Örnek: konfigürasyon yönetimi → nesne yönetimi → veritabanı → işletim sistemi. Bir başka ünlü örnek **OSI** ağ modeli. **Artı:** artımlı geliştirme, bir katman aynı arayüzle başka bir katmanla değiştirilebilir, değişiklik sadece komşuyu etkiler, **taşınabilir** (makineye bağlı şeyler iç katmanda, başka platforma geçerken sadece onu değiştir). **Eksi:** yapılandırmak zor (bazen üst katman alt katmanlara "delip geçmek" zorunda kalır), **performans** (istek her katmanda tekrar yorumlanır).

Alıştırma 11.3 tam olarak bu 3 modelin **artı ve eksilerini tablo halinde** istiyor. Bölüm sonunda tablo halinde hazırladım.

📝 Deftere not

- ⭐ **System organisation:** sistemi yapılandırmanın temel stratejisi, mimari tasarımın **başında** karar verilir.
- ⭐⭐ **3 organizasyon stili: Repository** (paylaşılan veri deposu) · **Client-server** (paylaşılan hizmetler ve sunucular) · **Layered / abstract machine** (katmanlar). Ayrı ya da **birlikte** kullanılabilir.
- ⭐ Alt sistemler arası veri paylaşımının **2 yolu:** (1) **merkezi veritabanı** = **repository model**, (2) her alt sistem **kendi DB'si** + **mesajlaşma**.
- ⭐ **Repository**, verinin bir alt sistemde üretilip başkasında kullanıldığı sistemlere uygun. Ör: komuta-kontrol, yönetim bilgi sistemleri, **CAD**, **CASE araç takımları**.
- **Şekil 11.2:** project repository + design editor, code generator, design translator, program editor, design analyser, report generator. İlk CASE deposu: **ICL** (1970'ler). **Buxton**'ın **Stoneman** (Ada) ortamı.
- ⭐⭐ **Repository artı ve eksileri:**
  - (+) Büyük veriyi paylaşmak **verimli**, açık aktarım yok · (–) alt sistemler **ortak veri modeline** uymalı (uzlaşma, performans düşebilir, uymayan yeni alt sistemi entegre etmek zor).
  - (+) Veri üretenler verinin **nasıl kullanıldığıyla ilgilenmez** · (–) **evrim zor** (büyük veriyi yeni modele çevirmek pahalı ya da imkânsız).

    - (+) Yedekleme, güvenlik, erişim kontrolü, kurtarma **merkezi** (depo yöneticisinin işi) · (–) alt sistemlerin farklı ihtiyaçlarına rağmen **aynı politika** dayatılır.
    - (+) Paylaşım şema üzerinden görünür, uyumlu **yeni araç eklemek kolay** · (–) depoyu birden çok makineye **dağıtmak zor** (veri tekrarı, tutarsızlık).
- ⭐ Normal depo **pasif** (kontrol alt sistemlerde). Blackboard model (yapay zekâdan): belirli veri hazır olunca alt sistemleri **tetikler**, verinin yapısı zayıfsa uygun.
- ⭐⭐ Client-server model: sistem = **hizmetler + sunucular + istemciler**. **3 bileşen:**
    - **Servers:** hizmet sunar (print server, file server, compile server).
    - **Clients:** hizmetleri çağırır, genelde kendi başına alt sistem, birden çok örnek eşzamanlı çalışabilir.
    - **Network:** erişimi sağlar (şart değil ama pratikte dağıtık).
- İstemci sunucu adlarını bilmeli. **Sunucu istemcilerin kimliğini ya da sayısını bilmez**. Erişim: **remote procedure call** ya da **request-reply** protokolü (**http**). İstemci istek gönderir ve **cevabı bekler**.
- ⭐ **Şekil 11.3, film ve fotoğraf kütüphanesi:** istemciler internet üzerinden catalogue, video, picture ve web sunucularına bağlanır. Video: hızlı, senkron, düşük çözünürlük, sıkıştırma. Fotoğraf: **yüksek çözünürlük**, ayrı sunucu. İstemci = web tarayıcısıyla entegre arayüz.
- ⭐⭐ **Client-server artı: dağıtık** mimari, ağdaki işlemcileri etkili kullanır, **yeni sunucu eklemek** ve sunucuları **şeffaf yükseltmek** kolay.
- ⭐⭐ **Client-server eksi:** yeni sunucunun faydası için istemci ve sunucularda değişiklik gerekebilir · sunucular arası **ortak veri modeli yok** (her sunucu kendi modelini kurabilir, performans için iyi) · **XML** dönüşümü kolaylaştırır ama **verimsizdir**, performans sorunu.
- ⭐⭐ Layered model (**abstract machine model**): sistem, her biri hizmet sunan **katmanlar**. Her katman = **soyut makine**, "dili" bir sonraki katmanı gerçekleştirmek için kullanılır. Ör: **OSI** modeli, Buxton'ın **APSE** 3 katmanı.
- ⭐ **Şekil 11.4 (sürüm yönetim sistemi):** Configuration management layer → Object management layer → Database system layer → Operating system layer.
- ⭐⭐ **Layered artıları: artımlı geliştirme** (katman bitince hizmetleri açılabilir) · **değiştirilebilir ve taşınabilir** (arayüz aynıysa katman **başkasıyla değiştirilebilir**) · değişiklik sadece **komşu katmanı** etkiler · makine bağımlılıkları **iç katmanda**, bu yüzden **çok platform** kolay.
- ⭐⭐ **Layered eksileri:** yapılandırmak **zor** (iç katmanların temel hizmetleri her seviyede gerekir, üst katman komşuları **delip geçer (punch through)**, model bozulur) · **performans** (istek birden çok katmanda yorumlanır). Çözüm: doğrudan iç katmanlarla iletişim.

Sayfa 252–255 — 11.3 Modular decomposition styles · Object-oriented (Şekil 11.5) · Function-oriented pipelining (Şekil 11.6)

📖 Sayfanın özeti

ALT SİSTEM VE MODÜL

Genel sistem organizasyonu seçildikten sonra alt sistemlerin **modüllere ayrıştırılması** yaklaşımına karar verilir. Sistem organizasyonu ile modüler ayrıştırma arasında katı bir ayrım yoktur (11.2'deki stiller bu seviyede de uygulanabilir), ama modüllerdeki bileşenler genelde alt sistemlerden **daha**

küçüktür, bu da başka ayrıştırma stillerine izin verir. Alt sistem ve modül arasında net bir ayrım yoktur, ama yazar şöyle düşünmeyi faydalı buluyor: **1) Alt sistem (sub-system)**, çalışması diğer alt sistemlerin sunduğu hizmetlere **bağlı olmayan**, kendi başına bir sistemdir. Alt sistemler modüllerden oluşur ve diğer alt sistemlerle iletişim için tanımlı **arayüzleri** vardır. **2) Modül (module)**, normalde diğer modüllere bir ya da daha fazla hizmet sunan bir sistem bileşenidir. Diğer modüllerin hizmetlerini kullanır. Normalde **bağımsız bir sistem sayılmaz**. Modüller genelde birkaç daha basit bileşenden oluşur.

2 ANA STRATEJİ

Bir alt sistemi modüllere ayrıştırmanın iki ana stratejisi vardır: **1) Object-oriented decomposition:** sistem **birbiriyle iletişim kuran nesneler** kümesine ayrıştırılır. **2) Function-oriented pipelining:** sistem, girdi verisini alıp çıktı verisine **dönüştüren** fonksiyonel modüllere ayrıştırılır. Nesne yönelimli yaklaşımda modüller, özel (private) durumları ve bu durum üzerinde tanımlı işlemleri olan **nesnelerdir**. Boru hattı modelinde modüller **fonksiyonel dönüşümlerdir**. İki durumda da modüller **sıralı bileşenler** ya da **süreçler** olarak gerçekleştirilebilir.

EŞZAMANLILIĞA ERKEN BAĞLANMA

Sistemde **eşzamanlılığa (concurrency) erken bağlanmaktan** kaçınılmalıdır. Eşzamanlı bir sistem tasarımından kaçınmanın avantajı, sıralı programların paralel sistemlerden daha kolay tasarlanması, gerçekleştirilmesi, doğrulanması ve test edilmesidir. Süreçler arası zaman bağımlılıklarını resmileştirmek, kontrol etmek ve doğrulamak zordur. En iyisi sistemleri modüllere ayrıştırmak, sonra **gerçekleştirme sırasında** bunların sıralı mı paralel mi çalışacağına karar vermektir.

11.3.1 NESNE YÖNELİMLİ AYRIŞTIRMA (ŞEKİL 11.5)

Nesne yönelimli bir mimari model, sistemi iyi tanımlanmış arayüzleri olan **gevşek bağlı (loosely coupled)** nesneler kümesi olarak yapılandırır. Nesneler diğer nesnelerin sunduğu hizmetleri çağırır (nesne modelleri Bölüm 8, OO tasarım Bölüm 14). Şekil 11.5, bir **fatura işleme sistemi** için nesne yönelimli mimari modeldir: sistem müşterilere fatura kesebilir, ödemeleri alabilir, bu ödemeler için makbuz ve ödenmemiş faturalar için hatırlatma gönderebilir. UML gösteriminde nesne sınıflarının adları ve öznitelikleri vardır, işlemler (varsa) dikdörtgenin alt kısmında tanımlanır. **Kesikli oklar** bir nesnenin başka bir nesnenin özniteliklerini ya da hizmetlerini kullandığını gösterir. Sınıflar: **Customer** (müşteri no, ad, adres, kredi süresi), **Payment** (fatura no, tarih, tutar, müşteri no), **Invoice** (fatura no, tarih, tutar, müşteri; işlemler: issue, sendReminder, acceptPayment, sendReceipt), **Receipt** (fatura no, tarih, tutar, müşteri no).

AÇIKLAMA

Nesne yönelimli ayrıştırma nesne sınıfları, öznitelikleri ve işlemleriyle ilgilenir. Gerçekleştirildiğinde bu sınıflardan nesneler oluşturulur ve nesne işlemlerini koordine etmek için bir kontrol modeli kullanılır. Bu örnekte **Invoice** sınıfının sistem işlevselliğini gerçekleştiren çeşitli işlemleri vardır. Bu sınıf müşterileri, ödemeleri ve makbuzları temsil eden diğer sınıfları kullanır.

OO: AVANTAJ VE DEZAVANTAJ

Avantajlar iyi bilinir: nesneler **gevşek bağlı** olduğu için bir nesnenin gerçekleştirmesi diğer nesneleri etkilemeden **değiştirilebilir**. Nesneler çoğu zaman **gerçek dünya varlıklarının** temsilleridir, bu yüzden sistemin yapısı kolay **anlaşılır**. Bu gerçek dünya varlıkları farklı sistemlerde kullanıldığı için nesneler **yeniden kullanılabilir**. Mimari bileşenlerin doğrudan gerçekleştirilmesini sağlayan nesne yönelimli programlama dilleri vardır. **Dezavantajlar:** hizmetleri kullanmak için nesneler diğer nesnelerin **adına ve arayüzüne açıkça başvurmak** zorundadır. Önerilen sistem değişikliklerini karşılamak için bir arayüz değişikliği gerekirse, değişen nesnenin **bütün kullanıcıları** üzerindeki etkisi

değerlendirilmelidir. Nesneler küçük ölçekli gerçek dünya varlıklarıyla temiz eşleşebilir, ama **daha karmaşık varlıkları** nesne olarak temsil etmek bazen zordur.

11.3.2 FONKSİYON ODAKLI BORU HATTI

Fonksiyon odaklı boru hattı ya da **veri akışı (data-flow)** modelinde fonksiyonel dönüşümler girdilerini işleyip çıktı üretir. Veri birinden diğerine akar ve sıra boyunca ilerlerken dönüşür. Her işleme adımı bir **dönüşüm (transform)** olarak gerçekleştirilir. Girdi verisi bu dönüşümlerden geçerek çıktıya dönüşür. Dönüşümler **sıralı ya da paralel** çalışabilir. Veri her dönüşüm tarafından **öğe öğe** ya da **tek bir toplu (batch)** halinde işlenebilir. Dönüşümler ayrı süreçler olarak gösterildiğinde bu modele, **Unix** sistemindeki terminolojiden dolayı bazen **pipe and filter** stili denir. Unix, veri kanalı görevi gören **borular (pipes)** ve fonksiyonel dönüşüm olan bir komutlar kümesi sunar. Bu modele uyan sistemler Unix komutlarını borular ve Unix kabuğunun kontrol olanaklarıyla birleştirerek gerçekleştirilebilir. **"Filter"** terimi kullanılır, çünkü bir dönüşüm girdi veri akışından işleyebileceği veriyi **"süzer"**.

BATCH SEQUENTİAL

Bu boru hattı modelinin varyantları bilgisayarlar otomatik veri işleme için ilk kullanıldığından beri kullanılmaktadır. Dönüşümler **sıralı** ise ve veri **toplu (batch)** işleniyorsa bu mimari modele **batch sequential** model denir. Bölüm 13'te anlatıldığı gibi **faturalama sistemleri** gibi veri işleme sistemleri için yaygın bir mimaridir. Veri işleme sistemleri genelde çok sayıda girdi kaydı üzerinde basit hesaplamalardan türeyen birçok çıktı raporu üretir.

ŞEKİL 11.6: FATURA SİSTEMİ BORU HATTI

Bir kurum müşterilere fatura kesmiştir. Haftada bir, yapılan ödemeler faturalarla **eşleştirilir**. Ödenmiş faturalar için **makbuz** kesilir, izin verilen ödeme süresi içinde ödenmemiş faturalar için **hatırlatma** gönderilir. Akış: Invoices → Read issued invoices → Identify payments (Payments girdisiyle) → (Issue receipts → Receipts) ve (Find payments due → Issue payment reminder → Reminders). Bu, fatura sisteminin sadece bir kısmının modelidir (fatura kesmek için başka dönüşümler gerekir). Nesne yönelimli eşdeğerinden farkına dikkat: **nesne modeli daha soyuttur**, çünkü işlemlerin **sırası** hakkında bilgi içermez.



Şekil 11.6: Fatura işleme sisteminin boru hattı modeli

PİPELİNE AVANTAJLARI

1) Dönüşümlerin **yeniden kullanımını** destekler. **2) Sezgiseldir**: birçok insan işini girdi ve çıktı işleme olarak düşünür. **3)** Yeni dönüşümler ekleyerek sistemi **geliştirmek** genelde kolaydır. **4)** Eşzamanlı ya da sıralı sistem olarak gerçekleştirmesi **basittir**.

PİPELİNE DEZAVANTAJLARI

Bu stilin ana sorunu, bütün dönüşümlerin tanıyabileceği **ortak bir veri aktarım formatı** olması gerekliliğidir. Her dönüşüm ya işleyeceği verinin formatı konusunda iletişim kurduğu dönüşümlerle anlaşmalı ya da iletilen bütün veri için standart bir format dayatılmalıdır. Dönüşümler bağımsız ve yeniden kullanılabilir olduğunda tek uygulanabilir yol ikincisidir. Unix'te standart format basitçe bir **karakter dizisidir**. Her dönüşüm girdisini ayrıştırmalı (parse) ve çıktısını anlaşılmış biçime çevirmelidir (unparse). Bu sistem yükünü artırır ve **uyumsuz veri formatları** kullanan dönüşümleri entegre etmenin imkânsız olması anlamına gelebilir. Ayrıca **etkileşimli sistemleri** boru hattı modeliyle yazmak zordur, çünkü işlenecek bir veri akışı gerekir. Basit metin girdi ve çıktısı bu şekilde modellenebilir, ama grafik kullanıcı arayüzlerinin fare tıklamaları ya da menü seçimleri gibi **olaylara** dayanan daha

karmaşık G/Ç formatları ve kontrolü vardır. Bunu boru hattı modeline uyumlu bir forma çevirmek zordur.

👩‍🏫 Hoca anlatımı

Önce **alt sistem ve modül farkı**: **Alt sistem** kendi başına çalışabilir, başkasına muhtaç değil (bir arabanın motoru gibi). **Modül** ise tek başına bir sistem sayılmaz, başka modüllerle birlikte çalışır (motordaki bir piston gibi).

Alt sistemi modüllere bölmenin **2 yolu**:

1) Object-oriented = sistemi **"şeylere"** böl: Müşteri, Fatura, Ödeme, Makbuz. Her nesnenin kendi bilgisi (öznitelik) ve yapabildikleri (işlem) var. **Artı:** gevşek bağlı (birini değiştir, diğerleri etkilenmez), gerçek dünyaya benzer (anlaşılır), yeniden kullanılabilir. **Eksi:** nesneler birbirinin **adını ve arayüzünü bilmek** zorunda (arayüz değişirse kullanan herkes etkilenir), karmaşık varlıkları nesneyle göstermek zor.

2) Function-oriented pipelining = sistemi **"işlemlere"** böl, bir **fabrika üretim bandı** gibi: hammadde girer, her istasyon bir işlem yapar, sonunda ürün çıkar. Faturalar okunur → ödemeler bulunur → makbuz ya da hatırlatma çıkar. Unix'te buna **pipe and filter** denir (`cat dosya | grep hata | sort` gibi). Toplu ve sıralı işlenirse **batch sequential** (faturalama sistemleri). **Artı (4):** dönüşümler yeniden kullanılır, sezgiseldir, yeni adım eklemek kolay, sıralı ya da paralel yapmak basit. **Eksi:** herkesin anlayacağı **ortak veri formatı** lazım (Unix'te düz metin, her adım ayrıştırıp yeniden yazar, bu yavaşlatır), ve **etkileşimli sistemlere (GUI) uygun değil** (fare tıklamaları bir "akış" değildir).

Önemli fark: nesne modeli **daha soyuttur**, çünkü işlemlerin **sırasını** göstermez. Boru hattı ise sırayı açıkça gösterir.

Bir de şu kural: **eşzamanlılığa erken karar verme!** Önce modüllere böl, paralel mi sıralı mı çalışacağına gerçekleştirmede karar ver, çünkü sıralı programlar test etmesi çok daha kolaydır.

📝 Deftere not

- Organizasyon seçilince alt sistemleri **modüllere** ayırma kararı. Katı ayrım yok, ama modüller alt sistemlerden **küçük**.
- ⭐⭐ **Alt sistem ve modül farkı:**
  - **Sub-system:** kendi başına sistem, çalışması diğer alt sistemlere **bağlı değil**. Modüllerden oluşur, tanımlı **arayüzleri** var.
  - **Module:** diğer modüllere hizmet sunan ve onların hizmetlerini kullanan bileşen. **Bağımsız sistem sayılmaz**, daha basit bileşenlerden oluşur.
- ⭐⭐ **2 ayrıştırma stratejisi:**
  - Object-oriented decomposition: **iletişim kuran nesneler**. Modül = **private state + işlemler**.
  - Function-oriented pipelining: girdiyi çıktıya **dönüştüren** fonksiyonel modüller. Modül = **fonksiyonel dönüşüm**.

  İkisi de **sıralı bileşen** ya da **süreç** olarak gerçekleştirilebilir.
- ⭐ **Eşzamanlılığa erken bağlanma.** Sıralı programlar tasarlamak, gerçekleştirmek, doğrulamak ve test etmek daha kolay. Önce modüllere ayır, sıralı ya da paralele **gerçekleştirmede** karar ver.

- ⭐ **11.3.1 OO model:** iyi tanımlı arayüzlü **loosely coupled** nesneler, birbirinin hizmetini çağırır.
- **Şekil 11.5 (fatura sistemi):** Customer, Payment, **Invoice** (issue, sendReminder, acceptPayment, sendReceipt), Receipt. UML: ad + öznitelikler + işlemler (alt kısım). **Kesikli ok** = başka nesnenin öznitelik ya da hizmetini kullanır.
- ⭐⭐ **OO avantajları: gevşek bağlı** (gerçekleştirme diğerlerini etkilemeden değişir) · **gerçek dünya varlıkları**, yapı kolay **anlaşılır** · **yeniden kullanılabilir** · OO dilleri doğrudan gerçekleştirir.
- ⭐⭐ **OO dezavantajları:** nesneler diğerlerinin **adına ve arayüzüne açıkça başvurur** → arayüz değişirse **tüm kullanıcılara** etkisi değerlendirilmeli · **karmaşık varlıkları** nesne olarak göstermek zor.
- ⭐⭐ **11.3.2 Function-oriented pipeline / data-flow model:** dönüşümler girdi işler, çıktı üretir, veri akarken **dönüşür**. Her adım = **transform**. Sıralı ya da paralel, **öğe öğe** ya da **batch**.
- ⭐ Ayrı süreçler olunca: pipe and filter stili (**Unix**: pipes = veri kanalları, komutlar = dönüşümler). **Filter** = girdiden işleyebileceğini süzer.
- ⭐ Sıralı + toplu = batch sequential model. **Faturalama sistemleri** gibi veri işleme sistemlerinde yaygın.
- **Şekil 11.6:** Invoices → read issued invoices → identify payments (+ Payments) → issue receipts → Receipts / find payments due → issue payment reminder → Reminders.
- ⭐ OO model pipeline'dan **daha soyut**: işlemlerin **sırasını** içermez.
- ⭐⭐ **Pipeline avantajları (4):** (1) dönüşümlerin **yeniden kullanımı**, (2) **sezgisel** (girdi-çıktı düşüncesi), (3) yeni dönüşüm ekleyerek **evrim kolay**, (4) eşzamanlı ya da sıralı gerçekleştirmesi **basit**.
- ⭐⭐ **Pipeline dezavantajları: ortak veri aktarım formatı** şart (Unix: **karakter dizisi**, her adım **parse / unparse** → ek yük, uyumsuz formatlar entegre edilemez) · **etkileşimli sistemler** zor (GUI = olaylar: fare tıklaması, menü seçimi, akışa çevrilemez).

Sayfa 256–260 — 11.4 Control styles · Centralised control (Şekil 11.7, 11.8) · Event-driven (Şekil 11.9, 11.10)

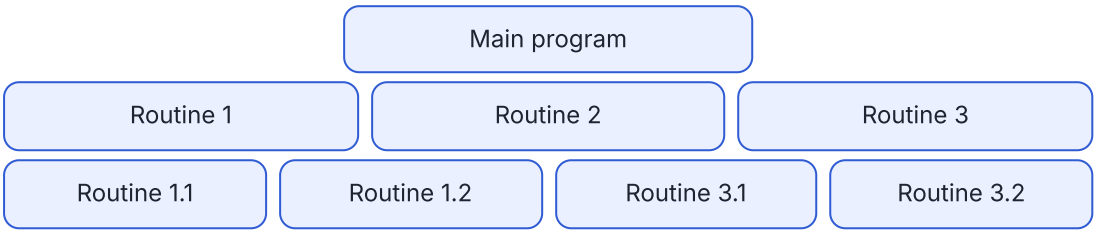
📖 Sayfanın özeti

KONTROL STİLLERİ

Sistemi yapılandırma modelleri sistemin alt sistemlere nasıl ayrıldığıyla ilgilidir. Bir sistem olarak çalışabilmeleri için alt sistemler, hizmetlerinin **doğru yere, doğru zamanda** ulaşması için kontrol edilmelidir. Yapısal modeller kontrol bilgisi **içermez (ve içermemelidir)**. Mimar, alt sistemleri kullanılan yapı modelini tamamlayan bir **kontrol modeline** göre düzenlemelidir. Mimari seviyedeki kontrol modelleri alt sistemler arası **kontrol akışıyla** ilgilenir. **2 genel kontrol stili** vardır: **1) Centralised control:** bir alt sistem kontrolün **genel sorumluluğunu** taşır, diğer alt sistemleri başlatır ve durdurur. Kontrolü başka bir alt sisteme devredebilir ama bu sorumluluğun kendisine geri dönmesini bekler. **2) Event-based control:** kontrol bilgisi bir alt sisteme gömülü olmak yerine her alt sistem **dışarıdan üretilen olaylara** yanıt verebilir. Bu olaylar diğer alt sistemlerden ya da sistemin ortamından gelebilir. Kontrol stilleri yapısal stillerle **birlikte** kullanılır. Anlatılan bütün yapısal stiller merkezi ya da olay tabanlı kontrolle gerçekleştirilebilir.

11.4.1 MERKEZİ KONTROL

Bir alt sistem **sistem denetleyicisi (controller)** olarak atanır ve diğer alt sistemlerin çalışmasını yönetmekten sorumludur. Kontrol edilen alt sistemlerin sıralı mı paralel mi çalıştığına göre **iki sınıf** vardır: **1) Call-return model:** bilinen **yukarıdan aşağı alt program (subroutine)** modelidir. Kontrol bir alt program hiyerarşisinin **tepesinden** başlar ve alt program çağrılarıyla ağacın alt seviyelerine geçer. Sadece **sıralı** sistemlere uygulanabilir. **2) Manager model: eşzamanlı (concurrent)** sistemlere uygulanabilir. Bir bileşen **sistem yöneticisi** olarak atanır ve diğer süreçlerin başlatılmasını, durdurulmasını ve koordinasyonunu kontrol eder. Süreç, diğer süreçlerle paralel çalışabilen bir alt sistem ya da modüldür. Bu modelin bir biçimi, bir yönetim rutininin bazı durum değişkenlerinin değerlerine göre belirli alt sistemleri çağırdığı **sıralı** sistemlerde de uygulanabilir. Bu genelde bir **case** ifadesiyle gerçekleştirilir.



Şekil 11.7: Call-return kontrol modeli

CALL-RETURN AYRINTILARI

Ana program Rutin 1, 2 ve 3'ü çağırabilir, Rutin 1 Rutin 1.1 ya da 1.2'yi, Rutin 3 Rutin 3.1 ya da 3.2'yi çağırabilir, vb. Bu, **program dinamiğinin** bir modelidir, **yapısal bir model değildir** (Rutin 1.1'in Rutin 1'in parçası olması gerekmez). Bu model C, Ada ve Pascal gibi programlama dillerine gömülüdür. Kontrol hiyerarşide üst seviyedeki bir rutinden alt seviyedekine geçer, sonra rutinin çağrıldığı noktaya **geri döner**. O an çalışan alt program kontrolden sorumludur, başka rutinleri çağırabilir ya da kontrolü üst rutine geri verebilir. Programın başka bir noktasına dönmek kötü programlama stilidir. Bu model modül seviyesinde fonksiyonları ya da nesneleri kontrol etmek için kullanılabilir. Programlama dillerindeki alt programlar doğal olarak fonksiyoneldir, ama birçok nesne yönelimli sistemde nesnelerdeki işlemler (metotlar) prosedür ya da fonksiyon olarak gerçekleştirilir (ör. bir Java nesnesi başka bir nesneden hizmet isteyince ilgili metodu çağırır). Bu modelin **katı ve sınırlı** yapısı hem bir **güç** hem bir **zayıflıktır**: kontrol akışlarını analiz etmek ve sistemin belirli girdilere nasıl yanıt vereceğini bulmak nispeten basit olduğu için güçtür. Normal çalışmanın **istisnalarını** ele almak zahmetli olduğu için zayıflıktır.

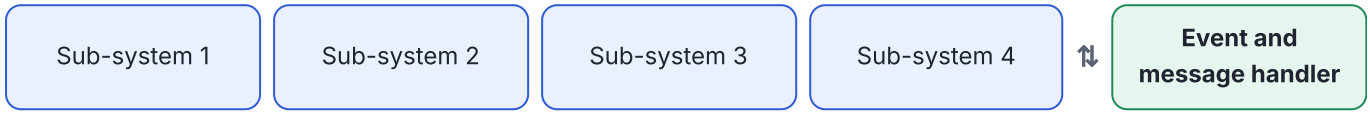
MANAGER MODEL (ŞEKİL 11.8)

Şekil 11.8 eşzamanlı bir sistem için **merkezi yönetim modelini** gösterir. Bu model çok sıkı zaman kısıtları olmayan **"yumuşak" gerçek zamanlı** sistemlerde sık kullanılır. Merkezi denetleyici, **sensörler ve aktüatörlerle** ilişkili süreçlerin çalışmasını yönetir (Bölüm 15'teki bina izleme sistemi bu modeli kullanır). Bileşenler: **system controller** merkezde, etrafında sensor processes, actuator processes, computation processes, user interface ve fault handler. Denetleyici süreci, süreçlerin ne zaman başlatılıp durdurulacağına **sistem durum değişkenlerine** göre karar verir. Diğer süreçlerin işlenecek bilgi üretip üretmediğini kontrol eder ya da işlenmek üzere onlara bilgi aktarır. Denetleyici genelde olayları ya da durum değişikliklerini yoklamak için sensörleri ve diğer süreçleri **sürekli döngüyle yoklar (polling)**. Bu yüzden bu modele bazen **event-loop** modeli denir.

11.4.2 OLAY GÜDÜMLÜ SİSTEMLER

Merkezi kontrol modellerinde kontrol kararları genelde bazı sistem **durum değişkenlerinin** değerleriyle belirlenir. Olay güdümlü kontrol modelleri ise **dışarıdan üretilen olaylarla** yönlendirilir. Bu bağlamda **olay** sadece ikili bir sinyal değildir, belirli değer aralığı alabilen bir sinyal ya da menüden

gelen bir komut girdisi olabilir. Olay ile basit bir girdi arasındaki fark, olayın **zamanlamasının** onu işleyen sürecin kontrolü **dışında** olmasıdır. Birçok tür olay güdümlü sistem vardır: kullanıcı arayüzü olaylarının düzenleme komutlarını gösterdiği **editörler**, bir koşulun doğru olmasının bir eylemi tetiklediği yapay zekâdaki **kural tabanlı üretim sistemleri** ve bir nesnenin özniteliğinin değişmesinin bazı eylemleri tetiklediği **aktif nesneler**. Bu bölümde **iki olay güdümlü model** anlatılıyor: **1) Broadcast models:** bir olay **bütün alt sistemlere yayınlanır**. O olayı işleyecek şekilde programlanmış herhangi bir alt sistem yanıt verebilir. **2) Interrupt-driven models:** sadece **gerçek zamanlı** sistemlerde kullanılır. Dış **kesmeler (interrupts)** bir kesme işleyicisi tarafından algılanır ve işlenmek üzere başka bir bileşene aktarılır. Broadcast modelleri ağdaki farklı bilgisayarlara **dağıtılmış** alt sistemleri entegre etmede etkilidir. Interrupt modelleri **katı zamanlama** gereksinimli gerçek zamanlı sistemlerde kullanılır.



Şekil 11.9: Seçici yayına (selective broadcasting) dayalı kontrol modeli

BROADCAST AYRINTILARI

Broadcast modelinde alt sistemler belirli olaylara **ilgi kaydeder (register)**. Bu olaylar gerçekleştiğinde kontrol, olayı işleyebilecek alt sisteme geçer. Bu modelin merkezi modelden farkı, kontrol politikasının olay ve mesaj işleyicisine **gömülü olmamasıdır**: alt sistemler hangi olaylara ihtiyaç duyduklarına karar verir, olay ve mesaj işleyici bu olayların onlara gönderilmesini sağlar. Bütün olaylar bütün alt sistemlere yayınlanabilir, ama bu çok fazla işlem yükü getirir. Daha çok, olay ve mesaj işleyici alt sistemlerin ve onların ilgilendiği olayların bir **kaydını (register)** tutar. Alt sistemler olay üretir (ör. bazı verilerin işlenmeye hazır olduğu). Olay işleyici olayları algılar, olay kaydına bakar ve olayı ilgilendiğini belirten alt sistemlere iletir. Kullanıcı arayüzü olaylarıyla yönlendirilen PC tabanlı basit sistemlerde fare, klavye vb. olaylarını dinleyip bunları daha özel komutlara çeviren açık **olay dinleyici (event-listener)** alt sistemleri vardır. Olay işleyici genelde **noktadan noktaya (point-to-point)** iletişimi de destekler: bir alt sistem açıkça başka bir alt sisteme mesaj gönderebilir. Bu modelin varyasyonları (ör. Field ortamı, HP Softbench) yazılım mühendisliği ortamlarında araç etkileşimlerini kontrol etmek için kullanılmıştır. Bölüm 12'deki **ORB (Object Request Broker)**'lar da dağıtık nesne iletişimi için bu kontrol modelini destekler.

BROADCAST: AVANTAJ VE DEZAVANTAJ

Avantajı: evrim nispeten basittir. Belirli olay sınıflarını işleyecek yeni bir alt sistem, olaylarını olay işleyiciye kaydettirerek entegre edilebilir. Herhangi bir alt sistem diğer bir alt sistemi onun **adını ya da yerini bilmeden** etkinleştirebilir. Alt sistemler dağıtık makinelerde gerçekleştirilebilir ve bu dağıtım diğer alt sistemlere **şeffaftır**. **Dezavantajı:** alt sistemler olayların **işlenip işlenmeyeceğini ya da ne zaman işleneceğini bilmez**. Bir alt sistem bir olay ürettiğinde o olaya hangi alt sistemlerin ilgi kaydettiğini bilmez. Farklı alt sistemlerin aynı olaylara kaydolması da oldukça mümkündür, bu da olayı işlemenin sonuçları kullanılabilir hale geldiğinde **çatışmalara** yol açabilir.

INTERRUPT-DRİVEN (ŞEKİL 11.10)

Dışarıdan üretilen olayların **çok hızlı** işlenmesini gerektiren gerçek zamanlı sistemler olay güdümlü olmalıdır. Ör: bir arabadaki emniyet sistemlerini kontrol eden gerçek zamanlı sistem olası bir çarpışmayı algılayıp sürücünün kafası direksiyona çarpmadan **hava yastığını** şişirmelidir. Bu hızlı yanıtı sağlamak için kesme güdümlü kontrol kullanılmalıdır. Şekil 11.10'da bilinen sayıda kesme türü vardır ve her tür için bir **işleyici (handler)** tanımlıdır. Her kesme türü, işleyicisinin adresinin saklandığı bellek konumuyla ilişkilidir (**interrupt vector**). Belirli türde bir kesme alındığında bir donanım anahtarı

kontrolün hemen işleyicisine geçmesini sağlar. Bu işleyici, kesmenin işaret ettiği olaya yanıt olarak başka süreçleri başlatabilir ya da durdurabilir. Bu model çoğunlukla bir olaya **anında yanıt** gereken gerçek zamanlı sistemlerde kullanılır. **Merkezi yönetim modeliyle birleştirilebilir**: merkezi yönetici sistemin normal çalışmasını yönetir, acil durumlar için kesme tabanlı kontrol kullanılır.



Şekil 11.10: Kesme güdümlü kontrol modeli

INTERRUPT: AVANTAJ VE DEZAVANTAJ

Avantajı: olaylara **çok hızlı yanıt** verilebilmesi. **Dezavantajları:** programlaması **karmaşık** ve **doğrulaması zordur**. Sistem testi sırasında kesme zamanlama kalıplarını tekrar oluşturmak imkânsız olabilir. Bu modelle geliştirilmiş sistemleri **değiştirmek zor** olabilir. Kesme sayısı **donanımla sınırlıysa**, bu sınıra ulaşıldığında başka türde olay işlenemez. Bu sınır bazen birkaç olay türünü tek bir kesmeye eşleyerek aşılabilir, işleyici hangi olayın olduğunu sonra çözer. Ama tek tek kesmelere **çok hızlı** yanıt gerekiyorsa kesme eşleme pratik olmayabilir.

👩‍🏫 Hoca anlatımı

Yapı modelleri "parçalar neler" der, **kontrol modelleri** ise "**kim kimi ne zaman çalıştırıyor**" der. İkisi birlikte kullanılır.

2 ana stil: Centralised = bir **patron** var, herkese ne yapacağını o söyler. **Event-based** = patron yok, herkes kendi ilgilendiği **olaya** tepki verir.

Centralised'ın 2 türü:

Call-return = şirket hiyerarşisi. Genel müdür bir müdüre iş verir, müdür bir şefe iş verir, şef işi bitirince müdüre **geri döner**, müdür genel müdüre döner. Programlamadaki fonksiyon çağrısı tam olarak budur. Sadece **sıralı** sistemlerde. **Güç:** akışı izlemek kolay. **Zayıflık:** istisnalar (beklenmedik durumlar) zahmetli.

Manager model = bir **orkestra şefi**. Aynı anda çalan (paralel) müzisyenleri yönetir, kimin ne zaman başlayıp duracağına karar verir. Sürekli etrafı yoklar ("sensörden yeni veri var mı?"), bu yüzden **event-loop** modeli de denir. "Yumuşak" gerçek zamanlı sistemlerde kullanılır (ör. bina izleme).

Event-based'ın 2 türü:

Broadcast = **okul anonsu**. Anons herkese gider, ilgilenen tepki verir. Daha akıllı versiyonu (Şekil 11.9): alt sistemler "beni şu olaylar ilgilendiriyor" diye **kaydolur**, olay işleyici olayı sadece onlara gönderir. Instagram'da birini takip etmek gibi: sadece takip ettiklerinin bildirimi sana gelir. **Artı:** yeni alt sistem eklemek kolay (sadece kaydol), kimin nerede olduğunu bilmeye gerek yok, dağıtık sistemlere uygun. **Eksi:** olayı gönderen, onu kimin ve ne zaman işleyeceğini bilmez, aynı olayı birden çok kişi işlerse çatışma olabilir.

Interrupt-driven = **yangın alarmı**. Ne yapıyorsan bırakıp hemen tepki verirsin. Gerçek zamanlı ve hayati sistemler için: **hava yastığı** örneğini unutma, sürücünün kafası direksiyona çarpmadan açılmalı. **Artı: çok hızlı**. **Eksi:** programlaması ve test etmesi zor (kesmelerin zamanlamasını testte tekrar oluşturamazsın), değiştirmek zor, kesme sayısı donanımla sınırlı.

Alıştırma 11.6 ve 11.7 bu konudan: maaş bordrosu (batch) → **call-return**. Farklı firmaların araçları birlikte çalışacak → **broadcast**. TV kumandası → **interrupt-driven**.

📝 Deftere not

- ⭐ Alt sistemler hizmetleri **doğru yere, doğru zamanda** vermek için kontrol edilmeli. Yapısal modeller **kontrol bilgisi içermez**. Mimar bir **kontrol modeli** ekler (alt sistemler arası **kontrol akışı**).
- ⭐⭐ **2 genel kontrol stili:**
  - Centralised control: bir alt sistem **genel kontrol sorumluluğunu** taşır, diğerlerini başlatır ve durdurur. Kontrolü devredebilir ama geri bekler.
  - Event-based control: her alt sistem **dışarıdan gelen olaylara** yanıt verir (diğer alt sistemlerden ya da ortamdan).

  Kontrol stilleri **yapısal stillerle birlikte** kullanılır.
- ⭐⭐ **Merkezi kontrolün 2 türü:**
  - Call-return model: **yukarıdan aşağı subroutine** modeli. Kontrol hiyerarşinin tepesinden başlar, alt programlara iner, çağrılan noktaya **geri döner**. Sadece **sıralı** sistemler. C, Ada, Pascal'a gömülü.
  - Manager model: **eşzamanlı** sistemler. Bir bileşen **sistem yöneticisi**: süreçleri başlatır, durdurur, koordine eder. Sıralı sistemde: durum değişkenine göre **case** ifadesi.
- ⭐ **Şekil 11.7:** Main program → Routine 1, 2, 3 → Routine 1.1, 1.2, 3.1, 3.2. **Program dinamiği** modeli, **yapısal model değil**.
- ⭐ Call-return'ün **katı yapısı**: **güç** = kontrol akışını analiz etmek kolay. **Zayıflık** = **istisnaları** ele almak zahmetli. Modül seviyesinde fonksiyon ve nesneler için de kullanılır (Java metot çağrısı).
- ⭐⭐ **Şekil 11.8, merkezi kontrol (gerçek zamanlı): system controller** + sensor processes, actuator processes, computation processes, user interface, fault handler. **"Soft" real-time** sistemlerde. Denetleyici **durum değişkenlerine** göre süreçleri başlatır ve durdurur, sensörleri sürekli **yoklar (polling)** → event-loop model. Ör: bina izleme sistemi.
- ⭐ **Merkezi:** karar = **durum değişkenleri**. **Olay güdümlü:** karar = **dış olaylar**.
- ⭐ **Event:** sadece ikili sinyal değil (değer aralığı, menü komutu). Olay ile girdi farkı: olayın **zamanlaması** onu işleyen sürecin **kontrolü dışında**.
- Olay güdümlü sistem türleri: **editörler** (UI olayları), **kural tabanlı üretim sistemleri** (yapay zekâ), **aktif nesneler**.
- ⭐⭐ **2 olay güdümlü model:**
  - Broadcast models: olay **tüm alt sistemlere** yayınlanır, işleyebilen yanıt verir. **Dağıtık** alt sistemleri entegre etmede etkili.
  - Interrupt-driven models: sadece **real-time**. Dış **kesmeleri** kesme işleyicisi algılar, başka bileşene aktarır. **Katı zamanlama** gereksinimi.
- ⭐ **Şekil 11.9, selective broadcasting:** alt sistemler olaylara **ilgi kaydeder**. **Event and message handler** bir **kayıt (register)** tutar, olayı sadece ilgilenenlere iletir. Kontrol politikası işleyicide **gömülü değil**. **Point-to-point** mesajı da destekler. Ör: event-listener (fare, klavye), Field, HP Softbench, **ORB**'lar.
- ⭐⭐ **Broadcast avantajı: evrim basit** (yeni alt sistem olaylarını kaydettirir) · başka alt sistemi **adını ya da yerini bilmeden** etkinleştirir · dağıtım **şeffaf**.
- ⭐⭐ **Broadcast dezavantajı:** alt sistem olayın **işlenip işlenmeyeceğini ve ne zaman işleneceğini bilmez** · kimin kayıtlı olduğunu bilmez · aynı olaya birden çok kayıt → **çatışma**.

- ⭐⭐ **Interrupt-driven (Şekil 11.10):** çok hızlı yanıt gereken real-time (ör. **hava yastığı**). Interrupts → **interrupt vector** (işleyici adresleri) → handlers → processes. Donanım anahtarı kontrolü **hemen** işleyiciye geçirir. **Merkezi modelle birleşebilir** (normal çalışma merkezi, acil durumlar kesmeyle).
- ⭐⭐ **Interrupt avantajı: çok hızlı yanıt**. **Dezavantajları:** programlaması **karmaşık**, **doğrulaması zor** (zamanlama kalıpları testte tekrarlanamaz), **değiştirmesi zor**, kesme sayısı **donanımla sınırlı** (birden çok olayı tek kesmeye eşleme, ama çok hızlı yanıtta pratik değil).

Sayfa 260–265 — 11.5 Reference architectures · OSI (Şekil 11.11) · ECMA CASE modeli (Şekil 11.12) · Key Points · Exercises

📖 Sayfanın özeti

ALANA ÖZGÜ MİMARİLER

Yukarıdaki mimari modeller **genel** modellerdir, birçok uygulama sınıfına uygulanabilir. Bunlara ek olarak belirli bir uygulama alanına **özgü** mimari modeller de kullanılabilir. Bu sistemlerin örnekleri ayrıntıda farklı olsa da ortak mimari yapı yeni sistemler geliştirirken yeniden kullanılabilir. Bunlara **alana özgü mimariler (domain-specific architectures)** denir. **İki tür** vardır: **1) Generic models:** bir dizi gerçek sistemden çıkarılan **soyutlamalardır**. Bu sistemlerin temel özelliklerini kapsarlar. Ör: gerçek zamanlı sistemlerde veri toplama ya da izleme sistemleri için genel mimari modeller olabilir (Bölüm 13). **2) Reference models: daha soyuttur** ve daha geniş bir sistem sınıfını tanımlar. Tasarımcılara o sistem sınıfının genel yapısı hakkında bilgi vermenin bir yoludur. Genelde uygulama alanının **incelenmesinden** türetilirler. Sistemlerin içerebileceği bütün özellikleri içeren **idealleştirilmiş** bir mimariyi temsil ederler.

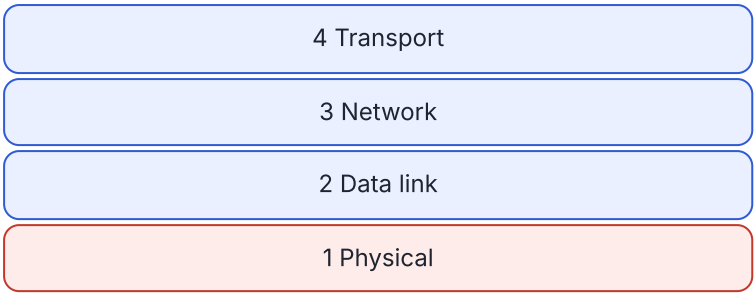
FARK VE KULLANIM

Bu iki model türü arasında elbette kesin bir ayrım yoktur, genel modeller de referans model işlevi görebilir. Yazar bunları ayırıyor, çünkü **genel modeller tasarımda doğrudan yeniden kullanılabilir**. **Referans modeller** ise normalde alan kavramlarını **iletmek** ve olası mimarileri **karşılaştırmak ya da değerlendirmek** için kullanılır. Referans mimariler normalde gerçekleştirmeye giden bir yol sayılmaz. Asıl işlevleri alana özgü mimarileri **tartışmak** ve bir alandaki farklı sistemleri **karşılaştırmaktır**. Referans model karşılaştırma için bir **sözlük (vocabulary)** sağlar ve sistemlerin değerlendirilebileceği bir **temel** görevi görür.

OSI MODELİ (ŞEKİL 11.11)

OSI, açık sistemler arası bağlantı (open systems interconnection) için **7 katmanlı** bir modeldir. Katmanlar (alttan üste): **1 Physical, 2 Data link, 3 Network, 4 Transport, 5 Session, 6 Presentation, 7 Application**. İki uç sistem 7 katmanın hepsine sahiptir, aradaki ağ düğümü sadece alt üç katmanı (physical, data link, network) kullanır ve hepsi iletişim ortamına bağlıdır. Katmanların tam işlevleri burada önemli değil. Özünde **alt katmanlar fiziksel bağlantıyla**, **orta katmanlar veri aktarımıyla**, **üst katmanlar** ise standartlaştırılmış belgeler gibi anlamlı uygulama bilgisinin aktarımıyla ilgilenir.





Şekil 11.11: OSI referans modeli (alt 3 katman = ağ düğümü de kullanır)

OSI: AMAÇ VE GERÇEK

OSI tasarımcılarının çok pratik bir hedefi vardı: uyumlu sistemlerin birbiriyle iletişim kurabilmesi için bir **gerçekleştirme standardı** tanımlamak. Her katman sadece **altındaki katmana** bağlı olmalıydı. Teknoloji geliştikçe bir katman, diğer katmanları kullanan sistemleri etkilemeden **şeffaf şekilde yeniden gerçekleştirilebilmeliydi**. Pratikte ise katmanlı yaklaşımın **performans sorunları** bu hedefi zayıflattı. Ağlar arasındaki büyük farklar yüzünden basit bağlantı imkânsız olabilir. Her katmanın fonksiyonel özellikleri iyi tanımlanmış olsa da **fonksiyonel olmayan özellikleri tanımlanmamıştır**. Geliştiriciler kendi üst seviye olanaklarını gerçekleştirip modeldeki **katmanları atlamak** ya da sistem performansını artırmak için standart dışı özellikler tasarlamak zorunda kalır. Sonuç olarak modeldeki bir katmanın şeffaf şekilde değiştirilmesi pratikte **neredeyse hiç mümkün değildir**. Ama bu, modelin **faydasını ortadan kaldırmaz**: sistemler arası iletişimin soyut yapılandırılması ve sistematik gerçekleştirilmesi için bir **temel** sağlar.

ECMA CASE REFERANS MODELİ (ŞEKİL 11.12)

Önerilen başka bir referans model **CASE ortamları** içindir (**ECMA**, Brown ve arkadaşları). Bir CASE ortamının sağlaması gereken **beş hizmet kümesini** belirler. Ayrıca bu hizmetleri kullanan tek tek CASE araçları için **"tak-çalıştır" (plug-in)** olanakları sağlamalıdır (şekilde araç yuvaları, tool slots). **5 hizmet seviyesi: 1) Data repository services:** veri öğelerinin ve ilişkilerinin **saklanması ve yönetimi**. **2) Data integration services:** grupları yönetmek ya da aralarında ilişki kurmak. Bunlar ve veri deposu hizmetleri ortamda **veri entegrasyonunun** temelidir. **3) Task management services: süreç modellerinin** tanımlanması ve uygulanması. **Süreç entegrasyonunu** destekler. **4) Message services:** araç-araç, ortam-araç ve ortam-ortam **iletişimi**. **Kontrol entegrasyonunu** destekler. **5) User interface services: kullanıcı arayüzü** geliştirme olanakları. **Sunum entegrasyonunu** destekler.

ECMA MODELİNİN KULLANIMI

Bu referans model herhangi bir CASE ortamına **nelerin dahil edilebileceğini** söyler, ama bir referans mimarinin her özelliğinin gerçek mimari tasarımlarda bulunmayacağını vurgulamak önemlidir. Bu, bir sistem tasarımı hakkında "veri deposu hizmetleri nasıl sağlanıyor?" ya da "sistem görev yönetimi sağlıyor mu?" gibi **sorular sorabileceğimiz** anlamına gelir. Bu referans mimarinin asıl değeri yine entegre CASE araç ve ortamlarını **sınıflandırmak ve karşılaştırmak** için bir araç olmasıdır. Ayrıca eğitimde bu ortamların temel özelliklerini vurgulamak ve onları genel bir şekilde tartışmak için kullanılabilir.

KEY POİNTS (BÖLÜM ÖZETİ)

Yazılım mimarisi sistemi yapılandırmanın temel çerçevesidir, performans, güvenlik ve availability gibi özellikler kullanılan mimariden etkilenir. Mimari tasarım kararları uygulamanın türü, sistemin dağıtımı, kullanılacak mimari stiller ve mimarinin nasıl belgelenip değerlendirileceği hakkındaki kararları içerir. Mimari tasarım sürecinde yapısal model, kontrol modeli ve ayrıştırma modeli gibi farklı modeller geliştirilebilir. Sistemin organizasyon modelleri repository, client-server ve soyut makine modellerini içerir: repository modelleri veriyi ortak bir depo üzerinden paylaşır, client-server modelleri genelde

veriyi dağıtır, soyut makine modelleri katmanlıdır ve her katman temel katmanının olanaklarıyla gerçekleştirilir. Ayrıştırma stilleri nesne yönelimli ve fonksiyon odaklı ayrıştırmayı içerir: boru hattı modelleri fonksiyoneldir, nesne modelleri kendi durumlarını ve işlemlerini tutan gevşek bağlı varlıklara dayanır. Kontrol stilleri merkezi kontrol ve olay tabanlı kontrolü içerir: merkezi modellerde kontrol kararları sistem durumuna göre, olay modellerinde dış olaylar sistemi kontrol eder. Referans mimariler alana özgü mimarileri tartışmak ve mimari tasarımları değerlendirip karşılaştırmak için bir araç olarak kullanılabilir.

FURTHER READİNG

Bass ve arkadaşlarının *Software Architecture in Practice*, Bosch'un *Design and Use of Software Architectures* (ürün hattı mimarileri) ve Shaw ile Garlan'ın ilk yazılım mimarisi kitabı *Software Architecture: Perspectives on an Emerging Discipline*. *Sınav için önemli değil.*

ALIŞTIRMALAR (KONULARI)

#	Konu
11.1	Sistem mimarisini tanımlamalar yazılmadan **önce** tasarlamak neden gerekebilir?
11.2	Availability ve security en önemli gereksinimler olduğunda mimari tasarımda ne tür **çatışmalar** çıkabilir?
11.3	Bu bölümdeki **yapısal modellerin** avantaj ve dezavantajlarını gösteren bir tablo hazırlayın.
11.4	Şu sistemler için uygun yapısal model önerin: tren istasyonunda otomatik **bilet satış** sistemi, birçok katılımcının video, ses ve veriyi gördüğü **video konferans** sistemi, koridor gibi açık alanları temizleyen, duvar ve engelleri algılayan **robot süpürge**.
11.5	Yukarıdaki sistemler için seçtiğiniz modele dayalı bir mimari tasarlayın.
11.6	Gerçek zamanlı sistemler genelde olay güdümlüdür. Hangi durumda gerçek zamanlı bir sistem için **call-return** kontrol modeli önerirsiniz?
11.7	Şu sistemler için uygun **kontrol modeli** önerin: çalışma saatleri ve ücretlerle **maaş bordrosu** basan toplu işlem sistemi, farklı firmaların ürettiği ama birlikte çalışması gereken **yazılım araçları**, uzaktan kumandaya yanıt veren **TV denetleyicisi**.
11.8	Tek makine ve dağıtık sürümler için **data-flow** ve **object** modellerinin dağıtılabilirlik açısından artı ve eksileri.
11.9	İki entegre CASE araç takımını karşılaştırmak için bir **CASE referans modelini** nasıl kullanırsınız?
11.10	Müşteriyle bağımsız çalışıp mimari tasarlayan ayrı bir "**yazılım mimarı**" mesleği olmalı mı? Zorlukları neler olur?

👨‍🏫 Hoca anlatımı

Generic ve reference model farkı: Generic = gerçek sistemlerden çıkarılmış **hazır kalıp**, doğrudan kullanırsın (bir ev planı şablonu gibi). **Reference** = daha soyut, **ideal** bir model. Doğrudan inşa etmek için değil, **karşılaştırmak, değerlendirmek ve konuşmak** için kullanılır (bir "ideal ev nasıl olmalı" rehberi gibi: "bu evde mutfak var mı, banyo var mı?" diye kontrol edersin).

OSI en ünlü referans model. 7 katmanı aşağıdan yukarı ezberlemek için klasik hatırlatıcı: "**P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way" → Physical, Data link, Network, Transport, Session,

Presentation, Application. Alt katmanlar = fiziksel bağlantı, orta = veri aktarımı, üst = uygulama bilgisi.

OSI'nin dersi: **teoride** her katman bağımsız değiştirilebilir. **Pratikte** performans sorunları yüzünden geliştiriciler katmanları atlar, bu yüzden şeffaf değiştirme neredeyse hiç olmaz. Ama model yine de çok faydalı (düşünme ve konuşma çerçevesi). Bu, layered modelin dezavantajlarının gerçek hayattaki örneği.

ECMA CASE modeli: bir CASE ortamının 5 hizmeti. Her biri bir entegrasyon türünü destekler: **Data repository + data integration → veri entegrasyonu**, **Task management → süreç entegrasyonu**, **Message → kontrol entegrasyonu**, **User interface → sunum entegrasyonu**. Alıştırma 11.9 bunu soruyor: iki CASE ürününü karşılaştırmak için bu 5 hizmet üzerinden "bu ürün bunu nasıl sağlıyor?" diye tek tek bakarsın.

Alıştırma 11.4 kısa cevaplar: bilet makinesi → **client-server** (makineler istemci, merkezi sunucu) ya da basit sistemse **repository**. Video konferans → **client-server** (dağıtık, birden çok katılımcı). Robot süpürge → **layered** ya da olay güdümlü (sensör, kontrol, motor katmanları).

📝 Deftere not

- ⭐ Domain-specific architectures: belirli uygulama alanına özgü, ortak yapı yeniden kullanılır.
- ⭐⭐ **2 tür:**
  - Generic models: gerçek sistemlerden **soyutlama**, temel özellikleri kapsar. Ör: real-time veri toplama ve izleme sistemleri (Bölüm 13). **Tasarımda doğrudan yeniden kullanılır.**
  - Reference models: **daha soyut**, daha geniş sistem sınıfı. Alan **incelemesinden** türer, **idealleştirilmiş** mimari. **İletişim**, **karşılaştırma ve değerlendirme** için.
- ⭐ Referans mimariler gerçekleştirme yolu **değil**. İşlevi: alana özgü mimarileri **tartışmak**, sistemleri **karşılaştırmak**. Karşılaştırma için **sözlük (vocabulary)** + değerlendirme **temeli**.
- ⭐⭐ **OSI (Şekil 11.11):** açık sistemler arası bağlantı için **7 katman**: 1 Physical · 2 Data link · 3 Network · 4 Transport · 5 Session · 6 Presentation · 7 Application. Ara ağ düğümü sadece alt 3 katman. **Alt** = fiziksel bağlantı, **orta** = veri aktarımı, **üst** = anlamlı uygulama bilgisi.
- ⭐ OSI amacı: **gerçekleştirme standardı**, her katman sadece **altındakine** bağlı, katman **şeffaf değiştirilebilir**.
- ⭐⭐ OSI gerçeği: katmanlı yaklaşımın **performans sorunları** + ağ farkları + katmanların **NFR'leri tanımsız** → geliştiriciler **katmanları atlar**, standart dışı özellik ekler → şeffaf değiştirme **neredeyse hiç mümkün değil**. Ama model hâlâ **faydalı** (soyut yapı ve sistematik iletişim temeli).
- ⭐⭐ **ECMA CASE referans modeli (Şekil 11.12):** 5 hizmet + araçlar için **plug-in** (tool slots):
  - **Data repository services:** veri öğeleri ve ilişkilerin saklanması ve yönetimi.
  - **Data integration services:** grupları yönetme, ilişki kurma. (1 + 2 = **veri entegrasyonu**)
  - **Task management services:** süreç modellerinin tanımı ve uygulanması → **süreç entegrasyonu**.
  - **Message services:** araç-araç, ortam-araç, ortam-ortam iletişimi → **kontrol entegrasyonu**.
  - **User interface services:** UI geliştirme → **sunum entegrasyonu**.
- Referansın her özelliği gerçek tasarımda olmaz. "Veri deposu nasıl sağlanıyor?", "görev yönetimi var mı?" gibi **sorular sorulabilir**. Asıl değer: CASE ortamlarını **sınıflandırma ve karşılaştırma** +

eğitim.

- ⭐⭐ **KEY POINTS:** mimari = sistemi yapılandırmanın **temel çerçevesi**, performans, güvenlik, availability'yi etkiler · kararlar: uygulama türü, dağıtım, stiller, belgeleme ve değerlendirme · modeller: **yapısal, kontrol, ayrıştırma** · organizasyon: **repository** (ortak depo), **client-server** (genelde dağıtık veri), **abstract machine** (katmanlı) · ayrıştırma: **OO** (gevşek bağlı, kendi durumu ve işlemleri) ve **function-oriented / pipeline** · kontrol: **centralised** (sistem durumu) ve **event-based** (dış olaylar) · referans mimariler: tartışma, değerlendirme, karşılaştırma.
- ⭐ Sınavda en olası: **11.3** (artı ve eksi tablosu), **11.4** (yapısal model seçimi), **11.7** (kontrol modeli seçimi), **11.2** (çatışma), **11.1**.
- ⭐ **11.7 kısa cevaplar:** maaş bordrosu (batch) → **call-return** · farklı firmaların araçları → **broadcast** · TV kumandası → **interrupt-driven**.

BÖLÜM 11 SONU: TEKRAR KISMI

Terim sözlüğü · ⭐ Mutlaka bil · Karşılaştırma tablosu · Olası sınav soruları · Mini test

A Terim sözlüğü

Bölüm 11'deki tüm terimler

İngilizce	Türkçe	Anlamı	Sayfa
software architecture	yazılım mimarisi	Sistemi yapılandırmanın temel çerçevesi	242
architectural design	mimari tasarım	Alt sistemleri ve kontrol / iletişim çerçevesini belirleme	242
sub-system	alt sistem	Kendi başına çalışabilen sistem parçası	242, 252
stakeholder communication	paydaş iletişimi	Mimarinin 1. avantajı	242
system analysis	sistem analizi	Mimarinin 2. avantajı	242
large-scale reuse	büyük ölçekli yeniden kullanım	Mimarinin 3. avantajı	242
product-line architecture	ürün hattı mimarisi	İlişkili sistemlerde aynı mimari	242, 245
complexity management	karmaşıklık yönetimi	Ayrıntıyı gizleyip ana soyutlamalara odaklanma	242
large-grain / fine-grain components	büyük taneli / ince taneli bileşenler	Büyük ve az parça / küçük ve çok parça	242–243
redundant components	yedekli bileşenler	Availability için	243
self-contained	kendi kendine yeten	Bağımsız bileşen	243
block diagram	blok diyagramı	Kutu = alt sistem, ok = veri / kontrol	243–244
box-and-line diagram	kutu-çizgi diyagramı	Basit mimari gösterim	244
packing robot	paketleme robotu	Şekil 11.1 örneği	243–244
localised change	yerel değişiklik	Değişikliğin tek alt sistemde kalması	244
architectural style	mimari stil	Sistem organizasyonu kalıbı (Garlan & Shaw)	246
composite architecture	bileşik mimari	Birden çok stilin birleşimi	246

static structural model	statik yapısal model	Ayrı geliştirilecek alt sistemler	246
dynamic process model	dinamik süreç modeli	Çalışma zamanındaki süreçler	246
interface model	arayüz modeli	Alt sistemlerin sunduğu hizmetler	246
relationship model	ilişki modeli	Alt sistemler arası veri akışı vb.	246
distribution model	dağıtım modeli	Alt sistemlerin bilgisayarlara dağılımı	246
ADL (architectural description language)	mimari tanımlama dili	Components + connectors, uzman dili	246–247
components / connectors	bileşenler / bağlayıcılar	ADL'lerin temel öğeleri	247
system organisation	sistem organizasyonu	Sistemi yapılandırmanın temel stratejisi	247
repository model	depo modeli	Paylaşılan merkezi veritabanı	247
project repository	proje deposu	CASE araç takımının merkezi	248
schema / data model	şema / veri modeli	Deponun ortak veri yapısı	248
blackboard model	kara tahta modeli	Veri hazır olunca alt sistemleri tetikleyen aktif depo	249
client-server model	istemci-sunucu modeli	Hizmetler, sunucular, istemciler, ağ	249
server / client	sunucu / istemci	Hizmet sunan / hizmet isteyen	249
remote procedure call	uzak prosedür çağrısı	İstemcinin sunucuya erişim yolu	249
request-reply protocol	istek-cevap protokolü	Ör. http	249
XML	XML	Şema dönüşümü kolay ama verimsiz veri gösterimi	250
layered model / abstract machine model	katmanlı model / soyut makine modeli	Her katman bir hizmet kümesi	250
APSE	Ada programlama destek ortamı	Buxton'ın 3 katmanlı modeli	251

portable	taşınabilir	Farklı platforma kolay geçen	251
punch through	delip geçmek	Katmanları atlayarak alt katmana erişmek	252
modular decomposition	modüler ayrıştırma	Alt sistemleri modüllere bölme	252
module	modül	Bağımsız sistem sayılmayan, hizmet sunan bileşen	252
object-oriented decomposition	nesne yönelimli ayrıştırma	İletişim kuran nesnelere bölme	252–253
loosely coupled	gevşek bağlı	Birbirine az bağımlı	253
concurrency	eşzamanlılık	Paralel çalışma (erken karar verme)	253
attributes / operations	öznitelikler / işlemler	Nesne sınıfının bilgisi ve davranışları	253
invoice / receipt / reminder	fatura / makbuz / hatırlatma	Örnek fatura sistemi	253–255
function-oriented pipelining	fonksiyon odaklı boru hattı	Veriyi dönüştüren adımlar dizisi	254
data-flow model	veri akış modeli	Pipeline'ın diğer adı	254
transform	dönüşüm	Bir işleme adımı	254
pipe and filter	boru ve süzgeç	Unix tarzı pipeline	254
batch sequential	toplu sıralı	Sıralı + toplu veri işleme (faturalama)	254
parse / unparse	ayrıştırma / geri biçimleme	Pipeline'da veri formatı yükü	255
control styles	kontrol stilleri	Alt sistemler arası kontrol akışı	256
centralised control	merkezi kontrol	Bir alt sistem diğerlerini yönetir	256
event-based control	olay tabanlı kontrol	Alt sistemler dış olaylara tepki verir	256
call-return model	çağır-dön modeli	Yukarıdan aşağı subroutine, sıralı	256–257
subroutine	alt program	Çağrılan rutin	256

manager model	yönetici modeli	Eşzamanlı süreçleri yöneten merkezi bileşen	256
state variables	durum değişkenleri	Merkezi kontrol kararlarının dayanağı	256–257
sensors / actuators	sensörler / eyleyiciler	Gerçek zamanlı sistem bileşenleri	257–258
soft real-time	yumuşak gerçek zamanlı	Çok sıkı zaman kısıtı olmayan	257
polling / event-loop	yoklama / olay döngüsü	Denetleyicinin sürekli kontrolü	257
event	olay	Zamanlaması işleyenin kontrolü dışındaki sinyal	258
broadcast model	yayın modeli	Olay alt sistemlere yayınlanır	258
event and message handler	olay ve mesaj işleyici	Olayları ilgilenenlere ileten bileşen	259
register (interest)	ilgi kaydı	Alt sistemin hangi olayları istediği	259
point-to-point	noktadan noktaya	Doğrudan mesaj	259
ORB (Object Request Broker)	nesne istek aracısı	Dağıtık nesne iletişimi	259
interrupt-driven model	kesme güdümlü model	Kesmelerle hızlı yanıt (real-time)	258, 260
interrupt / interrupt handler / interrupt vector	kesme / işleyici / kesme vektörü	Kesme ve işleyici adresleri	260
reference architecture	referans mimari	Karşılaştırma ve değerlendirme için ideal model	260–261
domain-specific architecture	alana özgü mimari	Belirli alana özgü yapı	261
generic model	genel model	Gerçek sistemlerden soyutlama, doğrudan kullanılır	261
OSI reference model	OSI referans modeli	7 katmanlı ağ modeli	261
ECMA reference architecture	ECMA referans mimarisi	CASE ortamları için 5 hizmet	262
tool slots / plug-in	araç yuvaları / eklenti	CASE araçlarının takıldığı yerler	262

B ⭐ Mutlaka bil

deftere elle yaz

Bölüm 11'den sınavda en çok çıkabilecekler

1. **Mimari tasarım tanımı** (alt sistemler + kontrol ve iletişim çerçevesi), tasarımın ilk aşaması (s.242)
2. **3 avantaj (Bass):** stakeholder communication, system analysis, large-scale reuse (s.242)
3. **NFR'ye göre mimari:** performance (large-grain, az iletişim), security (katman, değerli en içte), safety (tek alt sistem), availability (redundant), maintainability (fine-grain, paylaşılan veri yok) + **çatışma** (s.242–243)
4. **Blok diyagramı** ve Bass'ın eleştirisi / yazarın savunması (s.243–244)
5. **8 karar sorusu** (s.245)
6. **Mimari stil** tanımı, 3 karar grubu: organizasyon, ayrıştırma, kontrol (s.246)
7. **5 mimari model:** static, dynamic, interface, relationship, distribution · **ADL** (s.246–247)
8. **Repository**: tanım, örnek (CASE), **4 artı / 4 eksi** · blackboard (s.247–249)
9. **Client-server**: 3 bileşen, request-reply, film kütüphanesi örneği, artı / eksi (s.249–250)
10. **Layered**: soyut makine, Şekil 11.4, artılar (artımlı, değiştirilebilir, taşınabilir) / eksiler (punch through, performans) (s.250–252)
11. **Alt sistem ve modül farkı** (s.252)
12. **OO decomposition** artı / eksi · **pipeline** (pipe and filter, batch sequential) 4 artı / 2 eksi (s.253–255)
13. **Centralised** (call-return, manager) ve **event-based** (broadcast, interrupt-driven) + her birinin artı / eksisi (s.256–260)
14. **Generic ve reference model** farkı · **OSI 7 katman** + pratik sorunları · **ECMA 5 hizmeti** (s.260–263)

C Karşılaştırma tablosu (Alıştırma 11.3)

deftere geçir

Model	Avantajlar	Dezavantajlar
Repository	Büyük veriyi paylaşmak verimli · veri üreten, kullanımla ilgilenmez · yedekleme, güvenlik, kurtarma merkezi · uyumlu yeni araç eklemek kolay	Ortak veri modeli = uzlaşma, performans düşebilir · evrim zor · herkese aynı politika · dağıtmak zor
Client-server	Dağıtık, ağdaki işlemcileri kullanır · yeni sunucu eklemek ve sunucuyu şeffaf yükseltmek kolay	Ortak veri modeli yok · değişiklik istemcileri de etkileyebilir · XML dönüşümü verimsiz
Layered	Artımlı geliştirme · arayüz aynıysa katman değiştirilebilir · değişiklik sadece komşuyu etkiler · taşınabilir, çok platform	Yapılandırmak zor (punch through) · çoklu yorumlama yüzünden performans
Object-oriented	Gevşek bağlı, nesne değişince diğerleri etkilenmez · anlaşılır (gerçek dünya) · yeniden kullanılabilir · OO dilleri	Nesneler birbirinin adını ve arayüzünü bilmeli · karmaşık varlıkları göstermek zor
Pipeline	Dönüşümler yeniden kullanılır · sezgisel · yeni dönüşüm eklemek kolay · sıralı ya da paralel kolay	Ortak veri formatı şart (parse / unparse yükü) · etkileşimli (GUI) sistemlere uygun değil
Call-return	Kontrol akışını analiz etmek kolay	İstisnaları ele almak zahmetli · sadece sıralı
Broadcast	Evrim kolay (kayıt yeter) · ad ve yer bilmeden etkinleştirme · dağıtım şeffaf	Olayın işlenip işlenmeyeceği ve ne zaman işleneceği bilinmez · çatışma
Interrupt-driven	Çok hızlı yanıt	Karmaşık programlama · doğrulaması ve değiştirmesi zor · kesme sayısı donanımla sınırlı

D Olası sınav soruları

İngilizce cevap kalıbı + Türkçesi

Q1. What is architectural design? What are the advantages of explicitly designing a software architecture?

Mimari tasarım nedir, avantajları? (s.242)

> *Architectural design is the initial design process of identifying the sub-systems of a system and establishing a framework for sub-system control and communication. Its output is a description of the software architecture. Advantages: 1) stakeholder communication: it is a high-level presentation for discussion, 2) system analysis: early decisions affect performance, reliability and maintainability, 3) large-scale reuse: similar systems can use the same architecture.*

Mimari tasarım, sistemin alt sistemlerini belirleyip alt sistemlerin kontrolü ve iletişimi için bir çerçeve kuran ilk tasarım sürecidir. Çıktısı yazılım mimarisinin tanımıdır. Avantajları: 1) paydaş iletişimi: tartışma için üst seviye sunum, 2) sistem analizi: erken kararlar performans, güvenilirlik ve bakımı etkiler, 3) büyük ölçekli yeniden kullanım: benzer sistemler aynı mimariyi kullanabilir.

Q2. How do non-functional requirements influence the architecture?
NFR'ler mimariyi nasıl etkiler? (s.242–243)

> *Performance: localise critical operations in a small number of large-grain sub-systems with little communication. Security: use a layered structure with critical assets in the innermost layers. Safety: put safety-related operations in a single sub-system or a small number of sub-systems. Availability: include redundant components so they can be replaced without stopping the system. Maintainability: use fine-grain, self-contained components and avoid shared data structures. These can conflict, for example large-grain components help performance but fine-grain components help maintainability.*

Performans: kritik işlemleri az sayıda büyük taneli, az iletişimli alt sistemde topla. Güvenlik: kritik varlıklar en iç katmanda olacak şekilde katmanlı yapı. Emniyet: emniyet işlemlerini tek ya da az sayıda alt sistemde topla. Erişilebilirlik: sistemi durdurmadan değiştirilebilen yedekli bileşenler. Bakım: ince taneli, bağımsız bileşenler, paylaşılan veri yapısı yok. Bunlar çatışabilir: büyük taneli bileşen performansa, ince taneli bakıma yardım eder.

Q3. Explain why design conflicts might arise when availability and security are the most important requirements. (Ex. 11.2)
Availability ve security çatışması

> *Availability needs redundant components and the ability to replace and update components without stopping the system, so there are more components and more access paths. Security needs a layered structure with critical assets protected in the innermost layers and strong validation, which limits access and adds checks. More redundancy and easy replacement can create more vulnerabilities, while strict security checks can slow down or block recovery. A compromise must be found.*

Availability, yedekli bileşenler ve sistemi durdurmadan bileşen değiştirme ister, yani daha çok bileşen ve erişim yolu demektir. Security ise kritik varlıkların en içte korunduğu katmanlı yapı ve sıkı doğrulama ister, erişimi kısıtlar ve kontrol ekler. Fazla yedeklilik ve kolay değiştirme daha çok zafiyet yaratabilir, sıkı güvenlik kontrolleri ise kurtarmayı yavaşlatabilir. Bir uzlaşma bulunmalıdır.

Q4. What fundamental questions must be answered during architectural design?
Mimari tasarımda hangi sorular cevaplanır? (s.245)

> *Is there a generic application architecture to use as a template? How will the system be distributed across processors? Which architectural styles are appropriate? What is the fundamental approach to structure the system? How will the structural units be decomposed into modules? What strategy will control the units? How will the design be evaluated? How should it be documented?*

Şablon olabilecek genel bir uygulama mimarisi var mı? Sistem işlemcilere nasıl dağıtılacak? Hangi mimari stiller uygun? Sistemi yapılandırmanın temel yaklaşımı ne? Yapısal birimler modüllere nasıl ayrılacak? Birimler hangi stratejiyle kontrol edilecek? Tasarım nasıl değerlendirilecek? Nasıl belgelenecek?

Q5. What architectural models may be developed?
Hangi mimari modeller geliştirilebilir? (s.246)

> *1) A static structural model showing the sub-systems or components. 2) A dynamic process model showing how the system is organised into processes at run-time. 3) An interface model defining the services of each sub-system. 4) Relationship models showing relationships such as data flow. 5) A distribution model showing how sub-systems are distributed across computers.*

1) Alt sistemleri ya da bileşenleri gösteren statik yapısal model. 2) Çalışma zamanındaki süreçleri gösteren dinamik süreç modeli. 3) Her alt sistemin hizmetlerini tanımlayan arayüz modeli. 4) Veri akışı gibi ilişkileri gösteren ilişki modelleri. 5) Alt sistemlerin bilgisayarlara dağılımını gösteren dağıtım modeli.

Q6. Describe the repository model. What are its advantages and disadvantages?
Repository modeli (s.247–249)

> *In the repository model, all shared data is held in a central database that all sub-systems can access. It is used in CASE toolsets, CAD and management information systems. Advantages: it is efficient for sharing large amounts of data, producers do not need to know how data is used, backup and security are centralised, and new tools are easy to integrate. Disadvantages: sub-systems must agree on a common data model, evolution is difficult, the same policy is forced on all sub-systems, and it is difficult to distribute.*

Repository modelinde bütün paylaşılan veri, bütün alt sistemlerin erişebildiği merkezi bir veritabanında tutulur. CASE araç takımları, CAD ve yönetim bilgi sistemlerinde kullanılır. Avantajlar: büyük veriyi paylaşmak verimli, üreten kullanımla ilgilenmez, yedekleme ve güvenlik merkezi, yeni araç eklemek kolay. Dezavantajlar: ortak veri modeli şart, evrim zor, herkese aynı politika, dağıtmak zor.

Q7. Describe the client-server model. What are its advantages and disadvantages?
Client-server modeli (s.249–250)

> *The system is organised as a set of servers that offer services, a set of clients that call these services, and a network. Clients use a request-reply protocol such as http. The main advantage is that it is a distributed architecture: it is easy to add new servers and upgrade them transparently. The disadvantages are that there may be no shared data model across servers, and changes to clients and servers may be required to get the full benefit of a new server.*

Sistem; hizmet sunan sunucular, bu hizmetleri çağıran istemciler ve bir ağ olarak düzenlenir. İstemciler http gibi istek-cevap protokolü kullanır. Ana avantajı dağıtık bir mimari olmasıdır: yeni sunucu eklemek ve şeffaf yükseltmek kolaydır. Dezavantajları: sunucular arasında ortak veri modeli olmayabilir ve yeni sunucunun faydası için istemci ve sunucularda değişiklik gerekebilir.

Q8. Describe the layered (abstract machine) model. What are its advantages and disadvantages?

Katmanlı model (s.250–252)

> *The system is organised into layers, each providing a set of services and acting as an abstract machine for the layer above. Examples are the OSI model and a version management system (configuration management, object management, database, operating system). Advantages: it supports incremental development, a layer can be replaced if its interface stays the same, changes affect only adjacent layers, and it is portable. Disadvantages: it can be difficult to structure systems this way, outer layers may need to punch through to inner layers, and performance can suffer because of multiple levels of interpretation.*

Sistem, her biri hizmet sunan ve üstündeki katman için soyut makine gibi davranan katmanlara ayrılır. Örnekler OSI modeli ve sürüm yönetim sistemidir (konfigürasyon, nesne yönetimi, veritabanı, işletim sistemi). Avantajlar: artımlı geliştirme, arayüz aynıysa katman değiştirilebilir, değişiklik sadece komşu katmanları etkiler, taşınabilir. Dezavantajlar: bu şekilde yapılandırmak zor olabilir, dış katmanlar iç katmanlara delip geçmek zorunda kalabilir, çoklu yorumlama yüzünden performans düşebilir.

Q9. Compare object-oriented decomposition and function-oriented pipelining.

OO ve pipeline karşılaştırması (s.252–255)

> *In object-oriented decomposition, the system is a set of loosely coupled objects with private state and operations. It is understandable and reusable, but objects must know each other's names and interfaces. In function-oriented pipelining, the system is a set of transformations that process input data into output data, like Unix pipes and filters. It supports reuse of transformations and is intuitive, but needs a common data format and is not suitable for interactive systems. The object model is more abstract because it does not show the sequence of operations.*

Nesne yönelimli ayrıştırmada sistem, özel durumu ve işlemleri olan gevşek bağlı nesnelerdir. Anlaşılır ve yeniden kullanılabilir, ama nesneler birbirinin adını ve arayüzünü bilmelidir. Fonksiyon odaklı boru hattında sistem, girdiyi çıktıya dönüştüren dönüşümlerdir (Unix pipe and filter gibi). Dönüşümlerin yeniden kullanımını destekler ve sezgiseldir, ama ortak veri formatı ister ve etkileşimli sistemlere uygun değildir. Nesne modeli işlemlerin sırasını göstermediği için daha soyuttur.

Q10. Describe the centralised control models.

Merkezi kontrol modelleri (s.256–257)

> *In centralised control, one sub-system controls the others. 1) The call-return model is a top-down subroutine model: control starts at the top of a hierarchy and passes down through subroutine calls, then returns. It is only for sequential systems. It is easy to analyse but exceptions are awkward to handle. 2) The manager model is for concurrent systems: one component is a system manager that starts, stops and coordinates other processes, often by polling sensors in an event loop. It is used in soft real-time systems.*

Merkezi kontrolde bir alt sistem diğerlerini kontrol eder. 1) Call-return modeli yukarıdan aşağı alt program modelidir: kontrol hiyerarşinin tepesinden başlar, çağrılarla aşağı iner ve geri döner. Sadece sıralı sistemler içindir. Analiz etmesi kolay ama istisnaları ele almak zahmetlidir. 2) Manager modeli eşzamanlı sistemler içindir: bir bileşen diğer süreçleri başlatan, durduran ve koordine eden yöneticidir, genelde sensörleri bir olay döngüsünde yoklar. Yumuşak gerçek zamanlı sistemlerde kullanılır.

Q11. Describe the event-driven control models.

Olay güdümlü kontrol modelleri (s.258–260)

> *Event-driven models are driven by externally generated events. 1) In broadcast models, sub-systems register an interest in events and an event and message handler passes the events to them. Evolution is simple and sub-systems do not need to know each other's names or locations, but a sub-system does not know if or when its events will be handled. 2) In interrupt-driven models, used in real-time systems, interrupts are detected by an interrupt handler and control is transferred immediately. They give very fast responses but are complex to program and difficult to validate.*

Olay güdümlü modeller dış olaylarla yönlendirilir. 1) Yayın modellerinde alt sistemler olaylara ilgi kaydeder ve olay ve mesaj işleyici olayları onlara iletir. Evrim basittir, alt sistemler birbirinin adını ya da yerini bilmez, ama bir alt sistem olayının işlenip işlenmeyeceğini ve ne zaman işleneceğini bilmez. 2) Gerçek zamanlı sistemlerde kullanılan kesme güdümlü modellerde kesmeler işleyici tarafından algılanır ve kontrol hemen aktarılır. Çok hızlı yanıt verir ama programlaması karmaşık ve doğrulaması zordur.

Q12. Suggest an appropriate control model for: a batch payroll system, tools from different vendors that must work together, a TV controller. (Ex. 11.7)

Kontrol modeli seçimi

> *Batch payroll system: call-return model, because it is a sequential system with no external events. Tools from different vendors: broadcast model, because tools can register for events and do not need to know each other's names or locations. TV controller: interrupt-driven model, because it must respond quickly to signals from the remote control.*

Toplu maaş bordrosu: call-return, çünkü dış olayı olmayan sıralı bir sistemdir. Farklı firmaların araçları: broadcast, çünkü araçlar olaylara kaydolur ve birbirinin adını ve yerini bilmesi gerekmez. TV denetleyicisi: interrupt-driven, çünkü kumandadan gelen sinyallere hızlı yanıt vermelidir.

Q13. What is the difference between generic models and reference models? Describe the OSI model.

Generic ve reference model, OSI (s.260–262)

> *Generic models are abstractions from real systems and can be reused directly in a design. Reference models are more abstract and idealised. They describe a larger class of systems and are used to communicate domain concepts and to compare and evaluate architectures. The OSI model is a seven-layer reference model for open systems interconnection: physical, data link, network, transport, session, presentation and application. In practice, performance problems mean that layers are often bypassed, so transparent replacement of a layer is hardly ever possible.*

Genel modeller gerçek sistemlerden soyutlamalardır ve tasarımda doğrudan yeniden kullanılabilir. Referans modeller daha soyut ve idealdir, daha geniş bir sistem sınıfını tanımlar ve alan kavramlarını iletmek, mimarileri karşılaştırmak ve değerlendirmek için kullanılır. OSI, açık sistemler arası bağlantı için 7 katmanlı bir referans modeldir: fiziksel, veri bağı, ağ, taşıma, oturum, sunum ve uygulama. Pratikte performans sorunları yüzünden katmanlar çoğu zaman atlanır, bu yüzden bir katmanın şeffaf değiştirilmesi neredeyse hiç mümkün olmaz.

Q14. What services does the ECMA reference model for CASE environments define? How can it be used? (Ex. 11.9)

ECMA CASE modeli (s.262–264)

> *It defines five services: data repository services, data integration services, task management services, message services and user interface services. It should also provide plug-in facilities for CASE tools. To compare two CASE toolsets, we can ask for each toolset how it provides each of these services, for example "how are data repository services provided?" and "does it provide task management?".*

Beş hizmet tanımlar: veri deposu, veri entegrasyonu, görev yönetimi, mesaj ve kullanıcı arayüzü hizmetleri. CASE araçları için eklenti olanağı da sağlamalıdır. İki CASE ürününü karşılaştırmak için her biri için bu hizmetleri nasıl sağladığını sorarız, örneğin "veri deposu hizmetleri nasıl sağlanıyor?" ve "görev yönetimi var mı?".

E Mini test

cevaplar en altta

1. The output of the architectural design process is a description of the:
   a) test plan
   b) software architecture
   c) user interface
   d) project schedule
2. If performance is critical, the architecture should use:
   a) many fine-grain components
   b) a few large-grain components
   c) shared data structures only
   d) many layers
3. If security is critical, the architecture should use:
   a) a layered structure with critical assets in the innermost layers
   b) a broadcast model
   c) redundant components only
   d) no layers

4. If maintainability is critical, you should use:
   a) large-grain components
   b) shared data structures
   c) fine-grain, self-contained components
   d) one big module
5. A pattern of system organisation such as client-server is an:
   a) architectural style
   b) ADL
   c) interrupt
   d) reference model
6. The basic elements of ADLs are:
   a) classes and objects
   b) components and connectors
   c) layers and pipes
   d) events and handlers
7. All shared data held in a central database is the:
   a) client-server model
   b) repository model
   c) layered model
   d) pipeline model
8. A repository that triggers sub-systems when data becomes available is a:
   a) blackboard model
   b) call-return model
   c) OSI model
   d) batch model
9. In the client-server model, servers:
   a) must know all clients
   b) need not know the identity or number of clients
   c) call the clients
   d) cannot be distributed
0. "Punching through" adjacent layers is a disadvantage of the:
   a) repository model
   b) layered model
   c) broadcast model
   d) object model
1. Unix pipes and filters are an example of:
   a) object-oriented decomposition
   b) function-oriented pipelining
   c) interrupt-driven control
   d) repository model
2. A main problem of the pipeline model is:
   a) objects must know each other's names
   b) a common data format is needed
   c) too fast responses
   d) no reuse
3. The call-return model is applicable only to:
   a) sequential systems
   b) concurrent systems
   c) distributed systems
   d) real-time systems
4. A central controller that polls sensors in a loop uses the:
   a) broadcast model
   b) manager (event-loop) model
   c) pipeline model
   d) repository model
5. In the broadcast model, a sub-system that generates an event:
   a) knows exactly who will handle it
   b) does not know if or when it will be handled
   c) must handle it itself
   d) stops all sub-systems
6. An airbag control system should use:
   a) call-return control
   b) interrupt-driven control
   c) batch sequential
   d) repository model

7. Models used mainly to compare and evaluate architectures are:

a) generic models
b) reference models
c) dynamic models
d) distribution models

8. How many layers does the OSI reference model have?

a) 4
b) 5
c) 7
d) 9

Cevap anahtarı: 1-b · 2-b · 3-a · 4-c · 5-a · 6-b · 7-b · 8-a · 9-b · 10-b · 11-b · 12-b · 13-a · 14-b · 15-b · 16-b · 17-b · 18-c